



On retraining intervals and sequence lengths for machine learning models in foreign exchange rate forecasting

by
Cassius Ray Vlok

Thesis presented in partial fulfilment of the requirements for the degree of
Master of Commerce (Operations Research) in the Faculty of
Economic and Management Sciences at
Stellenbosch University

Supervisor: Prof SE Visagie

March 2024

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: March 2024

Copyright © 2024 Stellenbosch University

All rights reserved

Abstract

The foreign exchange market is non-stationary, highly volatile, noisy and non-linear, making it challenging for time series predictions. Market conditions constantly evolve, and finding a robust model that can capture current patterns while adjusting to emerging ones is difficult. Machine learning (ML) models are prone to a phenomenon known as model decay, meaning performance tends to worsen over time as the characteristics of the data set begin to change. Additionally, ML models require sufficient data to learn from, and make accurate predictions.

This study investigates the effects of varying how frequently a model is retrained (retraining interval) and the number of lagged data points to use as features (sequence length). The effects are measured on three ML models: a deep neural network capable of processing sequential time series data called long-short term memory (LSTM), an ensemble learning algorithm, random forests (RF), combining multiple decision trees to form a final output and a support vector machine (SVM) model capable of mapping non-linear data into higher dimensions to perform linear regression.

Each model's hyperparameters were optimised by a sequential model-based optimiser, Bayesian optimisation, to best fit the USD/ZAR exchange rate. The results from the calibration indicate that the activation function, ReLu, caused problems with convergence in the LSTM model, and the sigmoid and polynomial kernel functions led to poor results for the SVM model. The RF model was the most consistent and was less sensitive to the hyperparameters used.

The retraining intervals tested were yearly, bi-yearly, quarterly and monthly. The sequence lengths tested ranged from one to ten previous days used as features. The LSTM model results showed that the best mean squared error (MSE) comes from the 12-month retraining interval, which outperformed the 6, 3 and 1-month intervals by 0.51%, 3.06% and 5.53%, respectively. Sequence lengths had a smaller impact on the LSTM models, with the best MSE values from sequence lengths above 3. The 9-day sequence length had the lowest MSE of 0.2157, which was a 1.76% improvement over the worst MSE from 1 day.

Retraining intervals had a greater effect on the RF models, with the MSE of the 12-month intervals having a 1.54%, 6.35% and 10.71% improvement over the 6, 3, and 1-month intervals. RF models using the 1-day sequence length had a 1.54% lower MSE than the 2nd best sequence length. The sequence lengths of 2–10 days had similar performance with all these MSE values within 0.85% of each other.

The SVM had the lowest MSE of all the models at 0.02068, with the sequence length being the more critical hyperparameter to consider. Using a 1-day sequence length saw a 1.19% improvement over the 2nd best sequence length of 2 days and was 2.13% lower than the worst sequence length of 10 days. The MSE values from the different retraining intervals were within 0.14%, indicating minimal effects from this hyperparameter.

Opsomming

Die buitelandse valutamark is nie-stasionêr, hoogs wisselvallig, met baie geraas en nie-lineêr, wat dit uitdagend maak vir vooruitskattings. Marktoestande verander voortdurend, en dit is moeilik om 'n robuuste model te vind wat huidige patrone kan vasvang terwyl dit aanpas by ontwikkelende patrone. Masjienleer (ML) modelle is geneig tot 'n verskynsel bekend as modelverval, wat beteken dat die prestasie oor tyd versleg soos die eienskappe van die datastel begin verander. Daarbenewens vereis ML-modelle voldoende data om van te leer en akkurate voorspellings te maak.

Hierdie studie ondersoek die effekte van hoe gereeld 'n model heropgelei word (heropleidingsinterval) en die terugkykvenster (die aantal datapunte terug in die geskiedenis) om as kenmerke te gebruik (reekslengte). Die effek word gemeet op drie ML-modelle: 'n diep neurale netwerk in staat om tydreekse te verwerk, genaamd lang-korttermyngeheue (LSTM), 'n saamgestelde-leer algoritme, willekeurige woude (RF), wat verskeie besluitboomstrukture kombineer om 'n finale afvoer te vorm, en 'n ondersteuningsvektor masjien (SVM) model wat in staat is om nie-liniêre data in hoër dimensies te projekteer vir lineêre regressie.

Elke model se hiperparameters is geoptimeer deur 'n opeenvolgende modelgebaseerde optimeerder, Bayes-optimering, om die beste modelpassing vir die USD/ZAR wisselkoers te verkry. Die resultate van die kalibrasie dui aan dat die aktiveringsfunksie, ReLu, probleme veroorsaak het met konvergensie in die LSTM-model, en die sigmoid- en polinoomkernfunksies gelei het tot swak resultate vir die SVM-model. Die RF-model was die mees konsekwente en was minder sensitief vir die gebruikte hiperparameters.

Die heropleidingsintervalle wat getoets is, was jaarliks, twee maal per jaar, kwartaalliks en maandeliks. Die reekslengtes wat getoets is, het gewissel van een tot tien vorige dae wat as kenmerke gebruik is. Die resultate vir die LSTM-model het die beste gemiddelde fout kwadaat (MSE) getoon uit die 12-maande heropleidingsinterval, wat die 6, 3 en 1-maand-intervalle respektiewelik met 0.51%, 3.06% en 5.53% oortref. Reekslengtes het 'n kleiner impak op die LSTM-modelle gehad; die beste MSE-waardes was almal van reekslengtes bo 3. Die 9-daagse reekslengte het die laagste MSE van 0.2157 getoon, wat 'n 1.76% verbetering was oor die slegste MSE van 1 dag.

Heropleidingsintervalle het 'n groter effek gehad op die RF-modelle, met die MSE van die 12-maande-intervalle wat 'n 1.54%, 6.35% en 10.71% verbetering getoon het oor die 6, 3, en 1-maand-intervalle. RF-modelle wat die 1-daagse reekslengte gebruik het, 'n 1.54% laer MSE as die 2^{de} beste reekslengte. Die reekslengtes van 2 tot 10 dae het soortgelyke prestasie gehad, met al hierdie MSE-waardes binne 0.85% van mekaar.

Die SVM het die laagste MSE van 0.02068 van alle modelle gehad, met die reekslengte as 'n meer kriteke hiperparameter om te oorweeg. Die gebruik van 'n 1-dag-reekslengte het 'n 1.19% verbetering getoon oor die 2^{de} beste reekslengte van 2 dae en was 2.13% laer as die slegste reekslengte van 10 dae. Die MSE-waardes van die verskillende heropleidingsintervalle was binne 0.14%, wat dui op minimale effekte van hierdie hiperparameter.

Acknowledgements

The author wishes to acknowledge the following people for their various contributions towards the completion of this work:

- My supervisor, Prof SE Visagie, for his guidance, supervision and interesting discussions during my time as a postgraduate student.
- My parents, Burt and Linda, for their endless support and motivation.
- My friend, Timothy Rimell, for his friendship and encouragement throughout the many years of the project.
- The Department of Logistics at Stellenbosch University for their computer facilities.
- The Stellenbosch University Postgraduate Office for their generous financial support towards the project.

Any opinions or findings in this thesis are those of the author and do not necessarily reflect the view of Stellenbosch University.

Contents

1	Introduction	1
1.1	Background of the foreign exchange market	1
1.2	Trading on the foreign exchange market	2
1.3	Machine learning	4
1.4	Model decay	5
1.5	Problem description	6
1.6	Thesis scope	6
1.7	Thesis objectives	6
1.8	Thesis layout	7
2	Literature	9
2.1	Financial time series forecasting	9
2.2	Deep learning models	10
2.3	Support vector machines	14
2.4	Random forests	17
2.5	Summary	18
3	Data and feature generation	19
3.1	Data set	19
3.2	Data Cleaning	21
3.3	Baseline features	21
3.3.1	Open, high, low, close (OHLC)	21
3.3.2	Daily change	22
3.3.3	Day of the week	22
3.3.4	Next day's open price	23
3.4	Technical analysis based features	23
3.4.1	Moving average	24
3.4.2	Moving average convergence divergence	25

3.4.3	Average true range	25
3.4.4	Average directional index	27
3.4.5	Relative strength index	29
3.4.6	True strength index	31
3.4.7	Stochastic oscillator	31
3.4.8	Bollinger bands	32
3.4.9	Keltner Channel	34
3.5	Labels	35
3.6	Summary of features and parameter values	35
4	Data processing and modelling	37
4.1	Feature engineering	39
4.1.1	Data preprocessing	39
4.1.2	Feature scaling	42
4.1.3	Converting to supervised learning	43
4.2	Walk-forward validation	45
4.3	Models considered	46
4.4	Hyperparameter calibration	47
4.4.1	Bayesian optimisation	48
4.4.2	Cross-validation	49
4.5	Testing parameters	51
4.5.1	Retrain interval	51
4.5.2	Sequence length	52
4.6	Test sets	52
4.6.1	Test set 2018	53
4.6.2	Test set 2019	53
4.6.3	Test set 2020	54
4.6.4	Test set 2021	56
4.7	Result metrics	57
4.7.1	Statistical metrics	57
4.7.2	Threshold accuracy	58
4.7.3	Trading strategy/profit	58
4.7.4	Maximum drawdown	60
4.7.5	Number of trades	61
4.7.6	Average profit per trade	61
4.7.7	Profitable trades	61

4.8	Chapter summary	61
5	Statistical forecasting	63
5.1	Naïve model	63
5.2	Naïve results	63
5.3	ARIMA model	65
5.4	ARIMA Results	70
5.5	Chapter summary	72
6	Long short-term memory neural networks	73
6.1	Artificial neural networks	73
6.2	Recurrent neural networks	75
6.3	Long short-term memory	76
6.4	Hyperparameters	77
6.5	Hyperparameter calibration results	80
6.6	Results	84
6.7	Chapter summary	91
7	Random forests	93
7.1	Decision trees	93
7.2	Random forests (ensemble learning)	94
7.3	Hyperparameters	96
7.4	Hyperparameter calibration results	97
7.5	Results	100
7.6	Chapter summary	108
8	Support vector machines	109
8.1	Introduction	109
8.2	The kernel	110
8.3	Formulation	110
8.4	Hyperparameters	112
8.5	Hyperparameter calibration results	114
8.6	Results	117
8.7	Chapter summary	126
9	Model Comparison	127
9.1	Accuracy metrics	127

9.2	Profit metrics	129
9.3	Summary	131
10	Conclusion	133
10.1	Summary of findings	133
10.2	Recommendations	136
10.3	Achievement of objectives	136
10.4	Ideas for future work	137
	Appendix	139
A	Hyperparameter calibration results	139
A.1	LSTM	139
A.2	RF	142
A.3	SVM	146
A.4	ARIMA	149
B	Model results	153
B.1	LSTM	153
B.1.1	Test set 2018	153
B.1.2	Test set 2019	158
B.1.3	Test set 2020	162
B.1.4	Test set 2021	166
B.1.5	Test set 2018–2021	170
B.2	RF	174
B.2.1	Test set 2018	174
B.2.2	Test set 2019	178
B.2.3	Test set 2020	182
B.2.4	Test set 2021	186
B.2.5	Test set 2018–2021	190
B.3	SVM	194
B.3.1	Test set 2018	194
B.3.2	Test set 2019	198
B.3.3	Test set 2020	202
B.3.4	Test set 2021	206
B.3.5	Test set 2018–2021	210

List of Figures

1.1	An example of a foreign exchange price quote for the USD/ZAR currency pair.	2
1.2	An illustration of long and short trades.	3
1.3	The different fields within artificial intelligence.	4
1.4	The different types of machine learning applications.	5
2.1	The breakdown of feature sets used in literature to forecast financial markets.	10
2.2	The hybrid LSTM model proposed by Yildirim <i>et al.</i> [76].	12
2.3	The results for the sequence lengths tested by Pongsena <i>et al.</i> [54].	13
2.4	The results for the retraining intervals tested by Pongsena <i>et al.</i> [54].	14
2.5	The different types of deep learning models used for financial market predictions [31].	15
2.6	The NMSE using different kernel functions found by Kamruzzaman <i>et al.</i> [33].	16
3.1	The daily closing price of the USD/ZAR exchange rate.	20
3.2	An example of a candle stick plot for the USD/ZAR exchange rate.	22
3.3	An illustration of simple moving averages.	24
3.4	An illustration of the moving average convergence divergence.	26
3.5	An illustration of the average true range.	27
3.6	An illustration of the average directional index.	29
3.7	An illustration of the relative strength index.	30
3.8	An illustration of the true strength index.	32
3.9	An illustration of stochastic oscillators.	33
3.10	An illustration of Bollinger bands.	34
4.1	The flow of the modelling process used.	38
4.2	An illustration of the ML clipping phenomenon.	39
4.3	The difference between the closing price feature set distributions.	40
4.4	The difference between standardisation and min-max scaling.	44
4.5	The conversion of time series data into supervised ML data.	45

4.6	An illustration of rolling window walk-forward validation.	46
4.7	An illustration of expanding window walk-forward validation.	47
4.8	An illustration of the USD/ZAR 2018 market conditions.	53
4.9	An illustration of the USD/ZAR 2019 market conditions.	54
4.10	An illustration of the USD/ZAR 2020 market conditions.	55
4.11	An illustration of the USD/ZAR 2021 market conditions.	56
4.12	An illustration of maximum drawdown.	60
5.1	The results of the naïve forecast for a subset of 2018.	65
5.2	The results of the naïve forecast cumulative profit from 2018 to 2021.	66
5.3	The original closing prices used to calibrate the ARIMA model.	68
5.4	The transformed closing prices used to calibrate the ARIMA model.	69
5.5	The ARIMA(4,0,3) forecasted closing prices versus the actual closing prices. . . .	71
5.6	The ARIMA model's cumulative profit during 2018–2021 of the USD/ZAR dataset. .	72
6.1	A schematic representation of a single perception.	74
6.2	A schematic representation of a deep neural network architecture.	74
6.3	A schematic representation of unrolling a recurrent neural network through time. .	75
6.4	A schematic representation of an LSTM cell.	76
6.5	The different popular activation functions available for the LSTM NN.	78
6.6	An illustration of early stopping.	79
6.7	The best LSTM architecture found through Bayesian optimisation.	81
6.8	The convergence graph of the LSTM hyperparameter calibration.	82
6.9	The box and whisker plots for LSTM activation functions and initialisers.	83
6.10	The Pearson correlation coefficient matrix for LSTM hyperparameters.	83
6.11	The heatmaps of LSTM models' MSE and MAPE in 2018–2021.	85
6.12	The heatmaps of LSTM models' threshold accuracies in 2018–2021.	85
6.13	The heatmaps of LSTM models' cumulative profit and max drawdown in 2018–2021. .	86
6.14	The results of the best LSTM models' trading performance for 2018–2021.	86
6.15	The heatmaps of LSTM models' profitable and number of trades in 2018–2021. . .	87
6.16	The heatmaps of LSTM models' MSE and cumulative profit in 2018.	89
6.17	The heatmaps of LSTM models' MAPE and cumulative profit in 2019.	89
6.18	The heatmaps of LSTM models' cumulative profit and profitable trades in 2020. . .	90
6.19	The best cumulative profit of different retraining intervals for the LSTM model. .	90
7.1	A schematic representation of a single decision tree.	94
7.2	A schematic representation of the training process of random forests.	95

7.3	The box and whisker plots for RF's bootstrap hyperparameter values.	98
7.4	The Pearson correlations matrices for the RF hyperparameters.	99
7.5	The RF scatter plots of hyperparameters max features and min samples per leaf.	99
7.6	The heatmaps of RF models' MSE and MAPE in 2018–2021.	100
7.7	The MSE through time of the best RF models.	101
7.9	Comparing the different threshold accuracies for the RF model.	102
7.8	The heatmaps of RF models' threshold accuracies in 2018–2021.	103
7.10	The heatmaps of RF models' cumulative profit and max drawdown in 2018–2021.	104
7.11	The heatmaps of RF models' profitable and number of trades in 2018–2021.	105
7.12	The results of the best RF models' trading performance for 2018–2021.	106
7.13	The heatmaps of RF models' threshold accuracy and cumulative profit in 2018.	107
7.14	The heatmaps of RF models' MSE and cumulative profit in 2019.	107
7.15	The MSE of RF models over time for the lowest MSE models.	108
7.16	The heatmaps of RF models' MAPE and profitable trades in 2021.	108
8.1	An example of the SVM algorithm used for classification and regression.	110
8.2	An example of how the SVM uses the kernel function to map data.	111
8.3	The SVM scatter plots of gamma, reg (C) and epsilon.	115
8.4	The box and whisker plots for the SVM's kernel hyperparameters values.	116
8.5	The Pearson correlations matrices for the SVM hyperparameters.	117
8.6	The heatmaps of SVM models' MSE and MAPE in 2018–2021.	118
8.7	The heatmaps of SVM models' threshold accuracies in 2018–2021.	119
8.8	The heatmaps of SVM models' cumulative profit and max drawdown in 2018–2021.	120
8.9	The results of the best SVM models' trading performance for 2018–2021.	120
8.10	The heatmaps of SVM models' profitable and number of trades in 2018–2021.	121
8.11	Comparing the different profitable trades percentage for the SVM model.	122
8.12	The heatmaps of SVM models' MAPE and cumulative profit in 2018.	123
8.13	The heatmaps of SVM models' MSE and profitable trades in 2019.	124
8.14	The MSE through the 2019 testing period of selected SVM models.	124
8.15	The heatmaps of SVM models' MSE for 2020 and 2021.	125
8.16	The heatmaps of SVM models' cumulative profit for 2020 and 2021.	125
9.1	The threshold accuracies _{0.025} for all models.	129
9.2	The highest cumulative profit for all models during the 2018–2021 testing period.	130
B.1	The heatmaps of the LSTM models' MSE and MAPE in 2018.	154
B.2	The heatmaps of the LSTM models' threshold accuracies in 2018.	154

B.3	The heatmaps of the LSTM models' profit and max drawdown in 2018.	155
B.4	The heatmaps of the LSTM models' trading risk-related metrics in 2018.	156
B.5	The results of the best LSTM model's trading performance in 2018.	157
B.6	The heatmaps of the LSTM models' MSE and MAPE in 2019.	158
B.7	The heatmaps of the LSTM models' threshold accuracies in 2019.	159
B.8	The heatmaps of the LSTM models' profit and max drawdown in 2019.	159
B.9	The heatmaps of the LSTM models' trading risk-related metrics in 2019.	160
B.10	The results of the best LSTM model's trading performance for 2019.	161
B.11	The heatmaps of the LSTM models' MSE and MAPE in 2020.	162
B.12	The heatmaps of the LSTM models' threshold accuracies in 2020.	163
B.13	The heatmaps of the LSTM models' profit and max drawdown in 2020.	163
B.14	The heatmaps of the LSTM models' trading risk-related metrics in 2020.	164
B.15	The results of the best LSTM model's trading performance for 2020.	165
B.16	The heatmaps of the LSTM models' MSE and MAPE in 2021.	166
B.17	The heatmaps of the LSTM models' threshold accuracies in 2021.	167
B.18	The heatmaps of the LSTM models' profit and max drawdown in 2021.	167
B.19	The heatmaps of the LSTM models' trading risk-related metrics in 2021.	168
B.20	The results of the best LSTM model's trading performance for 2021.	169
B.21	The heatmaps of the LSTM models' MSE and MAPE in 2018–2021.	170
B.22	The heatmaps of the LSTM models' threshold accuracies in 2018–2021.	171
B.23	The heatmaps of the LSTM models' profit and max drawdown in 2018–2021. . .	171
B.24	The heatmaps of the LSTM models' trading risk-related metrics in 2018–2021. .	172
B.25	The results of the best LSTM model's trading performance for 2018–2021. . . .	173
B.26	The heatmaps of the RF models' MSE and MAPE in 2018.	174
B.27	The heatmaps of the RF models' threshold accuracies in 2018.	175
B.28	The heatmaps of the RF models' profit and max drawdown in 2018.	175
B.29	The heatmaps of the RF models' trading risk-related metrics in 2018.	176
B.30	The results of the best RF model's trading performance for 2018.	177
B.31	The heatmaps of the RF models' MSE and MAPE in 2019.	178
B.32	The heatmaps of the RF models' threshold accuracies in 2019.	179
B.33	The heatmaps of the RF models' profit and max drawdown in 2019.	179
B.34	The heatmaps of the RF models' trading risk-related metrics in 2019.	180
B.35	The results of the best RF model's trading performance for 2019.	181
B.36	The heatmaps of the RF models' MSE and MAPE in 2020.	182
B.37	The heatmaps of the RF models' threshold accuracies in 2020.	183

B.38 The heatmaps of the RF models' profit and max drawdown in 2020.	183
B.39 The heatmaps of the RF models' trading risk-related metrics in 2020.	184
B.40 The results of the best RF model's trading performance for 2020.	185
B.41 The heatmaps of the RF models' MSE and MAPE in 2021.	186
B.42 The heatmaps of the RF models' threshold accuracies in 2021.	187
B.43 The heatmaps of the RF models' profit and max drawdown in 2021.	187
B.44 The heatmaps of the RF models' trading risk-related metrics in 2021.	188
B.45 The results of the best RF model's trading performance for 2021.	189
B.46 The heatmaps of the RF models' MSE and MAPE in 2018–2021.	190
B.47 The heatmaps of the RF models' threshold accuracies in 2018–2021.	191
B.48 The heatmaps of the RF models' profit and max drawdown in 2018–2021.	191
B.49 The heatmaps of the RF models' trading risk-related metrics in 2018–2021.	192
B.50 The results of the best RF model's trading performance for 2018–2021.	193
B.51 The heatmaps of the SVM models' MSE and MAPE in 2018.	194
B.52 The heatmaps of the SVM models' threshold accuracies in 2018.	195
B.53 The heatmaps of the SVM models' profit and max drawdown in 2018.	195
B.54 The heatmaps of the SVM models' trading risk-related metrics in 2018.	196
B.55 The results of the best SVM model's trading performance for 2018.	197
B.56 The heatmaps of the SVM models' MSE and MAPE in 2019.	198
B.57 The heatmaps of the SVM models' threshold accuracies in 2019.	199
B.58 The heatmaps of the SVM models' profit and max drawdown in 2019.	199
B.59 The heatmaps of the SVM models' trading risk-related metrics in 2019.	200
B.60 The results of the best SVM model's trading performance for 2019.	201
B.61 The heatmaps of the SVM models' MSE and MAPE in 2020.	202
B.62 The heatmaps of the SVM models' threshold accuracies in 2020.	203
B.63 The heatmaps of the SVM models' profit and max drawdown in 2020.	203
B.64 The heatmaps of the SVM models' trading risk-related metrics in 2020.	204
B.65 The results of the best SVM model's trading performance for 2020.	205
B.66 The heatmaps of the SVM models' MSE and MAPE in 2021.	206
B.67 The heatmaps of the SVM models' threshold accuracies in 2021.	207
B.68 The heatmaps of the SVM models' profit and max drawdown in 2021.	207
B.69 The heatmaps of the SVM models' trading risk-related metrics in 2021.	208
B.70 The results of the best SVM model's trading performance for 2021.	209
B.71 The heatmaps of the SVM models' MSE and MAPE in 2018–2021.	210
B.72 The heatmaps of the SVM models' threshold accuracies in 2018–2021.	211

B.73	The heatmaps of the SVM models' profit and max drawdown in 2018–2021. . . .	211
B.74	The heatmaps of the SVM models' trading risk-related metrics in 2018–2021. . .	212
B.75	The results of the best SVM model's trading performance for 2018–2021. . . .	213

List of Tables

1.1	An example of the different types of currency pairs in each forex category.	2
2.1	The different tests of feature combinations used by Von Stein [70].	11
2.2	Features used by Yıldırım <i>et al.</i> [76] to train each of the LSTM models.	12
2.3	The results for the ANN and SVM found by Kamruzzaman and Sarker [32].	15
2.4	The results found by Patel <i>et al.</i> [52] for SVM compared to SVM-SVM models.	17
2.5	The prediction accuracy for five stocks using RF found by Basak <i>et al.</i> [3].	17
2.6	The results for six data sets using RF and SVM found by Derbentsev <i>et al.</i> [15].	18
3.1	An example of the type of financial data available from FxPro.	20
3.2	An example of the one-hot encoding technique.	23
3.3	A description of the different ADX values.	28
3.4	The summary of all features.	35
4.1	The scaling of features with fixed ranges.	41
4.2	The scaling of BB and Keltner channels into a single feature.	41
4.3	A summary of key statistical measures for the unscaled feature set.	42
4.4	A summary of key statistical measures for the scaled features set.	43
4.5	The different models considered in this study.	46
4.6	The training and validation sets used for cross-validation.	51
4.7	The search spaces for retraining interval and sequence length.	51
4.8	An example of the 70/15/15 test split for the 6-month retrain interval.	52
4.9	An example of calculating threshold accuracy.	58
4.10	An example of the trading data used to calculate the spread.	59
4.11	The number of profitable trading days within the testing period of 2018–2021.	61
5.1	Example of using the naïve forecasting model.	64
5.2	The forecasting results of the naïve model.	64
5.3	The search spaces for the ARIMA hyperparameter calibration.	68

5.4	The top five parameter calibration results for the ARIMA model.	69
5.5	The Ljung-Box test performed on the first 8 lags from the ARIMA(4,0,3).	70
5.6	The ARIMA models results for the USD/ZAR testing period.	70
6.1	The LSTM hyperparameter calibration parameters and best values found.	80
6.2	The worst five hyperparameter calibration iterations for the LSTM model.	82
6.3	The best five hyperparameter calibration iterations for the LSTM model.	82
6.4	The LSTM summarised results for retrain interval and sequence length.	88
7.1	The RF hyperparameter calibration parameters and best values found.	97
7.2	The top five hyperparameter calibration iterations for the RF model.	97
7.3	The top two important features considered by the RF model.	102
7.4	Summary of the averages from each parameter on selected performance metrics. .	105
8.1	The SVM hyperparameter calibration parameters and best values found.	114
8.2	The top five hyperparameter calibration iterations for the SVM model.	114
8.3	The worst five hyperparameter calibration iterations for the SVM model.	115
8.4	Summary of the averages from each parameter on selected performance metrics. .	122
9.1	The lowest MSE values for all models during the 2018–2021 testing period. . . .	128
9.2	The lowest MAPE values for all models during the 2018–2021 testing period. . .	128
9.3	The highest cumulative profit and best maximum drawdown for all models. . . .	129
9.4	The highest profitable trades and number of trades for all models.	131
9.5	The best models for each performance metric over the 2018–2021 testing period.	131
10.1	The search spaces for retraining interval and sequence length.	134
10.2	The best retrain intervals and sequence lengths for the ML models.	135
A.1	The LSTM hyperparameter calibration best hyperparameter values.	139
A.2	The hyperparameter calibration results for the LSTM model (a).	139
A.3	The hyperparameter calibration results for the LSTM model (b).	140
A.4	The hyperparameter calibration results for the LSTM model (c).	141
A.5	The hyperparameter calibration results for the LSTM model (d).	142
A.6	The RF hyperparameter calibration best hyperparameter values.	142
A.7	The hyperparameter calibration results for the random forest model (a).	143
A.8	The hyperparameter calibration results for the random forest model (b).	144
A.9	The hyperparameter calibration results for the random forest model (c).	145
A.10	The SVM hyperparameter calibration best hyperparameter values.	146

A.11 The hyperparameter calibration results for the SVM (a).	146
A.12 The hyperparameter calibration results for the SVM (b).	147
A.13 The hyperparameter calibration results for the SVM (c).	148
A.14 The ARIMA hyperparameter calibration best hyperparameter values.	149
A.15 The hyperparameter calibration results for the ARIMA model (a).	149
A.16 The hyperparameter calibration results for the ARIMA model (b).	150
A.17 The hyperparameter calibration results for the ARIMA model (c).	151
B.1 The summary of the LSTM model's performance in 2018.	157
B.2 The summary of the LSTM model's performance in 2019.	161
B.3 The summary of the LSTM model's performance in 2020.	165
B.4 The summary of the LSTM model's performance in 2021.	169
B.5 The summary of the LSTM model's performance in 2018–2021.	173
B.6 The summary of the RF model's performance in 2018.	177
B.7 The summary of the RF model's performance in 2019.	181
B.8 The summary of the RF model's performance in 2020.	185
B.9 The summary of the RF model's performance in 2021.	189
B.10 The summary of the RF model's performance in 2018–2021.	193
B.11 The summary of the SVM model's performance in 2018.	197
B.12 The summary of the SVM model's performance in 2019.	201
B.13 The summary of the SVM model's performance in 2020.	205
B.14 The summary of the SVM model's performance in 2021.	209
B.15 The summary of the SVM model's performance in 2018–2021.	213

List of Acronyms

ACF: Autocorrelation Function
ADX: Average Directional Index
AI: Artificial intelligence
ANN: Artificial Neural Network
ARIMA: Autoregressive Integrated Moving Average
ATR: Average True Range
BB: Bollinger Bands
BPTT: Backpropagation Through Time
CART: Classification and Regression Trees
CNN: Convolutional Neural Network
DNN: Deep Neural Network
DT: Decision Tree
EI: Expected Improvement
EMH: Efficient Market Hypothesis
EUR: Euro
GDP: Gross Domestic Product
GP: Gaussian Process
JSE: Johannesburg Stock Exchange
LSTM: Long Short-Term Memory
MACD: Moving Average Convergence Divergence
MAE: Mean Absolute Error
MAPE: Mean Absolute Percentage Error
ML: Machine Learning
MSE: Mean Square Error

NMSE: Normalised Mean Square Error

NN: Neural Network

NYSE: New York Stock Exchange

OHLC: Open, High, Low and Closing Prices

PACF: Partial Autocorrelation Function

PCA: Principle Component Analysis

ReLU: Rectified Linear Unit

RF: Random Forest

RNN: Recurrent Neural Network

RSI: Relative Strength Index

SGD: Stochastic Gradient Descent

SMA: Simple Moving Average

SMBO: Sequential Model-Based Optimisation

SVM: Support Vector Machine

TSI: True Strength Index

USD: United States Dollar

ZAR: South African Rand

CHAPTER 1

Introduction

Contents

1.1	Background of the foreign exchange market	1
1.2	Trading on the foreign exchange market	2
1.3	Machine learning	4
1.4	Model decay	5
1.5	Problem description	6
1.6	Thesis scope	6
1.7	Thesis objectives	6
1.8	Thesis layout	7

Global trade is vital to expanding and developing a country's economy. Many countries rely heavily on imports and exports as their primary source of economic activity. For instance, South Africa's total exports of goods and services such as precious metals, fossil fuels and agricultural goods contribute 31.19% to the GDP, while imports account for 25.02% [73]. For all these transactions between two countries, there needs to be an exchange of currency. The relative price between two currencies is determined through the foreign exchange market. Therefore, countries that rely on global trade depend on foreign exchange prices, directly impacting the associated profit and costs.

1.1 Background of the foreign exchange market

The foreign exchange market, referred to as the forex/fx market, is a decentralised market where all currency pairs are bought and sold [45]. Contrary to traditional stocks that trade on a centralised (singular) exchange. For example, the New York Stock Exchange (NYSE) is where American-listed companies are traded, and the Johannesburg Stock Exchange (JSE) is where South African-listed companies are traded. A singular exchange does not bind the forex market, meaning prices on different platforms for a currency pair may differ. Therefore, the "true" price of a currency pair is the aggregated price of all of the exchanges whereby the currency pairs are traded.

The forex market operates 24 hours a day, five days a week, excluding the weekends. The trading of currencies means that one currency is exchanged for another. Therefore, the buying of one currency equates to the selling of another. When looking at the prices for a currency pair,

the price is quoted in terms of 1 unit of the base currency that can be exchanged for x units of the quoted currency. Figure 1.1 contains an example of the price quote for the USD/ZAR currency pair, in which the United States dollar (USD) is the base currency and South African rand (ZAR) is the quote currency. Thus, for every US\$1 exchanged (sold), ZAR 19.60 will be received (bought).

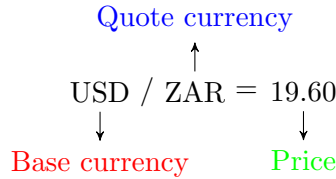


FIGURE 1.1: An example of a foreign exchange price quote for the USD/ZAR currency pair.

Three types of currency pairs exist in forex: major, minor and exotics [12]. The major currency pairs are more frequently traded and have the highest volume. These pairs involve the US dollar, which makes up 80% of all forex trading volume [44]. Table 1.1 contains a few examples of the different currency pairs in each category. The EUR/USD currency pair is the most popular trading pair that accounts for nearly 20% of all forex transactions [44].

Major pairs	Minor pairs	Exotic pairs
USD/EUR	EUR/GBP	USD/ZAR
USD/JPY	EUR/AUD	USD/HKD
GDP/USD	GBP/JPY	JPY/NOK
USD/CHF	NZD/JPY	NZD/SGD

TABLE 1.1: An example of the different types of currency pairs in each forex category.

Minor currency pairs, or cross-currency pairs, do not include the US dollar. They combine two major currencies, excluding the USD, like EUR/GBP or NZD/JPY. Although minor currency pairs have lower trading volumes, trading opportunities still exist. The exotic currency pairs, such as the USD/ZAR or JPY/NOK, contain one major currency and one from an emerging economy. Exotics have less liquidity due to lower trading volumes and carry increased risk with these less frequently traded currencies. However, for experienced traders, they can offer unique profit opportunities.

Understanding the differences between the types of currency pairs is beneficial, as each has its own set of characteristics, risks and potential rewards. The major pairs are more stable, while minor and exotic pairs have higher volatility, making them attractive to risk-seeking traders [12].

1.2 Trading on the foreign exchange market

Forex exhibits the same characteristics as any other market: supply and demand determine the price. These are affected by interest rates, economic factors, market sentiment, news, and market conditions. There are monetary opportunities for buying and selling currency pairs; if movements in these prices are in the correct speculated direction, a profit is made. Adversely, movements in the incorrect direction will result in a loss. The forex market has the additional benefit of generating returns regardless of price direction. Therefore, predictive power in these markets is a valuable commodity. The two types of trades based on the speculated direction of the market are:

1. Long trades (go long): Buy a currency to sell it at a higher price.

$$\text{Profit/loss (\%)} = \frac{\text{selling price} - \text{purchase price}}{\text{purchase price}} \cdot 100.$$

2. Short trades (go short): Sell a currency to buy it back at a lower price.

$$\text{Profit/loss (\%)} = \frac{\text{selling price} - \text{purchase price}}{\text{selling price}} \cdot 100.$$

Figure 1.2 contains an example of the long and short trades within the USD/ZAR currency pair. On 2017-06-21, the price was speculated to go up, and a long trade was entered at ZAR 13.07 per USD. The price increased, and the position was exited on 2017-07-06 at ZAR 13.4285 per USD, resulting in a 2.74% profit. Similarly, on 2017-08-11, a sell order was placed at ZAR 13.44 per USD on the suspicion of declining prices. The position was exited on 2017-09-05 at ZAR 12.90 per USD, netting a 4.02% profit.

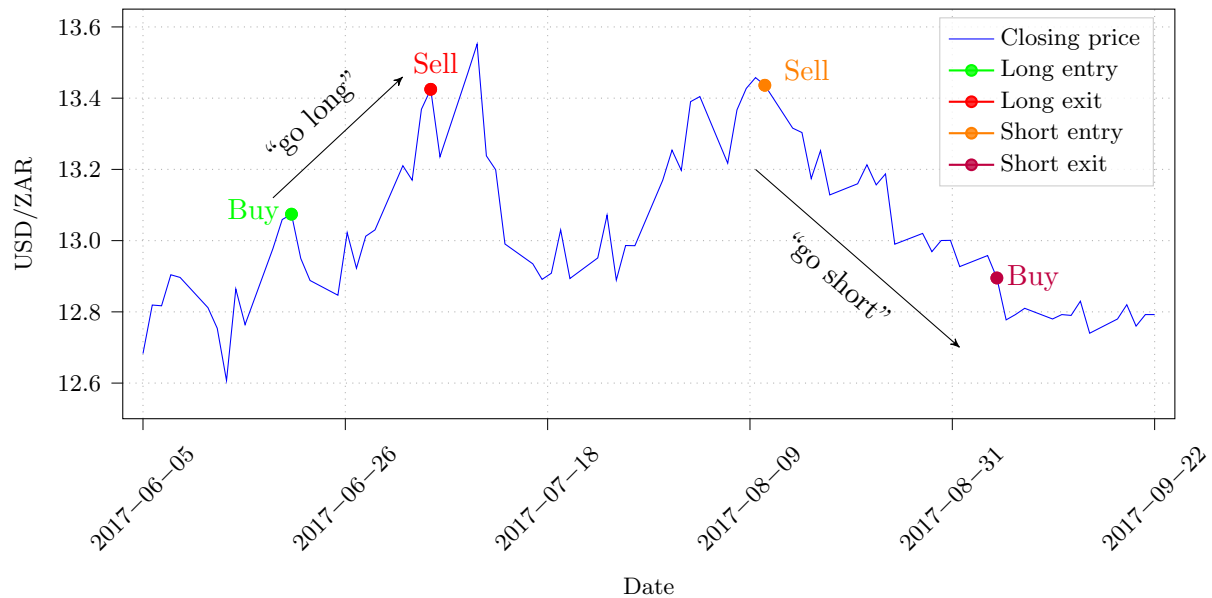


FIGURE 1.2: An illustration of the different types of long and short trades that a trader can use to profit in foreign exchange markets.

Both trades in Figure 1.2 resulted in profits; however, being profitable in forex trading is surprisingly difficult. Trading platforms are often required to disclose the percentage of customers who lose money. Popular online brokers, FxPro, FXTM and IG claim that over 81.85%, 81% and 68% of customers lose money on their platforms [22, 23, 30]. Numerous factors can cause these losses, such as emotion, greed, human error and unforeseen market movements. Therefore, removing the ‘human’ element from trading is advantageous as algorithms and models do not have emotional or irrational biases.

Accurately predicting the price direction of a currency pair allows an individual, company or enterprise to make decisions in the present that will either generate profit or save money. Additionally, if the magnitude of the price movement is known, trading strategies based on this information can be enhanced.

1.3 Machine learning

The collective term for all computer systems that can mimic human intelligence using logic is called artificial intelligence (AI) [24]. Machine learning (ML) is often used interchangeably with AI; however, they are fundamentally different. ML is a subset of AI that describes statistical techniques whereby machines improve at tasks through experience. Therefore, ML can learn from data without being explicitly programmed.

There exists a subset of ML models that draws inspiration from the biological processes of the human brain. These models fall into the deep learning category and focus on artificial networks with multiple layers of neurons connected in parallel. Deep learning developments in AI have led to strong performance in numerous fields. They can capture patterns and characteristics previously not capable with statistical methods. Figure 1.3 contains a schematic representation of the different subsets of AI.

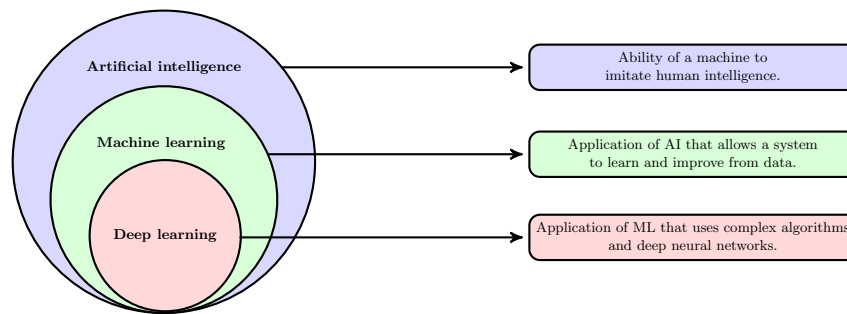


FIGURE 1.3: A schematic representation and explanation of the different fields within artificial intelligence.

Two fundamental components define how ML models operate and the applications to which they can be applied. These components are,

1. **Features:** These are the input variables used in ML models. They are measurable properties or characteristics used to describe the data set. Features can be in the form of numerical or categorical data.
2. **Labels:** This is the target or output variable(s) the ML model tries to predict. Each label corresponds to a feature set. Labels can be numerical or categorical.

Four main ML learning algorithms are available based on the characteristics of the data set and the aim of the problem trying to be solved. Figure 1.4 contains a schematic representation of three of these algorithms since one combines the two, and how the inputs are used to form a corresponding output. The algorithms are summarised as follows:

1. **Supervised learning:** Algorithms using features and labels, typically for classification and regression tasks. The goal is to map the set of features to the respective label using a predictive model while minimising the prediction error.
2. **Unsupervised learning:** These contain only features and no labels, commonly used for clustering tasks. Since no label is associated with a feature, the goal is to separate similar features into different classes/clusters. Additionally, anomaly detection or dimensionality reduction uses unsupervised learning.

3. **Reinforcement learning:** A learning system called an agent that can perform specific actions in a predefined environment. Each action has a reward or penalty associated with it. The goal is to find the optimal actions to maximise the reward.
4. **Semi-supervised:** This combines supervised and unsupervised learning techniques. Typically, it begins with unsupervised data, and similar features are clustered together. Then, labels are added manually after the clustering.

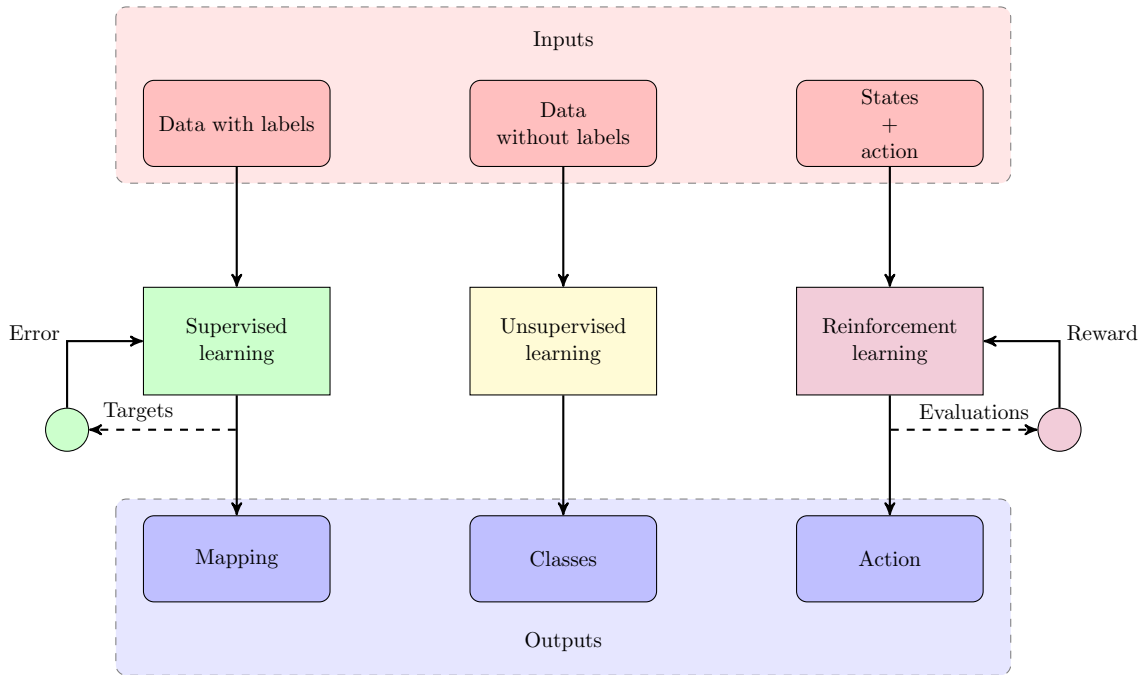


FIGURE 1.4: A schematic representation of the different types of machine learning applications.

These four learning algorithms expand the application of ML to numerous different fields, including financial markets. Foreign exchange data is noisy, non-stationary, and inherently non-linear, making it challenging to model using traditional forecasting techniques. Therefore, complex models are required to detect relationships and extract information from these data sets. The emergence of state-of-the-art ML models has paved the way to challenge the *efficient market hypothesis*, which states that prices are uncorrelated and price discounts everything [42], meaning markets can not be exploited. Thus, no predictive power exists in prices.

1.4 Model decay

One of the caveats of ML models is that they do not remain stable forever [24]. Suppose they are used to model continuous time series data such as foreign exchange rates. The data set can and will likely evolve, thus changing characteristics and exhibiting new patterns. The original ML model trained on old data might demonstrate good performance or profitability for a trading strategy initially; however, as time progresses and the underlying data set changes, the model's performance will likely degrade based on the differences between the training and testing data.

This problem is referred to as model decay. It describes how a model's accuracy worsens as data changes. Model decay becomes increasingly more apparent when data is sourced from continuous data streams rather than static databases, especially in the case of financial markets. Therefore,

expecting data distributions to remain stable over long periods is unrealistic. Retraining the model at regular intervals, varying the features used or using a completely different type of ML model may be necessary to offset this decay.

1.5 Problem description

The aim of this study is to investigate the impact of four different retraining intervals, yearly, bi-yearly, quarterly and monthly, and ten different sequence lengths ranging from one to ten previous days of data used as features. The impact is measured for three different ML models on forecasting the daily closing prices of the USD/ZAR foreign exchange rate. Each ML model's architecture and respective hyperparameters will be calibrated using Bayesian optimisation, ensuring a fair evaluation of results. The ML models will then be tested with different combinations of retraining intervals and sequence lengths across four years of unseen data to evaluate their effects on varying market conditions.

1.6 Thesis scope

Studies involving ML are prone to having broad scopes since multiple different areas of development can be optimised. It is often infeasible to solve every component within an ML pipeline while simultaneously understanding the causal effects between components due to these models' 'black box' nature. Therefore, certain pipelines in the ML process need to be fixed, with experimentation focused on specific areas. The scope of this study is limited to the following:

1. Optimisation of feature parameters generated from technical analysis is not considered.
2. The architecture and hyperparameters for each ML model will be calibrated once and remain constant throughout testing.
3. All predictions generated from the ML models will be used as is via the trading strategy. Therefore, each model's trading strategy will not be optimised to ensure comparable results.
4. Code optimisation for training and testing the ML models is not considered.

1.7 Thesis objectives

The following objectives are pursued in this study.

1. Provide a background to foreign exchange markets, ML, and the problem addressed.
2. Discuss existing literature on forecasting financial time series data with ML models.
3. Review, clean, validate and describe the data set used.
4. Based on technical analysis and literature performance, generate features for ML models.
5. Preprocess the data set into a viable format for all models.
6. Discuss the modelling process used in this study.

7. Select and describe appropriate ML and statistical models for time series forecasting.
8. Perform hyperparameter calibration and interpret the results on all applicable models.
9. Evaluate the performance of statistical models.
10. Investigate the ML model's forecasting performance regarding the different retraining intervals and sequence lengths.
11. Discuss the comparative performance between the different models.
12. Make recommendations based on results, summarise the study and discuss ideas for future research.

1.8 Thesis layout

In this chapter, an overview of the foreign exchange market and ML is given. Additionally, the study's problem, scope and objectives are discussed. The remainder of this study will be structured as follows. Chapter 2 contains a review of the relevant literature surrounding ML techniques for forecasting financial time series data.

In Chapter 3, the data set used and the generation of the selected technical indicators are described. The outline of the modelling process, with a focus on feature engineering, parameter calibration and result metrics shared by the ML models, is given in Chapter 4.

In Chapter 5, the statistical forecasting techniques implemented and a discussion of their results are given. Chapter 6, 7, and 8 contain an overview of the implemented ML models, LSTM, RF and SVM, respectively. Each model's background is discussed, focusing on its usage, development, architecture and hyperparameters. The discussion follows the interpretation and evaluation of the hyperparameter calibration results and concludes with the results for the different retraining intervals and sequence lengths during the testing period.

Chapter 9 contains a comparison between the best-performing models from each implemented model. In Chapter 10, the study is concluded, highlighting key findings and summarising the results. Additionally, areas of further work and improvement of the study are given.

CHAPTER 2

Literature

Contents

2.1	Financial time series forecasting	9
2.2	Deep learning models	10
2.3	Support vector machines	14
2.4	Random forests	17
2.5	Summary	18

Machine learning has become a popular research topic in recent years, with applications in numerous fields, including financial markets. The stock market shares similarities with the foreign exchange market, and the literature will be extended to overview both markets. The aim of this chapter is to discuss the relevant literature concerning financial time series predictions using ML models. The chapter is broken up into an overview of financial time series and the application of various machine learning models to predict these data sets.

2.1 Financial time series forecasting

The ability to predict financial markets and generate consistent profits directly contradicts the efficient market hypothesis (EMH). Malkiel [42] states that prices reflect all publicly available information in an efficient market, and it is impossible to outperform the market consistently. The EMH has been challenged throughout the literature, with numerous studies indicating profitable returns on the stock and foreign exchange markets [31]. The resurgence of profitable returns is likely due to the increased computational efficiency of machine learning models and the increasing availability of information they can process.

The literature is evenly divided into classifying directional price movements (classification) and directly forecasting prices (regression). In a literature survey of ML applications in the stock and foreign exchange markets conducted by Hu *et al.* [28], regression-based forecasts accounted for 52%, where classification was 48%, of the 86 papers reviewed. Both methods can be incorporated into a trading strategy; however, albeit more challenging, regression can further enhance a trading strategy with information regarding the magnitude of price movements.

Historical prices regarding a specific stock or currency pair are almost always used as the primary data source [31]. These prices include the open, high, low, and closing prices, as well as volume. Additionally, the feature set incorporates four primary data sources to add explanatory power:

technical indicators, sentiment analysis, macroeconomic data and fundamental data. Figure 2.1 contains the percentage of papers each source of data was used in through the literature survey conducted by Jiang [31], with technical indicators the most widely used due to their simplicity and availability.

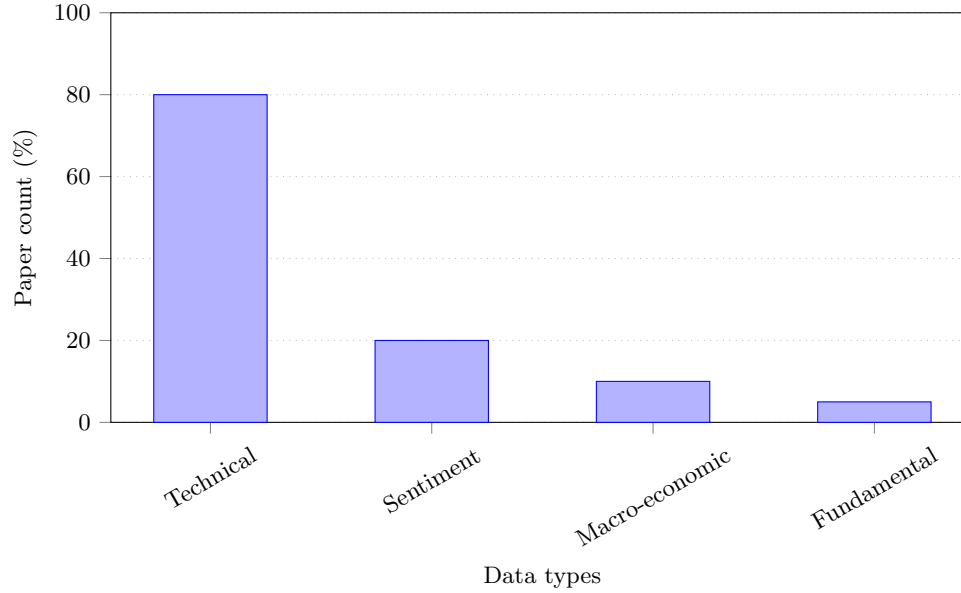


FIGURE 2.1: The breakdown of feature sets used in literature to forecast financial markets [31].

2.2 Deep learning models

The inception of deep learning models led to usages in healthcare and medical diagnostics, with convolutional neural networks (CNN) being used for medical image classification [78]. Other fields include natural language processing with the breakthrough of the BERT transformer used for sentiment analysis [16]. These advancements are attributed to the sophisticated architectures that allow them to capture deeper patterns sequentially extracted through each layer in the deep network, thus making them suitable candidates for modelling complex financial time series data.

Features characterise and provide explanatory insights into data sets, making them essential to the ML pipeline. Von Stein [70] investigated feature importance on three deep learning models used for foreign exchange rate forecasting. Additionally, the effects of the 12 different combinations of features, given in Table 2.1, were tested on various model architecture setups. The author forecasted the daily closing prices of the USD/ZAR currency pair, with the periods 2014-01-01 to 2018-12-31 used for training and validation while using 2019-01-01 to 2019-05-31 for testing.

The first model tested by Von Stein [70] was the long short-term memory (LSTM) neural network (NN), introduced by Hochreiter and Schmidhuber [27] and popularised due to its advanced capabilities in processing sequential data. The second model expanded on from the work of Ni *et al.* [49] combines a convolutional neural network (CNN) and LSTM to form a hybrid model, CNN-LSTM. CNNs are widely used in computer vision, as they can efficiently process and analyse visual data for image classification, object detection and facial recognition [38]. However, they can be extended to detect patterns in financial markets. The third model utilises the idea of a multi-head. Thus, multiple CNN models corresponding to the number of inputs

Test	Number of features	Features
1	4	Open, high, low, close (OHLC)
2	6	OHLC + 9-day and 21-day simple moving averages
3	7	OHLC + Binary price changes
4	12	OHLC + Relative strength index
5	17	OHLC + Heikin-Ashi candlesticks
6	24	OHLC + Ichimoku Kinko Hyo
7	8	OHLC + Bollinger bands
8	1	OHLC + 3-month implied volatility
9	47	OHLC + Implied volatility and risk reversal
10	12	OHLC + 1 st and 2 nd differences
11	102	All features
12	31	Features determined by PCA

TABLE 2.1: The different tests of feature combinations used by Von Stein [70].

are used as the first layer, with their outputs used as inputs to an LSTM NN.

Von Stein [70] deepened the study by incorporating a dimensionality reduction technique known as principle component analysis (PCA) to reduce the number of features by removing irrelevant ones. The total feature set consisted of 102 features, 59 being linearly dependent and thus removed. The remaining 43 were subject to PCA, after which only 31 remained. These new features formed test 12 in Table 2.1.

The results for each model were based on the threshold accuracy metric of R0.025 introduced by the author. This metric measures the percentage of model predictions within a specific threshold value of actual closing prices. The 12 different tests were conducted on each model while performing a 2-D grid search on the changing the number of layers ranging from 3 to 27 and the number of neurons in each layer ranging from 25 to 1 000 for each model.

The LSTM model had the lowest accuracy of the three models, with its best accuracy of 22.4% coming from test 1. The CNN-LSTM model also performed the best only using the four basic features of the opening, high, low and closing prices and achieving the 2nd best performance overall of 24.8%. The multi-head CNN-LSTM model had the best prediction accuracy of 25.6% using test 7's features.

Interestingly, the PCA feature set had opposing effects on the results, with test 12 having the worst performance for all three models. Test 12 for the LSTM, CNN-LSTM and multi-head CNN-LSTM resulted in a threshold accuracy of 8.8%, 4.4% and 9.6%, respectively. A notable finding from the study was the drop in threshold accuracy of the LSTM model when its architecture setup exceeded 12 layers. The performance dropped from $\approx 20\%$ for layers 3–11 to $\approx 7\%$ with LSTM models exceeding 12 layers, indicating this hyperparameter should be carefully set before training.

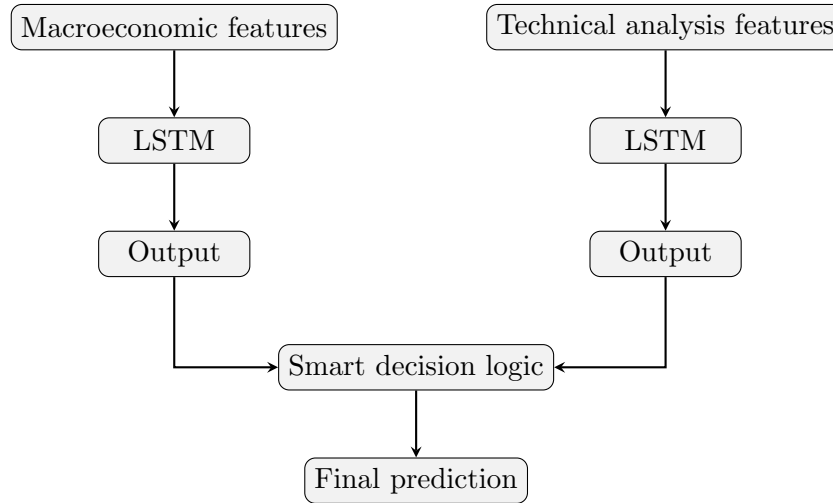
Yıldırım *et al.* [76] proposed a hybrid LSTM NN to forecast the binary directional movement of the EUR/USD foreign exchange pair's closing price over 1, 3 and 5 days. The hybrid model consisted of two separate LSTM NNs with the same architecture but trained on different datasets. The first LSTM was trained using macroeconomic data (LSTM-ME) from the United States and Europe, such as interest rates, inflation rates and index funds. The second LSTM was trained on popular technical indicators (LSTM-TA), such as moving averages, Bollinger bands, and relative strength index. Table 2.2 contains a complete list of the features used.

The outputs of the individual LSTM models, LSTM-ME and LSTM-TA, are combined using a smart decision logic function to form a final prediction for the hybrid LSTM model as seen in Figure 2.2. The study also included another LSTM model trained with the entire dataset, which

Macroeconomic LSTM features	Technical indicator LSTM features
Close EUR/USD	OLHC EUR/USD
Close S&P500	Moving average
Close DAX	Moving average convergence divergence (MACD)
Interest rates (Germany)	Rate of change (ROC)
Interest rates (Europe)	Momentum oscillator
FED interest rates	Relative strength index (RSI)
Inflation rate (EU)	Bollinger bands
Inflation rate (USA)	Commodity channel index (CCI)

TABLE 2.2: Features used by Yildirim *et al.* [76] to train each of the LSTM models.

combined macroeconomic and technical analysis features into one feature set (LSTM-ME-TA). The author's objective was to examine whether utilising a hybrid model would enhance the accuracy of predictions compared to using individual LSTM models and investigate the effect of fundamental analysis compared to technical analysis as features.

FIGURE 2.2: A schematic representation of the hybrid LSTM model proposed by Yildirim *et al.* [76].

Yildirim *et al.* [76] found minor differences in performance for 1-day forecasts between the two individually trained LSTM-ME and LSTM-TA, and the LSTM-ME-TA that utilises the entire feature set, with accuracies equating to 50.16%, 51.43% and 49.89% respectively. However, the hybrid LSTM model incorporating the smart decision logic had an impressive 73.09% accuracy. Therefore, the smart decision logic could filter out false positives from the LSTM-ME and LSTM-TA models, resulting in more accurate predictions. These findings were consistent for the 3-day and 5-day ahead forecasts, with the hybrid LSTM outperforming all other models.

When developing a deep neural network, it is essential to consider various factors related to its features and architecture. Pongsena *et al.* [54] addresses a subset of these factors in predicting the daily price direction of the EUR/USD foreign exchange pair from 2013 to 2018 using an LSTM NN. The areas addressed by the authors when developing their LSTM model were split up into three phases, each focusing on a component of the model:

1. **Phase 1:** The number of periods used at the time of prediction *i.e.*, the sequence length.
2. **Phase 2:** The number of data points for the respective training and testing sets, meaning how much information should be used to train the model.

3. **Phase 3:** The frequency of which the models are updated throughout the testing set *i.e.*, retraining interval.

The development of Pongsena *et al.* [54]’s LSTM was incrementally built following the sequential order of the phases. For example, phase 3 experiments used the best results from phases 1 and 2. This methodology introduces a dependency on subsequent phases and narrows the study’s results. The accuracy metric used was the binary classification accuracy defined as,

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}} \cdot 100 \quad (2.1)$$

where TP is the number of true positives, TN is the number of true negatives, FP is the number of false positives and FN is the number of false negatives. The accuracy is thus how many correct classifications the model made out of the total number of data points.

In phase 1, the authors tested sequence lengths ranging from 3 days to 30 days of historical data to use as features. The summarised results are given in Figure 2.3 with the sequence lengths of 9–12 days, resulting in the highest accuracy of 52%. The lower sequence lengths (3–8) still had relatively high accuracies at 50%. However, exceeding a sequence length of 14 decreased accuracy the more days used as features.

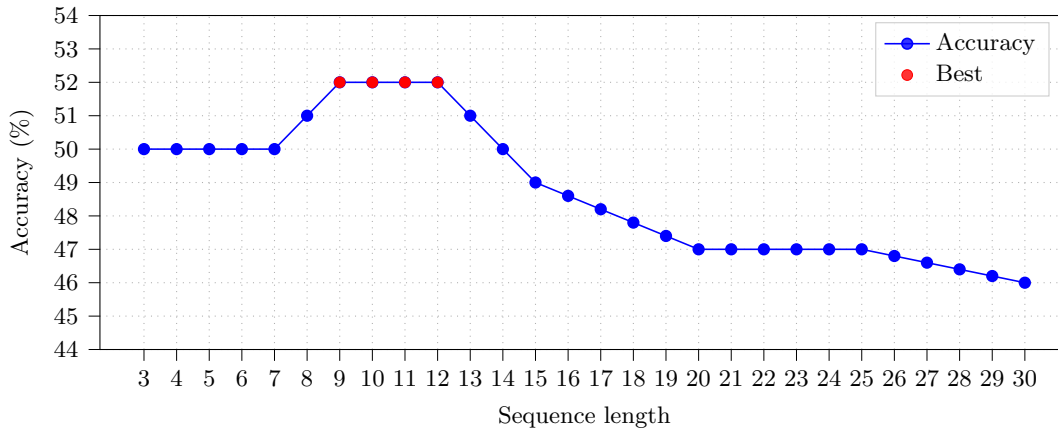


FIGURE 2.3: The results for the sequence lengths tested by Pongsena *et al.* [54].

The training and testing data split was experimented with in Phase 2. The two splits are defined as 5 years of training and 1 year of testing (long), with the other split being 1 year of training and 1 year of testing (short). Four splits were partitioned for different periods in the data set to ensure consistency. The findings were surprising as the shorter split, with less information, performed better than the longer split. However, the comparative difference showed an approximate 1.4% improvement over the longer split.

The retraining intervals considered were yearly, quarterly and monthly for phase 3. These experiments on the retraining interval used a sequence length of 10 from phase 1 and a short data split from phase 2. The monthly training interval had the highest accuracy at 61.25%, with the quarterly second at 57.92% and the worst performing interval, yearly at 50.72%. The accuracy findings through time are given in Figure 2.4. The shorter interval’s (monthly and quarterly) accuracy fluctuates far more than the yearly interval, with the quarterly having the worst performance in month 7. The conclusion was that shorter retraining intervals can adapt quicker to evolving market conditions but cause more volatility in accuracy. Longer intervals have worse accuracy but are more stable and less susceptible to short-term volatility.

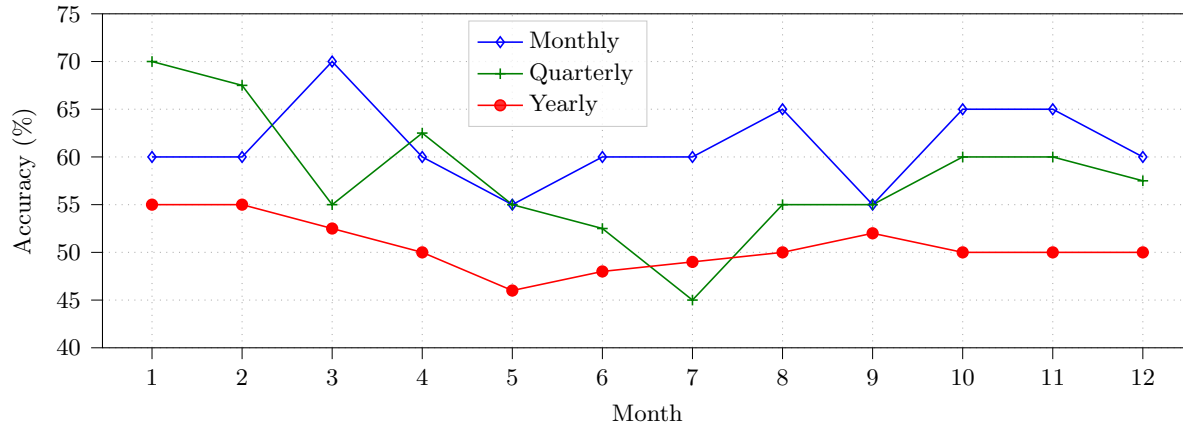


FIGURE 2.4: The results for the retraining intervals tested by Pongseena *et al.* [54].

There is no mention of the architecture or hyperparameter configuration of the LSTM model. Although not the paper's focus, it makes the reproducibility of the study difficult. This paper inspires new ideas to improve model performance, which deviates from the more frequently researched areas of feature engineering and model architecture. These areas investigated by Pongseena *et al.* [54] have a considerable effect on performance, notably for the retraining interval resulting in an 11.14% point increase in the accuracy by retraining every month as opposed to annually.

Siarni-Namini and Namin [65] compared the performance of LSTM models against the commonly used auto-regressive integrated moving averages (ARIMA) model to predict the monthly closing prices of five financial indexes, N225, IXIC, HIS, GSPC, DJ. The LSTM outperformed the ARIMA model for all data sets, having a lower RMSE of 87.45% on average for each data set. The authors also investigated the effects of varying the number of iterations used to train the LSTM model. They found that the LSTM models would increase in accuracy up to a specific point, after which the models began to overfit the training data and generalise poorly to the testing data.

The findings by Siarni-Namini and Namin [65] emphasise that the predictive performances of deep learning models outperform traditional forecasting techniques. However, careful consideration is still required when training the LSTM model, as incorrect configurations can lead to overfitting, resulting in a final model that generalises poorly.

In a literature survey conducted by Hu *et al.* [28] the most used deep learning model was the LSTM model. The LSTM model has good predictive performance, showcasing improved results compared to other models in financial market forecasting due to its ability to handle and process long sequences of sequential data. Therefore, unsurprisingly, this model is the most frequently used in literature, seeing over double the use case of its competitors illustrated in Figure 2.5. The conclusions drawn from literature are that LSTM models should always be considered and experimented with for time series predictions.

2.3 Support vector machines

Financial times series data is inherently non-linear, which makes regression notoriously tricky on these datasets. The development of support vector machines (SVM), which can map non-linear data into higher dimensions, has become a standard method for non-linear regression [41].

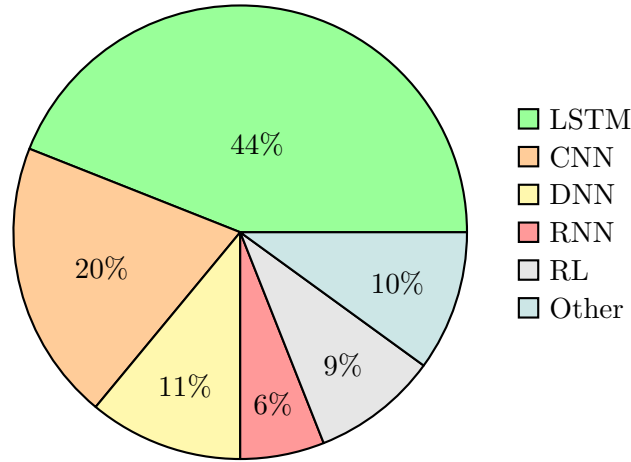


FIGURE 2.5: The different types of deep learning models used for financial market predictions [31].

Kamruzzaman and Sarker [32] compared the performance of artificial neural networks (ANN) to SVM models on predicting the weekly exchange rate of six currency pairs related to the Australian dollar (AUS). The authors used the previous week's exchange rate and five technical indicators, 5, 10, 20, 60 and 120 day moving averages, as features. There were 565 weekly data points from 1991-01 to 2000-07 for the US dollar (USD), British pound (GBP), Japanese Yen (JPY), Singapore dollar (SGD), New Zealand dollar (NZD) and Swiss Franc (CHF) against the AUS. The training and testing sets comprised 500 and 65 data points, respectively. Table 2.3 contains the comparative performance between the ANN and SVM model found by Kamruzzaman and Sarker [32]. The metrics used to evaluate performance were the normalised mean squared error (NMSE) and mean absolute error (MAE).

	GBP		USD		JPY	
Metric	ANN	SVM	ANN	SVM	ANN	SVM
NMSE	0.1578	0.1276	0.2624	0.2198	0.1264	0.1080
MAE	0.0030	0.0027	0.0035	0.0031	0.6243	0.5690
	SGD		NZD		CHF	
Metric	ANN	SVM	ANN	SVM	ANN	SVM
NMSE	0.2321	0.1512	0.0878	0.0880	0.0485	0.0442
MAE	0.0076	0.0068	0.0038	0.0037	0.0059	0.0054

TABLE 2.3: The forecasting results of six currency pairs to the Australian dollar for the ANN and SVM models found by Kamruzzaman and Sarker [32].

The authors found the SVM outperformed the ANN for five out of the six currency pairs, with the SVM having a 19.14%, 16.23%, 14.57%, 34.86% and 8.87% lower NMSE for the GBP, USD, JPY, SGD and CHF currencies, respectively. The only currency where the ANN had a lower NMSE was the NZD/AUS. However, it only resulted in a minimal 0.23% improvement, and the SVM had a lower MAE by 8.47%. Their research showcases the strong performance of SVM models in foreign exchange rate forecasting.

Kamruzzaman *et al.* [33] expanded on the previous research by Kamruzzaman and Sarker [32] using the same six currency pairs, feature set and result metrics. Since the SVM outperformed the ANN in their former study, this research aimed to further enhance the performance of the SVM model by investigating the effects of different kernel functions. The kernels investigated included the linear, polynomial, radial basis (RBF) and spline functions. Figure 2.6 contains

the NMSE results for the different kernel functions used to forecast the six currency pairs.

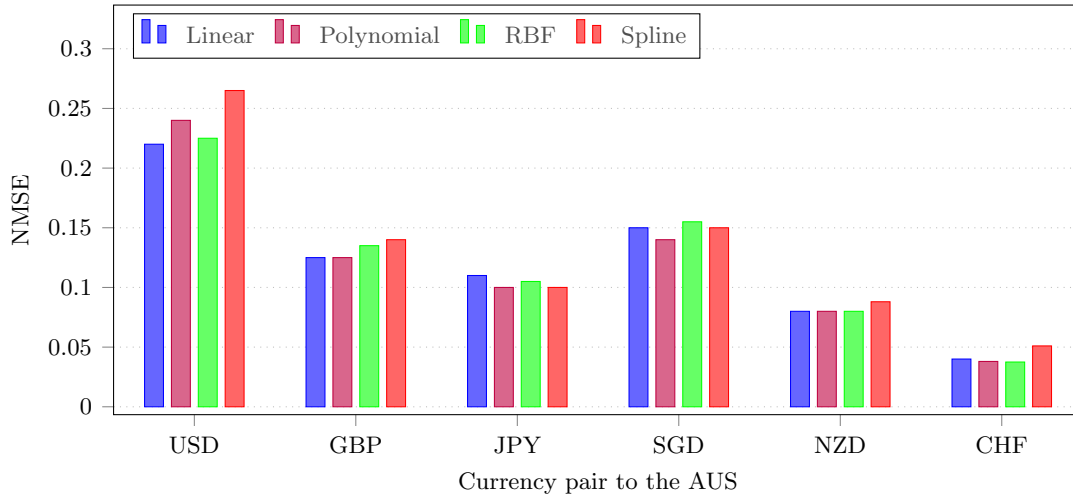


FIGURE 2.6: The NMSE for six currency pairs using different kernel functions found in the study by Kamruzzaman *et al.* [33].

The authors found that the polynomial kernel function had the lowest NMSE on average, except for the USD/AUS data set. The spline function was the worst-performing function but achieved the lowest NMSE for the JPY data set. They concluded that kernel selection should be chosen based on individual currency pair patterns, and no single kernel is suitable for all foreign currency pairs [33].

The use of hybrid models *i.e.*, using two model's predictions combined into one, is gaining popularity, with both Yildirim *et al.* [76] and Von Stein [70] using hybrid deep learning models. Patel *et al.* [52] incorporates this idea for SVM hybrid models and compares the performance of individual models against hybrid ones. The authors use an SVM model in stage 1 of a two-stage fusion approach. Stage 1's outputs are then fed into an ANN, random forest (RF), and SVM, resulting in the stage 2 models of SVM-ANN, SVM-RF, and SVM-SVM, respectively.

Patel *et al.* [52] tested the performance of these models on two Indian stock indexes, using ten years of historical data and seven technical indicators such as moving averages, moving-average convergence diverge and relative strength index. Additionally, the authors tested multiple forecast horizons ranging from 1 to 10, 15 and 30 days ahead.

Contrary to the notion that hybrid models improve performance. The SVM single model outperformed the hybrid SVM-SVM model for 1 to 4 days ahead and had lower root mean square error (RMSE) values as seen in Table 2.4. Notably, the one-day period had a 48.41% lower RMSE for the single SVM model over the hybrid SVM-SVM model. However, the more days ahead forecasted, the smaller the disparity between the SVM and SVM-SVM became. The SVM-SVM had a consistently lower RMSE for five to 30 days ahead.

The hybrid models for the ANN (SVM-ANN) and RF (SVM-RF) outperformed the single models, ANN and RF, for both data sets and all forecasting periods, respectively. The results of Patel *et al.* [52] indicate that the hybrid models increase performance for ANN and RF models for all use cases. However, using a hybrid model for the SVM models can worsen performance in specific scenarios. Therefore, careful evaluation is recommended when incorporating an SVM model in the second stage of a hybrid model.

The findings of the studies using SVM illustrate their strong performance in foreign exchange rate forecasting. Additionally, these studies highlight the importance of architecture setups with

Model	Days ahead forecasted											
	1	2	3	4	5	6	7	8	9	10	15	30
SVM	1.26	1.78	2.22	2.59	2.92	3.22	3.48	3.69	3.92	4.15	5.09	6.82
SVM-SVM	1.87	2.09	2.39	2.62	2.86	3.08	3.33	3.55	3.77	3.99	4.87	6.26
Improve (%)	48.41	17.42	7.66	1.16	-2.05	-4.35	-4.31	-3.79	-3.83	-3.86	-4.32	-8.21

TABLE 2.4: The RMSE results found by Patel *et al.* [52] for SVM compared to SVM-SVM models and the SVM's improvement (%) (Improve) over the SVM-SVM.

hyperparameters, such as the kernel function, which has notable effects on performance.

2.4 Random forests

Decision trees are a simple yet powerful ML model traditionally used for classification but can be expanded to regression tasks [24]. The ensemble of multiple decision trees forms the random forest (RF) model. These models eliminate the biases from individual trees, resulting in better generalisation without performance loss. The robustness of RF models containing no prerequisites concerning data characteristics makes them suitable for forecasting noisy financial time series data [55].

Basak *et al.* [3] evaluated the predictive performance of the RF tree-based learning model on classifying the daily directional movement of ten stocks. The authors used six technical indicators to extract information and gain insight into possible price movements from the daily closing prices. RF models have the attribute to rank features based on performance, and they found the on-balance-volume (OBV) technical indicator to have the highest importance in the feature set. The prediction horizons ranged from 3 to 90 days forecasted ahead. The results based on the accuracy, defined in equation (2.1), for five of the ten stocks are summarised in Table 2.5

Stock	Forecast horizon						
	3	5	10	15	30	60	90
AAPL	65.26	72.55	78.80	82.01	85.34	90.44	93.02
AMS	65.88	68.8	75.44	79.60	83.52	89.24	90.47
AMZN	67.09	72.49	77.43	80.35	86.48	89.44	94.79
FB	67.59	74.15	81.39	86.06	89.89	89.63	94.76
MSFT	67.30	72.68	77.39	81.66	86.15	90.14	92.24
Average	66.62	72.13	78.09	81.94	86.28	89.78	93.06

TABLE 2.5: The prediction accuracy (%) for five stocks with varying forecasting horizons using RF found by Basak *et al.* [3].

Interestingly, the accuracy improves as the forecast horizon increases, even though less information is available and the risk of price uncertainty is increased. The 90-day saw impressive results with high accuracies of 93.06% on average. However, the study fails to incorporate ‘dummy’ classifiers *i.e.*, simply predicting that all stocks will increase/decrease over the different horizons. The lack of comparisons questions the RF performance, and further experimentation should be conducted. The RF models still show strong predictive performance, albeit the lack of comparison; a classification accuracy above 65% for all stocks justifies the consideration of RF models for financial time series data.

RF models are a valuable benchmark for more sophisticated models, such as deep learning and SVM, as they are computationally fast to train. Rabbi *et al.* [58] implemented LSTM, SVM and

RF models to forecast twenty-two currency pairs relative to the USD. The data set consisted of the daily closing prices of each currency pair from 2000 to 2019. The authors used the MSE and MAPE metrics to evaluate the forecasting results. The LSTM model had the best accuracy in terms of MAPE overall, achieving the lowest MAPE for all twenty-two currency pairs. However, the RF model had the lowest MSE for fifteen currency pairs, indicating that the RF models generalise better on average.

Derbentsev *et al.* [15] used RF and SVM models to predict the daily closing prices of two stock indexes, S&P 500 and NASDAQ, two cryptocurrencies, BTC/USD and ETH/USD, and the foreign exchange pair, EUR/USD. The authors used an 80/20 percentage split for each data set using historical prices from 2015-01-01 to 2020-06-30. Table 2.6 contains a summary of the results found using the MAPE and RMSE accuracy metrics for regression.

Data set	RF		SVM	
	MAPE	RMSE	MAPE	RMSE
EUR/USD	0.47	0.0067	0.40	0.0065
BTC/USD	2.65	321.8	0.13	106.5
ETH/USD	5.17	15.89	8.39	260.4
S&P 500	2.85	108.9	2.91	106.5
NASDAQ	2.51	289.1	2.77	340.3

TABLE 2.6: The results for six data sets using RF and SVM found by Derbentsev *et al.* [15].

The RF and SVM models show alternating performance on the different time series data. The SVM model had a lower RMSE for the EUR/USD and BTC/USD than the RF model by 2.99% and 66.9%, respectively. At the same time, the RF model had better MAPE values for the ETH/USD, S&P 500 and NASDAQ by 3.22%, 0.06% and 0.26%, respectively. Derbentsev *et al.* [15]’s findings indicated that the more simplistic RF model could outperform the complex SVM model on specific data sets, and both should be incorporated for forecasting time series data spanning over multiple markets.

2.5 Summary

The literature on applying ML methods to financial time series data suggests several areas for improvement and expansion. The papers containing the most applicable information to this study were by Pongsena *et al.* [54] and Von Stein [70]. These two research papers experimented with architecture setups, model training parameters, and feature importance. Both papers are within the foreign exchange markets, and Von Stein [70] uses the same underlying data set USD/ZAR. These papers are expanded on by addressing similar research questions of Pongsena *et al.* [54] to the same deep neural network, LSTM, and two additional ML models, random forests (RF) and support vector machines (SVM), which, from the literature, have shown strong predictive performance.

CHAPTER 3

Data and feature generation

Contents

3.1	Data set	19
3.2	Data Cleaning	21
3.3	Baseline features	21
3.3.1	Open, high, low, close (OHLC)	21
3.3.2	Daily change	22
3.3.3	Day of the week	22
3.3.4	Next day's open price	23
3.4	Technical analysis based features	23
3.4.1	Moving average	24
3.4.2	Moving average convergence divergence	25
3.4.3	Average true range	25
3.4.4	Average directional index	27
3.4.5	Relative strength index	29
3.4.6	True strength index	31
3.4.7	Stochastic oscillator	31
3.4.8	Bollinger bands	32
3.4.9	Keltner Channel	34
3.5	Labels	35
3.6	Summary of features and parameter values	35

Selecting a good set of features for a prediction model is crucial since all models can only learn from the information available. Géron [24] reinforces the saying, ‘garbage in, garbage out’, meaning that a model can only learn if it contains enough relevant features and not many irrelevant ones. This chapter describes the data set and the generation of features used for all models in the remainder of this study.

3.1 Data set

All data used in this study comes from FxPro [22], an online trading broker where numerous financial instruments, such as bonds, stocks and foreign exchange pairs, can be bought and

sold. FxPro [22] provides the functionality to access historical prices to analyse patterns, train models, or back-test trading strategies. Utilising data from a genuine online trading broker is advantageous as it relates more closely to real-world trading scenarios, allowing direct application of a trading strategy or results from a predictive model to the broker's trading platform.

The time series data set used was the daily exchange rate of the United States dollar to the South African rand currency pair, which will be referred to as “USD/ZAR” for the remainder of the study. The time horizon chosen for the USD/ZAR was from 2012-01-01 to 2021-12-31, as some data points that predate 2012-01-01 were missing from the data set. This 10-year period contains 2076 data points for which the markets were open, and an overview of the series during the respective period is displayed in Figure 3.1. The closing price range is between ZAR7.43 per USD on 2012-03-1 and ZAR 19.09 per USD on 2020-04-23, with an average daily change of 0.0286%. The data set extracted from FxPro [22] contains the daily date and the corresponding

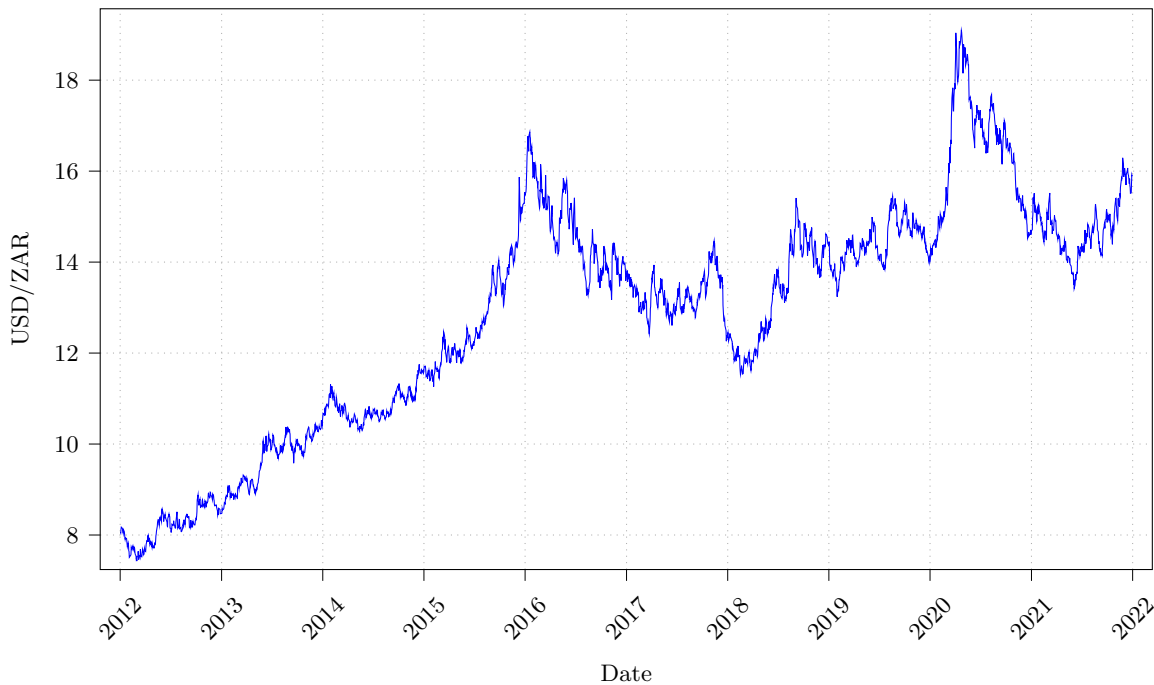


FIGURE 3.1: The daily closing price of the USD/ZAR exchange rate from 2012-01-02 to 2021-12-31.

open price, high price, low price, closing price and trading volume. However, this study excludes trading volume from the data set due to the decentralisation of the foreign exchange market. Therefore, the volume represented on FxPro [22] is a partial overview of the entire market and could introduce potential biases. Table 3.1 contains an example of the type of historical financial data used from FxPro [22].

Date	Open	High	Low	Close	Volume (\$ '00 000)
2021/01/18	15.18	15.37	15.15	15.15	389 664
2021/01/19	15.15	15.18	14.91	14.97	369 879
2021/01/20	14.97	15.01	14.85	14.89	329 578
2021/01/21	14.89	15.01	14.76	14.93	358 647

TABLE 3.1: An example of the type of financial data available from FxPro for the USD/ZAR foreign currency pair.

3.2 Data Cleaning

Data cleaning refers to checking a data set for possible errors, inconsistencies and corrupted data points and correcting them. These errors could consist of incorrect data points; in the case of financial data, this could be the high or low prices for a particular date being lower or higher than another price, respectively. Another caveat when dealing with time series data is that missing data points can disrupt the continuity of price movements, resulting in false price changes that do not accurately reflect the actual price changes over time. For example, if the closing price on 2014-01-20 was ZAR 10 per USD and a month of data was missing, the following data point available was 2014-02-20 with a closing price of ZAR 12 per USD. This change would indicate a 20% price movement between two consecutive days if inputted into a model. Therefore, if missing data points exist, either fill in these data points from external sources, interpolate between data points, only include the data set without missing data, or discard the data set and choose a new time series without missing data.

The USD/ZAR data set contained no missing data points, and data points were consistent with their attributes from 2012-01-01 to 2021-12-31, verified in *Python* [56] using the *pandas* library. Due to financial markets being closed on certain holidays, for example, 2020-12-25 (Christmas Day), these days were still present in the data set. They contained prices with zero change from the previous day, *i.e.*, the current day's open, high, low and closing prices were equal to the previous day's corresponding prices. These days were removed from the data set since retaining them would imply that no change had occurred between two consecutive days.

3.3 Baseline features

Features are the primary source of information available to a model, irrespective of how intricate and complex the architecture is; without a good selection of relevant features, the model will produce poor results. The relevant literature contains two main categories of data to predict foreign exchange rates, namely the use of fundamental analysis and technical analysis [76]. The former uses macroeconomic indicators such as GDP, unemployment rates and company reports, while the latter uses market prices to derive signals about where prices might go. Most fundamental data is not readily available and is usually released monthly, quarterly or yearly. When working with daily prices like the USD/ZAR data set, having data with varying time horizons can cause issues when building prediction models. This raises a new problem in finding techniques to transform longer time frames into a daily time frame accurately. However, the problem falls outside of this study's scope, thus assuming that all the necessary information is already incorporated into prices, allowing for extracting features from the data set using technical analysis alone.

3.3.1 Open, high, low, close (OHLC)

Open, high, low, close or OHLC, as abbreviated, refers to the respective prices for a given time series data within a specific time frame, which is daily in this study. These values summarise the overall movement of prices from the beginning to the end of a trading day. Candlestick plots are standard financial plots that visualise these OHLC values as seen in Figure 3.2, with green candlesticks for trading days where the closing price is higher than the open price (price increase) and red candle sticks for a price decrease. The longer the candlestick and length of the shaded bar indicate a greater fluctuation of prices throughout a trading day. For example, on 2016-02-26,

the total price movement was 5.1% (low to high), as opposed to the previous day of 2016-02-25 of only 1.64%, with the former date having a longer candlestick. The remaining features selected, derived from the OHLC values of the USD/ZAR exchange rate, comprise of popular technical indicators that show reasonable predictive performance in previous research [47, 69, 70]. The mathematical equations will use the following notation for all remaining technical indicators. Let

- o_t be the open price at time step t ,
- h_t be the high price at time step t ,
- ℓ_t be the low price at time step t and
- c_t be the close price at time step t .



FIGURE 3.2: An example of a candle stick plot for the USD/ZAR exchange rate from 2016-01-21 to 2016-03-30.

3.3.2 Daily change

Daily change is the percentage difference between the opening and closing prices on any given day. This metric tracks the overall change in the price and would simulate the profit (loss) if a financial instrument was bought at the open price and sold at the closing price. The daily change for time step t is calculated as

$$D_{c_t} = \frac{c_t - o_t}{o_t}. \quad (3.1)$$

3.3.3 Day of the week

Unlike the stock market, which operates between certain hours during the same period, the foreign exchange markets are open continuously from Monday to Friday. The day-of-the-week effect, as referred to in literature, is an overlooked effect that impacts the volatility and returns of the market depending on what day of the week it is [19, 20, 68]. Since the day of the week

is categorical and simply using the string ‘Monday’ as an input, a model will not know how to interpret this value. Therefore, one-hot-encoding using `sci-kit learn` [53] will transform the day of the week into a binary feature vector as seen in Table 3.2, where the position of the 1 in the input vector will denote which day it is. The feature vector will be the one-hot-encoded columns Monday, Tuesday, Wednesday, Thursday, and Friday in Table 3.2.

Date	Day of the week	Monday	Tuesday	Wednesday	Thursday	Friday
2021/01/18	Monday	1	0	0	0	0
2021/01/19	Tuesday	0	1	0	0	0
2021/01/20	Wednesday	0	0	1	0	0
2021/01/21	Thursday	0	0	0	1	0

TABLE 3.2: An example of the day of the week transformed into a binary feature vector using one-hot encoding.

3.3.4 Next day’s open price

In financial markets, the opening price of a financial instrument may differ from the previous period’s closing price. This variation arises from after-hour trades that can cause price fluctuations before the market opens. Therefore, providing the model access to the following day’s open price will eliminate discrepancies caused by after-market actions. This data is essential for making accurate predictions and developing effective trading strategies, as all trades and predictions are made at the start of each trading day based on the open price.

3.4 Technical analysis based features

A popular method used to analyse financial markets and predict future price movements based on historical prices is technical analysis [47]. The analysis involves investigating patterns, trends and signals generated by technical indicators, of which thousands are available. Each technical indicator aims to capture different market aspects and price information. The indicators fall into four categories: trend, momentum, volatility, and volume. Trend indicators help identify and confirm the direction of the market. The momentum-based indicators measure the strength of price changes and indicate the direction of price movement. High momentum levels suggest prices will continue to move in the same direction, while low momentum indicates minimal continuous movement in a specific direction. Volatility indicators measure the deviation of prices from the mean, indicating the range within which prices are likely to fluctuate. High volatility suggests larger price movements, while lower volatility suggests small ones. Finally, volume indicators measure the number and size of transactions during a fixed period. However, this study excludes volume indicators due to the decentralised nature of the foreign exchange market, which might lead to a biased data set based on the source of information.

When incorporating technical indicators in trading or prediction models, combining multiple indicators to analyse different market aspects and filter false signals from a single indicator is the best practice. This study employs well-known technical indicators from the literature, including some features used by Von Stein [70], who used the same foreign exchange pair, USD/ZAR, in their research.

3.4.1 Moving average

The moving average is the mean of a fixed-length window rolled through time, removing the last time step and appending the new time step to the window. The moving average is useful for smoothing out volatility and noise in the time series, thus extracting the market's underlying trend [14]. Let $S\overline{m}_t(r)$ be the simple moving average (SMA) calculated at the end of time step t using the previous r time steps calculated as

$$S\overline{m}_t(r) = \frac{1}{r} \sum_{i=t-r+1}^t c_i. \quad (3.2)$$

Equation (3.2) calculates a simple moving average (SMA) since all values used to compute the average have equal weighting. Many technical indicators build on moving averages to first filter out noise in the prices before additional calculations, such as Bollinger bands [8] and Keltner channels [34]. Figure 3.3 contains the 9-day and 21-day SMA of the USD/ZAR closing prices, typically the default values used in literature. Traders can interpret market trends by comparing

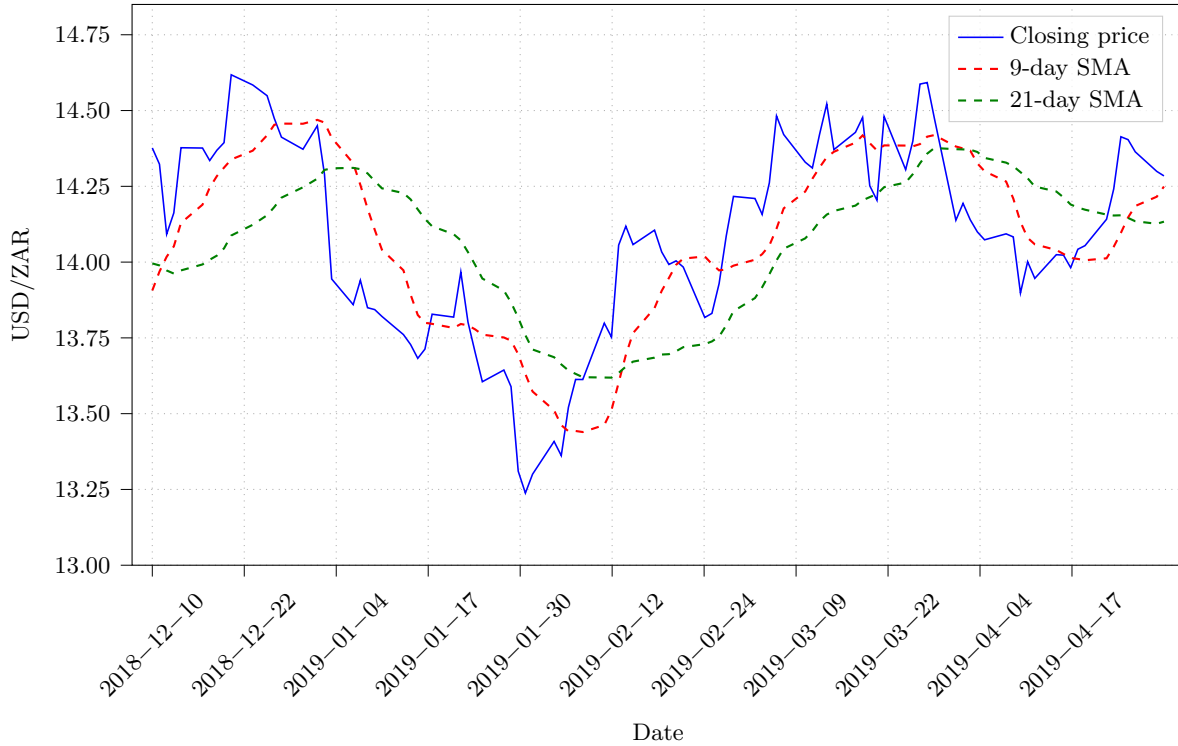


FIGURE 3.3: An illustration of the 9-day and 21-day simple moving averages plotted over the closing price of the USD/ZAR exchange rate from 2018-12-10 to 2019-04-30.

the SMA to market prices. For example, 2018-12-22 in Figure 3.3, the closing price exceeds the SMA, indicating an upward trend in the markets [14]. Traders also use the SMA relative to other SMAs or prices to generate market signals depending on specific phenomena. For example, the cross-over strategy entails when a shorter period moving average crosses a longer moving average line, signalling a change in the trend [47]. Figure 3.3 contains an illustration of this phenomenon on 2019-01-06, where the 9-day SMA crosses the 21-day SMA, signalling that the market is changing from an upward to a downward trend.

3.4.2 Moving average convergence divergence

The moving average convergence divergence (MACD) introduced by Gerald Appel [1] is a trend-following momentum indicator, meaning the indicator calculates the underlying trend first and then calculates the trend's strength, *i.e.*, momentum. The MACD represents the convergence (C) and divergence (D) of the exponential moving averages (MA) used in its calculations. The term 'convergence' refers to a scenario where price movements align with the direction of momentum, meaning increasing prices alongside rising momentum. This alignment often suggests that the underlying trend is likely to continue. Conversely, 'divergence' occurs when price movements deviate from the direction of momentum, which may involve prices moving in the opposite direction as momentum decreases or decelerates. Divergence can be an early warning sign of a potential trend reversal [2].

The calculation of MACD involves subtracting a longer-term exponential moving average (EMA), which reacts slower to price changes, from a shorter-term EMA, which reacts faster to price changes. Let ${}^E\overline{m}_t(\alpha, \mathbf{x})$ be the EMA of time series $\mathbf{x}$ at the end of time step t and weight α , the formulas is given as

$${}^E\overline{m}_t(\alpha, \mathbf{x}) = \alpha x_t + (1 - \alpha)x_{t-1} \quad \text{when } t \geq 1, \quad (3.3)$$

with ${}^E\overline{m}_0(\alpha, \mathbf{x}) = x_0$. The MACD, $M_d(r_1, r_2, \mathbf{x})$, at the end of time step t over the previous r time steps of time series $\mathbf{x}$ is then calculated as

$$M_d(r_1, r_2, \mathbf{x}) = {}^E\overline{m}_t(\alpha_1, \mathbf{x}) - {}^E\overline{m}_t(\alpha_2, \mathbf{x}), \quad (3.4)$$

with smoothing factors $\alpha_1 = 2/(r_1 + 1)$, $\alpha_2 = 2/(r_2 + 1)$ with $r_1 < r_2$ and $\mathbf{x}$ being the time series of the closing price.

The MACD indicates the direction and strength of the trend, with positive values indicating an upward trend and negative values a downward trend. There is no fixed range for MACD values, but the further they are from zero, the stronger the trend.

The reversal of the trend from positive to negative on 2017-11-28 when the MACD crosses the baseline is illustrated in Figure 3.4. A common strategy among traders is to use the MACD to enter a position (either buy or sell) when the MACD crosses the baseline the first time and exit their position when the MACD crosses the baseline for a second time [14]. By adopting this strategy, a trader would have entered a sell order on 2017-11-28 at the price of ZAR 13.65 per USD and exited their position on 2018-03-20 at the price of ZAR 11.95 per USD, resulting in a profit of 12.45% assuming no transaction costs.

Figure 3.4 also contains an example of divergence between prices and momentum on 2017-12-29, where the MACD has a low of -0.3815 , indicating a strong downward price trend. However, the MACD started to increase while remaining negative, suggesting that the downward trend was still present, but the momentum was decreasing. Thus signalling a possible trend reversal, confirmed by the closing price trending upwards two months later.

3.4.3 Average true range

The Average True Range (ATR) is a volatility indicator proposed by J. Welles Wilder [71]. According to Wilder, the range, defined as "the distance the price moves per increment of time," is directly related to market volatility. Thus, determining the market's range can help extract its underlying volatility. The term 'true range' describes the maximum price movement from the current day or the previous day's closing price to the highest or lowest price of the

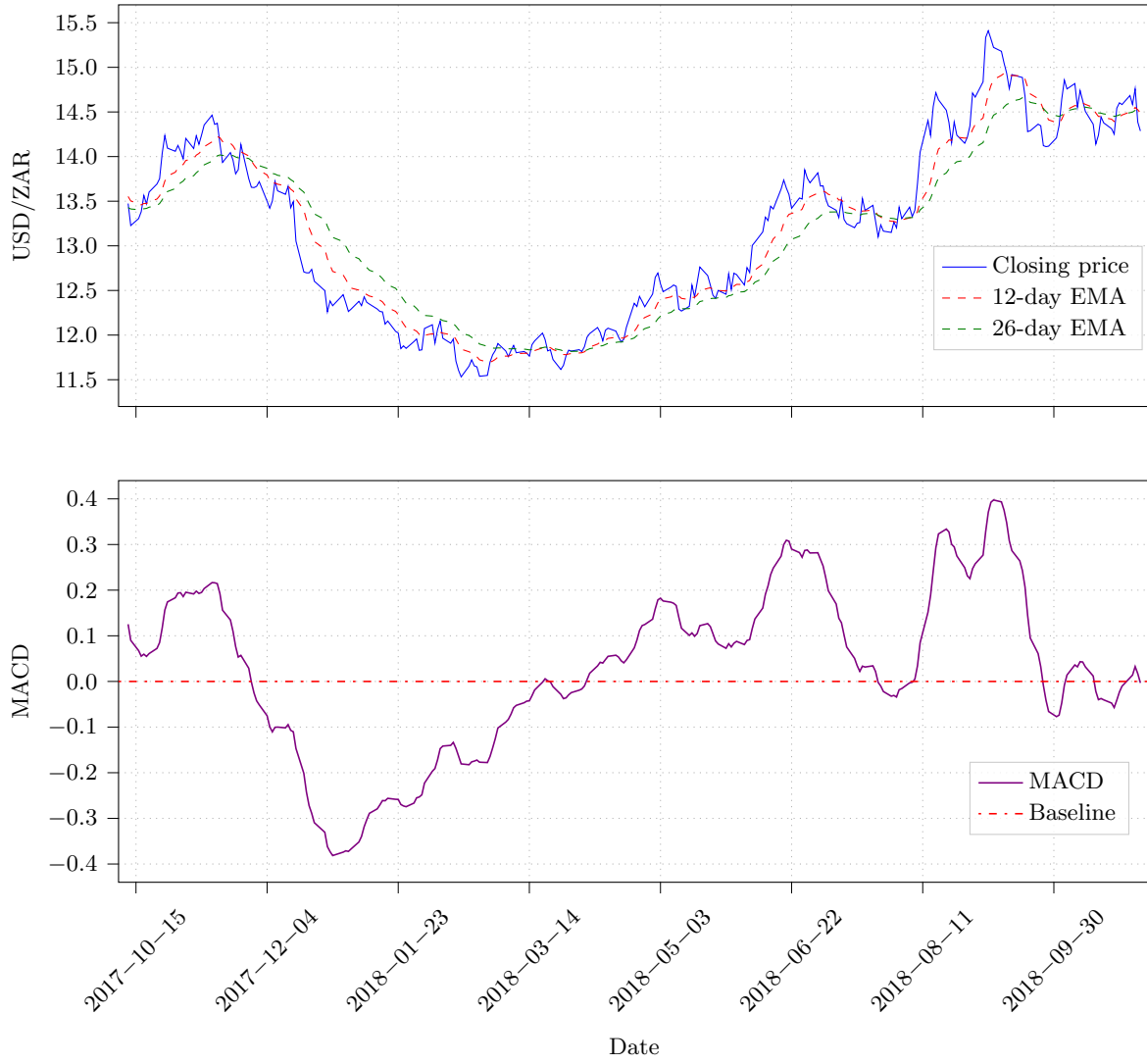


FIGURE 3.4: An illustration of the MACD with $r_1 = 12$ and $r_2 = 26$ plotted over the closing price of the USD/ZAR exchange rate (c_t) from 2017-10-12 to 2018-11-02.

current day, considering after-market price changes [71]. The true range of each time step t can be calculated as

$$T_{r_t} = \max \left\{ (h_{t-1} - \ell_{t-1}); |h_{t-1} - c_{t-2}|; |l_{t-1} - c_{t-2}| \right\}. \quad (3.5)$$

The ATR, $T_{\bar{r}_t}(r)$, at the end of time step t over the previous r time steps is then calculated as

$$T_{\bar{r}_t}(r) = \frac{1}{r} \sum_{i=t-r+1}^t T_{r_i} \quad (3.6)$$

The ATR value represents the average volatility over the selected time horizon and needs to be compared to previous ATR values to interpret the market volatility. While ATR does not indicate the direction of price movement, traders can adjust their position size or take-profit and stop-loss levels to account for higher or lower volatility [69].

Figure 3.5 contains an illustration of the ATR over the USD/ZAR closing prices with a maximum ATR value of 0.2581 on 2017-12-18, indicating the most volatile period in Figure 3.5's

time horizon. The sudden increase of ATR on 2017-12-18 was caused by the closing price decreasing 3.35% from the previous day compared to the average daily change of 0.76% for the preceding period, suggesting increased market volatility. Traders may increase their position size to exploit the increased price fluctuations. However, the increased volatility was temporary, and subsequent days showed a decrease in ATR value, indicating a more stable market.

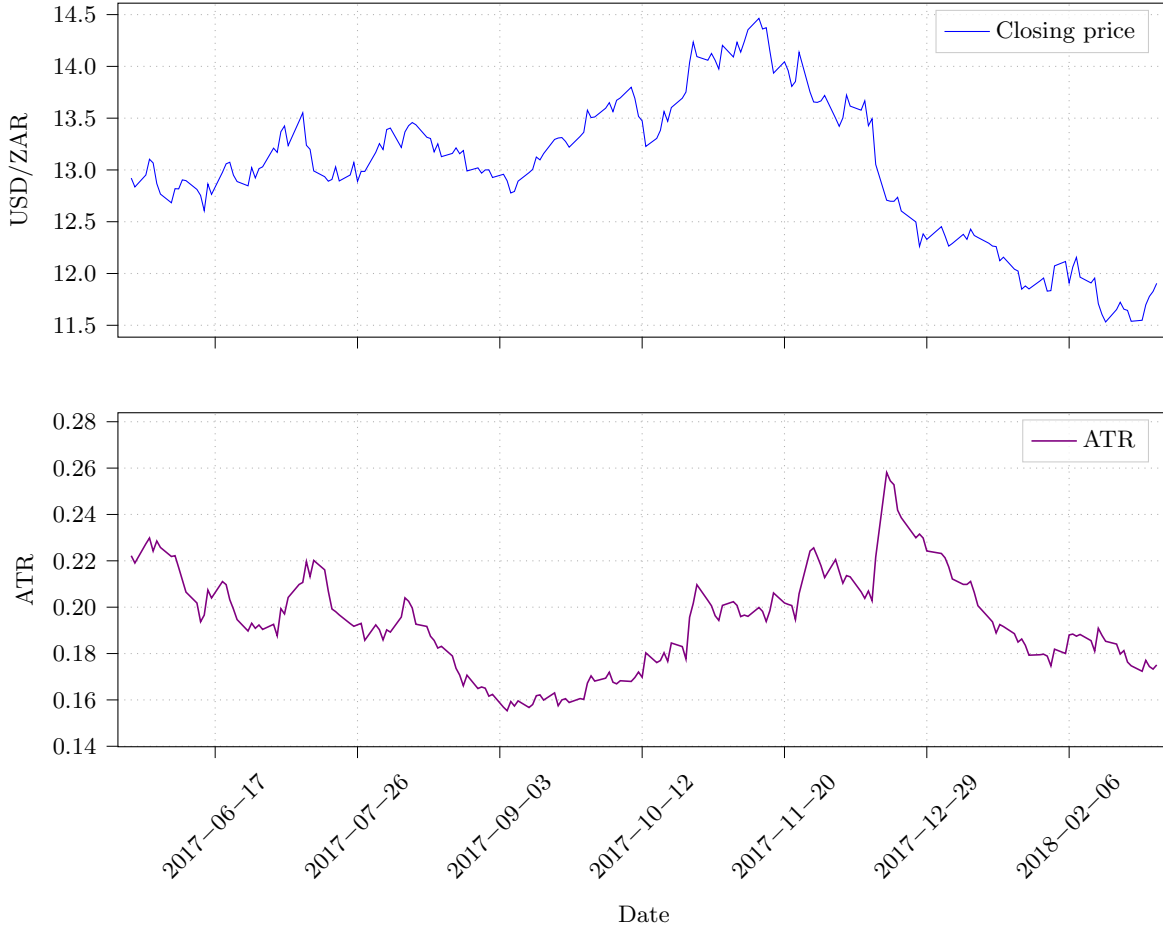


FIGURE 3.5: An illustration of the 14-day ATR plotted over the closing price of the USD/ZAR exchange rate from 2017-05-25 to 2018-03-02.

3.4.4 Average directional index

The average directional index (ADX) developed by J. Welles Wilder [71] combines three technical indicators, namely the positive/negative directional movement (+DM/−DM), the positive/negative directional index (+DI/−DI) and the average true range (ATR). The first step is to capture the daily movements in prices by calculating the +DM, Pm_t , and −DM, Nm_t , for each day t formulated as

$$^Pm_t = \begin{cases} h_t - h_{t-1} & \text{if } (h_t - h_{t-1}) > (l_{t-1} - t_i) \\ 0 & \text{otherwise, and} \end{cases} \quad (3.7)$$

$$^Nm_t = \begin{cases} l_{i-t} - l_t & \text{if } (l_{t-1} - l_t) > (h_t - h_{t-1}) \\ 0 & \text{otherwise.} \end{cases} \quad (3.8)$$

Then determine the average of both directional indices, $^P\overline{m}_t(r)$ and $^N\overline{m}_t(r)$ at the end of each time step t using the previous r time steps as

$$^P\overline{m}_t(r) = \frac{1}{r} \sum_{i=t-r+1}^t ^Pm_i, \quad \text{and} \quad (3.9)$$

$$^N\overline{m}_t(r) = \frac{1}{r} \sum_{i=t-r+1}^t ^Nm_i. \quad (3.10)$$

Let $^T\overline{r}_t(r)$ be the ATR at the end of time step t using the previous r time steps calculated using equation (3.6) given in §3.4.3. Once the +DM, −DM and ATR have been calculated, the +DI, $^Pd_t(r)$, and −DI, $^Nd_t(r)$, for each day t using the previous r days can be calculated by multiplying the smoothed moving average of each respective directional index divided by the ATR for the same period by 100. The formulas are

$$^Pd_t(r) = \frac{100}{1 + \frac{^P\overline{m}_t(r)}{^T\overline{r}_t(r)}}, \quad \text{and} \quad (3.11)$$

$$^Nd_t(r) = \frac{100}{1 + \frac{^N\overline{m}_t(r)}{^T\overline{r}_t(r)}}. \quad (3.12)$$

Equations (3.11) and (3.12) measure the strength of an upward and downward trend, respectively. Lastly, the ADX, $^Ad_t(r)$, at the end of day t over the previous r days can be calculated by further smoothing the +DI and −DI and dividing by the ATR, given as

$$^Ad_t(r) = \left(\frac{\left| \frac{^Pd_t(r) - ^Nd_t(r)}{^Pd_t(r) + ^Nd_t(r)} \right|}{^T\overline{r}_t(r)} \right) \cdot 100. \quad (3.13)$$

The ADX is a tool used to assess market trend strength, but not direction [71]. Traders can compare the +DI with the −DI to determine the direction of the trend. The ADX values range between 0 and 100 with varying trend strength and direction levels defined by Wilder [71] and shown in Table 3.3.

ADX value	Trend strength	Direction of trend
0–25	Absent or weak trend	
25–50	Strong trend	$\left\{ \begin{array}{l} \text{Positive if } +\text{DM} > -\text{DM} \\ \text{Negative if } +\text{DM} < -\text{DM} \end{array} \right.$
50–75	Very strong trend	
75–100	Extremely strong trend	

TABLE 3.3: A description of the different ADX values with their varying trend strengths and direction.

Figure 3.6 shows the ADX value exceeding 50 with the $+\text{DM} > -\text{DM}$, indicating a very strong positive trend on 2016-01-20. A trader solely relying on the ADX will be inclined to believe that the market will continue to increase, and a price reversal is unlikely due to the ADX value. However, this is not the case in this example, as the price almost immediately declines after the ADX reaches above 50. Thus, this highlights the importance of incorporating more than one technical indicator and that even if all signs indicate a specific pattern, the market will not always exhibit the same response. The ADX values for the remainder of Figure 3.6 mainly oscillate below 25, with no lasting breakouts indicating an absent or weak trend.



FIGURE 3.6: An illustration of the 14-day average directional index plotted over the closing price of the USD/ZAR exchange rate from 2015-11-10 to 2016-08-17.

3.4.5 Relative strength index

The relative strength index (RSI), developed by J. Welles Wilder [71], measures the velocity of a market's directional price movement and is a momentum oscillator ranging from 0 to 100. The RSI compares the average gains with the average losses to generate signals indicating if the market is oversold or overbought. The convention in literature and among industry professionals is that an RSI value over 70 suggests that the current market is overbought, and a price correction leading to prices decreasing is likely. In contrast, an RSI value below 30 suggests that the current market is oversold, and a price correction leading to price increases is likely [14, 47, 71].

The relative strength index at the end of time step t over the previous r time steps is calculated using the average gain and loss over the same time steps. The average gain, $A\bar{g}_t(r)$, and average loss, $A\bar{l}_t(r)$, is calculated as

$$A\bar{g}_t(r) = \frac{1}{r} \sum_{i=t-r+1}^t \max\{c_i - o_i; 0\}, \quad \text{and} \quad (3.14)$$

$$A\bar{l}_t(r) = \frac{1}{r} \sum_{i=t-r+1}^t \max\{o_i - c_i; 0\}. \quad (3.15)$$

The relative strength index, $R_{st}(r)$, at the end of time step t over the previous r time steps can

then be calculated as

$$R_{S_t}(r) = 100 - \left[\frac{100}{1 + \frac{A_{\bar{g}_t(r)}}{A_{l_t(r)}}} \right] \quad \text{for } t \geq r. \quad (3.16)$$

The RSI can also be approximated for the first r time steps, using the average gain and loss up to that time step. Figure 3.7 contains an illustration of the RSI technical indicator over the USD/ZAR closing prices. On 2015-08-21, the RSI line crossed the theoretical overbought line, thus exceeding a value of 70. This RSI value suggests that a price reversal is likely at this point; however, the price continued to increase until 2015-09-06, reaching an RSI value of 85, which would further suggest a price correction. This time, the signal was correct, and the price fell 2.24% from 2015-09-06 to 2015-09-10 before the RSI value returned to a value between 30 and 70 for the remainder of the chosen time horizon, indicating no expected price movement.

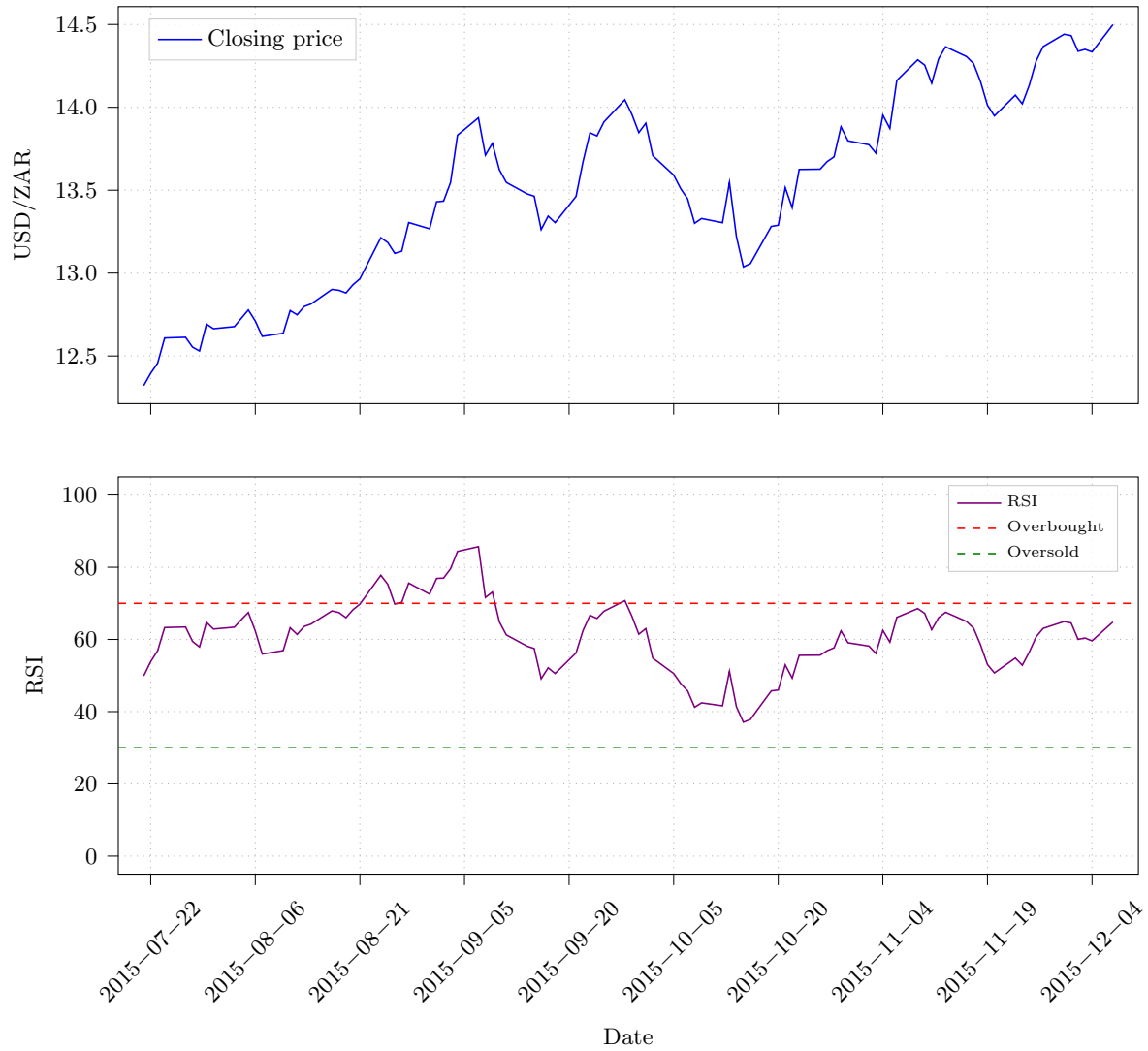


FIGURE 3.7: An illustration of the 14-day RSI plotted over the closing price of the USD/ZAR exchange rate from 2015-07-21 to 2015-12-07.

3.4.6 True strength index

The True Strength Index (TSI), developed by William Blau [6], is a momentum oscillator. The TSI measures the strength of price movements while effectively filtering out the noise, resulting in smoother price trends [14]. The TSI estimation involves a two-step process: first, by calculating the rate of change over a specified period, as defined in equation (3.17); second, by applying a double smoothing using two EMAs as defined in equation (3.3). Typically, the first EMA contains a longer period than the second EMA. The final TSI value derives from dividing the double-smoothed rate of change by the double-smoothed absolute value of the rate of change, thus accounting for negative price movements.

The rate of change, $R_{c_t}(r)$ at the end of time step t over the previous r time steps is calculated as

$$R_{c_t}(r) = \frac{c_t - c_{t-r+1}}{c_{t-r+1}} \cdot 100 \quad (3.17)$$

The TSI, $T_{s_t}(s_1, s_2, r)$, at the end of time step t over the previous r time steps is calculated as

$$T_{s_t}(s_1, s_2, r) = \frac{E\overline{m}_t(s_1, (E\overline{m}_t(s_2, R_{c_t}(r)))}{E\overline{m}_t(s_1, |(E\overline{m}_t(s_2, R_{c_t}(r)))|)}, \quad (3.18)$$

where s_1 and s_2 represent the number of days for which the second and first EMAs are calculated, with $s_1 < s_2$. The TSI ranges from -100 to 100 , providing insights into price momentum and direction. Values above 25 indicate strong positive momentum, while values below -25 suggest strong negative momentum [47].

Figure 3.8 contains an illustration of the 14-day TSI overlaid on the closing price of the USD/ZAR exchange rate from 2015-10 to 2016-12. In Figure 3.8, the TSI crosses above the strong positive line on 2015-12-09 and remains consistently above this threshold until crossing down again on 2016-01-29. This crossing above the positive line indicates a strong upward price momentum, potentially signalling increasing prices, confirmed by the closing price rising from 14.97 to 15.86 , a 5.95% increase in price over the same period. Traders often use TSI crossings as signals for potential trading strategies, with crossings above the positive line potentially signalling buy opportunities and crossings below the negative line indicating sell positions [47].

3.4.7 Stochastic oscillator

The Stochastic oscillator developed by George C. Lane [36] is a momentum oscillator that ranges between 0 and 100 . Like the RSI and TSI, stochastic oscillators indicate overbought and oversold conditions in the market. Colby [14] suggests that the stochastic oscillator indicates overbought and oversold conditions when they rise above 80 and fall below 20 , respectively. The stochastic oscillator, $O_{s_t}(r)$, at the end of time step t for the previous r time steps is calculated as

$$O_{s_t}(r) = \left(\frac{c_{t-1} - \min_i \{\ell_i | t-r+1 \leq i \leq t\}}{\max_i \{h_i | t-r+1 \leq i \leq t\} - \min_i \{\ell_i | t-r+1 \leq i \leq t\}} \right) \cdot 100. \quad (3.19)$$

Figure 3.9 contains a visual representation of the 14-day stochastic oscillator applied to the closing price of the USD/ZAR currency pair. Notably, the stochastic oscillator exhibits more frequent oscillations exceeding the overbought and oversold conditions than the RSI and TSI indicators. This increased frequency is due to the stochastic oscillator's sensitivity to short-term price changes [14]. As outlined by Lane [36], stochastic oscillator values exceeding its

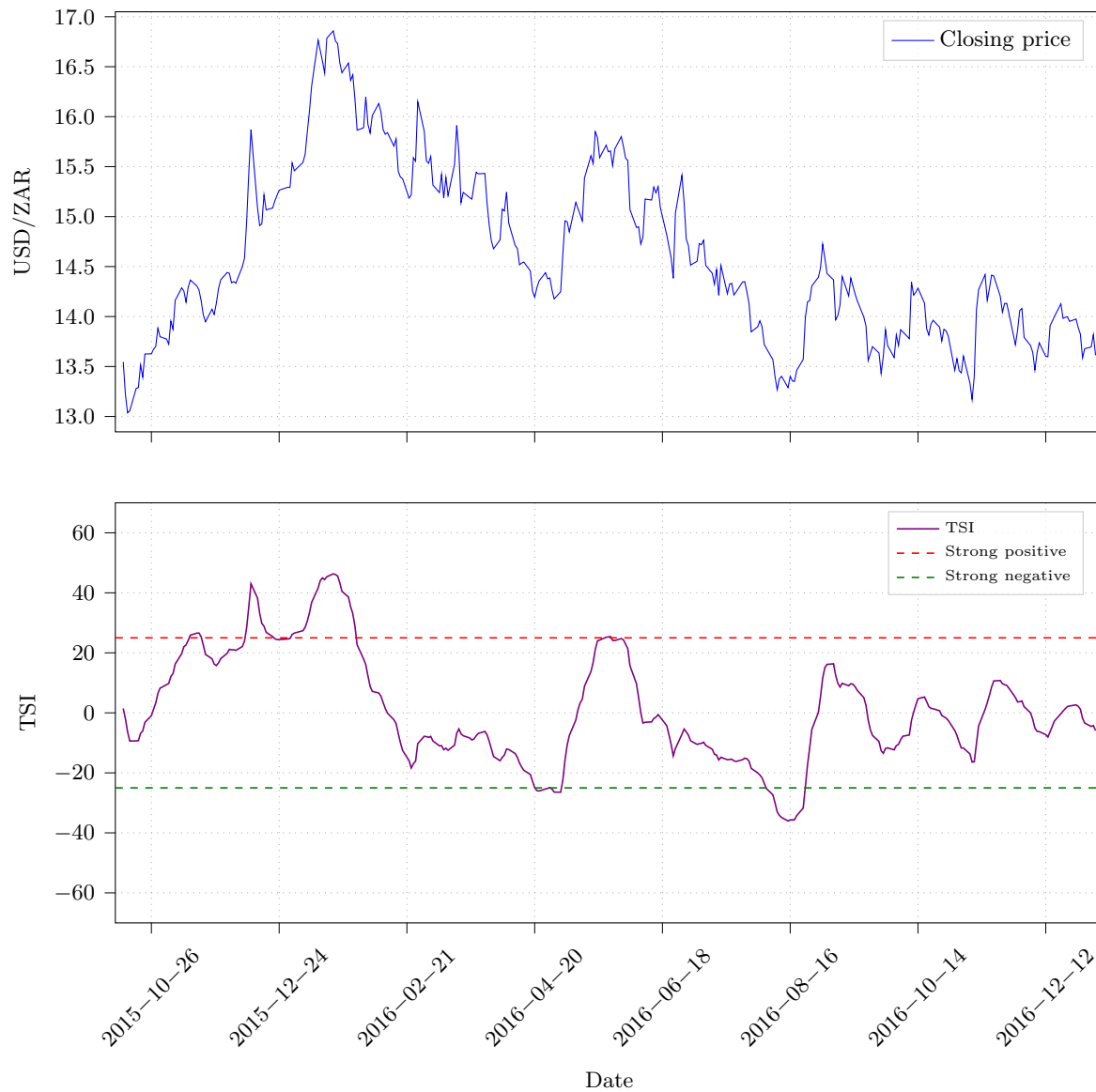


FIGURE 3.8: An illustration of the 14-day true strength index plotted over the closing price of the USD/ZAR exchange rate from 2015-10-13 to 2017-01-04.

overbought/oversold conditions (80/20) should not result in an immediate trading action but rather act as a ‘trigger’ that is ‘confirmed’ by another indicator such as ADX or SMA [36]. For instance, if the closing price reaches a new high while the stochastic oscillator fails to confirm the movement, this disparity could indicate a price divergence and signal a potential reversal. Figure 3.9 contains instances of such divergence: a new high in the 14-day closing price followed by subsequent highs, each associated with a lower stochastic oscillator value than the initial high. Traders may consider taking a short position in these instances.

3.4.8 Bollinger bands

Bollinger bands (BB) are a popular technical indicator used frequently in literature and among professional traders and fall under the volatility category [14, 47]. The indicator developed by trader John Bollinger [8] consists of three bands: a simple moving average in the middle and

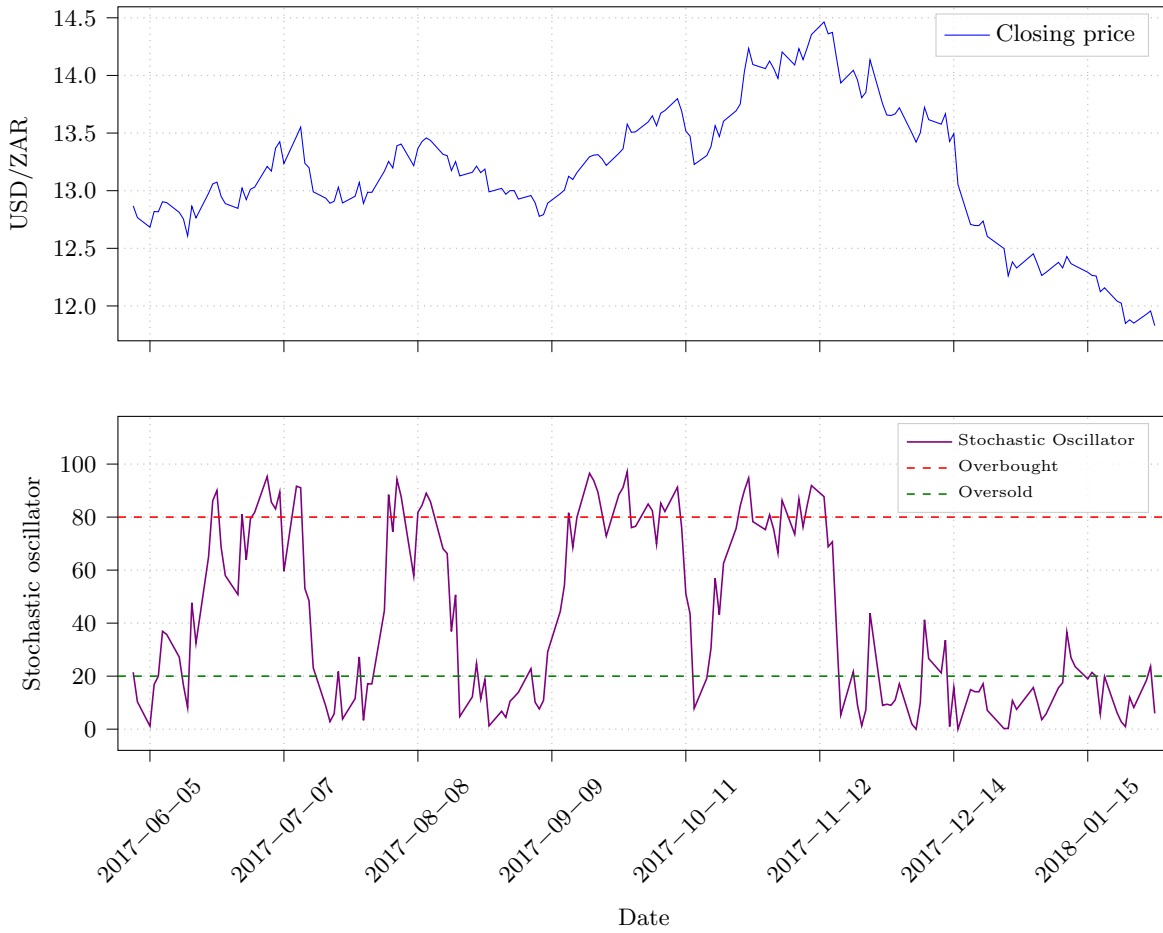


FIGURE 3.9: An illustration of the 14-day stochastic oscillator plotted over the closing price of the USD/ZAR exchange rate from 2017-06-01 to 2018-01-31.

two standard deviation bands above and below the simple moving average line. These bands will dynamically adjust to the changes in volatility, expanding when the market is more volatile and contracting when less volatile. Trading signals can be derived from these bands when the price of a market moves outside of the upper or lower bands, indicating overbought and oversold conditions, respectively [47]. The standard deviation, $\sigma_t(r)$, at the end of time step t of the closing prices over the previous r time steps is calculated as

$$\sigma_t(r) = \sqrt{\frac{1}{r} \sum_{i=t-r+1}^t \left(c_i - S\overline{m}_t(r) \right)^2}. \quad (3.20)$$

Let $U_{b_t}(r, s)$, $M_{b_t}(r)$ and $L_{b_t}(r, s)$ be the upper, middle and lower Bollinger bands, respectively, calculated at time step t over the previous r time steps and s standard deviations. The three Bollinger bands are then calculated as

$$M_{b_t}(r) = S\overline{m}_t(r), \quad (3.21)$$

$$U_{b_t}(r, s) = S\overline{m}_t(r) + s \cdot \sigma_t(r), \quad \text{and} \quad (3.22)$$

$$L_{b_t}(r, s) = S\overline{m}_t(r) - s \cdot \sigma_t(r). \quad (3.23)$$

In Figure 3.10, the closing price of the USD/ZAR on 2017-10-26 exceeded the upper BB, signalling the market was exhibiting overbought conditions. The price dropped by 1.01% the

following day, confirming the generated signal. The time horizon from 2017-12-13 to 2018-01-08 in Figure 3.10 has a wider band (distance from upper to lower BB), indicating a more volatile period in the market; therefore, larger price movements. The average absolute daily price change in this period was 0.9%. Compared to the subsequent period, 2018-01-09 to 2018-01-31, with a narrower band indicating less volatility, showed a reduction of 48.89% in average absolute price change at only 0.46%.



FIGURE 3.10: An illustration of 20-day Bollinger bands using 2 standard deviations plotted over the closing price of the USD/ZAR exchange rate from 2017-10-12 to 2018-03-02.

3.4.9 Keltner Channel

Developed by Chester W. Keltner [34], the Keltner channel is a volatility indicator similar to Bollinger bands, both consisting of three distinct bands or channels and sharing similar interpretations. Keltner channels differ from Bollinger bands by substituting the simple moving average with an exponential moving average and the standard deviation with the average true range in the formulations. The usage of the average true range, given in equation (3.6), provides a smoother representation of volatility and is less responsive to minor price fluctuations, thus filtering out possible false signals that Bollinger bands may generate [47]. However, incorporating both Bollinger bands and Keltner channels captures a more comprehensive view of market volatility.

Let $U_{k_t}(r, s)$, $M_{k_t}(r)$ and $L_{k_t}(r, s)$ be the upper, middle and lower Keltner channels respectively calculated at time step t over the previous r time steps and scaling factor s . The three Keltner

channels are then calculated as

$$M_{k_t}(r) = {}^E\overline{m}_t(r, \mathbf{c}_t), \quad (3.24)$$

$$U_{k_t}(r, s) = {}^E\overline{m}_t(r, \mathbf{c}_t) + s \cdot T_{\overline{r}_t}(r) \quad \text{and} \quad (3.25)$$

$$L_{k_t}(r, s) = {}^E\overline{m}_t(r, \mathbf{c}_t) - s \cdot T_{\overline{r}_t}(r) \quad (3.26)$$

3.5 Labels

In machine learning, a label is the output or target variable a model predicts based on the corresponding feature set. For time series data, the label is typically one or multiple quantities predicted n time steps into the future. Therefore, the quantity used can either be a feature or a label, depending on the current period. This study uses the **closing price**, $\hat{c}_{t+n}$, as the label where t is the current time step, with all information of prices available up to that point, predicted n time steps into the future. This study only considers predicting the daily closing price for the following day, $\hat{c}_{t+1}$, thus $n = 1$, for the following reasons:

1. Narrow the scope.
2. Have access to current information.
3. Mitigate price fluctuations due to unforeseen events.

3.6 Summary of features and parameter values

Table 3.4 contains a summary of all the features and their corresponding parameter values used. This study does not investigate the optimal choice of parameter values for these features, as it falls outside the scope. Instead, the selected parameter values were chosen on recommendations by the indicator's inventors and preferred values used by traders. The features and their associated parameter values will remain constant throughout the remainder of this study, with calibration and experimentation focused on the machine learning models that use these features.

Feature	Type	Parameters	Number of features
Open, high, low, close prices	Base	–	4
Daily change	Base	–	1
Next day's open price	Base	–	1
Day of the week	Base	–	5
Simple moving average	Trend	$r = \{9, 21\}$	2
Moving average convergence divergence	Trend	$r_1 = 12, r_2 = 26, x = \mathbf{c}_t$	1
Average directional index	Trend	$r = 14$	1
Relative strength index	Momentum	$r = 14$	1
True strength index	Momentum	$r = 14, s_1 = 13, s_2 = 25$	1
Stochastic oscillators	Momentum	$r = 14$	1
Average true range	Volatility	$r = 14$	1
Bollinger bands	Volatility	$r = 20, s = 2$	3
Keltner channel	Volatility	$r = 20, s = 2$	3
Total			25

TABLE 3.4: A summary of the data, technical indicators and their respective parameter values used as the features in this study.

CHAPTER 4

Data processing and modelling

Contents

4.1	Feature engineering	39
4.1.1	<i>Data preprocessing</i>	39
4.1.2	<i>Feature scaling</i>	42
4.1.3	<i>Converting to supervised learning</i>	43
4.2	Walk-forward validation	45
4.3	Models considered	46
4.4	Hyperparameter calibration	47
4.4.1	<i>Bayesian optimisation</i>	48
4.4.2	<i>Cross-validation</i>	49
4.5	Testing parameters	51
4.5.1	<i>Retrain interval</i>	51
4.5.2	<i>Sequence length</i>	52
4.6	Test sets	52
4.6.1	<i>Test set 2018</i>	53
4.6.2	<i>Test set 2019</i>	53
4.6.3	<i>Test set 2020</i>	54
4.6.4	<i>Test set 2021</i>	56
4.7	Result metrics	57
4.7.1	<i>Statistical metrics</i>	57
4.7.2	<i>Threshold accuracy</i>	58
4.7.3	<i>Trading strategy/profit</i>	58
4.7.4	<i>Maximum drawdown</i>	60
4.7.5	<i>Number of trades</i>	61
4.7.6	<i>Average profit per trade</i>	61
4.7.7	<i>Profitable trades</i>	61
4.8	Chapter summary	61

This chapter contains an overview of the data processing and modelling outlined in Figure 4.1. The overview continues with the pipeline’s feature scaling and preprocessing step, following the data and feature generation steps discussed in Chapter 3. Since all implemented machine learning models follow the same modelling process, the process is discussed in general, elaborating on the parameter calibration technique, different test sets and result metrics used by each model. Therefore, each model will be self-contained in separate chapters concerning its background, development and results of the discussed sections.

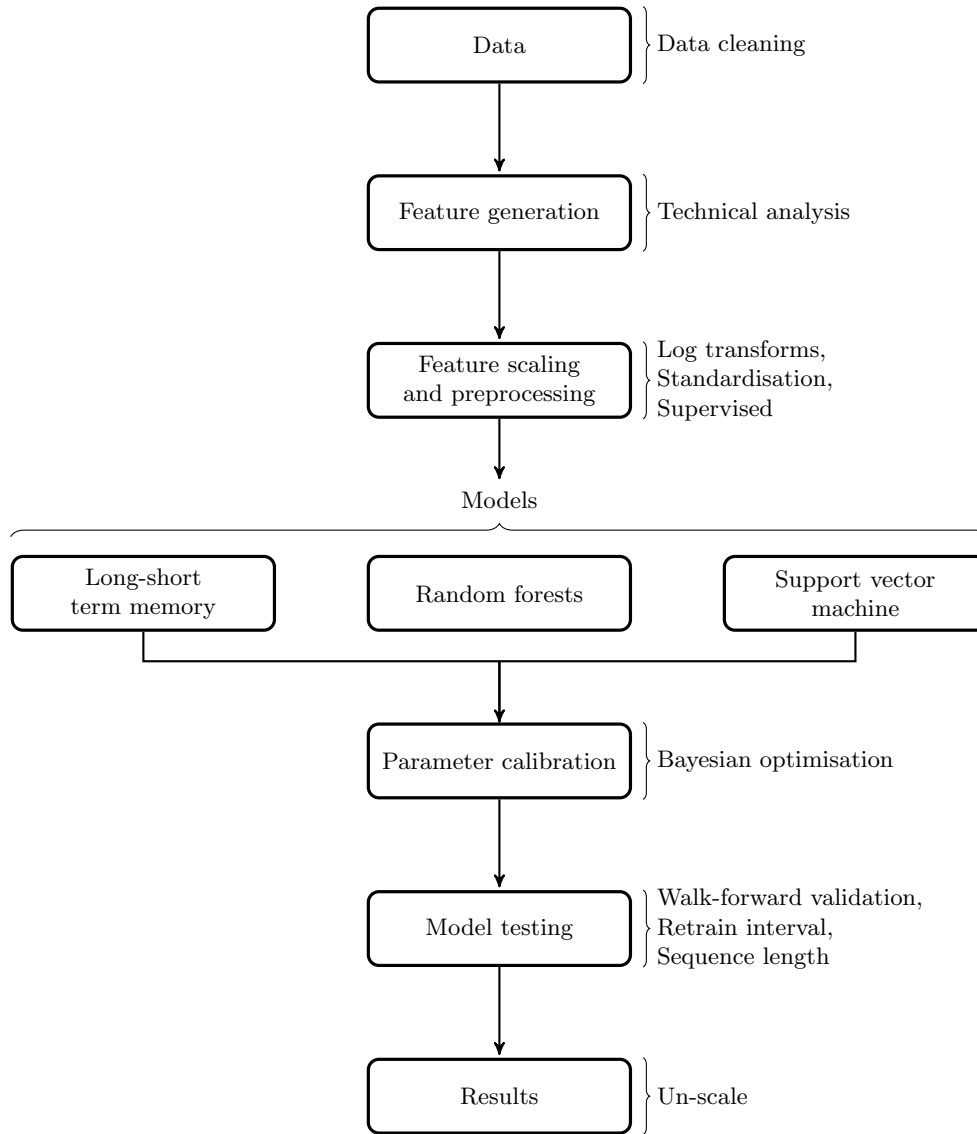


FIGURE 4.1: A schematic representation of the modelling process used in the study.

Data preprocessing, model implementation, parameter calibration, model training, and testing were done on a single computer running Windows 10 64-bit operating system equipped with a 3.7GHz AMD Ryzen 5900x CPU, Gigabyte GeForce RTX 3080 Nvidia graphics card, 64GB 3200MHZ DDR4 RAM by Gskills, and a 2TB Samsung 980 Pro PCIe 4.0 NVMe SSD. The models were developed using Python 3.9 [56] in conjunction with the scikit-learn [53], NVIDIA cuDNN [51], and Keras libraries [13].

4.1 Feature engineering

Feature engineering is an essential step in the machine learning pipeline and refers to the *ad hoc* process of selecting, generating or transforming features from data to improve model performance [77]. The generated features in Table 3.4 require further modification to extract more information and transform certain features into a preferred range. Machine learning models perform best when interpolating data, and through experimentations on the data set, the models had poor performance when extrapolating data. Difficulty extrapolating means that the models returned poor results when tested on features outside the range of the features used to train the model. The machine learning models experience a phenomenon where the outputs are clipped at the highest range of inputs received during training. Figure 4.2 shows an example of an LSTM model making predictions trained on the unscaled/un-processed feature set given in Table 4.3. The LSTM model cannot extrapolate the data since the range of features for the training set, notably the closing price, is between ZAR 8.48 per USD and ZAR 12.59 per USD, while the test set contained a range of ZAR 11.77 per USD and ZAR 16.86 per USD.

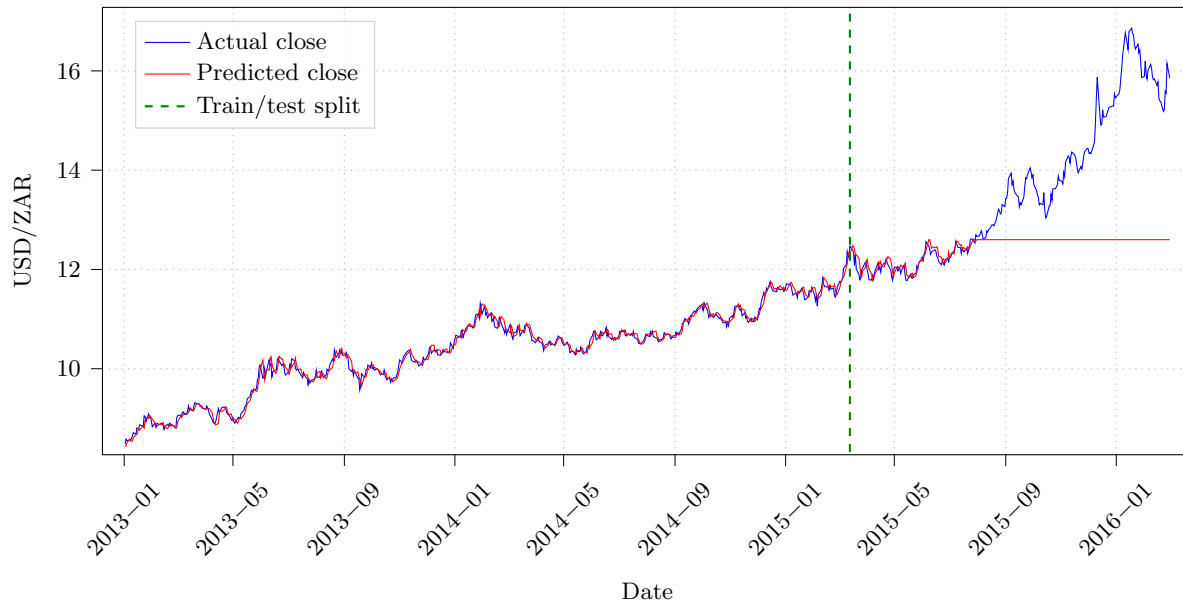


FIGURE 4.2: An illustration of an LSTM NN using unscaled data to predict the closing prices of the USD/ZAR currency pair trained on 2012-01-01 to 2015-03-31 and tested on 2015-04-01 to 2016-02-31.

This feature engineering focuses primarily on achieving a consistent scale among the chosen features, ensuring no or little mismatches between training and test sets without leaking data from the test set. This objective is achieved by transforming and scaling features through data preprocessing and standardisation.

4.1.1 Data preprocessing

All the baseline features, excluding the **Day of the week** (binary), contain continuous values with ranges that vary across different periods. Leaving the values unchanged will entice the problem of extrapolation for the training and test set in Figure 4.2. The values across all periods must be similar and comparable regardless of the range. A few options were available to choose from and explore, which involved transforming the features to a new value based on two subsequent periods through:

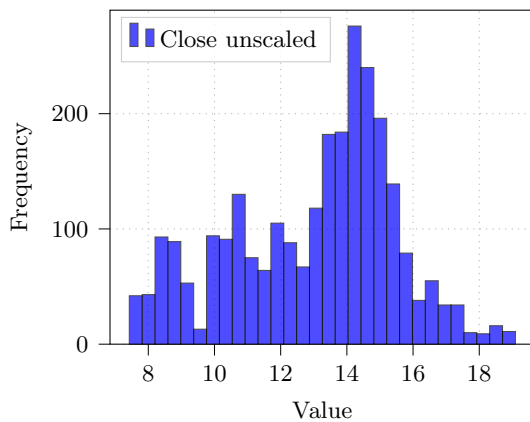
1. Differences,
2. Percentage differences, and
3. Log percentage differences.

The first method uses the difference between features for time t and $t - 1$; however, just using the differences encounters the same problem of mismatched scales. For example, if the price moves from ZAR 10.00 per USD to ZAR 11.00 per USD or ZAR 19.00 per USD to ZAR 20 per USD, they both indicate a change of ZAR 1 per USD, but the former corresponds to a 10% increase in price, while the latter is only a 5.23%. Therefore, smaller movements at lower price ranges have more effect on prices than the same magnitude of change at higher prices.

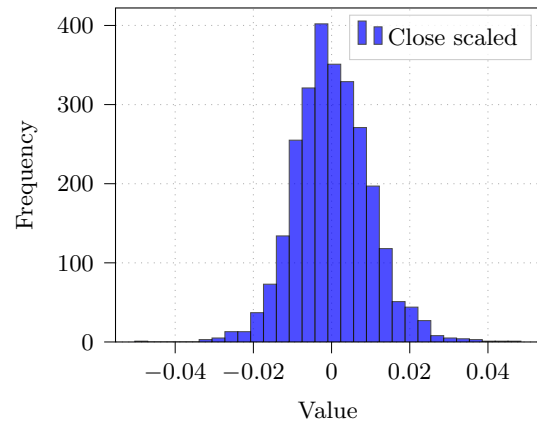
The second and third methods are similar, working on the percentage differences between features for subsequent periods, with the third incorporating logarithms. Fryzlewicz [21] states using log percentage differences for financial data has the advantage of having a mean close to zero, symmetric distribution and clustering volatility. Equation (4.1) is used to transform the features, **Open**, **High**, **Low**, **Close**, **Next day open** and **Daily change** (only taking the log of daily change, since it is already a percentage).

$$\text{Log \% difference} = \ln \left(\frac{q_t}{q_{t-1}} \right) \cdot 100, \quad (4.1)$$

where q_t is the feature value at time t and q_{t-1} is the feature value at time $t - 1$. The new distribution of these scaled features found in Table 4.4 confirms the advantages of having a mean close to zero, with the mean of all the feature sets approximately 0.0003. Figure 4.3 shows the differences in the distribution of values found in the **Close** feature set, where the distribution appears normal in the scaled data in Figure 4.3(b) compared to the random distribution of the unscaled data in Figure 4.3(a).



(a) The closing price feature set unscaled.



(b) The closing price feature set scaled using Log % returns.

FIGURE 4.3: The difference between the closing price feature set distributions before and after scaling from 2012 to 2021 of the USD/ZAR data set.

The SMA features, **9-day SMA** and **21-day SMA**, contain continuous values similar to the OHLC. Instead of using the log percentage differences between consecutive days for this indicator, the indicator is transformed to the percentage difference between the closing price and the corresponding SMA value. For example, if the 9-day SMA has a value of 13.5 and the closing

price is 14, the 9-day SMA will be converted to 0.037 since the closing price is 3.70% above the 9-day SMA. The SMA features are transformed into a scaled SMA, $S\overline{m}_t(r)'$, calculated as

$$S\overline{m}_t(r)' = \frac{c_t - S\overline{m}_t(r)}{S\overline{m}_t(r)} \cdot 100 \quad (4.2)$$

The original features set contains 25 features, of which 4 contain a predefined fixed range. These features are the oscillators, namely **ADX**, **RSI**, **TSI** and **stochastic oscillators**, which can easily be converted into the range 0–1 using min-max normalisation using the lower and upper bounds for each feature, respectively. The scaled feature set, X'_i , for feature i using min-max normalisation is calculated as

$$X'_i = \frac{X_i - X_{i_{\min}}}{X_{i_{\max}} - X_{i_{\min}}}, \quad (4.3)$$

where X_i is the original feature set of feature i with $X_{i_{\min}}$ and $X_{i_{\max}}$ corresponding the minimum and maximum values found in feature set. Table 4.1 contains each oscillator's transformations and updated scales.

Features	Initial range	Transformation	Final range
ADX	[0, 100]	$X_{\min} = 0, X_{\max} = 100$	[0, 1]
RSI	[0, 100]	$X_{\min} = 0, X_{\max} = 100$	[0, 1]
Stochastic oscillators	[0, 100]	$X_{\min} = 0, X_{\max} = 100$	[0, 1]
TSI	[-100, 100]	$X_{\min} = -100, X_{\max} = 100$	[0, 1]

TABLE 4.1: The scaling of features with a fixed range, namely *ADX*, *RSI*, *Stochastic oscillators* and *TSI*.

Bollinger bands and Keltner channels contain three indicators (features): the upper, middle and lower bands. These values are continuous and correlate with the OHLC and the respective band multiplier, thus needing to be scaled appropriately. BB and Keltner channels were converted to a single feature using the closing price, specifically the closing price to the upper and lower bands in terms of a percentage value. The percentage band for both BB and Keltner channels are calculated as,

$$\% \text{ band}_t = \left(\frac{c_t - l_{b_t}}{u_{b_t} - l_{b_t}} \right) \cdot 100. \quad (4.4)$$

where l_{b_t} and u_{b_t} are the upper and lower bands of either the BB or Keltner channel feature. Table 4.2 contains the interpretations of the different ranges for the newly transformed percentage band features.

Band range	Interpretation
% band < 0	Closing price is below the lower band
% band = 0	Closing price is at the lower band
$0 < \% \text{ band} \leq 0.50$	Closing price is between the lower band and middle band
$0.50 < \% \text{ band} < 1$	Closing price is between the upper band and middle band
% band = 1	Closing price is at the upper band
% band > 1	Closing price is above the upper band

TABLE 4.2: The scaling of *BB* and *Keltner channels* into a single feature.

The two remaining features, **MACD** and **ATR**, contain no fixed range. However, all the values are within the range between -1 and 1 and are thus left as is in preprocessing. A summary of the initial distribution for the original feature set is seen in Table 4.3 with the distributions of the feature set after data preprocessing was applied given in Table 4.4. The ranges of all the features appear to be similar after preprocessing, and all contain mean values between -1 and 1.

Features (unscaled)	Mean	Std	Min	25%	50%	75%	Max
Open	12.8987	2.4972	7.4268	10.8717	13.4647	14.6302	19.0727
High	13.0111	2.5288	7.5018	10.9562	13.6265	14.7484	19.3417
Low	12.8134	2.4694	7.3966	10.8212	13.3963	14.5372	18.8638
Close	12.9047	2.4965	7.4266	10.8791	13.4764	14.6313	19.0921
Daily change	0.0005	0.0096	-0.0329	-0.0057	0	0.0065	0.0496
Next days Open	12.8987	2.4972	7.4268	10.8717	13.4647	14.6302	19.0727
Monday	0.2002	0.4003	0	0	0	0	1
Tuesday	0.1995	0.3997	0	0	0	0	1
Wednesday	0.1998	0.4000	0	0	0	0	1
Thursday	0.2006	0.4005	0	0	0	0	1
Friday	0.1998	0.4000	0	0	0	0	1
9-day SMA	12.8921	2.4971	7.5174	10.8505	13.5072	14.6385	18.8929
21-day SMA	12.8729	2.4992	7.5593	10.8254	13.5014	14.6381	18.6731
MACD	0.0217	0.1439	-0.4250	-0.0606	0.0164	0.0979	0.7664
ADX	24.4205	9.6977	8.6552	17.1918	22.5715	28.9195	61.2280
RSI	51.6847	11.0843	21.9347	43.6796	51.5663	59.6409	85.7079
TSI	3.5063	18.2591	-43.2011	-9.0539	2.8159	16.7623	55.7123
Stochastic oscillator	47.6967	30.3925	0	18.8116	47.0063	75.5072	100
ATR	0.1980	0.0724	0.0823	0.1418	0.1894	0.2388	0.4753
Upper BB	13.2929	2.6431	7.6843	11.1204	13.9912	15.1025	19.3594
Middle BB	12.8745	2.4990	7.5527	10.8188	13.5000	14.6392	18.6824
Lower BB	12.4562	2.3687	7.3890	10.5761	13.0202	14.1155	18.1887
Upper Keltner channel	13.0768	2.5595	7.6682	10.9662	13.7340	14.8684	19.0448
Middle Keltner channel	12.8795	2.5008	7.5590	10.8297	13.5081	14.6460	18.6678
Lower Keltner channel	12.6822	2.4429	7.4494	10.6919	13.2912	14.4199	18.2951

TABLE 4.3: A summary of key statistical measures for the unscaled feature set generated from the USD/ZAR currency pair from 2012 to 2021.

4.1.2 Feature scaling

Machine learning algorithms tend to perform poorly when the features have different numerical scales [24]. Neural networks are susceptible to feature ranges and prefer a range of data between 0 and 1 or -1 and 1. For example, even in the preprocessed USD/ZAR feature set, the ranges differ drastically for the closing price, and Keltner channel % is seen in Table 4.4. The closing price range is $[-0.0503, 0.0484]$, while the Keltner channel % is $[-1.6519, 3.2343]$. The difference in range might signal to the models that the higher Keltner channel values are of more importance than the closing price since they contain a larger scale, which might not be the case. The two popular techniques of handling mismatched scales are min-max scaling and standardisation [24]. Figure 4.4 shows the different scales of each scaling technique applied to the Keltner channel feature set.

As given in equation (4.3), min-max scaling or normalisation refers to scaling values of a set to a specified range (typically 0 and 1) using the minimum and maximum values. Therefore, the scaled values of 0 and 1 represent the unscaled minimum and maximum values, respectively. The drawbacks of this technique are that it is sensitive to outliers, and the scale of the testing set is not guaranteed to be in the range of 0–1. For instance, if the maximum RSI value is 100 and the rest of the RSI values fall in the 0–20 range, min-max scaling would squish all other values to 0.0–0.20. The other caveat is that the min-max parameters are determined from the training data and not the test data; therefore, if the ranges are higher or lower in the test data, this would lead to feature values outside of the 0–1 bounds after normalisation is applied, which might lead to the ‘clipping’ phenomenon highlighted in Figure 4.2.

Features (scaled)	Transformation	Mean	Std	Min	25%	50%	75%	Max
Open	Log %	0.0003	0.0097	-0.0413	-0.0060	-0.0002	0.0062	0.0532
High	Log %	0.0003	0.0096	-0.0891	-0.0047	-0.0003	0.0048	0.1079
Low	Log %	0.0003	0.0081	-0.0420	-0.0044	0.0003	0.0046	0.0549
Close	Log %	0.0003	0.0097	-0.0503	-0.0060	-0.0003	0.0063	0.0484
Daily change	Log	0.0005	0.0096	-0.0335	-0.0057	0.0000	0.0064	0.0484
Next day open	Log %	0.0003	0.0097	-0.0413	-0.0060	-0.0002	0.0062	0.0532
Monday	-	0.2002	0.4003	0	0	0	0	1
Tuesday	-	0.1995	0.3997	0	0	0	0	1
Wednesday	-	0.1998	0.4000	0	0	0	0	1
Thursday	-	0.2006	0.4005	0	0	0	0	1
Friday	-	0.1998	0.4000	0	0	0	0	1
9-day SMA	% from close	-0.0012	0.0152	-0.0754	-0.0104	-0.0003	0.0091	0.0530
21-day SMA	% from close	-0.0029	0.0237	-0.1071	-0.0181	-0.0014	0.0141	0.0734
MACD	-	0.0217	0.1439	-0.4250	-0.0606	0.0164	0.0979	0.7664
ADX	Min-max	0.2442	0.0970	0.0866	0.1719	0.2257	0.2892	0.6123
RSI	Min-max	0.5168	0.1108	0.2193	0.4368	0.5157	0.5964	0.8571
TSI	Min-max	0.5175	0.0913	0.2840	0.4547	0.5141	0.5838	0.7786
Stochastic oscillator	Min-max	0.4770	0.3039	0.0000	0.1881	0.4701	0.7551	1.0000
ATR	-	0.1980	0.0724	0.0823	0.1418	0.1894	0.2388	0.4753
BB %	% band	0.5213	0.3316	-0.2550	0.2400	0.5282	0.7974	1.3515
Keltner channel %	% band	0.5737	0.7600	-1.6519	0.0004	0.5347	1.1374	3.2343

TABLE 4.4: A summary of key statistical measures for the scaled features set generated from the USD/ZAR currency pair from 2012 to 2021.

Standardisation works by subtracting and dividing the mean by the standard deviation for each feature in the feature set given in equation (4.5). This method standardises the feature set, meaning all features have a mean of zero and a standard deviation of 1. Unlike min-max normalisation, the features have no fixed range; however, each feature will have the same characteristics. The formula for standardisation is calculated as

$$Z_i = \frac{X_i - \mu_i}{\sigma_i}, \quad (4.5)$$

where,

Z_i is the standardised feature set i ,

X_i is the original feature set i ,

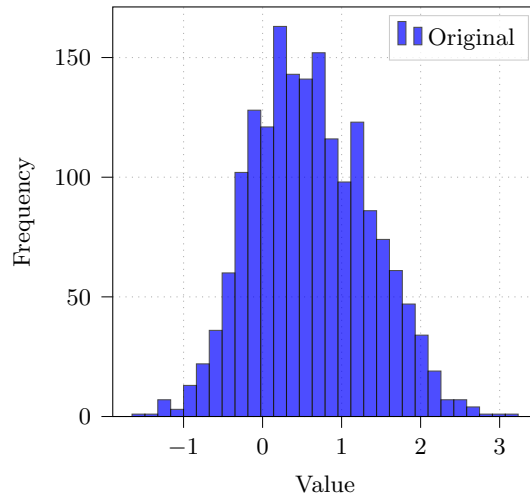
μ_i is the mean of feature set X_i , and

σ_i is the standard deviation of feature set X_i .

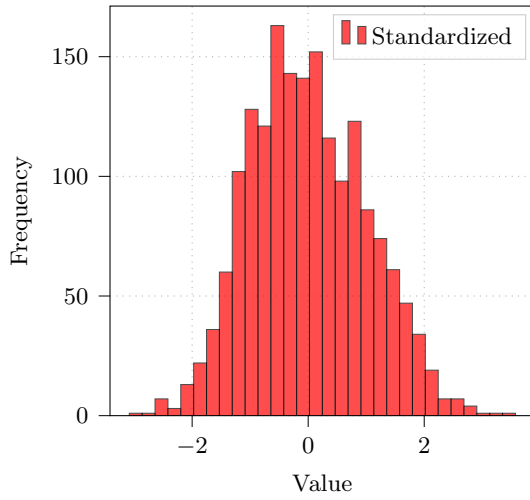
Experimentation using both methods were investigated but not reported on as this is outside the scope of the study, with standardisation having better performance overall than min-max scaling and the chosen method for scaling each train, validation and test split of the data set. The standardisation parameters, μ and σ , are always calculated on the training set and then applied to the corresponding validation and test sets. This approach ensures no information leaking from the validation and test set into a model's training.

4.1.3 Converting to supervised learning

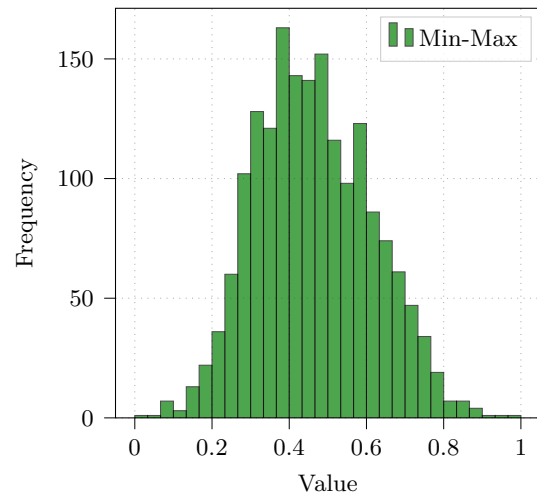
The training process for a supervised machine learning model requires both a feature and its corresponding label. For time series data, absolute values are the target quantity, particularly for regression. This study's target quantity is the next day's closing price. At the same time, the inputs are the current day's closing price and other features in Table 4.4. The label thus becomes



(a) The original unscaled Keltner channel after pre-processing.



(b) The Keltner channel feature scaled using standardisation.



(c) The Keltner channel feature scaled using min-max normalisation.

FIGURE 4.4: The difference between standardisation and min-max scaling of the Keltner channel feature from 2012 to 2021 of the USD/ZAR data set.

the closing price in the feature set shifted one day forward. A window generator function was implemented to split the data into features and labels at a timestep. The window generator function takes the following parameters:

1. Feature width (the number of sequences used as features).
2. Label width (the number of sequences to predict).
3. Look ahead period (the number of sequences ahead skipped before starting the forecast).

Figure 4.5 illustrates splitting up a sequence of times series data into the feature set and label set using the window generator function for a single label. This process is repeated incrementally through the data set to build the entire feature and label set.

The function allowed testing of the abovementioned parameters and their effect on the forecasting performance. The label width and look ahead period were fixed at a value of one, meaning only one label and one period ahead of the most recent observation will be predicted. These were fixed to align with the prerequisite of having the most information available and mitigating the effects of price fluctuations. The drawback is that any trading strategy using the forecast is limited to a single-day forecast, not allowing longer-term trades or insights as to where prices might go over a longer horizon. The feature width, a parameter investigated by Von Stein [70], will also be investigated across the different models, ranging from one day of input data used up to using ten days of input data as features.

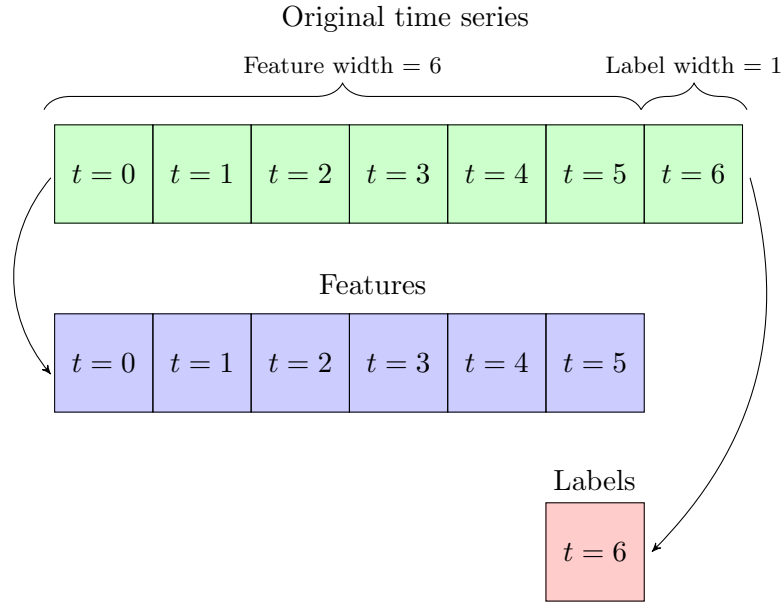


FIGURE 4.5: A schematic representation of converting a sequence of time series data into supervised machine learning data with a feature and label set.

4.2 Walk-forward validation

The idea behind walk-forward validation is to simulate ‘walking’ through the test set day by day with only having access to information up to the current day. This technique allows inputs from the test set to be used as features without leaking data into the training/validation sets. It does not use the values in the validation and test set to update a model’s parameters. The values in the validation and test set get appended to the features set while ‘walking’ through the sets. Therefore, keeping the temporal structure of the time series data intact to ensure no data gets leaked before that day has passed.

Figure 4.6 contains an example of walk-forward validation and how the feature and label sets are incrementally updated through the available data periods. The process in Figure 4.6 is also known as a *rolling window* walk-forward validation approach since the window size is fixed and data is removed after each iteration. The alternative, an *expanding window*, is shown in Figure 4.7. The expanding window differs by not dropping data and continuously ‘growing’ the feature set. This method was experimented with but resulted in the machine learning models overfitting the datasets, as the training and testing ratio split would keep increasing and decreasing, respectively. Therefore, this study utilises a rolling window walk-forward validation approach throughout testing.

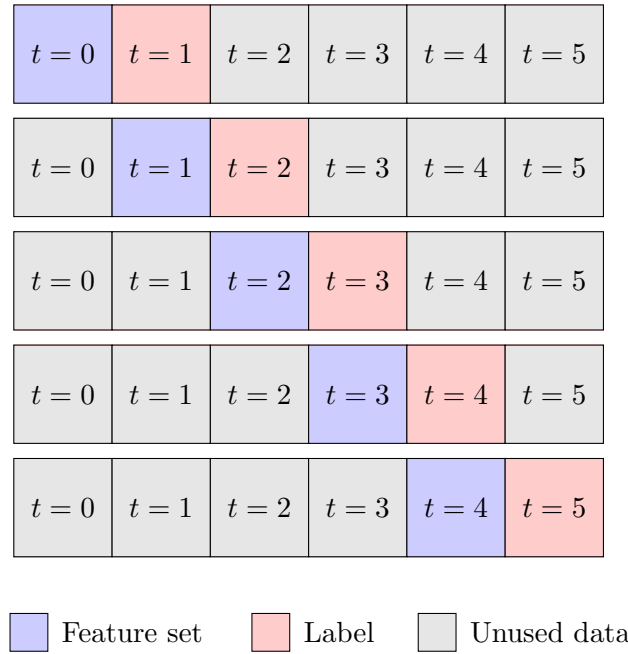


FIGURE 4.6: An illustration of how a **rolling** window walk-forward validation is used to roll through training and testing sets to develop the features (shaded blue) and label sets (shaded red) along with the unused data (shaded grey).

4.3 Models considered

The following models are considered in this study and broken up into two categories: ML models and statistical forecasting models. The ML models consist of a neural network that uses long short-term memory as cells, along with two popular ML models, random forests (RF) and support vector machines (SVM). The statistical forecasting models include the naïve forecast and auto-regressive moving average models (ARIMA), used to evaluate the performance of the machine learning models. Table 4.5 contains a summarised version of the models implemented in this study with a short overview of each model's type and complexity.

Model	Type	Complexity	Additional factors
LSTM	Neural network	High	Sequential data handling
Random forests	Ensemble learning	Medium	Feature importance, tree-based
Support vector machine	Supervised learning	Medium	Kernel selection, dimensionality reduction
Naïve forecast	Time series	Low	Simple, historical average
ARIMA	Time series	Medium	Autoregressive, moving average, seasonality

TABLE 4.5: A short overview of the different models considered to forecast the USD/ZAR foreign exchange pair.

Each model will be self-contained in its own chapter, with an overview of its background, workings and results for hyperparameter calibration and testing. The remaining sections give an overview of the hyperparameter calibration used by the ML models, the testing parameters to address the research question, the different years of market conditions accessed and the relevant result metrics to evaluate each model's performance.

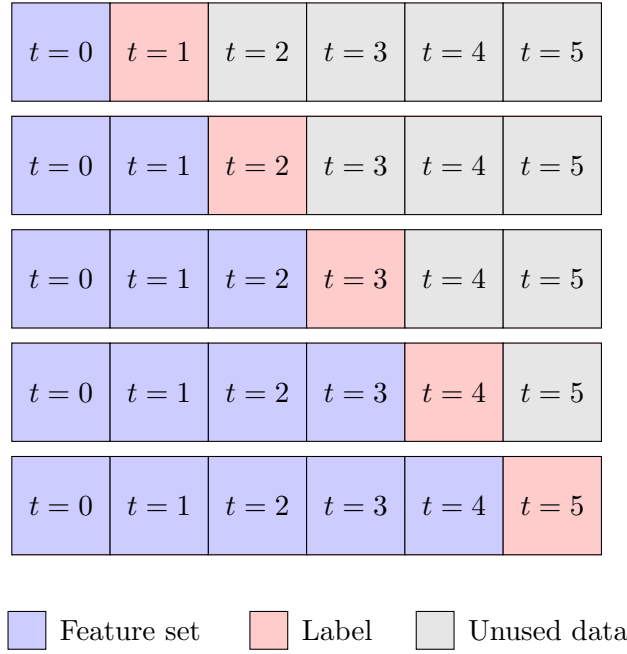


FIGURE 4.7: An illustration of how a **expanding** window walk-forward validation is used to step through and expand the training and testing sets to develop the features (shaded blue) and label sets (shaded red) along with the unused data (shaded grey).

4.4 Hyperparameter calibration

Two types of parameters exist in ML models. The first is model parameters that can be initialised and adjusted through data learning, for example, the LSTM model's neuron weights. The second is parameters that cannot be directly calculated from training on the data set and have to be set prior to training a model. These parameters correspond to the model's architecture and are called the model's *hyperparameters*. Hyperparameters can influence the results of ML models, and exploring a wide range of possible values is an essential process in the ML pipeline called hyperparameter calibration or optimisation.

Hyperparameter calibration finds the set of optimal hyperparameters that maximises the respective model's performance on an underlying dataset [29]. This step comes after data collection and features engineering; therefore, the hyperparameters are optimised for the specific features used. In general, hyperparameter calibration aims to find the optimal hyperparameter configuration, h^* , formulated as

$$h^* = \arg \min_{h \in H} g(h), \quad (4.6)$$

where h is the hyperparameter configuration that can take on any value in the search space H and $g(h)$ is the objective function to be minimised, such as mean squared error or absolute percentage error. Numerous techniques are available for hyperparameter calibration with manual testing, *i.e.*, trial-and-error approach, still prevalent in some studies. Manual tuning is ineffective for complex models with many parameters, where the causal relationship between each variable and the performance can not be easily estimated. Therefore, it is best to use automated hyperparameter calibration techniques, many of which exist. Hutter *et al.* [29] state the following reasons for conducting automated hyperparameter optimisation,

1. Reduces human effort.

2. Improves performance; different model architectures perform better on different data sets.
3. Models and research are reproducible, giving a fair comparison between ML models if each model has been calibrated to achieve its best performance.

There are numerous hyperparameter optimisation techniques to choose from. The most popular and used optimisation techniques to choose from, each with its own specific and recommended use cases as well as drawbacks, are,

- Grid search
- Random search
- Sequential model-based optimisation (SMBO)

Grid search is the most basic of all the calibration techniques and consists of defining a search space and incrementally stepping through every combination of hyperparameters [50]. Grid search is considered a brute force approach, with calibration times ‘exploding’ for every additional hyperparameter that needs calibration. The advantages are that a grid search can produce results similar to more sophisticated methods if fewer parameters exist and time complexity is not an issue. Few parameters and no computational restrictions are seldom the case with ML models, which consist of large hyperparameter search spaces and would not be feasible to explore the entire space in a reasonable time frame.

A simple alternative to grid search is *random search*. It works similarly to grid search by defining a search space of hyperparameters. However, rather than iteratively testing every possible combination of random search, it randomly explores different combinations of hyperparameter values [5]. The drawback of random search is that each iteration is independent of the last. Therefore, improvements are not guaranteed. Random search is often included in other more sophisticated hyperparameter calibration techniques as a starting point to explore larger search spaces, as it is computationally cheap and fast to run.

Sequential model-based optimisation (SMBO) is a sophisticated method for hyperparameter calibration that intelligently explores the search space to find optimal hyperparameters. Unlike grid and random search, it can adaptively choose new hyperparameter values based on the evaluations of past hyperparameter values, using a probabilistic model [64]. SMBOs are considered more efficient than grid and random search because they reduce the search space by only focusing on promising areas, making them helpful in optimising complex ML models with large hyperparameter spaces. Additionally, as SMBOs can learn from each iteration, thus minimising the total number of evaluations needed to find the optimal hyperparameters. This makes SMBOs, such as Bayesian optimisation, advantageous for computationally expensive models.

4.4.1 Bayesian optimisation

Bayesian optimisation is a popular iterative algorithm for hyperparameter optimisation. The main idea is to use a probabilistic surrogate model, typically a Gaussian process (GP), that approximates the actual function, which can be challenging to estimate. The surrogate model is used to guide the search for the optimal solution by trading off between searching unexplored areas (exploration) and zoning in on areas with good past performance (exploitation) [29]. Another critical component is the acquisition function, or selection function, which determines, given the set of already explored parameters, what the following parameter values should be to maximise this function, thus exploring these parameters in the next iteration.

Surrogate model

GP was chosen as the surrogate model, a probabilistic model representing the uncertainty in the objective function. Mathematically, a GP defines a distribution over functions. Given a set of observed data points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, where x_i is the input variable and y_i is the function evaluation. The GP defines the conditional distribution of the function values at any unobserved input x .

Acquisition function

Using the predictive distribution of the surrogate model, the acquisition function can quantify the utility of different parameter configurations and choose the configuration that maximises the function. The chosen acquisition function is the expected improvement (EI) defined as

$$\text{EI}(x) = E [\max(0, y - f(x_{\text{best}}))], \quad (4.7)$$

where y is the value predicted by the surrogate model at x , and $f(x_{\text{best}})$ is the best observed function value so far. The EI favours points where the surrogate predicts a higher value than the current best and accounts for uncertainty.

Implementation

This study uses Bayesian optimisation to find the best hyperparameters for the LSTM, RF and SVM models using the `bayesian-optimization` [50] library in Python with a pre-implementation available. Algorithm 1 contains the basic pseudo code for Bayesian optimisation. The parameters available to tune in the `bayesian-optimisation` function deal with the tradeoff of exploration and exploitation and have been set to

1. `number of random searches` = 50, and
2. `number of optimisation searches` = 100.

These parameter values mean the first 50 iterations of the Bayesian optimisation will be random to build up the surrogate model, equivalent to a random search. These initial 50 iterations do not take past hyperparameter sets into account. The subsequent 100 iterations will then perform Bayesian optimisation. The chosen values exhibit a reasonable tradeoff between exploration of the search space and exploitation of the best regions in the search space, along with the constraint of computer resources available since the time taken for the calibration ranged from 3 hours to 8 days.

4.4.2 Cross-validation

The main objective for forecasting regression-based models is how well they can generalise to unseen data [4]. It is not uncommon that a model, given a specific hyperparameter configuration, performs better on a certain period in the data set but poorly on all other periods. The aim is to search for a hyperparameter configuration that can generalise well to a wide set of data. *Cross-validation* is a technique that can aid in solving this problem.

Cross-validation is a statistical method used to compare and evaluate models by dividing data into training and validation segments [60]. The most popular cross-validation technique is k -fold cross-validation, which partitions the data set into k different training and validation splits.

Algorithm 1: Bayesian Optimisation**Input:** A surrogate model, acquisition function, supervised data D , total number of iterations N **Output:** Best hyperparameter configuration h^*

```

1: Initialise the surrogate model with observed data:  $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ 
2: for  $i = 1$  to  $N$  do
3:   Select the next sampling point by optimising the acquisition function:
4:    $x_{\text{next}} = \arg \max_x \alpha(x|D)$ 
5:   Evaluate the true objective function at the selected point:
6:    $y_{\text{next}} = f(x_{\text{next}}) + \epsilon$ , where  $\epsilon$  is observation noise
7:   Update the surrogate model with the new data point:
8:   Add  $(x_{\text{next}}, y_{\text{next}})$  to  $D$ 
9:   Update the surrogate model based on  $D$ 
10: end for

```

Cross-validation takes results for each fold and computes the average across all. Hyperparameter sets achieving the best average score over all the splits are preferred to hyperparameter configurations that might outperform in certain configurations but are worse on average.

With time series data, there is a problem with using existing techniques such as k -fold cross-validation. The problem is that the temporal structure of a time series should consistently be kept through partitioning the data set. Ignoring this premise can lead to test sets that predate the training set, meaning data from the future is being used to predict the past, thus potentially leaking information. Many studies in literature do not address this concern and apply pre-implemented cross-validation in programming languages such as Python [56]. For this reason, the folds for k -folds cross-validation are defined manually.

Hyperparameter Bayesian optimisation combined with manual k -fold cross-validation was used to find the best hyperparameter configuration for the LSTM, RF and SVM models. The splits are defined manually to avoid violating the temporal structure of the data set and are displayed in Table 4.6. The manual k -fold cross-validation incorporates the effect of the two testing parameters `sequence length` and `retrain interval`. Including these parameters simulates the test conditions and ensures no biases towards any singular value of either parameter towards the hyperparameter configuration of the ML model architectures.

Due to limited data predating 2012, the initial splits for the 12-monthly retrain interval are inconsistent with an 80% train and 20% validation split compared to the other splits in the table. This deviation was assumed to have a negligible impact on the parameter calibration results as the initial splits still contain sufficient data points to train the models.

Each model calibrated has a total number of 150 hyperparameter configurations tested through Bayesian optimisation, with each configuration tested 160 times with cross-validation. Four different retraining intervals with four different folds for each (total of 16 folds) and ten different sequence lengths were evaluated. A total of 24 000 models are trained for each of the three ML models, and results are aggregated into a single metric, mean squared error, to evaluate and rank the score of each hyperparameter configuration.

Hyperparameter calibration is only performed once and not repeated. Ideally, this process should be repeated across the different training, validation and test sets; however, this is not feasible due to computational resources and time constraints. Additionally, this study focuses not on the optimal hyperparameter configuration of the underlying ML models but on the effects of the retraining interval and sequence length used, which are incorporated into the parameter calibration. Thus, assuming all models will have adequate architectures throughout testing these parameters. The constant architecture for each model also ensures that the effects of the testing

Retrain interval	Train range	Validation range	Train split	Validation split
12-Monthly	2012-03-01–2013-12-31	2014-01-01–2014-12-31	65%	35%
	2012-03-01–2014-12-31	2015-01-01–2015-12-31	74%	26%
	2012-03-01–2015-12-31	2016-01-01–2016-12-31	79%	21%
	2013-01-01–2016-12-31	2017-01-01–2017-12-31	80%	20%
6-Monthly	2014-01-01–2015-12-31	2016-01-01–2016-06-30	80%	20%
	2014-07-01–2016-06-30	2016-07-01–2016-12-31	80%	20%
	2015-01-01–2016-12-31	2017-01-01–2017-06-30	80%	20%
	2015-07-01–2017-06-30	2017-07-01–2017-12-31	80%	20%
3-Monthly	2016-01-01–2016-12-31	2017-01-01–2017-03-31	80%	20%
	2016-04-01–2017-03-31	2017-04-01–2017-06-30	80%	20%
	2016-07-01–2017-06-30	2017-07-01–2017-09-30	80%	20%
	2016-10-01–2017-09-30	2017-10-01–2017-12-31	80%	20%
1-Monthly	2017-05-01–2017-08-31	2017-09-01–2017-09-30	80%	20%
	2017-06-01–2017-09-30	2017-10-01–2017-10-31	80%	20%
	2017-07-01–2017-10-31	2017-11-01–2017-11-30	80%	20%
	2017-08-01–2017-11-30	2017-12-01–2017-12-31	80%	20%

TABLE 4.6: A table of the different training and validation sets used for cross-validation in parameter calibration to incorporate the different retrain intervals.

parameters are not influenced by different hyperparameter configurations during the different testing periods, making the results between testing periods unbiased.

4.5 Testing parameters

The following two parameters, **retrain interval** and **sequence length**, are investigated to address the research question. The two parameters will be used in a 2-D grid search over the ranges of each parameter given in Table 4.7. The two parameters are applied across four distinct test sets, each simulating different market conditions to assess the impact of these parameters.

Parameter	Search space	Timeframe
Retrain interval	[12, 6, 3, 1]	Months
Sequence length	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]	Days

TABLE 4.7: The search spaces for retraining interval and sequence length used for the 2018–2021 testing period.

4.5.1 Retrain interval

A model’s **retraining interval** refers to how frequently the model parameters are updated or retrained throughout testing. For example, if there are four years of test data with the retrain interval set to twelve months. A model will initially be trained on the training data and used, without any parameter updates, to make predictions for the first year of test data. After the first year (twelve months) in the test set, the first year of test data is appended to the training set and the model is retrained with the new data. This process is repeated every 12 months, and the model parameters are updated four times with a twelve-month retrain interval.

Exploring the different retrain intervals determines if more frequent updates to the model parameters, such as every six months, three months or every month, will improve the model’s prediction performance. The main question investigated is whether updating model parameters

more frequently, which will provide the model with updated information on market conditions, will improve the prediction accuracy or make the model more sensitive to short-term market fluctuations. Thus trying to balance long-term stability and short-term information across different market conditions. The ranges were chosen based on the work of Pongsena *et al.* [54], with the addition of the bi-yearly interval.

4.5.2 Sequence length

Sequence length refers to the number of previous days used to predict the next day's value within a model, which also determines the number of features. For example, a sequence length of one uses the 22 preprocessed and scaled features in Table 4.4, and a sequence length of two will use double that amount, 22 from time step t plus 22 from time step $t - 1$, thus 44 features in total. Generally, the number of features is 22 times the **sequence length**.

Von Stein [70] also observed this parameter's significance, highlighting that different combinations of features yielded varying performance results depending on the number of preceding days used as inputs. This study expands on the work of Von Stein [70] by extending the range from 1–5 to 1–10 for a more comprehensive analysis. The upper limit of restricting the sequence length to 10 was based on the finding of Pongsena *et al.* [54]. The authors found decreased accuracy when exceeding a sequence length of 12 days, with 9, 10, 11 and 12 days containing the highest accuracy.

4.6 Test sets

The results are split into four sets, each corresponding to the year of the tested USD/ZAR data set. This is useful for the 6-month, 3-month and 1-month retrain interval since these parameters will further split each testing year into 2, 4, and 12 sets, respectively. Table 4.8 contains an example of the 6-month retrain interval splits for the four years of testing data. In this example, the 2018a and 2018b test set results are appended to form a single year, 2018. Regardless of the retraining interval, all results are annualised for the respective year they fall.

Datasets	Training	Validation	Test	Split
Test set 2018a	2015-03 – 2017-06	2017-07 – 2017-12	2018-01 – 2018-06	70/15/15
Test set 2018b	2015-09 – 2017-12	2018-01 – 2018-06	2018-07 – 2018-12	70/15/15
Test set 2019a	2016-03 – 2018-06	2018-07 – 2018-12	2019-01 – 2019-06	70/15/15
Test set 2019b	2016-09 – 2018-12	2019-01 – 2019-06	2019-07 – 2019-12	70/15/15
Test set 2020a	2017-03 – 2019-06	2019-07 – 2019-12	2020-01 – 2020-06	70/15/15
Test set 2020b	2017-09 – 2019-12	2020-01 – 2020-06	2020-07 – 2020-12	70/15/15
Test set 2021a	2018-03 – 2020-06	2020-07 – 2020-12	2021-01 – 2021-06	70/15/15
Test set 2021b	2018-09 – 2020-12	2021-01 – 2021-06	2021-07 – 2021-12	70/15/15

TABLE 4.8: An example of the training, validation, and test sets for each year of walk-forward validation used to evaluate the models for a 6-month retrain interval using a 70/15/15 split.

A summary of each test set for the different years is given below, with respect to overall price movements and market conditions deciphered through the use of the technical indicators chosen as features. The description aids in understanding model results compared to market conditions and finding a causal relationship between the price movements for a specific year and the tested parameters.

4.6.1 Test set 2018

The first year of testing data is the USD/ZAR data set from 2018-01-02 to 2018-12-31, with an opening price of ZAR 12.45 per USD and a closing price of ZAR 14.37 per USD. The market exhibits an overall upward trend with relatively low volatility based on the width of the BB bands in Figure 4.8. Two periods stand out with increasing levels of volatility, notably the periods from 2018-06-09 to 2018-07-12 and 2018-08-09 to 2018-09-12. These two periods contained larger increases in price compared to any other period in 2018, and both signalled overbought conditions according to the RSI levels in Figure 4.8. The average daily change for 2018 was 0.817%, with the lowest change of 0.0002% and the highest change of 3.41%. The minimum and maximum closing price values observed during this period were ZAR 11.53 per USD and ZAR 15.41 per USD, respectively.

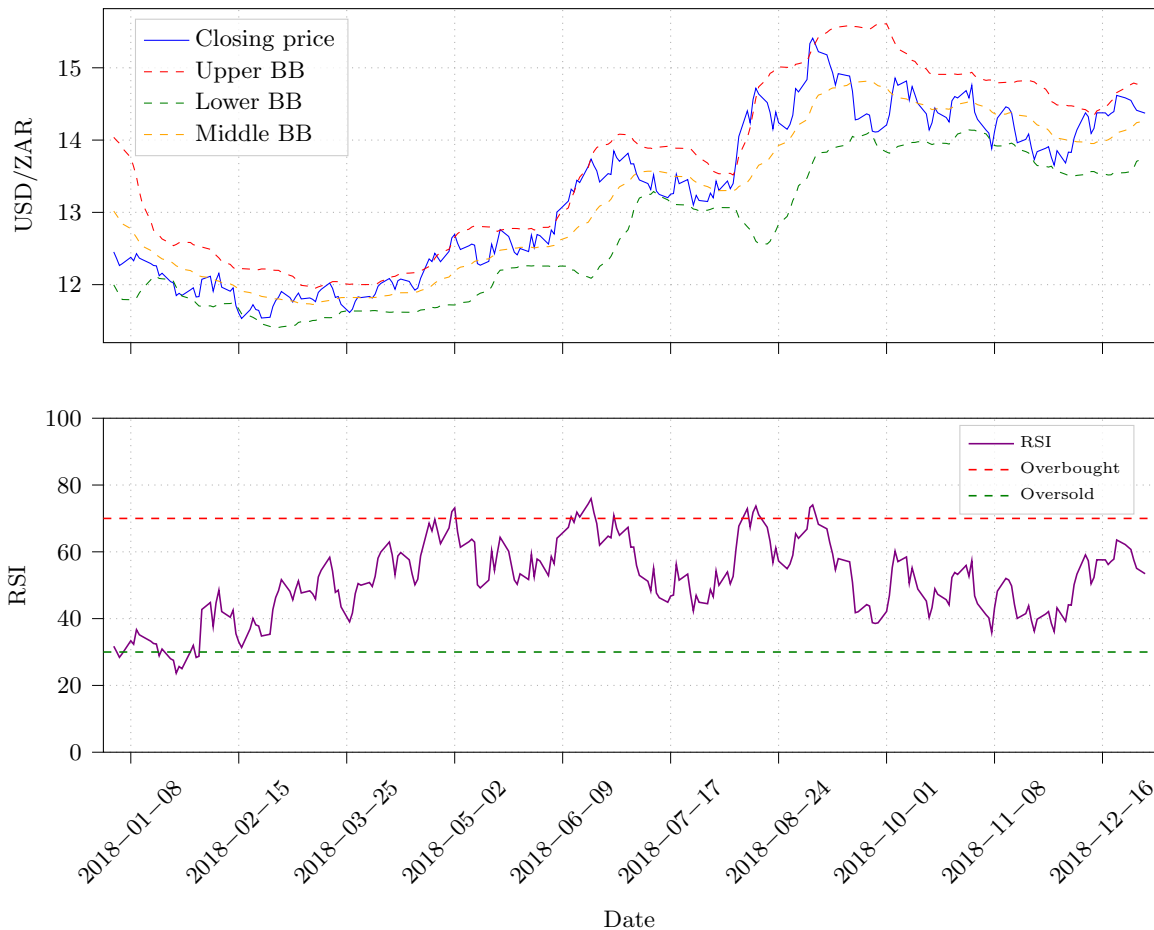


FIGURE 4.8: An illustration of the **2018 test set**, which is the closing price of the USD/ZAR from 2018-01-02 to 2018-12-30, overlaid with the 20-day Bollinger band using 2 standard deviations and a 14-day RSI technical indicators to display the market conditions.

4.6.2 Test set 2019

The test set for 2019 of the USD/ZAR data set contains the period from 2019-01-02 to 2019-12-31. This period saw the most frequent price reversals with no lasting trend based on the 12-day and 26-day SMA's in Figure 4.9. The set closed off at a lower closing price of ZAR 14.00 per USD, after opening at ZAR 14.45 per USD. The average daily change from the opening to

closing price was 0.62%, with the lowest change of 0.0007% and the highest change of 2.445%. The TSI indicator signalled strong positive momentum beginning on 2019-08-07 with a TSI value of 25.34 and reaching a maximum TSI value of 39.86 on 2019-08-19 before beginning to decrease, signalling a divergence in price. During this period, the closing price reached a maximum of ZAR 15.45 per USD. This strong momentum was short-lived, and the price gradually decreased until the period ended with a strong negative momentum, ending the testing set.

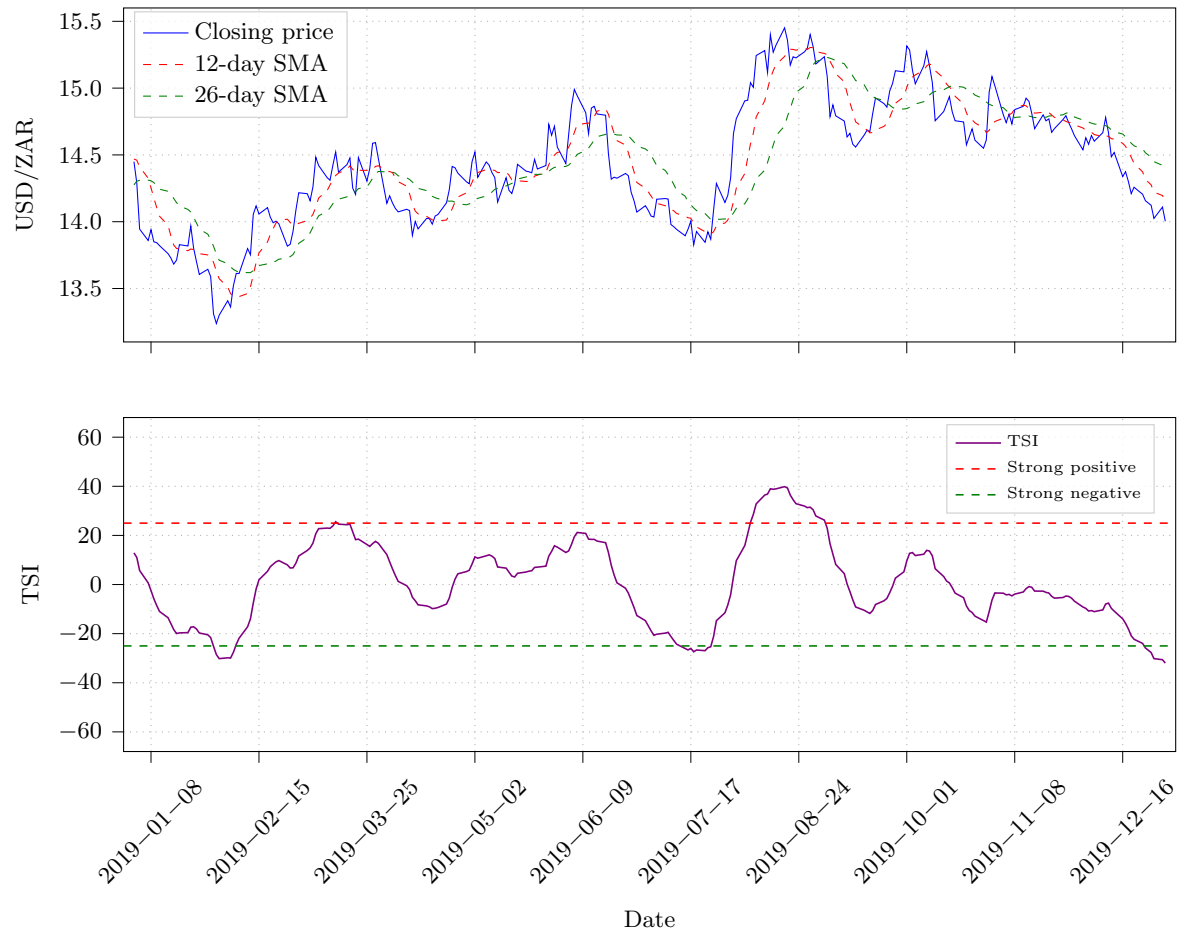


FIGURE 4.9: An illustration of the **2019 test set**, which is the closing price of the USD/ZAR from 2019-01-02 to 2019-12-31, overlaid with the 12-day and 26-day SMA along with the 14-day TSI technical indicators to display the market conditions.

4.6.3 Test set 2020

The 2020 test set contained the most extensive range of prices with a movement of ZAR 5.02 per USD from the lowest closing price of ZAR 14.07 per USD on the opening day, 2020-01-02, to the highest closing price of ZAR 19.09 per USD on 2020-04-23. This represents a price movement of 35.69% in just over four months, the biggest return in the shortest time in the entire dataset from 2012 to 2021. The MACD value during the same period confirms the strong trend observed. The MACD value starts to decrease from its peak of 0.7663 on 2020-04-06, signalling a divergence and a possible reversal in price, seen in Figure 4.10. This signal was correct as closing prices began to decrease.

This movement coincides with the same time as the global COVID-19 pandemic, which explains the high volatility and size of the movement. The United States dollar is considered a ‘safe

haven' and becomes stronger when there are high levels of uncertainty in the market and world conditions [39]. Standard & Poor's, a global credit ratings agency, considers South Africa a risky investment and at times of economic turmoil, for instance, a global pandemic, investors tend to pull out of weaker economies and invest in safer, less risky investments, such as the United States [74].

This short fluctuation in high prices eventually rebounded after 2020-04-25. An overshoot in one direction is likely to cause an overshoot in the opposite direction to balance out the markets, referred to as mean reversion and is a popular trading strategy among traders [63].

The prices came down from the peak gradually with a downward trend of the closing price below the 12-day and 26-day EMA for the remainder of the 2020 test set ending on 2020-12-31 at a closing price of ZAR 14.60 per USD. This set also contained the highest average daily change of 0.837%. It is expected that the models will struggle to predict the 2020 year as exogenous factors affecting the markets that are not predictable and referred to as a 'black swan' event.

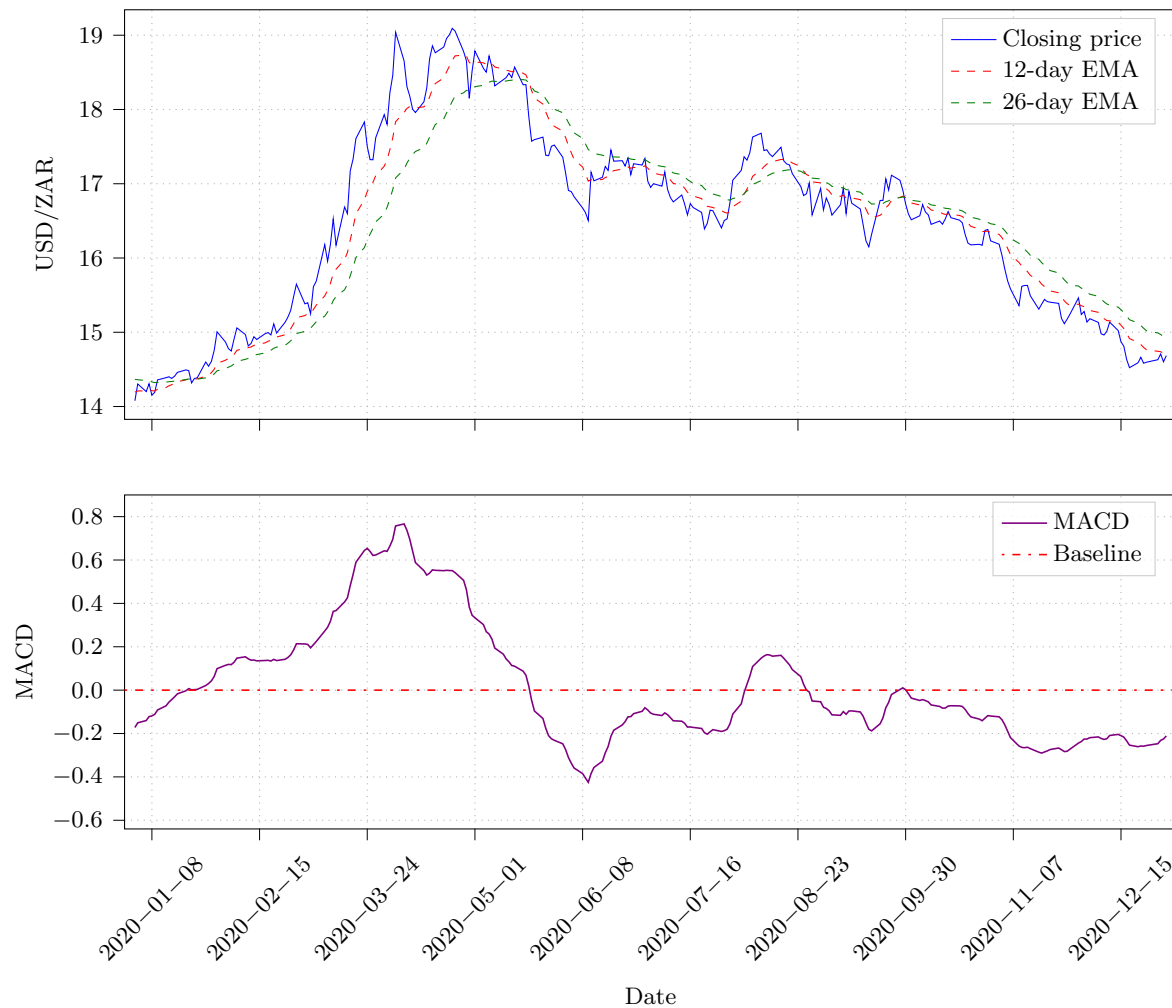


FIGURE 4.10: An illustration of the **2020 test set**, which is the closing price of the USD/ZAR from 2020-01-02 to 2020-12-31, overlayed with the 12-day and 26-day EMA and corresponding MACD technical indicators to display the market conditions.

4.6.4 Test set 2021

The final year of testing data is the 2021 USD/ZAR dataset from 2021-01-04 to 2021-12-31. This set follows from the 2020 set that has been appended to form part of the training/validation sets used to train the models for 2021. The newly formed training/validation set may hinder the model's ability to determine patterns in the market due to previous unstable and external factors that affected the 2020 set. The 2021 test set opens at a closing price of ZAR 14.71 per USD and a close price of ZAR 15.51 per USD, with an average daily change of 0.59%, the lowest of all the test sets.

The 2021 set exhibits frequent price fluctuations similar to the 2019 test set, with the Keltner channels signalling 76 days of overbought conditions and 43 days of oversold conditions seen in Figure 4.11, indicating high volatility during 2021. The ADX values show a strong negative trend beginning on 2021-04-21 since the $-DM$ is higher than the $+DM$. The ADX switches to a strong positive trend on 2021-06-16 when the $+DM$ crosses above the $-DM$.

The 2021 test is separated into two parts, with the year's first half exhibiting a downward trend and the second half an upward trend. The six-monthly retrain frequency is expected to perform the worst on this set since the model will be trained on a downward trend for the second half and then tested on an upward-trending market.

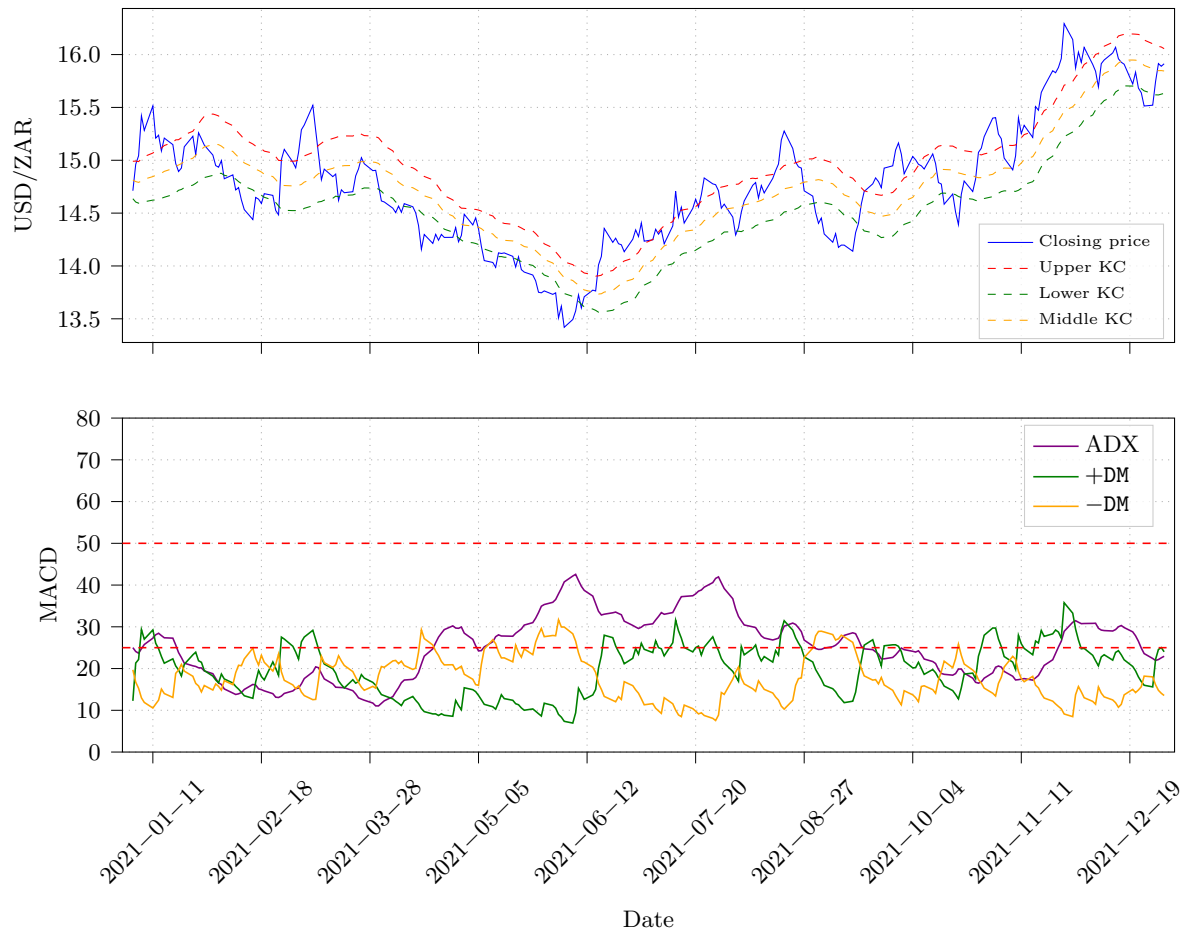


FIGURE 4.11: An illustration of the **2021 test set**, which is the closing price of the USD/ZAR from 2021-01-04 to 2021-12-31, overlaid with the 20-day Keltner channel and a 14-day ADX technical indicators to display the market conditions.

4.7 Result metrics

Result metrics are necessary for evaluating and comparing the effectiveness of a model's performance. In the literature survey conducted by Hu *et al.* [28], the authors found that the most common metrics used to assess performance for financial time series predictions are the root mean squared error, mean squared error, mean absolute percentage error, accuracy, profit and Sharpe ratio.

This study incorporated the following result accuracy metrics: mean square error (MSE), mean absolute percentage error (MAPE) and a newly defined metric threshold accuracy. The trading performance was also analysed through the following trading-related metrics: cumulative profit, maximum drawdown, average profit per trade, profitable trades and the number of trades. These metrics evaluate and investigate each model's accuracy and trading performance for the different combinations of **retrain interval** and **sequence length**. However, since all models are optimised on MSE, more importance is given to this metric.

The preprocessing and scaling of the label (predicted value) make the results of the implemented models challenging to interpret. Therefore, a series of transformations are applied to the label to convert it to its original form. First, the scaled and standardised predicted closing price, denoted as $\hat{c}_{t+1}''$, is destandardised using equation (4.8).

$$\hat{c}_{t+1}' = (\hat{c}_{t+1}'' \cdot \sigma_i) + \mu_i. \quad (4.8)$$

Next, the transformed predicted value is converted from the predicted log percentage closing price, $\hat{c}_{t+1}'$, to the predicted closing price by reversing the log transformation and adding the percentage predicted to the current day's closing price, c_t , via equation (4.9).

$$\hat{c}_{t+1} = c_t(\exp(\hat{c}_{t+1}') - 1) + c_t. \quad (4.9)$$

These transformations allow for more interpretable results and include predictions based on daily closing prices. This approach aligns with the results found in the literature and is consistent with the methodology used by Von Stein [70].

4.7.1 Statistical metrics

The following metrics are used to calculate the accuracy of both the fitted and forecasted data, focusing on the MSE for assessing performance between different models. These measures include the **mean square error** (MSE) and **mean absolute percentage error** (MAPE). Wilson and Keating [72] provides the formulas for each metric and are given below, with the following variables defined. Let,

- $\hat{c}_t$ be the predicted closing price at period t ,
- c_t be the actual closing price at period t , and
- n be the total number of time steps in the tested set.

$$\text{MSE} = \frac{1}{n} \sum_{t=1}^n (\hat{c}_t - c_t)^2. \quad (4.10)$$

$$\text{MAPE} = \frac{1}{n} \sum_{t=1}^n \left| \frac{\hat{c}_t - c_t}{c_t} \right| \cdot 100. \quad (4.11)$$

4.7.2 Threshold accuracy

Statistical metrics for regression, such as MSE and MAPE, can only evaluate how well a model performs compared to another model; these metrics alone are difficult to interpret since the scale of the data influences them. Classification metrics such as accuracy, which calculates the total number of correct predictions over the total number of predictions, make performance more interpretable and thus answer the question, on average, how accurate are the predictions? However, for regression models to predict any value with a 100% accuracy is an unrealistic standard. The threshold accuracy metric, adapted from the work by Von Stein [70] on the same type of data, calculates the accuracy of a predicted value within an absolute margin of the actual value. This simulates model performance comparable to a classification model. The same threshold values are carried over from Von Stein [70], which are R0.025, R0.05, R0.075 and R0.1. The mathematical formulation for the threshold accuracy metric for a specific r is defined as,

$$\text{Threshold accuracy}_r = \frac{1}{n} \sum_{t=1}^n T_t, \quad (4.12)$$

where,

$$T_t = \begin{cases} 1, & \text{if } |\hat{c}_t - c_t| \leq r, \\ 0, & \text{otherwise,} \end{cases} \quad (4.13)$$

with r being any of the following values [0.025, 0.05, 0.075, 0.1]. Table 4.9 contains an example of the actual and predicted values for six days of test data and the different thresholds that the predictions fall into, with a total threshold accuracy percentage for each threshold indicated.

Date	c_t	$\hat{c}_t$	$ \hat{c}_t - c_t $	Threshold accuracy			
				R0.025	R0.05	R0.075	R0.1
2019-02-08	13.61	13.62	0.01	1	1	1	1
2019-02-11	13.80	13.61	0.19	0	0	0	0
2019-02-12	13.75	13.81	0.06	0	0	1	1
2019-02-13	14.06	13.77	0.29	0	0	0	0
2019-02-14	14.12	14.08	0.04	0	1	1	1
2019-02-15	14.06	14.12	0.07	0	0	1	1
Totals	-	-	-	16.67%	33.33%	66.67%	66.67%

TABLE 4.9: An example of the threshold accuracy metric calculated for R0.025, R0.05, R0.075, R0.1.

4.7.3 Trading strategy/profit

Statistical metrics for forecasting are suitable for analysing models, testing for overfitting and underfitting, and allowing easy comparison between models. However, when dealing with financial markets *i.e.* trading, the main goal is profit. Therefore, the study also employs the measure: “How much money will the underlying model make if used to execute trades?”. The models’ predictions are incorporated into a trading strategy to simulate the profit (loss) each model would have made in a real-life scenario.

Hundreds of trading strategies are available, each with rules and conditions for executing trades. Common strategies include buying or selling if the price is predicted to increase or decrease respectively, buying or selling on specific triggers or more advanced strategies that incorporate risk management, with particular levels of take profit or stop-losses. Sizing of trades is another aspect to consider; usually, more volatility will encourage larger trades, while small volatility leads to smaller trades.

Optimising the trading strategy falls outside of the scope, but real-world trading scenarios are simulated as closely as possible with the addition of transaction fees and spread. Adding these costs addresses some of the shortcomings in the literature, where high profits are reported without the inclusion of any trading costs.

All these additional costs in executing a trade will start to chip away at profits, especially when price movements are low. For example, suppose a model can accurately predict the direction of the price movement, but the size of the movement is less than the spread. In that case, the overall profit from the transaction will result in a loss since the costs will outweigh the profit received, thus making it crucial only to execute trades where movements will be profitable after considering the associated costs. The trade size for every transaction placed will be 100 units of the quote currency; thus ZAR 100; this value allows the profit to be interpreted as a percentage throughout the different time horizons.

The trading strategy has three classes of actions to choose from at the start of the next trading day's open price, o_{t+1} . The decision of action will be based on the predicted closing price, $\hat{c}_{t+1}$, relative to the open and whether the price movement is bigger than the spread to negate the trading costs. The three actions are,

1. Go long: if the price is predicted to increase more than the spread
2. Do nothing: if the price movement does not exceed the spread
3. Go short: if the price is predicted to decrease more than the absolute spread

The spread calculation derives from the average percentage difference of the ask (lowest price seller is willing to sell) and bid (highest price a buyer will pay) prices for one day's hourly prices of the USD/ZAR currency pair obtained from FxPro [22]. The average spread found equates to 0.1%. Table 4.10 contains an example of this calculation. FxPro [22] have relatively small transaction costs at 0.0035%; however, these additional costs can be incorporated into the spread. Regardless of the volume at the time of execution, all transactions are assumed to be executed at the opening and closing prices found in the data. Therefore, the 0.1% accounts for total transaction costs per trade and is a more conservative estimation of the profit calculated.

Date	Time	Bid	Ask	Spread	Spread (%)
2022-02-16	08:00	15.792	15.986	0.014	0.0875
2022-02-17	09:00	15.981	15.994	0.013	0.0812
2022-02-18	10:00	16.015	16.027	0.012	0.0748
2022-02-19	11:00	16.054	16.072	0.018	0.1120
2022-02-20	12:00	16.025	16.044	0.019	0.1184
Average				0.0152	0.0948

TABLE 4.10: An example of the trading data used to calculate the spread of the USD/ZAR currency pair.

The total/cumulative profit (loss) is an accumulation of the daily profit, p_i , based on the trading strategy for time step i . The profit metric can be calculated as

$$\text{Cumulative profit} = \sum_{t=1}^n p_i, \quad (4.14)$$

where,

$$p_i = \begin{cases} (100 \times \frac{c_t - o_t}{o_t}) - s_t, & \text{if } \hat{c}_t - o_t > \hat{s}_t, \\ 0, & \text{if } |\hat{c}_t - o_t| \leq \hat{s}_t, \\ (100 \times \frac{o_t - c_t}{o_t}) - s_t, & \text{if } \hat{c}_t - o_t < -\hat{s}_t, \end{cases} \quad (4.15)$$

where s_t is the actual spread at time step t and $\hat{s}_t$ is the estimated spread. The former is calculated as $s_t = \frac{0.05(o_t + c_t)}{100} = 0.0005o_t + 0.0005c_t$. The latter was estimated solely from the opening price, as using the actual closing price to calculate the spread would leak information when making the trade. The estimation of s_t is given as $\hat{s}_t = \frac{0.1(o_t)}{100} = 0.001o_t$. Experimentation with different spread values found that $s_t \approx \hat{s}_t$ and using either spread value had minimal effect on the total profit.

4.7.4 Maximum drawdown

The profit metric can only determine how much money will be made from the beginning to the end of a specified period but does not contain any information regarding how that value came to be. The maximum drawdown metric gives insight into the risk and volatility of a model throughout a specified trading period. The maximum drawdown is defined as ‘the maximum observed loss from a peak to a trough of a portfolio before a new peak is attained’, meaning the largest consecutive loss incurred throughout a specified period. A lower maximum drawdown is desirable, indicating less risk for a trading strategy. The formula is given as

$$\text{Maximum drawdown} = -\max \left(\frac{\text{Peak} - \text{Trough}}{\text{Peak}} \right). \quad (4.16)$$

This metric makes it easier to compare two models with similar profits. For example, in Figure 4.12, both model A and model B end with the same cumulative profit of 30%; however, the cumulative profits drastically differ before the end of the period. Model A is more volatile with a maximum drawdown of -18.62% , while model B shows smaller consistent profits containing a maximum drawdown of -1.92% . Model B is thus preferred since it is a less risky strategy than model A.

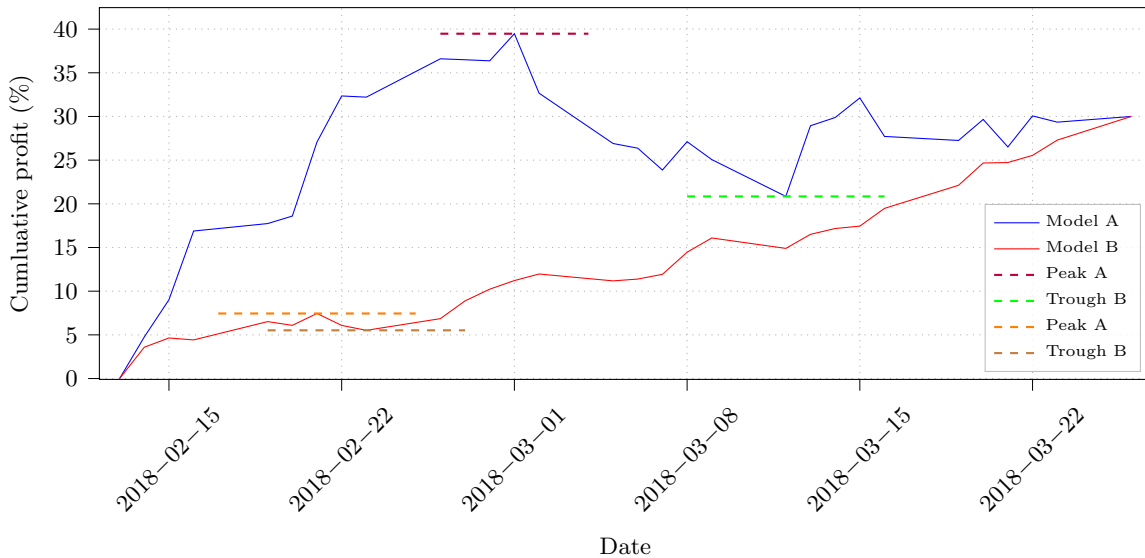


FIGURE 4.12: An example showing the volatility of a cumulative profit between two models and the different peaks and troughs used to calculate the maximum drawdown from 2018-02-13 to 2018-03-26.

4.7.5 Number of trades

The number of trades measures the total number of trades a model executes and indicates how often the predicted closing price movement is greater than the spread. The number of trades metric tallies the total number of long and short trades triggered by the trading strategy.

4.7.6 Average profit per trade

The average profit gives more information regarding the risk of a model's trading performance and is calculated by dividing the cumulative profit by the number of trades. Since this metric accounts for the number of trades a model executes, it can distinguish between models with similar profits.

For example, model A performs 20 trades and accumulates a total profit of 20%. Model B only executes five trades and ends with a total profit of 20%. The trading performance concerning total profit is identical, and choosing which model is better based on this metric alone is challenging. The average profit metric can be used to distinguish between the preferred model, model B, with a higher average profit per trade at 4% compared to model A of only 1%.

4.7.7 Profitable trades

Since the model does not execute trades daily and only when the trading strategy is triggered, the binary classification metric cannot evaluate how often the trades result in a profit. The profitable trades metric divides the number of times a model made a profitable trade, $p_i > 0$, over the total number of trades a model makes. This metric gives insight into the long-term performance of a model's trading strategy, and a profitable trade percentage over 50% is usually required to guarantee profits over a large number of trades.

Table 4.11 contains the number of days within each testing year with profitable trading days *i.e.*, where the price movement was greater than the spread. The average profitable trade for all years is 91.07%. Thus, incorporating the spread into the trading strategy aims to filter non-profitable trading days where price movements are smaller than the spread.

	2018	2019	2020	2021	All
Number of trading days	260	260	261	261	1042
Number of profitable trades	237	232	236	244	949
Profitable trades (%)	91.15	89.23	90.42	93.49	91.07

TABLE 4.11: *The number of profitable trading days within the testing period of 2018–2021 in the USD/ZAR data set.*

4.8 Chapter summary

The aim of this chapter was to explain the various feature engineering and scaling techniques used to transform the data set into a useable format for ML models. Additionally, the different components of the modelling pipeline such as walk-forward validation, selection of models, the various hyperparameter techniques used, analysis of the test set and the result metrics to evaluate model performance are all elaborated on. The subsequent chapters therefore build upon and utilise the sections contained within this chapter.

 CHAPTER 5

Statistical forecasting

Contents

5.1	Naïve model	63
5.2	Naïve results	63
5.3	ARIMA model	65
5.4	ARIMA Results	70
5.5	Chapter summary	72

Statistical forecasting models serve as valuable baselines to compare to machine learning models. These traditional forecasting methods formed the basis of predictive modelling before the emergence of machine learning techniques. Despite often being less complex, they can still be highly effective and occasionally outperform more complex machine learning models, depending on the data set. The advantage of these models is their ease of implementation, fewer model parameters, and faster production compared to machine learning models. This chapter presents two different statistical forecasting methods to measure the performance of the machine learning models.

5.1 Naïve model

The naïve forecast is considered the simplest form of forecasting; it assumes that the next period's forecast will be the same as the current period [72]. The formula for the naïve forecast can be mathematically represented by

$$\hat{c}_t = c_{t-1}, \quad (5.1)$$

where $\hat{c}_t$ represents the predicted closing price at period t and c_{t-1} represents the closing price at period $t - 1$. In terms of forecasting foreign exchange closing prices, the predicted closing price for the next period will be equal to the current closing price using the naïve forecast. Table 5.1 contains an example of using the naïve forecast on the closing prices of the USD/ZAR exchange pair, which is simply the closing prices shifted one period ahead.

5.2 Naïve results

The only input necessary to compute the naïve forecast is the closing prices of the USD/ZAR exchange rate; thus, excluding all of the other features in calculating the forecast. The results for

Date	Closing price (c_t)	Naïve forecast ($\hat{c}_t$)
2022/01/24	15.25	—
2022/01/25	15.22	15.25
2022/01/26	15.32	15.22
2022/01/27	15.44	15.32
2022/01/28	15.58	15.44

TABLE 5.1: *Example of using the naïve forecasting model to predict the closing prices of the USD/ZAR foreign exchange pair.*

the naïve forecast are given separately for each of the four years of testing data and a combined summary for all the years collectively in Table 5.2.

Due to the predicted closing price being the same as the previous closing price, the binary directional movement would indicate no directional change. The lack of bidirectional movement also caused the trading strategy to trigger only 19.29% of the time, resulting in 201 trades executed out of the 1042 total trading days.

Metric	Period				
	2018	2019	2020	2021	All
MSE	0.01961	0.01375	0.03329	0.01777	0.02111
MAPE	0.83258	0.63770	0.84475	0.72904	0.76107
Threshold of R0.025 (%)	11.97	15.06	11.54	11.92	12.62
Threshold of R0.05 (%)	27.03	32.05	23.08	25.38	26.88
Threshold of R0.075 (%)	42.86	51.35	35.00	40.38	42.39
Threshold of R0.1 (%)	53.28	63.32	44.62	52.31	53.37
Cumulative profit (%)	5.63	−4.03	8.15	6.53	16.27
Maximum drawdown (%)	−7.00	−6.12	−5.19	−2.24	−10.77
Average profit per trade (%)	0.0804	−0.0960	0.1567	0.1764	0.0809
Profitable trades (%)	42.86	42.86	46.15	48.65	44.78
Number of trades	70	42	52	37	201

TABLE 5.2: *The forecasting results for the USD/ZAR foreign exchange pair using the naïve forecast evaluated using statistical metrics.*

The naïve forecast for the USD/ZAR exchange rate is used as a benchmark for all the machine learning models. This method resulted in consistently low MSE values due to the daily closing prices exhibiting small daily changes on average. The naïve forecast for 2019 performed the best and had the lowest MSE, MAPE and threshold accuracies due to lower price volatility compared to years 2018 and 2020, but surprisingly performed the worst in trading, resulting in a −4.03% loss for the year. The poor trading performance resulted in an average correct trade percentage of 42.86% for the 42 trades executed, tied worse off all the years.

Figure 5.1 contains the actual and predicted closing prices of the first six months of the 2018 test set. The pattern of 1-day lagged prices in the plots is observed for all four testing periods since the naïve simply uses the previous day as the predicted value. The longer ‘jumps’ occurring every five data points in Figure 5.1 are the non-trading days *i.e.*, the weekends, when the foreign exchange markets are closed and no predictions are made for these days. These gaps are still included to keep the temporal structure of predictions intact.

The 2020 test set had the highest MSE of all the years but a similar MAPE to 2018 due to the higher closing prices observed in 2020, which would skew the MSE towards the higher side. This period also generated the most profit at 8.15%, although the profitable trades were only 46.15%, meaning out of the 52 trades made, only 24 made money. The lower profitable trades suggest

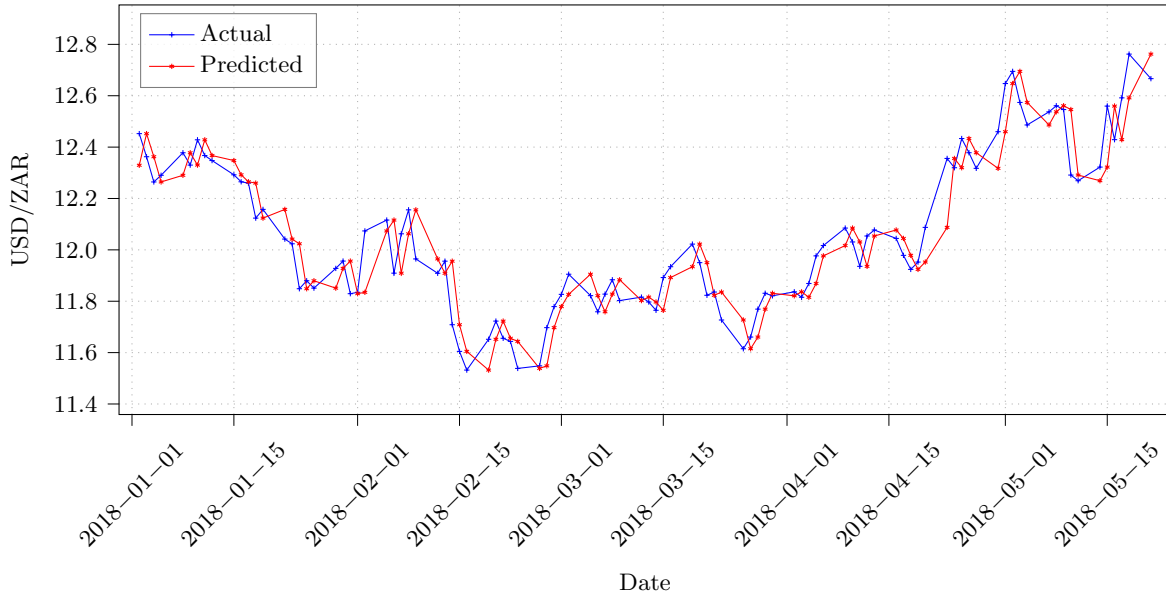


FIGURE 5.1: An illustration of the exhibited 1-day lagging pattern between the actual closing price and predicted closing price based on the naïve forecast of the USD/ZAR dataset from 2018-01-02 to 2018-05-20.

that the correct trades made by the naïve forecast, albeit fewer, negated and outweighed the more frequent losing trades. The overall trading performance for 2021 outperformed all other years, with the second highest profit of 6.53%, but achieved the highest profitable trade percentage and average profit per trade, with less volatility resulting in the lowest maximum drawdown of -2.24% .

The profit accumulated through the USD/ZAR test sets from 2018 to 2021 is given in Figure 5.2. The trading strategy begins with consecutive losses, resulting in the -7% drawdown, after which it makes a profit and breaks even halfway through 2018. The profit drifts sideways from 2018-08-10 to 2019-08-20, with winning and losing trades negating each other. The naïve forecast profits declining again before going on a winning streak starting from 2020-07-06. The profitable streak coincides with the USD/ZAR falling from its peak following the highly volatile period in the first half of 2020.

In summary, the naïve forecast executed 201 trades, of which 44.78% were profitable. The total profit was 16.27% with a maximum drawdown of -10.77% . However, achieving a return of 16.27% for an investment over four years is relatively poor, especially when incorporating additional factors such as interest rates and inflation. The profitable trade percentage for all four years being lower than 50% suggests that in the long run, given enough trades executed, the naïve forecast should not be considered solely to make trading decisions.

5.3 ARIMA model

Autoregressive integrated moving average (ARIMA), introduced by Box *et al.* [10], is a popular forecasting technique for uni-variate stationary time series. These models combine three components to capture and predict underlying patterns and trends in time-dependent data. These components are,

1. Autoregressive (AR): A regression model that incorporates the relationship between past

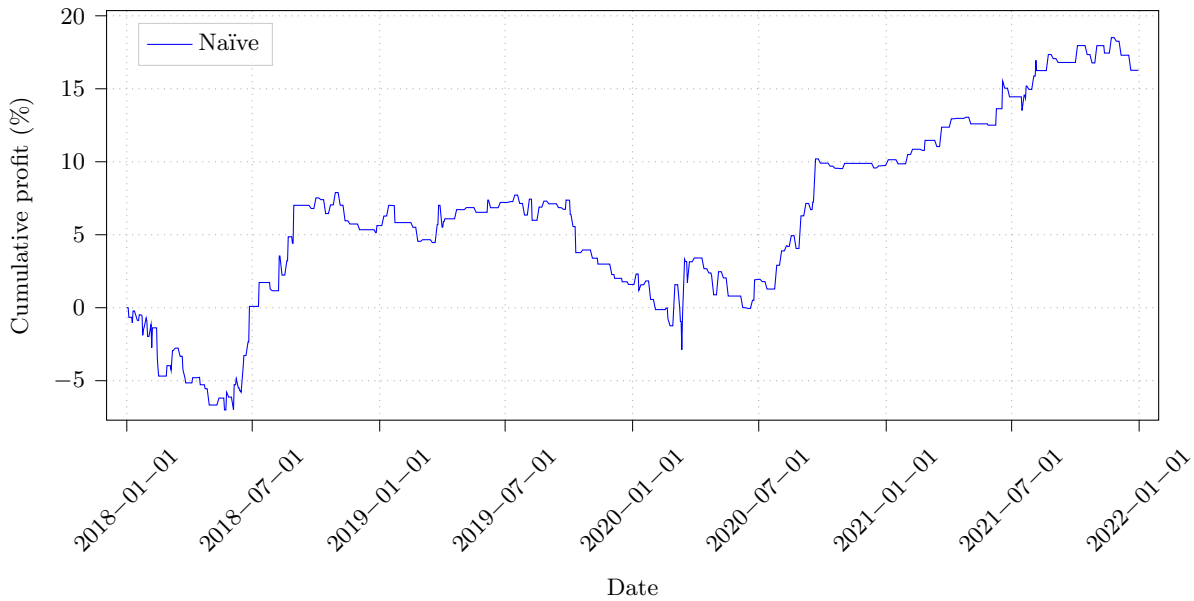


FIGURE 5.2: The **naïve** forecasts cumulative profit for the **2018–2021** testing period.

time series values.

2. Integrated (I): This transforms the time series into stationary data using the differences between values in the time series.
3. Moving average (MA): A method incorporating the relationship between time series values and the residual error terms after applying a moving average model to the past time series values.

ARIMA models follow the assumption that future values of an independent variable can be expressed as a linear combination of the past values and past errors of the same variable [72]. The three components describe the type of ARIMA model used; therefore, corresponding to the chronological order of the three components mentioned above, an ARIMA(p, d, q) is expressed as

$$y_t = \theta_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t - \theta_1 \varepsilon_{t-1} - \theta_2 \varepsilon_{t-2} - \cdots - \theta_q \varepsilon_{t-q} \quad (5.2)$$

where y_t is the actual value at time t and ε_t is the random error at time t . The integers p and q are the number of past observations used in the autoregressive and moving average components, respectively. The parameter d is the number of differences applied to the actual values before expressing the series according to equation (5.2). Finally, ϕ_i and θ_i are the coefficients estimated through maximum likelihood techniques, which are methods capable of determining parameters of a statistical model given an underlying data set [35].

The main advantage of using ARIMA is its use of univariate time series, making it computationally fast to run and requiring less data cleaning and processing than multivariate models [67]. The drawback is that ARIMA models are sensitive to parameter selection, which can be challenging and require manual tuning. Box *et al.* [10] developed a framework to build an ARIMA model outlined as,

1. **Identification:** This involves plotting the time series and visually inspecting the data for trends, seasonality, and irregular patterns. Additionally, checking for stationarity through

autocorrelation (ACF) and partial correlation (PACF) graphs, along with statistical checks such as the Dickey–Fuller test [18].

2. **Estimation:** Choosing the ARIMA parameters p, d, q through ACF and PACF graphs or parameter calibration techniques such as grid search. Estimating the coefficients, ϕ and θ through maximum likelihood techniques.
3. **Diagnostic:** Analyse the residuals of the fitted ARIMA model and ensure the residuals represent white noise; thus, no auto-correlation exists. The Ljung–Box statistical test can be used to determine the presence of autocorrelation [40]. Compare different ARIMA models based on statistical metrics such as MSE.

ARIMA identification

The training and validation sets used to identify and estimate the ARIMA model were the USD/ZAR closing price from 2012-05-01 to 2016-12-30 and 2017-01-02 to 2017-12-30, respectively. These splits represent an 80/20 percentage train/validation split, consistent with the hyperparameter calibration splits used for the ML models.

ARIMA models assume that the underlying time series is stationary, and since it is uni-variate, only the closing price was used from the feature set. A rolling walk-forward validation was applied to the validation set to keep the ARIMA forecasting horizon to 1 day; thus, a new ARIMA model was trained and forecast generated for each day in the validation set. This process was repeated incrementally to build up predicted values within the validation set to prevent data leakage and allow the ARIMA model access to recent observations.

The USD/ZAR closing prices in Figure 5.3(a) do not appear to be stationary, meaning no constant mean and variance, and an underlying trend is present. The ACF and PACF in Figures 5.3(b) and 5.3(c) also contain significant spikes in the lagged observations, which exceed the 95% confidence levels. The presence of non-stationarity was confirmed through a Dickey–Fuller hypothesis test for stationarity [18]. The hypotheses tested were the null hypothesis, H_0 , stating that a unit root is present, indicating non-stationary. The alternative hypothesis, H_a , states that no unit root is present and suggests the time series is stationary. The test over the training period returned a critical value, $z = 0.7326$. Since $z > 0.05$, H_0 can not be rejected, and the time series is considered non-stationary with a 95% confidence level.

The log percentage returns transformation in §4.1.1 calculated via equation (4.1) was used to make USD/ZAR closing prices stationary. The Dickey–Fuller test is repeated on the transformed data and returns a critical value, $z = 0.0$. Since $z < 0.05$, H_0 is rejected, and the transformed time series is assumed to be stationary with a 95% confidence level. The newly transformed series for the training set can be seen in Figure 5.4(a) with the corresponding ACF and PACF in Figures 5.4(b) and 5.4(c) confirming the assumption of stationarity.

ARIMA Estimation

Step 2 of Box *et al.*'s framework involves determining the hyperparameters of the ARIMA model [10]. The values of p, d, q were determined in two ways. They are, firstly, examining the ACF and PACF plots following the recommendations by Box *et al.* and choosing appropriate values based on the plots. The second method uses a grid search of viable parameter values since there are relatively few parameters to try, and the function is quick to evaluate. Then,

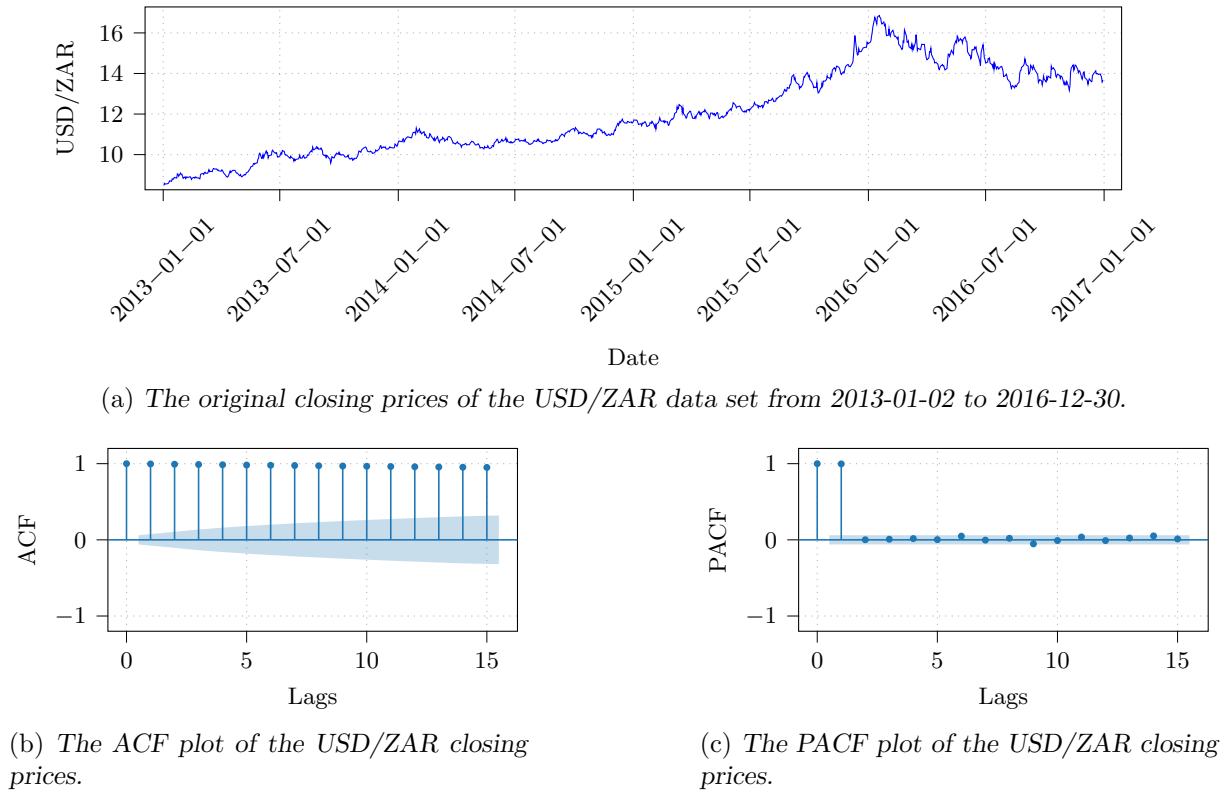


FIGURE 5.3: The USD/ZAR closing prices from 2012-05-01 to 2016-12-30, along with the corresponding ACF and PACF graphs that were used as the training set to calibrate the ARIMA model parameters.

stepping through the validation set and comparing the MSE of both results. Table 5.3 contains the ARIMA model's grid search space for the parameter values tested.

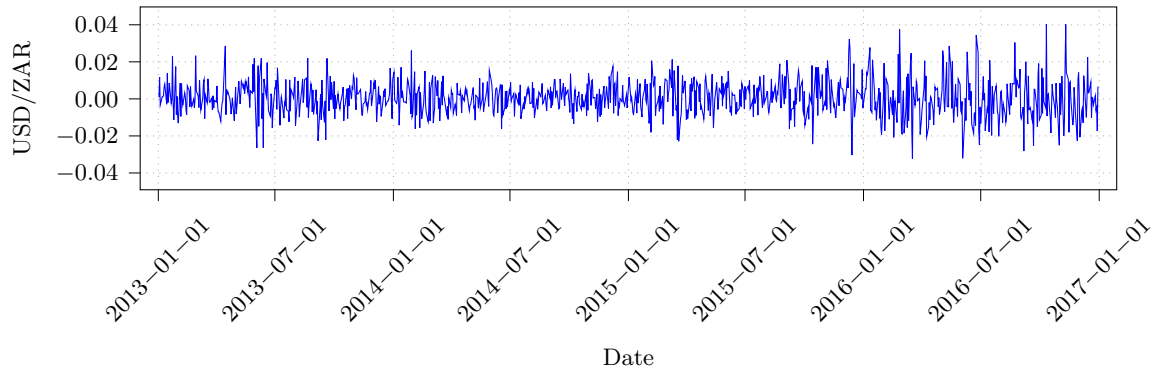
Parameter	Search space
p	[0, 1, 2, 3, 4, 5]
d	[0, 1, 2, 3, 4, 5]
q	[0, 1, 2, 3, 4, 5]

TABLE 5.3: The search spaces for the ARIMA hyperparameter calibration.

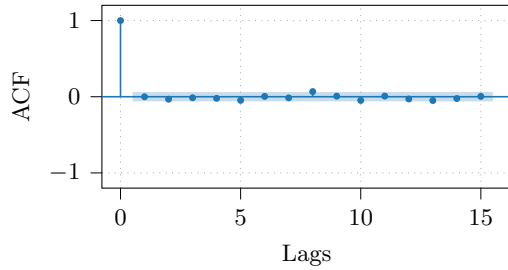
Investigating the ACF and PACF graphs, no significant ACF/PACF values exceeded the bounds of the 95% confidence level; therefore, the recommended ARIMA was of order (0, 0, 0). This parameter configuration falls within the search space defined in Table 5.3 and will be included in the grid search results.

The grid search, implemented in Python [56] using the `statsmodels.tsa.arima.model` package, consisting of 216 possible parameter configurations with a total run time of 10 hours and 57 minutes. Table 5.4 contains a summary of the top 5 ARIMA parameter values found for the MSE and profit metrics. The entire parameter calibration results are given in §A.4 within Tables A.15, A.16 and A.17.

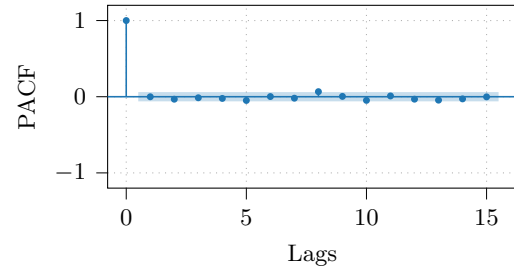
The best parameters were $(p, d, q) = (4, 0, 3)$, recording the lowest MSE of 0.01741 and the 18th highest total profit of 1.73%. The ARIMA(3,4,3) model achieved the highest profit of 16.43%; however, it contained an MSE 141.69% higher than the ARIMA(4,0,3) model at 0.4207. The ARIMA(0,0,0) model found through investigating the ACF and PACF plot ranked 14th overall with an MSE of 0.1755, only 0.84% lower than the best ARIMA model.



(a) The transformed log percentage closing prices of the USD/ZAR data set from 2013-01-02 to 2016-12-30.



(b) The ACF plot of the USD/ZAR closing prices.



(c) The PACF plot of the USD/ZAR closing prices.

FIGURE 5.4: The USD/ZAR closing prices transformed into log percentage closing price differences from 2012-05-01 to 2016-12-30 were used as the training set to calibrate the ARIMA model parameters.

Interestingly, the top 4 ARIMA hyperparameter configurations based on profit had poor MSE values compared to the top 4 configurations with the highest MSE and *vice versa*. A Pearson correlation coefficient test was computed to investigate the relationship between MSE and profit for the hyperparameter calibration [7]. The test returned a weak negative correlation of -0.116 with p -value = 0.087. Thus $p > 0.05$ and no statistically significant correlation exists with a 95% confidence level. This value suggests the ARIMA model struggles to execute profitable trades, regardless of accuracy, and only 11.11% of parameter combinations resulted in a profit.

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
148	4	0	3	0.01741	1.73	4.269
182	5	0	1	0.01742	-4.41	3.075
44	1	1	1	0.01745	3.53	2.025
45	1	1	2	0.01747	-5.11	2.241
46	1	1	3	0.01747	-5.11	2.241

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
136	3	4	3	0.04207	16.36	5.244
131	3	3	4	0.02978	14.35	4.865
173	4	4	4	0.04383	10.22	5.374
135	3	4	2	0.04642	5.97	4.622
47	1	1	4	0.01764	5.74	3.227

TABLE 5.4: The top five parameter calibration results based on MSE (left) and cumulative profit (right) for the ARIMA model.

ARIMA diagnostics

The best ARIMA(4,0,3) from the hyperparameter configuration was used to forecast the USD/ZAR test sets from 2018 to 2021. Step 3 of the Box-Jenkins terminology states to run diagnostic tests

on the model's residuals. Therefore, a Ljung–Box test testing for autocorrelation among the residuals was conducted [40]. The null hypothesis, H_0 , states the autocorrelations at all specified lags in the time series are zero (indicating the residuals are white noise). The alternative hypothesis, H_a , states that at least one autocorrelation in the specified lags is non-zero (indicating the presence of autocorrelation in the residuals). Table 5.5 contains the results Ljung–Box test results for the first six lags in each testing period and whether to reject H_0 . All lags in each test set contained p -values > 0.05 . Therefore, H_0 could be rejected for each set, and the residuals represent white noise for every ARIMA model.

Lags	Ljung–Box test											
	2018			2019			2020			2021		
	LB	p-value	H_0	LB	p-value	H_0	LB	p-value	H_0	LB	p-value	H_0
1	0.096	0.757	No	0.010	0.918	No	1.420	0.233	No	0.322	0.570	No
2	0.313	0.855	No	0.573	0.751	No	3.410	0.182	No	0.600	0.741	No
3	1.113	0.774	No	0.906	0.824	No	5.046	0.168	No	0.623	0.891	No
4	1.120	0.891	No	1.007	0.909	No	5.078	0.279	No	1.093	0.895	No
5	3.797	0.579	No	3.995	0.550	No	5.954	0.311	No	1.118	0.952	No
6	9.261	0.159	No	4.700	0.583	No	8.581	0.199	No	1.133	0.980	No

TABLE 5.5: The Ljung-Box test performed on the first 8 lags from the ARIMA(4,0,3) model for each period of testing to ensure the residuals are white noise.

5.4 ARIMA Results

Similar to the naïve forecast, the results for each testing period of the USD/ZAR from 2018 to 2021 obtained from the ARIMA(4,0,3) model are summarised in Table 5.6. The table contains the same metrics to compare and investigate the two models' differences.

Metric	Period				
	2018	2019	2020	2021	All
MSE	0.01933	0.01319	0.03391	0.01786	0.02108
MAPE	0.82270	0.62907	0.84237	0.72693	0.75532
Threshold of R0.025 (%)	13.90	15.83	11.92	12.69	13.58
Threshold of R0.05 (%)	27.41	32.82	24.23	26.15	27.65
Threshold of R0.075 (%)	41.70	50.58	35.77	41.54	42.39
Threshold of R0.1 (%)	52.90	64.86	45.00	54.23	54.24
Cumulative profit (%)	0.13	4.04	−1.58	−2.00	0.60
Maximum drawdown (%)	−9.49	−2.56	−5.02	−5.63	−9.49
Average profit per trade (%)	0.0022	0.0763	−0.0544	−0.0322	0.0029
Profitable trades (%)	37.29	43.40	48.28	46.77	43.35
Number of trades	59	53	29	62	203

TABLE 5.6: The forecasting results for the USD/ZAR foreign exchange pair using the ARIMA forecast evaluated using statistical metrics.

In 2018, the ARIMA model outperformed the naïve forecast, slightly improving MSE and MAPE by 1.43% and 1.20%, respectively. The improvement suggests a marginally better fit for the data. The average threshold was also higher than the naïve forecast for three out of the four accuracies (R0.025, R0.05, R0.075), indicating the ARIMA model was able to get closer to the actual closing prices more often. The trading performance was the poorest, with the number of profitable trades equating to 37.29% of the 59 trades; surprisingly, the profit generated by

the ARIMA model was breakeven at 0.13%, with a maximum drawdown of -9.49%. The high drawdown indicates consecutive periods of substantial losses.

The year 2019 saw a more noticeable improvement in the ARIMA model's performance, especially in terms of MSE, which decreased by approximately 4.0% compared to the naïve forecast (MSE 0.01319 compared to naïve 0.01375). Additionally, the MAPE dropped to 0.629, indicating better accuracy in predictions. The generated profit was 4.04% with a low average drawdown of -2.56, thus outperforming the only period in the naïve forecast that generated negative returns.

For 2020 and 2021, the ARIMA models MSE and MAPE were within 1.86% and 0.36% compared to the naïve forecast, respectively. Both periods resulted in a loss of profit, with 2020 resulting in a -1.58% loss even though it had the highest profitable trades percentage at 48.28%. 2020 also had the lowest number of trades at 29, with a -5.02 maximum drawdown. The 2021 period saw a -2% loss with a corresponding maximum drawdown of -5.63%.

Summary

Across all testing years, the ARIMA model's performance was similar to that of the naïve forecast regarding MSE and MAPE. The model showed a gradual improvement in prediction accuracy, as the MAPE was consistently lower than that of the naïve. Figure 5.5 illustrates the closer fit of predicted closing prices to the actual closing price of the USD/ZAR data set for 2020. Despite the improvement in accuracy, the ARIMA model's trading strategy resulted in only small profits or losses in some years. Additionally, consistent maximum drawdowns indicated periods of repeating losing trades. Given its final cumulative profit of 0.6% the ARIMA model should not be considered a viable model to trade with.

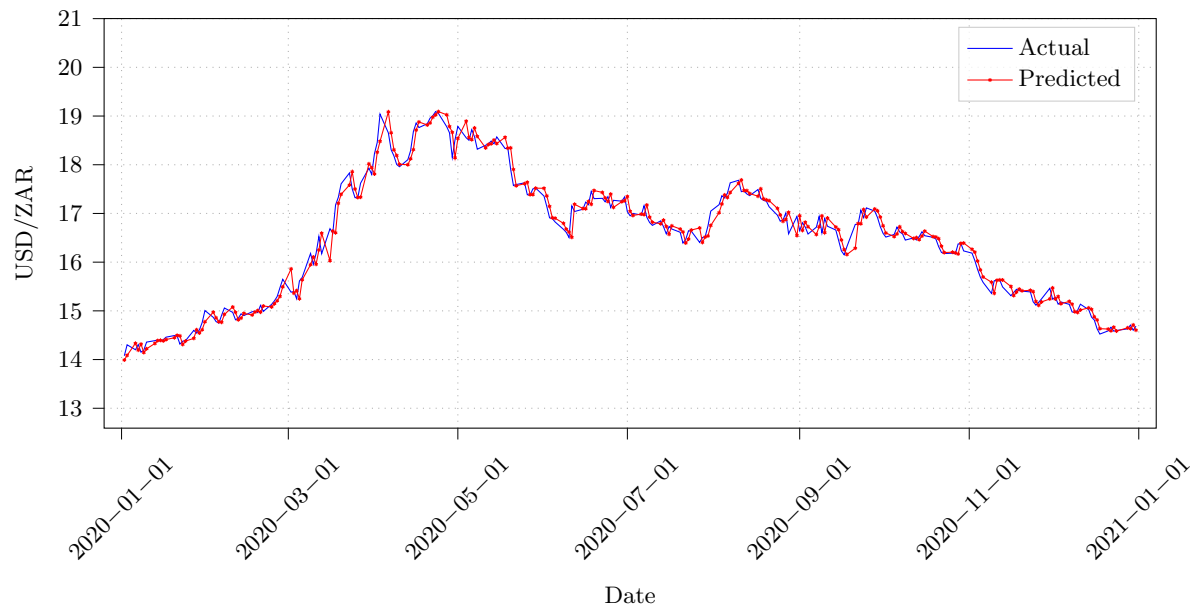


FIGURE 5.5: The ARIMA(4,0,3)'s forecasted closing prices plotted against the actual closing prices for 2020 of the USD/ZAR data set.

Figure 5.6 shows the profit generated for all testing years compared to the profit obtained from the naïve forecast. The ARIMA model was able to negate the opening loss the naïve model sustained; however, this was short-lived as the ARIMA model switched from profit to a loss on

the 2018-05-30 thus getting the 2018 max drawdown of -9.49% . The ARIMA recovered from this initial loss and generated consistently profitable trades from 2018-08-09 to 2019-08-20 while the naïve forecast drifted sideways. During the highly volatile second half of 2020, the ARIMA executed no trades. This stagnant trading could be due to the upward trending period that the model was trained on and then tested on the following downward trending period. Starting from 2020-07-13, both the naïve and ARIMA model had around 1.2% profit. The naïve model was able to capitalise on the increased volatility and generate a profit of 14.98% , whereas the ARIMA model drifted sideways and returned -0.66% until the end of 2021.

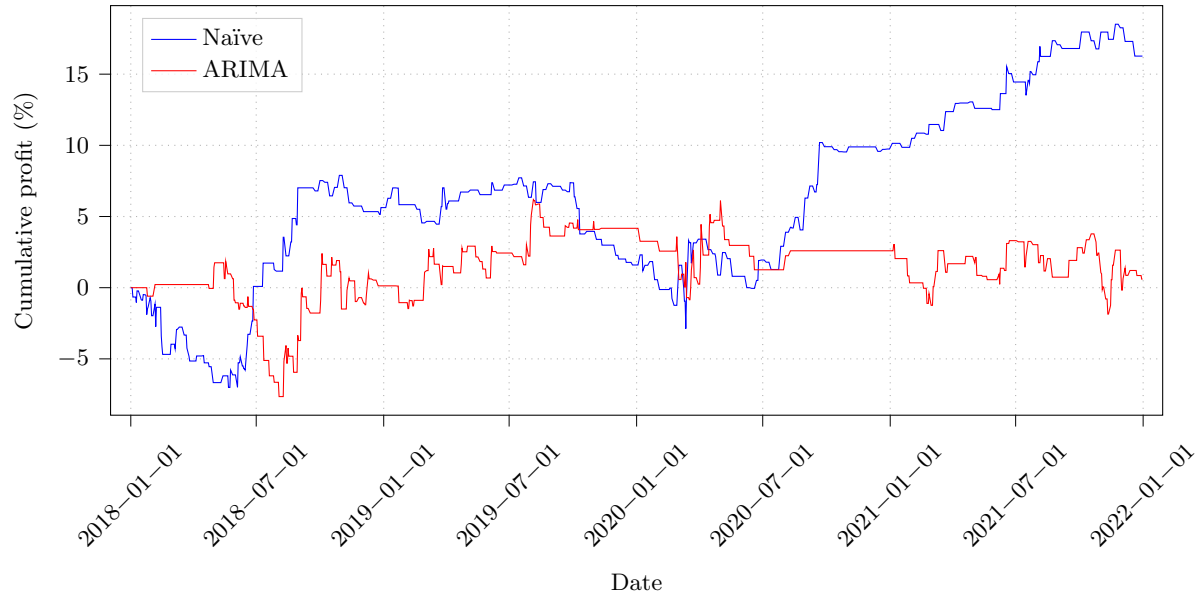


FIGURE 5.6: The **ARIMA** model's cumulative profit for the **2018–2021** testing set.

5.5 Chapter summary

Two statistical methods for forecasting are presented in this chapter, the naïve forecast and the ARIMA model. Both model's forecasting methodology and assumptions are explained and the results across each of the 4 years testing data are analysed. The analysis of results highlight the strengths and limitations of each approach to make informed decision in foreign exchange markets. Furthermore, the results obtained in this chapter are used as a benchmark against the more sophisticated ML models.

CHAPTER 6

Long short-term memory neural networks

Contents

6.1	Artificial neural networks	73
6.2	Recurrent neural networks	75
6.3	Long short-term memory	76
6.4	Hyperparameters	77
6.5	Hyperparameter calibration results	80
6.6	Results	84
6.7	Chapter summary	91

This chapter covers all sections related to the LSTM model. The chapter begins with fundamental preliminaries to understanding an LSTM model concerning artificial and recurrent neural networks. Then, an overview of an LSTM model architecture and training algorithm is given. The hyperparameter results are discussed, focusing on the hyperparameters with the biggest influence on the results. The chapter concludes by analysing the impact of the two parameters, **retraining interval** and **sequence length** for the 2018–2021 USD/ZAR data sets testing period.

6.1 Artificial neural networks

The human brain, comprised of billions of interconnected neurons, can process and learn information through these neurons connected in a complex parallel network. This biological system inspired the development of artificial neural networks (ANNs) that mimic these cognitive processes. McCulloch and Pitts introduced the simplest form of an ANN, the perceptron, which Frank Rosenblatt later refined [43, 61]. Figure 6.1 contains a schematic representation of how information is passed through a single perception. Inputs, x_i , with respective weights, w_i , compute a weighted sum passed through an activation function (here, a step function) to generate an output based on a threshold value, 0 in this example.

Figure 6.1 only contains a single perception; however, a deep neural network (DNN) is formed when multiple perceptions or other ANNs are stacked together and connected to multiple layers

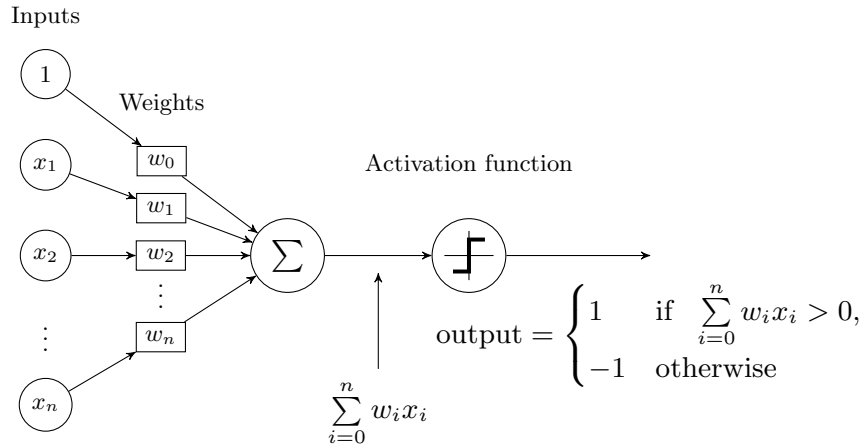


FIGURE 6.1: A schematic representation of a single perceptron used to build deep neural networks.

of perceptrons or any other ANN. Figure 6.2 illustrates the schematic representation of a typical DNN, with information passing from left to right, starting with the inputs, x_i , passing through the hidden layers of perceptions each containing an activation function and ending with the outputs, y_i .

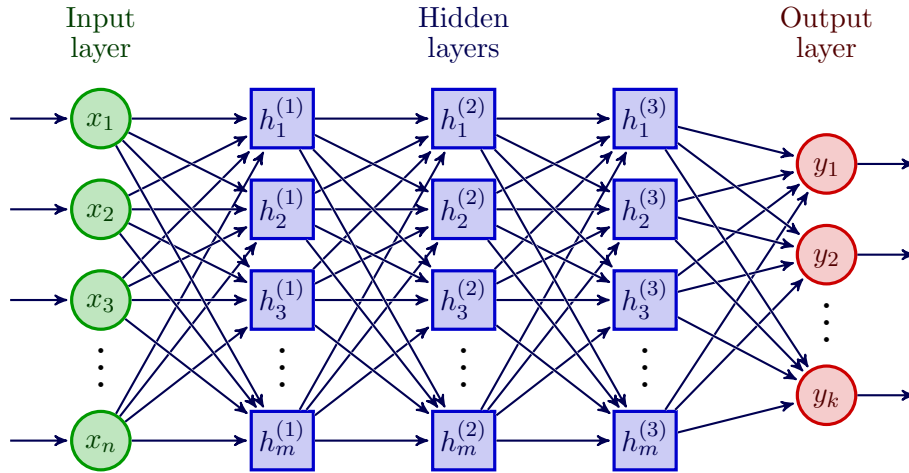


FIGURE 6.2: A schematic representation of a deep neural network architecture.

The weights between the connections of neurons for ANNs and DNNs are trained through an algorithm called *backpropagation* proposed by Rumelhart *et al.* [62]. The algorithm consists of three key phases to evaluate the predictions and adjust the weights of the individual neurons, summarised by Géron [24] as:

1. **Forward pass:** Computes predictions based on the current weights of the network and measures the error by evaluating the predicted labels compared to the actual labels (supervised learning).
2. **Backward pass:** Backtraces from the output layer through each layer towards the inputs in the network. This step calculates the gradients of the training metric (typically, MSE) concerning each weight. The gradients indicate the contribution each weight has to the error, thus measuring the sensitivity between outputs and changes in the weights.

3. **Gradient descent:** With the gradients calculated, the algorithm adjusts the weights to minimise the error.

The iterative process of forward and backward passes, followed by adjusting the weights through gradient descent, is repeated until the network error is minimised or reaches a predefined acceptable level of convergence. ANN's architecture and training procedure form the foundation for developing intricate NNs.

6.2 Recurrent neural networks

Traditional ANNs only pass information from the activation functions in one direction and are called feedforward NNs. A recurrent neural network (RNN) works similarly to ANNs, with the addition of being able to pass information backwards. Figure 6.3 demonstrates the concept of unrolling an RNN through time, where each RNN cell, A , at time t is a function of all the previous RNN cells. The output at layer, h_t , is thus dependent on previous observations.

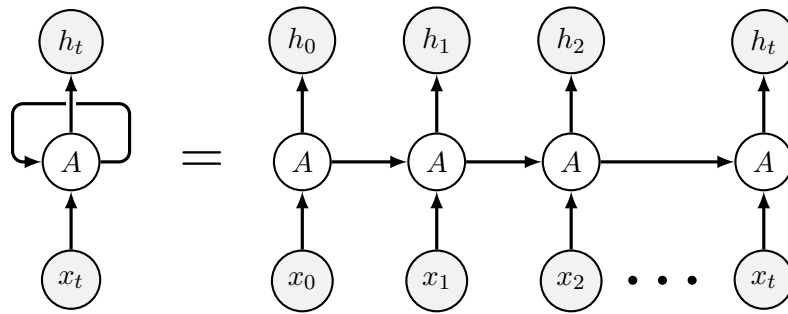


FIGURE 6.3: A schematic representation of how a recurrent neural network is unrolled through time.

This bidirectional flow, facilitated by loops within the network, allows the RNNs to handle sequential data effectively. Sequential data is data where the order of observations is important, and dependencies from past observations can influence future observations. Since the output of a recurrent neuron is a function of the input from all the previous time steps, there is a form of memory to each neuron [24]. This ability to retain memory from prior observations proves particularly effective in tasks such as speech recognition and analysing time series data [24].

RNNs use a training algorithm that accounts for the sequential nature of the network called *backpropagation through time* (BPTT). This algorithm involves first unrolling the network through time, depicted in Figure 6.3 and then applying the backpropagation in §6.1. The BPTT algorithm, in conjunction with RNNs, has two significant caveats, the first being the inability to capture long-term dependencies in the data. While RNNs can handle short sequences, usually ten steps long, the inability to remember more extended time patterns in the data set hinders the predictive performance of these models.

Secondly, the instability of weights during BPTT, known as the ‘exploding’ or ‘vanishing’ gradients problem, comes from iterative weight updates using gradient descent. Gradients with weights close to zero tend to diminish with each iteration, eventually converging towards zero and effectively disappearing from the network. Conversely, large weights lead to exponentially growing gradients, hindering convergence and poor predictions.

6.3 Long short-term memory

Hochreiter and Schmidhuber [27] proposed a new type of RNN designed to overcome the problems experienced with regular RNNs, called long short-term memory (LSTM). The new RNN architecture addresses the issues of the exploding/vanishing gradient problem and the inability to capture long-term dependencies. The LSTM cell consists of a memory cell, c_t , that captures the long-term dependencies in the data and the hidden states, h_t , which is more biased towards recent observations. These components are updated through three gates, attributed to how they handle the flow and retention of information. In Figure 6.4, the flow of information in the LSTM cell through the various different gates governing the memory of the cell is illustrated. The three gates are:

1. **Forget gate:** This gate utilises the previous memory cell, c_{t-1} , and the output from the previous hidden state, h_{t-1} , and current inputs, x_t , being passed through the sigmoid activation functions. The gate then determines how much information should be disregarded through the element-wise multiplication of these two components.
2. **Input gate:** The new data entering the cell and how much information should be added to the memory cell, c_t , are handled through this gate. The previous hidden cell, h_{t-1} , and the current input, x_t , are processed through two activation functions — sigmoid and hyperbolic tangent. The outputs from both activation functions are multiplied and added to the updated memory cell, c_{t-1} , that has already passed through the forget gate. The final result is the updated memory cell c_t , which contains the long-term memory.
3. **Output gate:** The final gate determines the information to pass to subsequent LSTM cells in the network. The updated memory cell, c_t , is passed through a hyperbolic tangent function. The previous hidden states, h_{t-1} , and inputs, x_t , are processed through the sigmoid activation function and combined with c_t to update the hidden cell, h_t , and output a value, y_t . Therefore, the output of a cell, y_t , is the corresponding input, x_i , of the cells in the following layer.

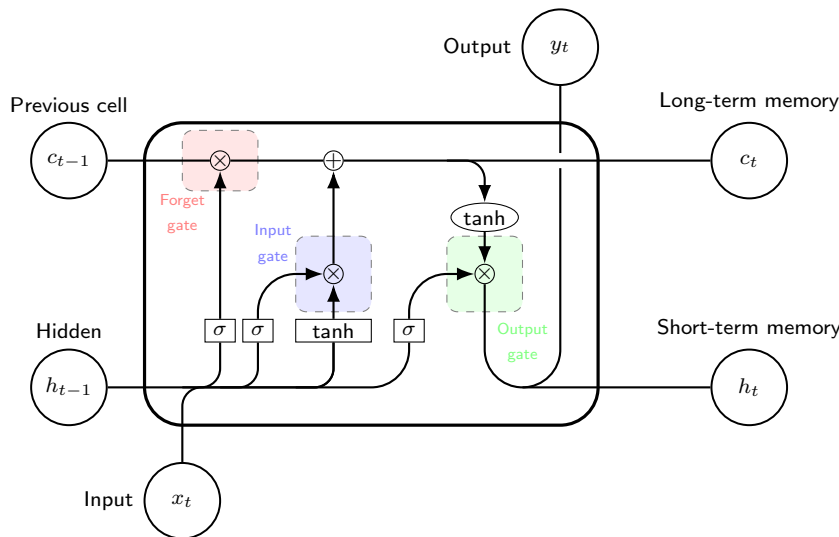


FIGURE 6.4: A schematic representation of an LSTM cell. The sigmoid activation function is represented by σ , and $\tanh$ represents the hyperbolic tangent activation function. The mathematical symbols $\oplus$ and $\otimes$ are the vector addition and element-wise multiplication, respectively.

LSTMs make a suitable choice for modelling time-series data as one of the advantages is that they can handle noisy, non-standard distributions and continuous values [27]. LSTM networks work well over a broad set of parameter settings, making them less sensitive to parameter tuning than other models to get relatively good performance. Von Stein [70] confirmed these findings when performing hyperparameter testing on the LSTM models architecture. Out of the 3 models that were considered (LSTM, CNN-LSTM and multi-head CNN-LSTM), the LSTM had higher accuracy than the other two models over a wider set of model parameters. Although the LSTM did not achieve the highest overall accuracy out of the 3 models, it was the most consistent and can be considered more reliable.

6.4 Hyperparameters

The LSTM model is highly customisable with a wide range of parameter values and settings, from the number of layers to the training algorithm of the model can all be adjusted. Different architecture setups can influence the LSTM model's performance and improve convergence during training. This section presents an overview of selected hyperparameters and best practices in literature for building, training and calibrating LSTM models.

Number of hidden layers

The number of hidden layers refers to the number of LSTM layers connected in parallel and determines the width of the network. Wider networks can extract more information from the data, with each layer capturing different patterns and improving stability [75]. The caveat of adding too many layers is the chance of overfitting the dataset, particularly for limited sequential data. Each added layer adds to the complexity and training times for the models since more layers equate to more weights that need to be updated. The range of hidden layers explored was between 3 and 10, as less than three layers were considered too shallow, and more than ten layers caused convergence issues in the findings of Von Stein [70].

Number of neurons per hidden layer

The depth of a network is determined by the number of LSTM cells (neurons) in a hidden layer within the network. The convention was to use a pyramid structure where the number of neurons in each hidden layer would get smaller the wider the network became; however, this practice has been abandoned. New best practices state that the number of neurons per layer can be equal for every hidden layer, achieving similar results to varying them between layers [24]. This assumption reduces the complexity of choosing the number of neurons in hidden layers. Therefore, the depth is consistent throughout each layer, and only one hyperparameter value needs adjusting. The values considered were layers ranging from 16 to 512 LSTM neurons.

Initialisers

The weights of a NN need a starting value before training commences. Different techniques have been developed throughout the years to initialise the weights of neurons to speed up and improve convergence. The most popular methods named after the authors who developed them are the initialisers Lecun normal [37], Glorot uniform [25] and He normal [26]. The effects of these three initialisers on the LSTM models are investigated.

Activation function

Different activation functions are available for the LSTM cell in Figure 6.4. The focus will only be on the activation function of the output gate (default tanh), with the input and forget gates left at their default activation functions, sigmoid and hyperbolic tangent. Figure 6.5 contains the different types of popular activation functions and their corresponding formulas of how the weights are clipped. These activation functions are responsible for the processing of information between LSTM cells in the network with the innovation of the rectified linear unit (ReLU) showing superior performance on time-series data [46, 59]. The activation functions experimented with are the tanh, sigmoid and ReLU functions.

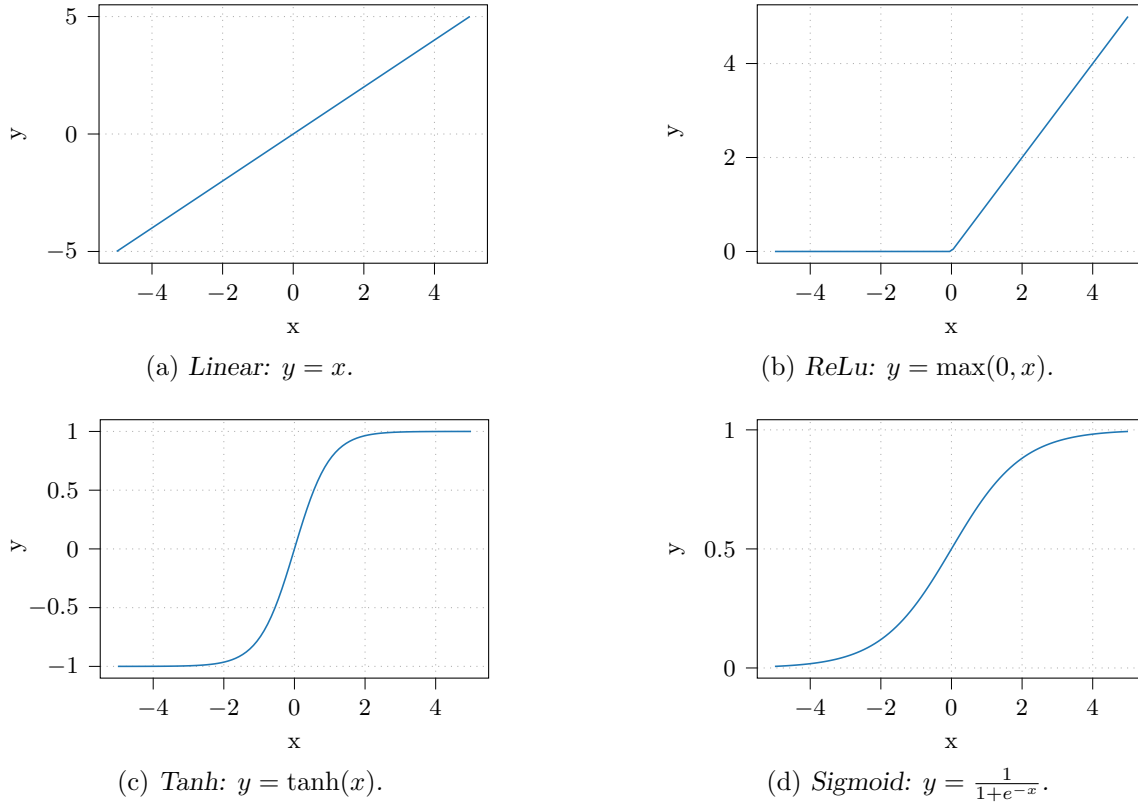


FIGURE 6.5: The different popular activation functions available for the LSTM NN.

Optimisers

The optimiser refers to the training algorithm for the LSTM model, which traditionally uses BPTT with stochastic gradient descent (SGD) to update the weights in the model. Improvements have been made to the training techniques with more sophisticated and faster convergence algorithms available. Popular optimisers, such as SGD with momentum, RMSprop, Adagrad and Adam have shown improved performance over the standard SGD optimiser [48]. All these optimisers have varying comparable performance depending on the underlying data set. However, to limit the scope and reduce the number of hyperparameters considered for calibration, the Adam optimiser was chosen and assumed to be sufficient to train the LSTM models. The Adam optimiser generally has fast convergence and good performance and is commonly the default optimiser in the literature surrounding LSTM models [57, 59, 76].

Epochs

During training, the entire dataset is fed into the model for a user-defined number of epochs. Each epoch includes a complete forward and backward pass and weight adjustments using the BPTT algorithm. A large number of epochs will complete more iterations of BPTT and possibly lead to overfitting by memorising the training data and being unable to generalise. A solution to configure the number of epochs while simultaneously preventing overfitting is to use a technique called *early stopping*. The number of epochs is set to 100 with the inclusion of early stopping.

Early stopping

Early stopping is a technique used to prevent NNs from overfitting the training data. After each epoch is finished, the model uses the validation set (not trained on) to evaluate the prediction accuracy of the trained model. If the validation accuracy (MSE) does not improve for n epochs, then the training of the NN will terminate, and the model will be restored to the best weights of the NN with the best validation accuracy. Figure 6.6 illustrates the convergence graph on the training and validation sets. The training error decreases for all epochs; however, the validation error increases after 500. This increase is due to the model overfitting the training data. Therefore, using early stopping, the NN weights will be restored to weights at the time of the best validation error at epoch 500. The early stopping criterion was chosen as 20 epochs, meaning training will terminate after 20 epochs if there is no improvement in the validation error.

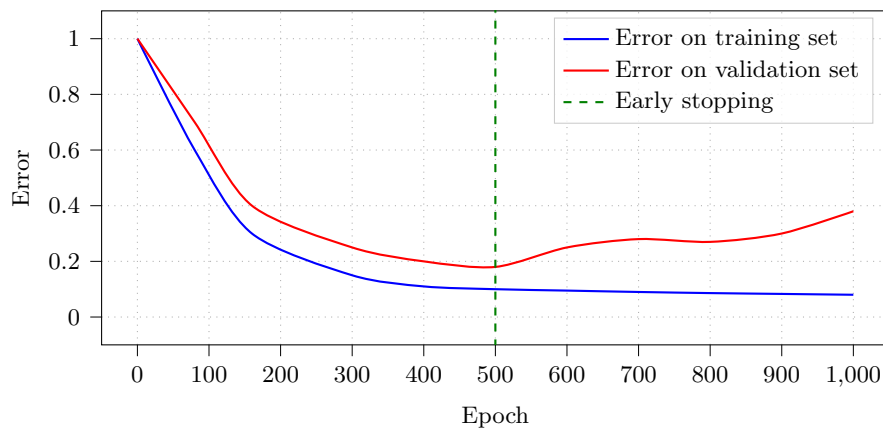


FIGURE 6.6: An example of using early stopping to train an NN. The error on the validation set (red) becomes larger when the network begins to overfit the training set. Although the error on the training set (blue) decreases, stopping training after 500 epochs will avoid overfitting.

Learning rate

Géron [24] considers the learning rate the most important parameter, suggesting the optimal learning rate is half the maximum learning rate. The learning rate determines the size of ‘jumps’ between model weights during training. With a lower learning rate, the ML model will take too long to converge and is prone to get stuck in local optima. A learning rate that is too big will ‘jump’ around the search space and be able to explore more solutions but fail to converge or focus on a specific solution. A technique implemented is to reduce the size of the learning rate when the training error does not increase, therefore reducing the step size and trying to get better

performance. The default learning rate is typically 0.001, but the ranges between 0.0001 and 0.1 are explored for further optimisation. Additionally, incorporating reduced learning rates. The learning rate is set to decrease by 10^{-1} if the training error does not improve after 10 epochs.

Drop out

A regularisation technique to prevent overfitting is to use a dropout layer at the end of the LSTM model. The dropout layer deactivates a percentage of the neurons in the last layer. The idea is that if the dropout is 20%, then only 80% of all the neurons are activated and used to make predictions. They must now learn more to compensate for the loss. Importantly, all neurons are used to make predictions on the testing sets using the LSTM models. Thus, dropout only applies to the training process. The dropout percentage considered for the last layer of the LSTM model is between 0% and 50%, thus ranging from no neurons deactivated to up to 50% of the neurons deactivated.

Batch size

The training process divides the data set into smaller parts called batches and computes BPTT on a single batch instead of the entire data set. Once all batches have been processed, then a single epoch is complete. The batch approach allows the models to gradually learn from the data set in feasible sizes and iteratively update weights throughout each epoch. Adjusting the batch size is a tradeoff between efficiency and training stability, as the larger the batch size, the faster the model trains, but with a higher chance of instability. The batch size ranges considered are between 16 and 128.

6.5 Hyperparameter calibration results

As discussed in §4.4, Bayesian optimisation with manual k -fold cross-validation was used to calibrate selected hyperparameters for the LSTM model. LSTM models contain far more hyperparameters than the abovementioned. However, these hyperparameters are considered more important, and the hyperparameters excluded will remain at their default values. Table 6.1 contains the search space used and the corresponding best values found through 150 iterations of Bayesian optimisation. The calibration took the longest out of all the considered models, resulting in a run time of 7 days, 23 hours and 52 minutes. The longer time is due to the complexity of the LSTM training algorithm and the computational limit of the computer used to train the LSTM models. The complete results for all 150 iterations can be found in §A.1 given in Tables A.2, A.3, A.4 and A.5.

Hyperparameter	Abbreviation	Search space	Best value
number of layers	n.l	[3, 4, 5, ..., 10]	3
number of nodes	n.n	[16, 17, 18, ..., 512]	95
activation function	act.f	[ReLu, sigmoid, tanh]	sigmoid
initialiser	init	[glorot_uniform, he_normal, lecun_normal]	glorot_uniform
dropout rate	drop	[0, ..., 0.5]	0.46%
batch size	b.s	[16, 17, 18, ..., 128]	83
learning rate	l.rate	[0.0001, ..., 0.1]	0.0664

TABLE 6.1: *The LSTM hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.*

The **number of layers** parameter corresponds to the number of LSTM hidden layers in the model's architecture, with 3 being the best. A depth, **number of nodes**, of 95 LSTM cells achieved the best test MSE. Figure 6.7 illustrates the best architecture found for the LSTM model, containing 3 hidden layers with 95 LSTM cells connected in parallel. The last LSTM layer contains a dropout percentage of 0.46% to regularise the model, thus preventing overfitting. The last layer is connected to a dense layer containing one cell with a linear activation function to allow the model to output a predicted value not confined by the bounds of a non-linear function.

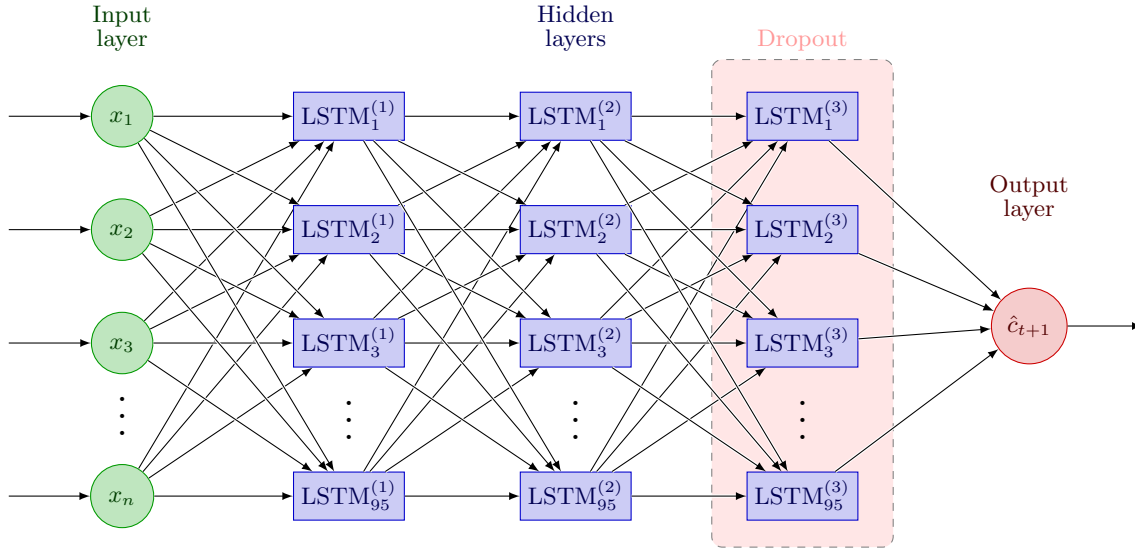


FIGURE 6.7: A schematic representation of the best LSTM architecture found through Bayesian optimisation.

The result for the hyperparameter calibration is broken up into categorical hyperparameters, **activation function** and **initialiser**, and the remaining numerical hyperparameters to investigate if any correlations between the parameters and the performance of the model can be found. Notably, there is little correlation between the variables and the profit achieved. This is likely because the model is optimised on the MSE, and incorporating a trading strategy is excluded from the model's optimisation process.

The ReLu activation functions had the most noticeable effect on the MSE, with the ReLu function responsible for the worst five iterations in the calibration having high MSE values above 9, as seen in Table 6.2. Comparing these poor MSE values to the best five iterations in Table 6.3, there are convergence issues with certain parameter configurations when the ReLu function is used. These results are surprising, given ReLu's performance in the literature [59]. Additionally, Figure 6.8 shows the convergence graph of the LSTM model's hyperparameter calibration, with the different activation functions used at each iteration colour-coded. The ReLu function appears in all but one outlier, with a clipped value of 0.0235.

Combining the ReLu activation function with a wide and deep LSTM NN, number of layers exceeding 8 (except for iteration 144), and neurons in each layer exceeding 300 produces poor prediction performance, with other hyperparameter values having a negligible effect on this combination. Therefore, avoiding this combination when working with the USD/ZAR dataset is recommended. However, The ReLu function had the 4th best prediction accuracy but was outperformed by the sigmoid and tanh functions.

Overall, the sigmoid function performed the best, and out of the top 50 parameter configurations,

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
33	8	410	ReLu	he	48.65	43	0.089	2E+51	-24.59	4E+12	-27.20	59
74	8	304	ReLu	glorot	1.20	38	0.079	1+E6	-6.55	59.85	-1.70	55
144	4	503	ReLu	lecun	15.36	32	0.086	4396	-6.57	0.105	-7.41	93
116	12	330	ReLu	lecun	0	46	0.1	25.11	-11.96	3678.8	-10.82	104
86	10	450	ReLu	glorot	42.08	27	0.067	9.02	-24.27	11.48	-22.56	80

TABLE 6.2: The worst five hyperparameter calibration iterations for the LSTM model found through 150 iterations of Bayesian optimisation.

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
30	3	95	sigm	glorot	0.46	83	0.066	0.02026	-0.97	0.01896	2.20	95
140	7	333	sig	he	0.02	31	0.013	0.02039	0.27	0.01897	0.59	86
34	5	28	tanh	glorot	24.62	80	0.072	0.02040	-2.24	0.01916	-0.01	31
69	10	70	ReLu	le	30.37	76	0.087	0.02043	-0.04	0.01900	0.52	103
75	3	218	sigm	glorot	50	75	0.012	0.02044	-3.19	0.01887	2.47	82

TABLE 6.3: The top five hyperparameter calibration iterations for the LSTM model found through 150 iterations of Bayesian optimisation.

the sigmoid activation function was used for 41 and achieved the top 2 results. Figure 6.9(a) shows the box and whisker plot, displaying the interquartile range of the different activation functions tested on MSE. The outliers have been clipped to an MSE value of 0.0235 to plot the diagrams on a comparable scale and reduce the influences of the outliers. The sigmoid contains the lowest interquartile range with only one outlier clipped to 0.0235, seen in Figure 6.8. The tanh function was second best and had no outliers; therefore, it was the most stable activation function. The ReLu, given the numerous outliers, has the worst range.

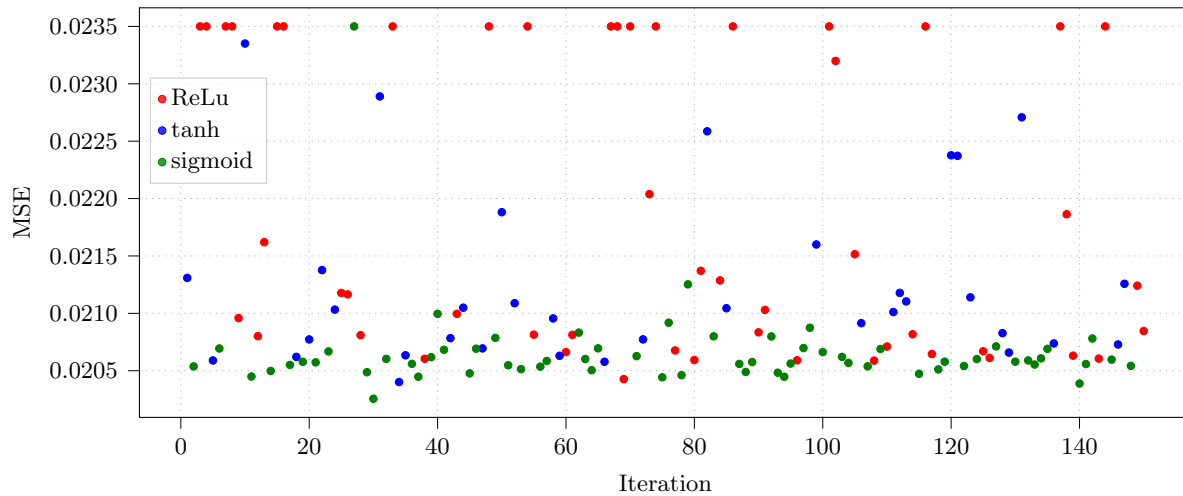


FIGURE 6.8: The convergence graph of the LSTM hyperparameter calibration with the outliers being clipped at an MSE of 0.0235 to illustrate which activation function, ReLu (red), tanh (blue), and sigmoid (green) produced the best results on a comparable scale.

This finding was consistent with Von Stein [70], who chose the ReLu activation function as default and, through experimentation of the LSTM model architecture, found poor performance dropping by over 100% in a threshold accuracy of 0.025 when the number of hidden layers

exceeded 12. This could be due to the activation function of ReLU illustrated in Figure 6.5(b), not clipping the neurons in the LSTM to a range between -1 and 1 or 0 and 1 , and the phenomenon of the exploding gradients is experienced in the model.

The initialisers Glorot uniform and Lecun normal had similar results and consistently outperformed the He normal initialiser. The box and whisker diagrams are given in Figure 6.9(b), illustrating the superior performance of the Glorot uniform and Lecun normal initialisers. The outliers are once again clipped to 0.0235 in this graph to see the difference on a comparable scale. The He normal has the widest interquartile range, indicating the worst MSE values from this initialiser compared to the other two. The Glorot uniform has a slightly lower interquartile range with a similar median to the Lecun normal. However, it contains the lowest MSE value out of the three initialisers but with more outliers than the Lecun.

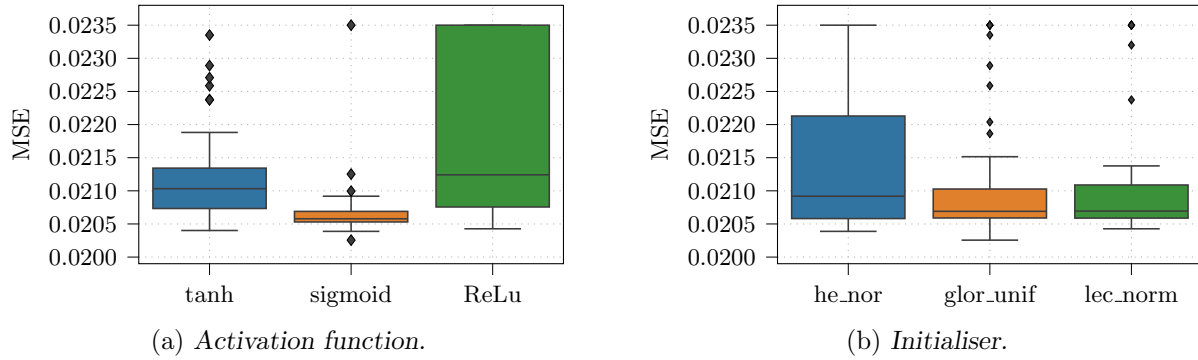


FIGURE 6.9: The box and whisker plots showing the distribution of the MSE found for the different activation functions (left) and the initialisers (right) through hyperparameter calibration of the LSTM model. The outliers have been clipped to 0.0235.

The numerical values had a negligible effect on the parameter calibration results, and no distinct correlations were found between any of the metrics tested. Figure 6.10 contains Pearson's correlation matrix between the numerical values and the metrics MSE and profit. All absolute correlation coefficients had values below 0.2, indicating a weak correlation between MSE. Once again, the profit and MSE are negatively correlated. Noticeably, the profit made on the training set is strongly correlated to the profit made on the testing set, indicating that a model capable of making a profit on seen data will continue to do so on unseen data.

n lay	1.00	0.09	-0.10	-0.06	-0.12	-0.15	-0.03	0.03	-0.21
n nod	0.09	1.00	-0.10	-0.28	-0.10	0.19	-0.28	0.10	-0.23
drop	-0.10	-0.10	1.00	0.13	0.23	0.02	-0.17	0.12	-0.07
b size	-0.06	-0.28	0.13	1.00	0.05	-0.12	0.14	-0.07	0.20
l rate	-0.12	-0.10	0.23	0.05	1.00	0.15	-0.29	0.10	-0.23
MSE TT	-0.15	0.19	0.02	-0.12	0.15	1.00	-0.54	0.18	-0.43
Profit TT	-0.03	-0.28	-0.17	0.14	-0.29	-0.54	1.00	-0.57	0.80
MSE TR	0.03	0.10	0.12	-0.07	0.10	0.18	-0.57	1.00	-0.59
Profit TR	-0.21	-0.23	-0.07	0.20	-0.23	-0.43	0.80	-0.59	1.00
	n lay	n nod	drop	b size	l rate	MSE TT	Profit TT	MSE TR	Profit TR

FIGURE 6.10: The Pearson correlation coefficient matrix between the numerical hyperparameters of the LSTM model and the metrics MSE and profit for the training (TR) and test (TT) sets.

6.6 Results

The LSTM model with the best architecture found in the hyperparameter calibration in Table 6.1 took 2 hours and 12 minutes to test all combinations of **retraining interval** and **sequence length** from 2018 to 2021 in the USD/ZAR data set. For a retrain interval of 12, meaning that every 12 months, the model is updated on new data (data is rolled one year forward in the data window). This means the training set gets a new year of data and drops its first year of data. The old testing year becomes the validation set, and the new test set is the new year of data.

There are 40 combinations of retraining interval (4 total) and sequence length (10 total). The LSTM model with a 12-month retrain interval is retrained 4 times since there are 4 years of testing data, resulting in 160 total models (16 for each sequence length). Similarly, the 6-month, 3-month and 1-month retrain intervals equate to 320, 640 and 1920 different LSTM models initialised and trained, respectively. Due to the high number of models, investigating each individual model's training and validation performance to address concerns of underfitting or overfitting is not possible. However, the selected architecture, notably the dropout layer with the addition of an early stopping training criterion, was assumed to negate the effects of overfitting the model. The only caveat left is underfitting, which was assumed to be negated by selecting the best hyperparameters.

The different LSTM models concerning their parameters **retrain interval** and **sequence length** are referred to as LSTM(r, s), with r being the different retrain intervals 12-monthly, 6-monthly, 3-monthly and 1-monthly and s being the different sequence lengths 1 through 10. For example, the LSTM(12, 4) is the LSTM model with a 12-month retrain interval and a sequence length of 4, meaning the model is retrained every 12 months and uses 4 previous days as features to predict the next day's closing price.

All test years combined

The retraining interval had a noticeable effect on the LSTM model results with the summarised results of the MSE and MAPE for the testing period 2018–2021 given in Figure 6.11. There is a clear distinction between the retraining interval and these performance metrics, with the less frequently trained models performing better than the more frequently updated models. The best MSE values thus came from the 12-month retrain interval in Figure 6.11(a) with the LSTM(12, 2) model having the lowest MSE, with a value of 0.02098. The LSTM(12, 2) marginally outperformed the MSE of the naïve and ARIMA model by 0.62% and 0.47%, respectively.

The sequence length had a subtle effect. However, only using the previous day (**sequence length** = 1) as a feature has the worst MSE for the 12-month and 6-month retrain interval. The 1-day sequence length for the 12-month retrain interval was 0.02168 and 2.24% worse than the average 12-month retrain interval across all sequence lengths. This poor performance of the 1-day sequence length was apparent in the 12-monthly retrain for the MAPE metric but not in the 6-month retrain interval in Figure 6.11(b).

The threshold accuracies summarised for all four years of testing given in Figure 6.12 show a more sporadic distribution of the results, and a pattern is less clear. For the smaller threshold values of R0.025 and R0.05 in Figures 6.12(a) and 6.12(b), a shorter retrain interval (more frequently updated) had the best overall accuracies. The LSTM(1, 10) had the highest R0.025 accuracy of 13.68%, and the R0.05 threshold showed the LSTM(3, 1) with the highest accuracy of 29.09%, 1.25% higher than the second highest in the same category. The higher threshold values of R0.075 and R0.1 showed consistent results with the MSE and MAPE metrics, with

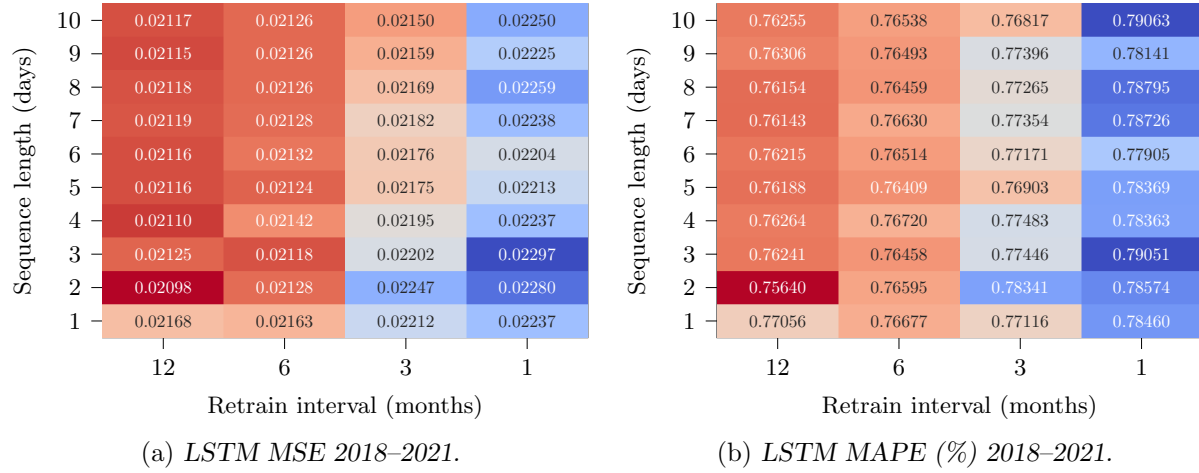


FIGURE 6.11: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' MSE (left) and MAPE (right), for the **2018-2021** test set.

higher accuracies of R0.075 and R0.1 in Figures 6.12(c) and 6.12(d) being obtained from less frequent retrain intervals of 12 and 6 months.

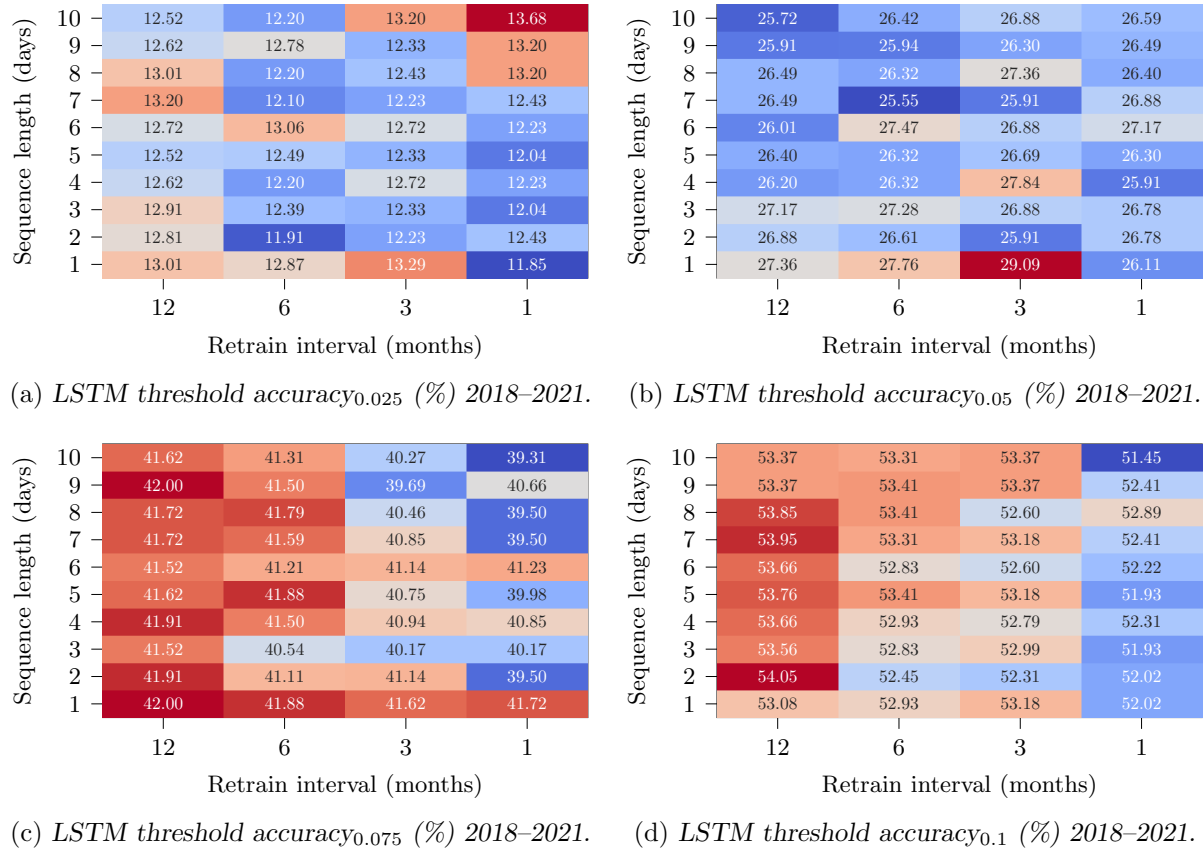


FIGURE 6.12: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' different threshold accuracies for the **2018-2021** test set.

Contrary to the hyperparameter results, there appears to be a stronger correlation between the MSE and profit, with more accurate LSTM models achieving a better cumulative profit, illustrated in Figure 6.13(a). The best profit came from the LSTM(12, 2), with a cumulative profit equating to 34.34%, which is 19.51% higher than the second-best profit model, LSTM(12, 9), at 14.83%. The 12-month showed superior performance to the other retraining intervals, resulting in a profit for 9 out of the 10 different sequence lengths from 2018 to 2021. The only model is LSTM (12, 2), nearly breaking even at a -2.67% loss. The 6-month retrain interval was less profitable, with only 7/10 sequence lengths making a profit. The poor trading results are showcased by the 3-month and 1-month resulting in large losses, with the average profit for these sequence lengths being -26.58% and -41.5% , respectively. The maximum drawdown in Figure 6.13(b) illustrates the volatility of the trading performance, with the results showing similar findings to the profit, with the longer retraining intervals (less frequent) having lower drawdowns.

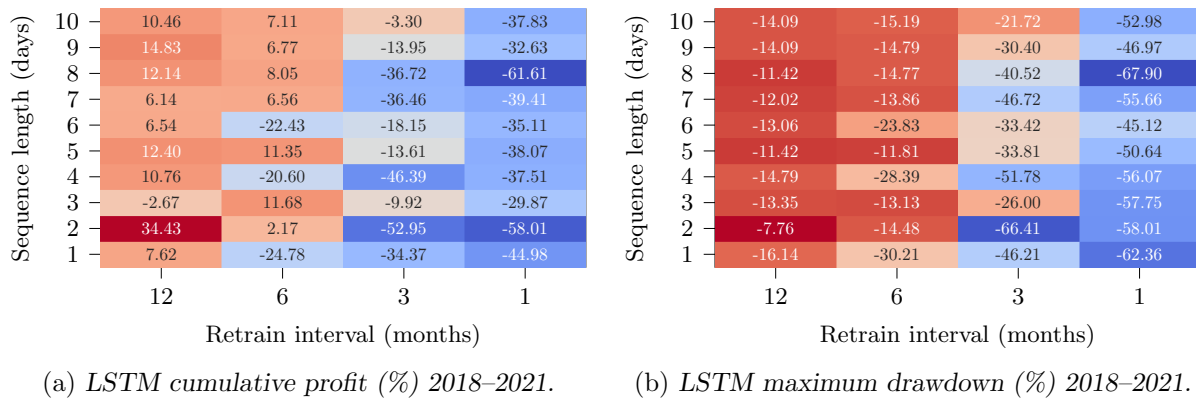


FIGURE 6.13: The heatmaps representing the effects of **retraining interval** and **sequence length** on the cumulative profit (left) and maximum drawdown (right) of the LSTM model in 2018–2021

Figure 6.14 illustrates the most profitable LSTM model, LSTM(12, 2), over the 2018 to 2021 test period. The model ends with a final cumulative profit of 34.34%, achieving more than double the profit of the naïve model at 16.27% and 33.74% higher than the 0.6% of the ARIMA.

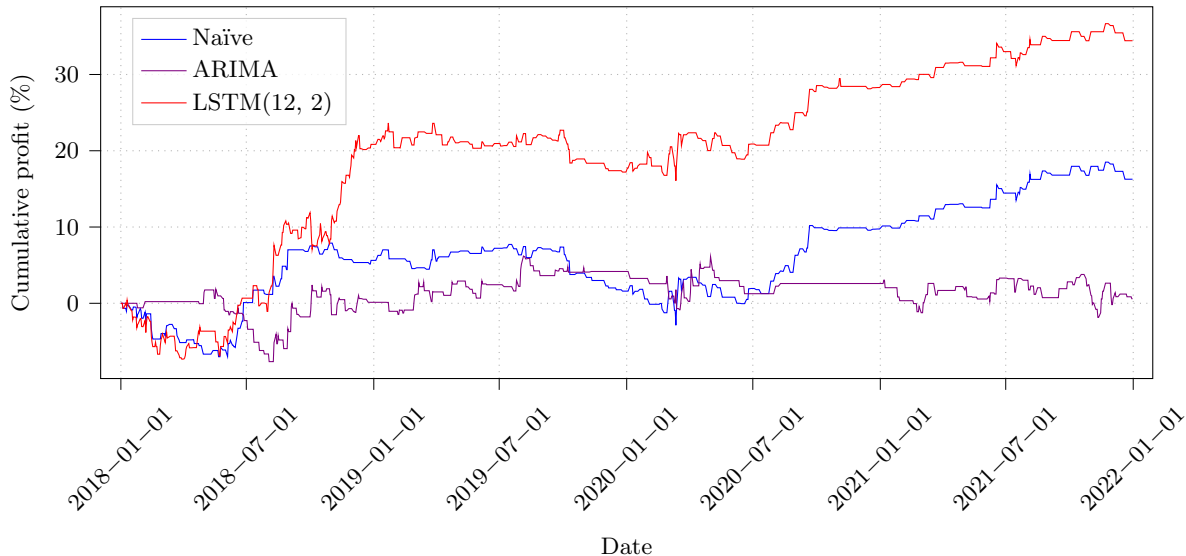


FIGURE 6.14: The results of the highest cumulative profit of the LSTM(12, 2) in 2018–2021.

The trading performance is further analysed through the profitable trades percentage and the number of trades executed metric, given in Figures 6.15(a). These metrics give insight into how profitable each trade is and how many trades the models make; therefore, how often the LSTM model predicts a change in price higher than the spread. As expected, the LSTM(12, 2) had the highest profitable trades percentage, achieving an impressive 51.0%, beating both the naïve and ARIMA models. The 1-month retrain interval had a better average profitable trades percentage in Table 6.4 at 45.02% than the 6-month and 3-month with 43.41% and 43.98%, respectively.

The higher profitable trades percentage is a surprise since the 1-month retrain interval had the worst average profit seen in Table 6.4. The number of trades executed in Figure 6.15(b) shows that the shorter retraining intervals trading more often, meaning the predicted closing price movement is larger than the lower retrain intervals to trigger the trading strategy (trade if absolute movement is greater than the spread). This indicates that the more frequent retrain intervals are more volatile than the higher ones. Since more trades are executed at a losing profitable trades percentage below 50%, the model cannot negate losing trades frequently enough to make a profit and thus results in large accumulated losses seen in Figure 6.13(a).

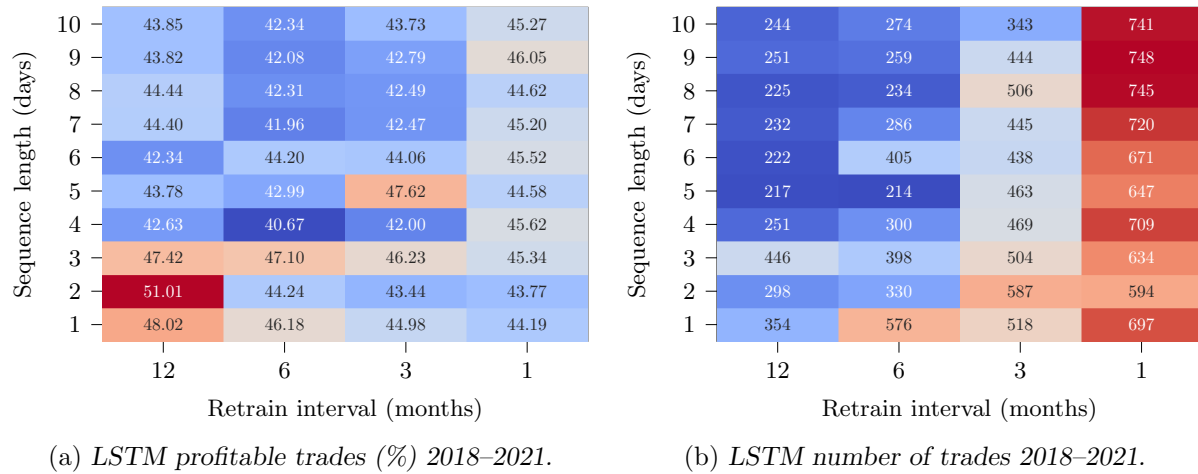


FIGURE 6.15: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' profitable trades percentage (left) and the number of trades (right) for the **2018–2021** test set.

Table 6.4 contains a summary of the different retrain intervals and sequence lengths tested and the averages across different metrics. As mentioned, the 12-month interval outperformed all other parameter values tested regarding MSE, MAPE, cumulative profit, and maximum drawdown and achieved the second-best profitable trades percentage. The LSTM model favoured higher sequence lengths from the summary in Table 6.4 with the sequence length of 9, resulting in the best MSE of 0.02156 for all the sequence lengths. These results suggest the LSTM models prefer more information since the higher retrain intervals contain more training data on which the LSTM model is trained. For example, for the 2018 test set, consistent with a 70/15/15 training/validation/test split of the data, a 12-month retrain interval will use 5 years of training data, 1 year of validation data (for early stopping) and 1 year of testing data. The 1-month training interval only uses 5 months of data to train, 1 month to validate, and 1 month to test, thus containing only 8.33% of the number of data points the 12-month interval LSTM model uses.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0212	0.76246	11.26	-12.81	45.17	274
	6	0.02131	0.76549	-1.41	-18.05	43.41	328
	3	0.02187	0.77329	-26.58	-39.70	43.98	472
	1	0.02244	0.78545	-41.50	-55.35	45.02	691
Seq len	1	0.02195	0.77327	-24.13	-38.73	45.84	536
	2	0.02188	0.77288	-18.59	-36.67	45.62	452
	3	0.02186	0.77299	-7.69	-27.56	46.52	496
	4	0.02171	0.77208	-23.44	-37.76	42.73	432
	5	0.02157	0.76967	-6.98	-26.92	44.74	385
	6	0.02157	0.76951	-17.29	-28.86	44.03	434
	7	0.02167	0.77213	-15.79	-32.07	43.51	421
	8	0.02168	0.77168	-19.53	-33.65	43.47	428
	9	0.02156	0.77084	-6.24	-26.56	43.69	426
	10	0.02161	0.77168	-5.89	-25.99	43.80	400

TABLE 6.4: The **LSTM** models' aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

There are opposing conclusions when comparing the effects of the **retraining interval** for the LSTM model in this study to the findings of Pongsena *et al.* [54]. They found an 11.14% improvement in binary classification accuracy from the 1-month retraining intervals over the 12-month intervals. In contrast, this study found that the 12-month retraining intervals improved the MSE by 5.53% over the 1-month intervals. Opposing results could be due to the differences in supervised learning techniques since the authors performed classification and this study used regression. However, considering the threshold accuracies, the average 12-month retraining interval LSTM models still had higher accuracies. The best sequence length obtained by Pongsena *et al.* [54] was 9–12 days. These results were consistent in this study, with the lowest MSE coming from the higher sequence lengths, with 9 days being the best. The discrepancies between the two studies could also be attributed to different hyperparameter configurations regarding the architecture of the LSTM models and the different data sets tested.

Test set 2018

The 2018 testing period was responsible for the LSTM(12, 2) model having superior performance in terms of MSE and profit at the end of the four years. This model achieved a profit of 20.84% for 2018, outperforming all other models by 11.42%. The MSE was also much lower at 0.01926, which, on average, was 1.18% lower than any other model. Figure 6.16 contains heatmaps of the LSTM(12, 2) performance regarding MSE and MAPE, standing out for the 2018 testing period.

Test set 2019

The period with the most price reversals and smallest range of prices, with minimal trends, was the 2019 period. The period had the best accuracy in terms of MAPE shown in Figure 6.17(a), with the 6-month retrain interval achieving the best MAPE. Conversely, the period also contained the worst cumulative profit, seen in Figure 6.17(b). The LSTM(6, 3) had the highest cumulative profit of only 2.8%. The frequent reversals and lack of trend enabled the LSTM models to predict values within smaller ranges but failed to accumulate continuous profitable trades.

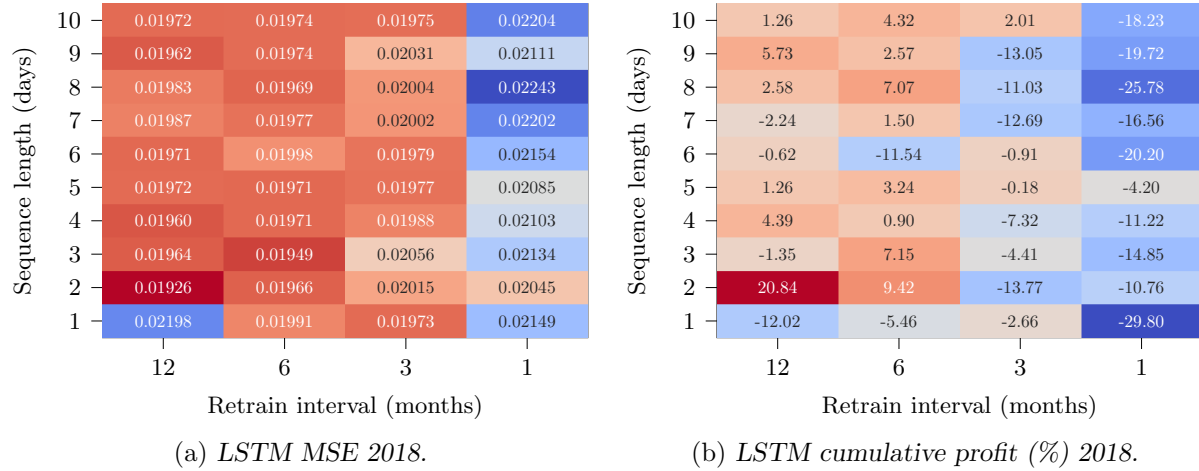


FIGURE 6.16: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' MSE (left) and the cumulative profit (right) for the **2018** test set.

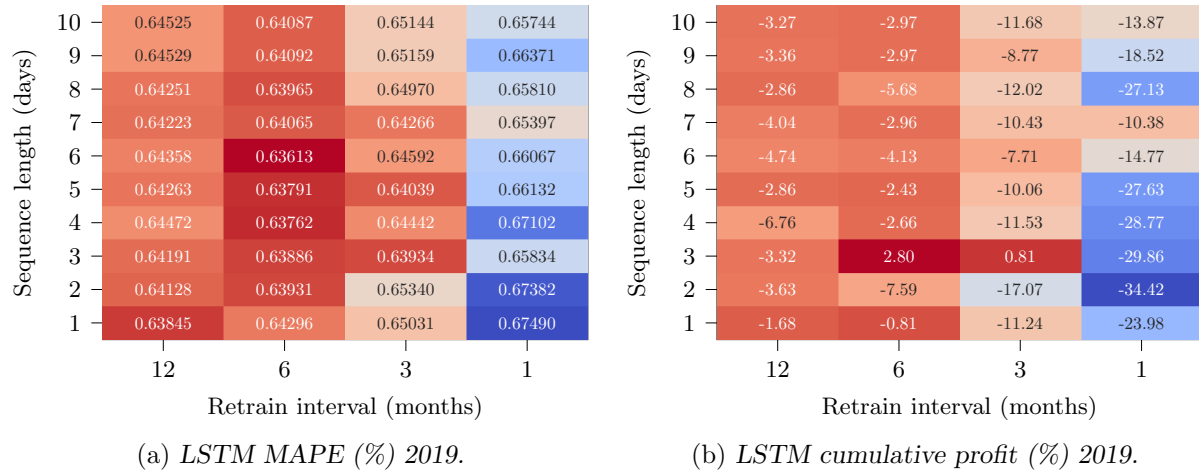


FIGURE 6.17: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' MAPE (left) and the cumulative profit (right) for the **2019** test set.

Test set 2020

In 2020, the market contained high volatility and rapid price movements within the first four months. This led to the 2020 period having the highest average MSE. Even with the worst accuracy metrics in the year, the models could still make profits, with the 1-month retraining interval having higher profits seen in Figure 6.18(a).

Interestingly, the 1-month retrain interval had the highest average profitable trades percentage with 51.33%, which was 5.5% higher than the following best average profitable trades percentage from the 12-month retraining interval of 45.82. The differences for the profitable trades percentage can be seen in Figure 6.18(b), with the most profitable model LSTM(1, 9) recording a profitable trades percentage of 56.59%.

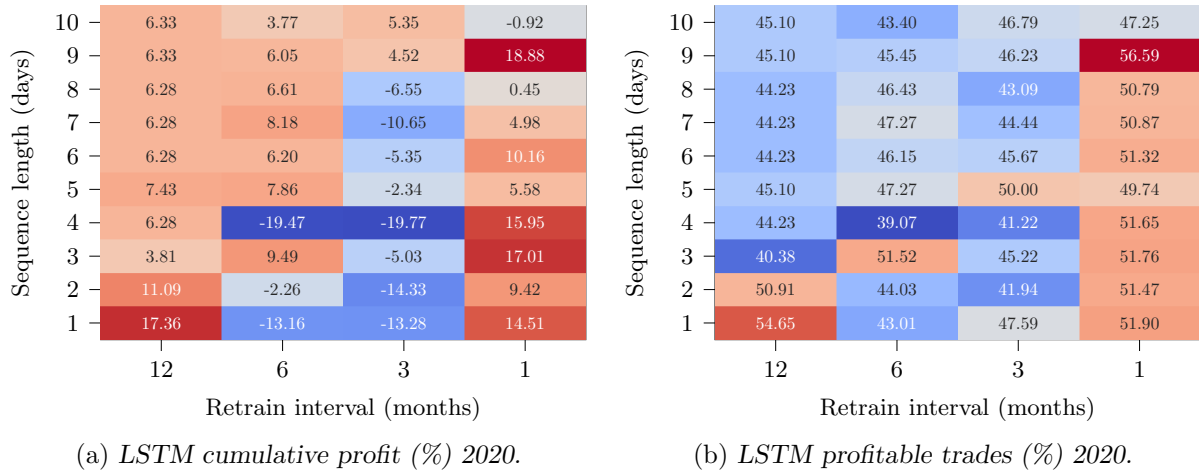


FIGURE 6.18: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **LSTM** models' cumulative profit (left) and the profitable trades percentage (right) for the **2020** test set.

This period coincided with five different LSTM models achieving a profit of over 14%, a respectable return and just under double that of the naïve forecast, with 4 out of the 5 coming from the 1-month retrain interval. These models were the LSTM(12, 1), LSTM(1, 1), LSTM(1, 3), LSTM(1, 4) and LSTM(1, 9) and their cumulative profits for 2020 are given in Figure 6.19. The 1-month retrain interval appears to exploit continuous trends as soon as the train, validation, and test set periods contain the same patterns. However, the 12-month retrain interval achieved similar profits with a smaller drawdown and is preferred overall since it is less risky.

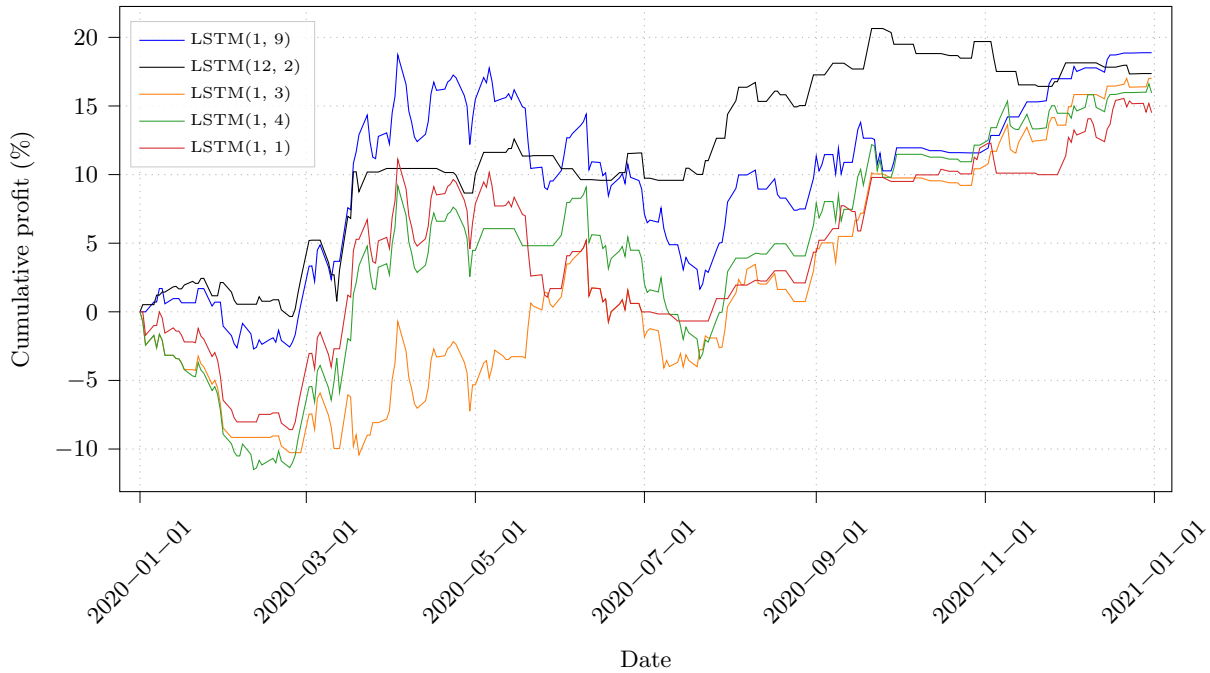


FIGURE 6.19: The cumulative profit for different combinations of retrain interval and sequence length of the **LSTM** model for the **2020** test set to illustrate the different volatilities of the combinations. Notably, the 12-month retrain interval has the lowest maximum drawdown.

Test set 2021

The 2021 year was consistent with the aggregated results of the LSTM model. The period again saw improvements in accuracy metrics over the longer retrain intervals of 12 months and 6 months. The results for this period are available in section §B.1, along with results for all testing periods for the LSTM model.

6.7 Chapter summary

In this chapter, the LSTM neural network was introduced, accompanied by a discussion of fundamental concepts of ANNs in §6.1 and RNNs in §6.2. The LSTM neural network's training algorithm and concepts of specialized gates pertaining to memory retention were elaborated on in §6.3. Various hyperparameters concerning LSTM models and the results from calibrating each of these parameters using Bayesian optimization were illustrated in §6.4 and §6.5, respectively. Finally, the chapter concluded with an evaluation of the overall results obtained through four years of testing data and the subsequent effects of the testing parameters on the LSTM model's performance in §6.6.

CHAPTER 7

Random forests

Contents

7.1	Decision trees	93
7.2	Random forests (ensemble learning)	94
7.3	Hyperparameters	96
7.4	Hyperparameter calibration results	97
7.5	Results	100
7.6	Chapter summary	108

This chapter contains an overview of the implemented random forest (RF) model. The chapter begins with an introduction to decision trees, the foundation of RFs. The architecture and training of RF models, with the results of the hyperparameter calibration, are then discussed. The chapter concludes with the results of the effects of **retraining interval** and **sequence length** on the 2018 to 2021 USD/ZAR test set.

7.1 Decision trees

Decision trees (DTs) are hierarchical structures commonly used for classification and regression problems [55]. As this study predicts continuous values instead of classes, the focus will be on regression-based DTs. These models can handle complex data without prerequisites such as linearity, stationarity, and constant scales, unlike ARIMA and LSTM [66]. They work by recursively partitioning data into smaller subsets until a ‘tree’ is formed, as seen in Figure 7.1.

The trees are ‘grown’ using the *classification and regression trees* (CART) algorithm. The algorithm splits a node (separates samples) using a single feature, k , and a threshold, q_k . The selection of which feature and threshold to choose is based on minimising the cost function, g , given as

$$g(k, q_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}, \quad (7.1)$$

where m is the total number of samples at the given node, $m_{\text{left}}/m_{\text{right}}$ are the number of samples in the left and right subsets, respectively and $\text{MSE}_{\text{left}}/\text{MSE}_{\text{right}}$ are the MSE values for the left and right subsets, respectively. This process is repeated recursively until either an error of zero or the **maximum depth** hyperparameter is reached.

Figure 7.1 contains a schematic representation of how an unlabelled instance transverses a trained DT to output a predicted value. The unlabelled instance is evaluated at node 1 using

feature x_1 with a threshold of 5. If the instance's x_1 feature is less than 5, then branch to the left (node 2), conversely branching to the right (node 3). Subsequently, further evaluations are performed down the tree until the unlabelled instance arrives at either of the four leaf nodes. The aggregated value of all the samples in the leaf node will be the predicted output from the model.

For example, if the unlabelled instance contains the feature vector $[x_1] = [6]$. The instance will transverse to node 3, since $6 > 5$ and then to leaf node 6, since $(6 < 8)$. The predicted label for the unlabelled instance will be 0.111, based on the average value of the 110 samples that fall into node 6.

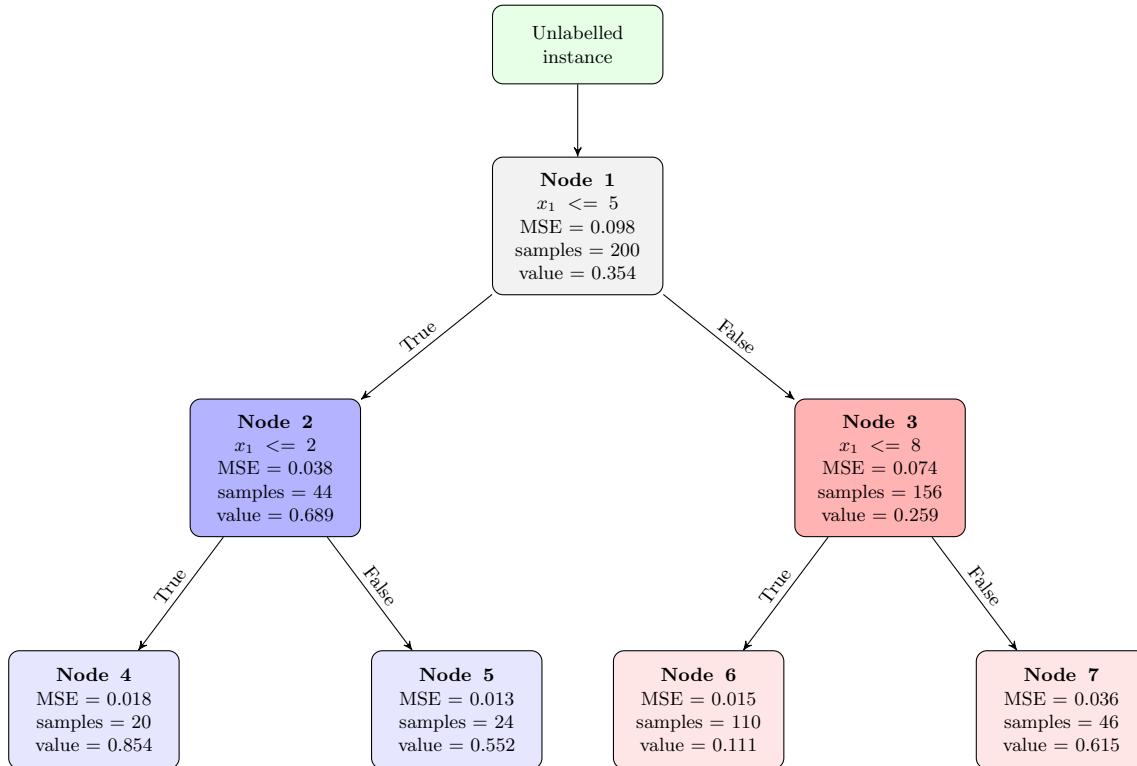


FIGURE 7.1: A schematic representation of a single decision tree (regression) and how splits are based on the feature x_1 and the various threshold values, q_1 . The decision tree's output is the aggregated value of all the samples in a leaf node.

The CART algorithm is greedy and thus selects the best splits at the higher levels but fails to consider the impact at the lower splits. These greedy algorithms produce 'reasonably good' solutions but often fail to converge, with the advantage of faster production times and the ability to handle big data sets [24]. DTs are also non-parametric models since all the parameter values are set before training commences and are prone to overfitting data [66]. Therefore, the degree of freedom controlled by the hyperparameters should be restricted before fitting data to prevent overfitting.

7.2 Random forests (ensemble learning)

The idea behind ensemble learning is to combine multiple 'weak' models to form a single 'strong' model. The combination of models removes potential biases and irregularities, such as overfitting, from the individual models. Thus, the aggregated model should have better predictions

that generalise well. Leo Breiman [11] introduced the random forest algorithm in 2001, which constructs multiple DTs and merges their outputs into a single prediction via averaging for regression.

Diversifying the DTs in a RF model is essential. Otherwise, each tree will be identical, and the outputs for a single tree will equal the outputs for all the aggregated trees. Two approaches are available to introduce randomness into the RF training process: bagging and pasting.

Bagging or bootstrap aggregating refers to sampling with replacement. This means the data set is split into random samples called bootstraps, with each split randomly chosen from the entire data set (with replacement). Thus, data points can overlap between the different splits. Each DT is now trained on a random bootstrap, which decreases the correlation among trees.

The other approach, *pasting*, refers to sampling without replacement. Like bagging, the data set is partitioned into random samples; however, data points can not overlap between splits. This means that each time a split is made, the data points are removed from the data set and cannot be chosen again.

The process of training a RF and how multiple DTs are combined on various subsets of the data through bagging or pasting and then aggregated together to form a prediction value is illustrated in Figure 7.2. The randomness of the DTs enables the RF to overcome the overfitting problem experienced by individual DTs by aggregating different decision boundaries. The RF's bias is similar to that of a single DT, meaning they tend to make the same number of errors on the training set. However, the variance is less, meaning the decision boundary generalises better [24].

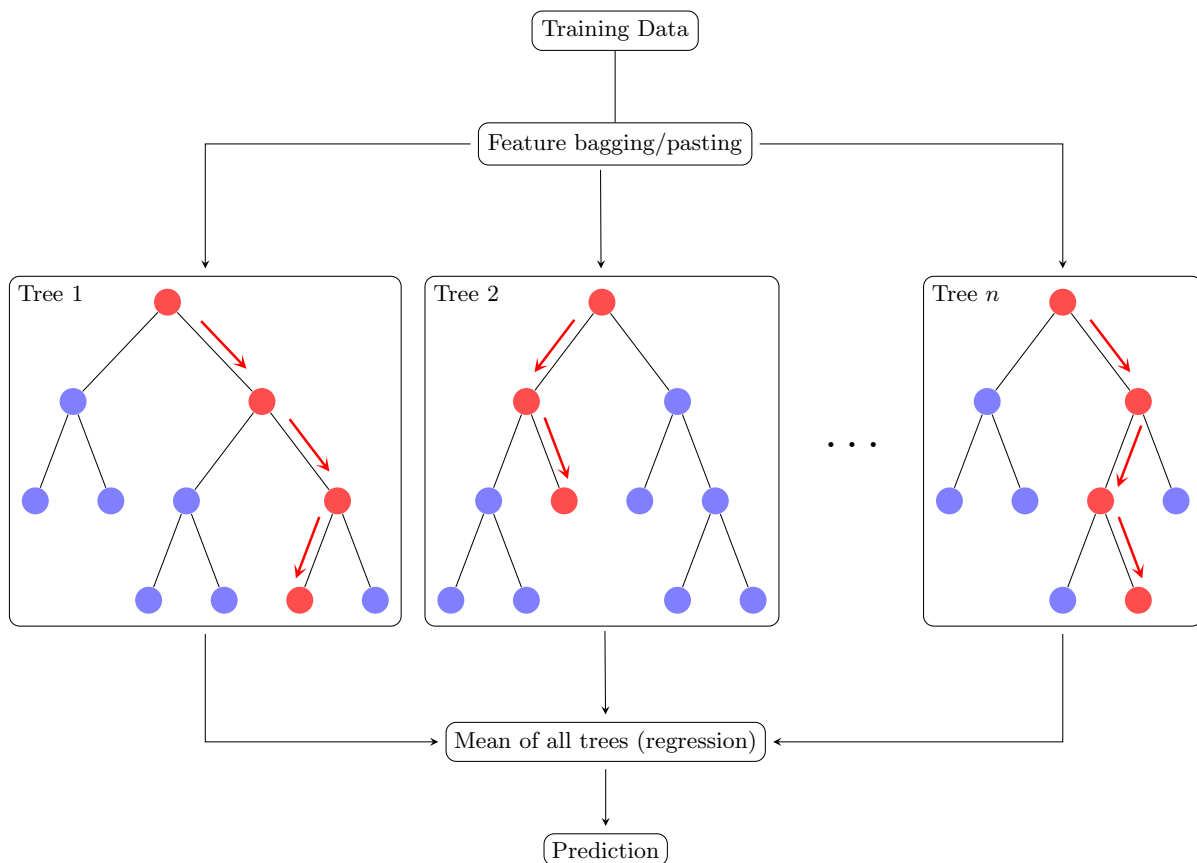


FIGURE 7.2: A schematic representation of the training process of random forests.

7.3 Hyperparameters

The architecture of a RF involves configuring the DTs within the model. The various hyperparameters used to configure the DTs are discussed in this section, highlighting their effects on the RF and the range that will be considered during hyperparameter calibration. Most of these parameters are to regularise the model *i.e.*, prevent overfitting and improve the robustness.

Number of trees

This refers to the total number of DTs utilised by the RF model. Generally, increasing the number of trees improves RF's performance and stability [55]. However, the increased number of DTs increases complexity and training times, with diminishing returns eventually reached after a certain number of trees. The number of DTs considered ranges from 1 to 1000.

Maximum depth of a tree

The maximum depth determines the number of nodes or levels in the DT. The deeper the tree grows, the more splits are made between the samples and the risk of overfitting increases. The maximum depth hyperparameter trades off between fitting the data set and generalisability and a depth between 1 and 200 is considered.

Maximum features

Only a random subset of features is considered for splitting samples, controlled by the maximum features hyperparameter. The maximum feature, given as a percentage, adds to the randomness and diversity of DTs within the RF [55]. Lower percentages will consider fewer features and result in less correlated trees, with higher percentages considering more features. At the extreme, max features set to 100% means all features are considered for each node to split the samples. The percentage of random features to consider when splitting samples is between 10% and 99%.

Minimum number of samples per split

This parameter enforces the minimum number of samples a node requires to perform a split while constructing a DT. It prevents isolating too few samples in nodes, which would have caused the model to overfit and generalise poorly. The range considered is between 2 and 50 samples per split.

Minimum number of samples per leaf

The leaf nodes in a DT are responsible for the final predictions and are the last nodes in the tree. Since the prediction of each tree is the mean of all samples in a leaf node (regression), too few samples in a leaf node will be biased to these samples. Once again, the minimum number of samples in each leaf node reduces the risk of overfitting, and the considered ranges are between 2 and 50 samples in a leaf node.

Bootstrap (bagging or pasting)

The training algorithm is adjustable, specifically whether to use sampling with or without replacement. The bagging and pasting sampling techniques, discussed in §7.2, are experimented with, and their effects on the RF models are evaluated. Notably, the bagging method is usually preferred when limited data is available [24].

7.4 Hyperparameter calibration results

The RF hyperparameters in §7.3 were calibrated using Bayesian optimisation with manual k -fold cross-validation discussed in §4.4. The hyperparameters are related to the individual DTs and their corresponding characteristics. Table 7.1 contains the search space used and the corresponding best values found through 150 iterations of Bayesian optimisation. The calibration was faster than the LSTM model, with a running time of 6 hours and 8 minutes. The shorter time is due to the simplicity of the training algorithm and the lack of weights that need adjusting. Therefore, building and training multiple DTs are computationally cheaper than an LSTM model with over 100 000 weights that need calibrating through the BPTT algorithm. The results for all iterations of the RF hyperparameter calibration can be seen in §A.2.

Hyperparameter	Abbreviation	Search space	Best value
number of trees	n.tree	[1, 2, 3, ..., 1000]	808
maximum features	m.feats	[0.1, ..., 0.99]	0.1579
maximum depth	m.d	[1, 2, 3, ..., 200]	34
minimum number of samples per split	m.s.s	[2, 3, 4, ..., 50]	48
minimum number of samples per leaf	m.s.l	[2, 3, 4, ..., 50]	47
bootstrap (with or without replacement)	boot	[true, false]	true

TABLE 7.1: The RF hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.

The RF models were the most stable of the three ML models, with no outliers observed during the hyperparameter calibration. The average MSE was 0.022425, with a standard deviation of 0.00122. Therefore, producing ‘reasonably good’ accuracy with the top 5 iterations given in Table 7.2. The average MSE of these 5 iterations was 0.02145, which is 5.25% higher than that of the top 5 LSTM iterations’ average MSE of 0.02038. The lower MSE values for the best iterations show the underlying data set’s complexity and suggest that more sophisticated models, such as LSTM, are required to learn the intricate patterns and map the features to labels.

N	Random forest parameters						Test set		Train set		Time (mins)
	n.tree	m.feats	m.d	m.s.s	m.s.l	boot	MSE	Profit	MSE	Profit	
90	808	0.1579	34	48	47	true	0.02143	-3.42	0.01789	18.09	2.99
105	340	0.1464	129	21	47	true	0.02144	-3.47	0.01790	18.23	2.35
35	161	0.1903	2	2	33	true	0.02145	-3.55	0.01790	15.35	1.67
100	490	0.1816	123	3	49	true	0.02145	-3.47	0.01791	17.35	2.78
141	260	0.1334	103	32	45	true	0.02146	-3.61	0.01790	17.13	2.10

TABLE 7.2: The top five hyperparameter calibration iterations for the RF model found through 150 iterations of Bayesian optimisation.

The only categorical hyperparameter is **bootstrap**, set to either true or false, deciding whether

the bagging or pasting approach is used for training. Therefore, a true value indicates the use of sampling with replacement (bagging) and a false value without replacement (pasting). The bagging approach outperformed pasting, with 49 out of the top 50 best MSE iterations using bagging. The disparity is confirmed through a box and whisker plot for the `bootstrap` hyperparameter on the MSE and cumulative profit, in Figures 7.3(a) and 7.3(b), respectively. The pasting technique (false) has higher interquartile ranges for both plots and contains more outliers. Notably, both bootstrap techniques were equally tested as there consist 74 and 76 iterations for bagging (true) and pasting (false), respectively. Therefore, both have had an approximate 50% split on the optimisation, and the results are not biased towards any particular value.

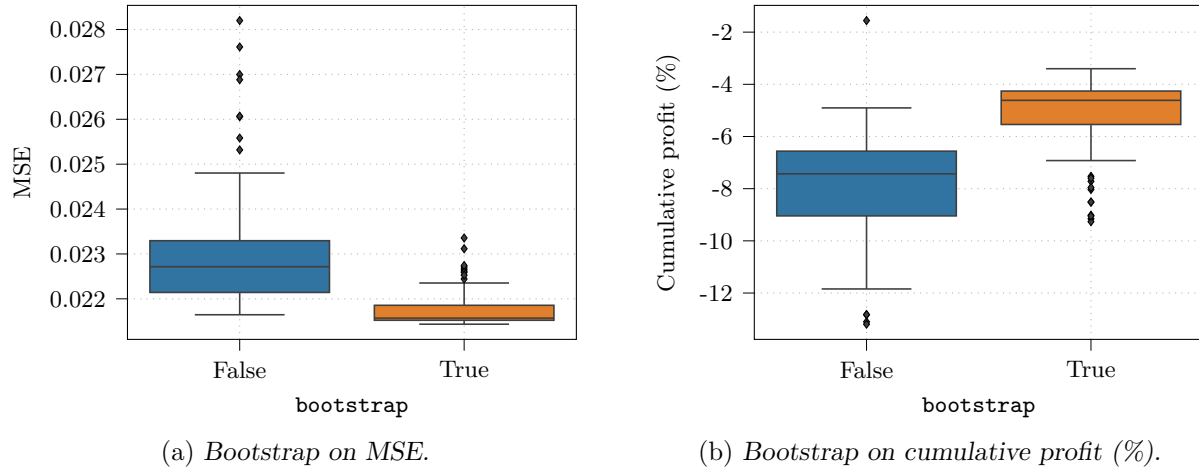
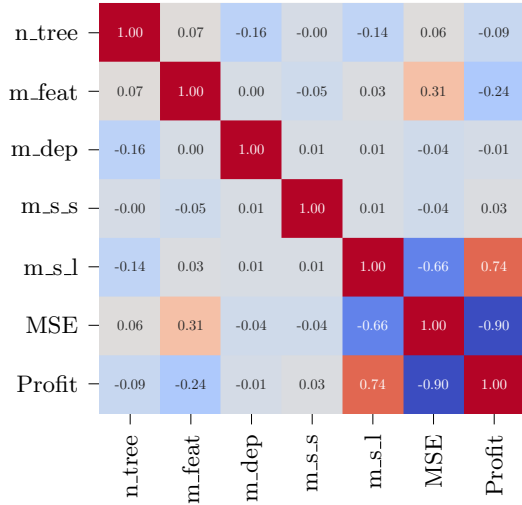


FIGURE 7.3: The box and whisker plots showing the distribution of the MSE (left) and cumulative profit (right) found for the `bootstrap` hyperparameter's categorical values.

The Pearson correlation matrices in Figure 7.4 contain the correlation between the numerical hyperparameters and the metrics MSE and profit. Figure 7.4(a) contains the correlation for all 150 iterations. The `minimum number of samples per split` has the highest correlation of -0.66 , indicating a strong negative correlation between these two variables. This is more than double that of the second most correlated hyperparameter `max feature` at 0.31 .

The correlations are also investigated after the false bootstrap values were removed (only bagging), in Figure 7.4(b). This removal inspects if the poor MSE from the pasting approach skewed the correlations. The correlation strengthens for the `minimum number of samples per split` to a near linear negative correlation of -0.93 , suggesting the model is more accurate when more samples are used to split the nodes. The `max feature` decreases from 0.3 to 0.05 , and the `number of trees` contains a higher positive correlation to the MSE of 0.3 from 0.06 . This higher value suggests that more DTs are likely overfitting the data set.

The scatter plots for the two highest correlated hyperparameters `minimum number of samples per split` and `max features` against the MSE and profit are given in Figure 7.5. Additionally, the iterations with a bootstrap value of true (bagging) are highlighted in blue, and a value of false (pasting) is highlighted in red. The separation of blue and red dots for all four plots in Figure 7.5 reinforces the superior performance of the bagging technique. The performance is likely attributed to the small number of data points available in the lower retraining intervals for the cross-validation, and sampling without replacement (pasting) was not feasible for the smaller data sets, thus building shallow trees that underfit the data set.



(a) All 150 iterations.

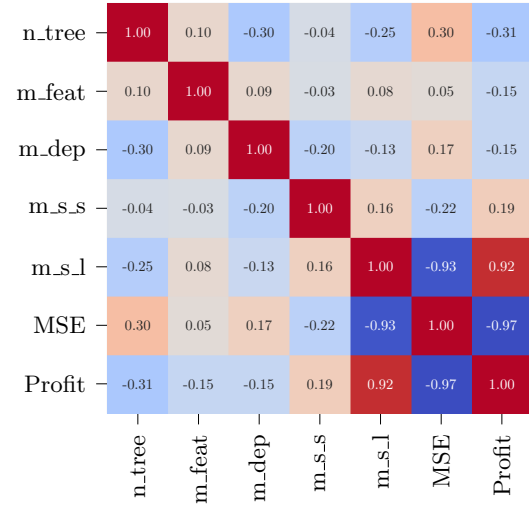
(b) Subset of iterations (*bootstrap* = true).

FIGURE 7.4: The Pearson correlations matrices containing the relationship between the hyperparameters and testing metrics for all iterations of Bayesian optimisation (left) and a subset only containing iterations where *bootstrap* = true, therefore bagging (right).

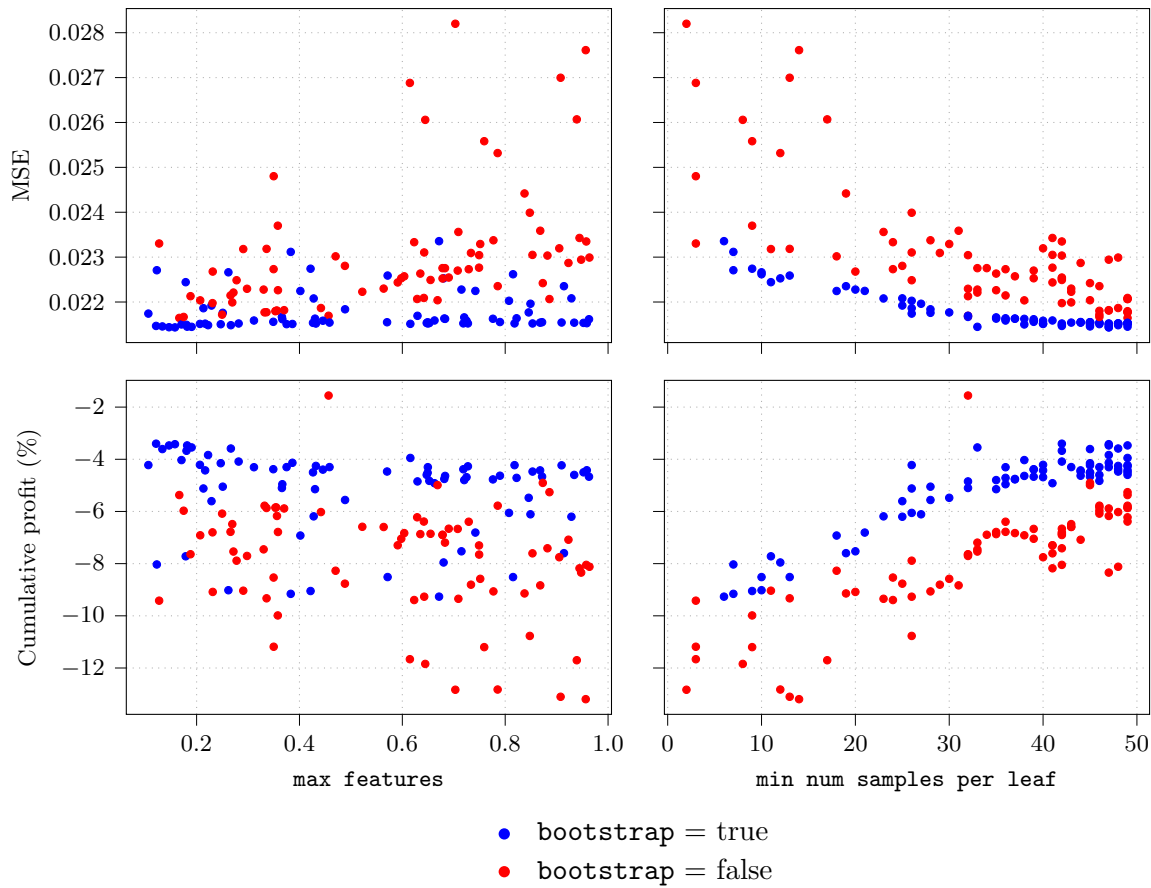


FIGURE 7.5: The scatter plots of the hyperparameters *max features* and *minimum number of samples per leaf*, against the MSE and cumulative profit from the RF models' hyperparameter calibration. The iterations are coloured depending on the value of *bootstrap*, with true (bagging) in blue and false (pasting) in red.

7.5 Results

The RF models with the best hyperparameter configuration in Table 7.1 completed the grid search of testing parameters in 10 minutes and 50 seconds. Similarly to the LSTM model, to distinguish between the different RF models for various combinations of **retrain interval** and **sequence length**, the notation RF(r , s) is used, with r being the retrain interval and s being the sequence length. The results for the RF models share similarities with the LSTM models' results, with the same convention of longer retraining intervals (less frequent updates) performing better in accuracy and profit-related metrics than the shorter retraining intervals (more frequent updates). The overview of the entire testing period of the USD/ZAR from 2018 to 2021, along with interesting findings for specific years, are discussed.

All test years combined

The **retraining interval** produced a more prominent effect on the accuracy than the LSTM model with a larger separation of MSE values between the different intervals in Figure 7.6(a). The range of MSE values is between the lowest MSE of 0.02096 of the RF(12, 3) to the highest of 0.02438 from the RF(1, 5) model, indicating a 14.08% difference. This high difference confirms the influence of the retraining interval, with the 12-month retrain interval MSE values being 1.54%, 6.45% and 10.71% lower on average than the 6-month, 3-month and 1-month intervals, respectively. The best MSE value from the RF(12, 3), 0.02096, narrowly outperforms the LSTM's best MSE, 0.02098, by 0.1%. The RF(12, 3), thus, also had a lower MSE than the naïve by 0.71% and the ARIMA by 0.57%.

The number of features used to make predictions of the RF model had a minimal impact compared to the number of times the models were retrained. The lower **sequence lengths**, especially for one previous day's of features, had better accuracy with the best MAPE coming from the RF(6, 1) seen in Figure 7.6(b).

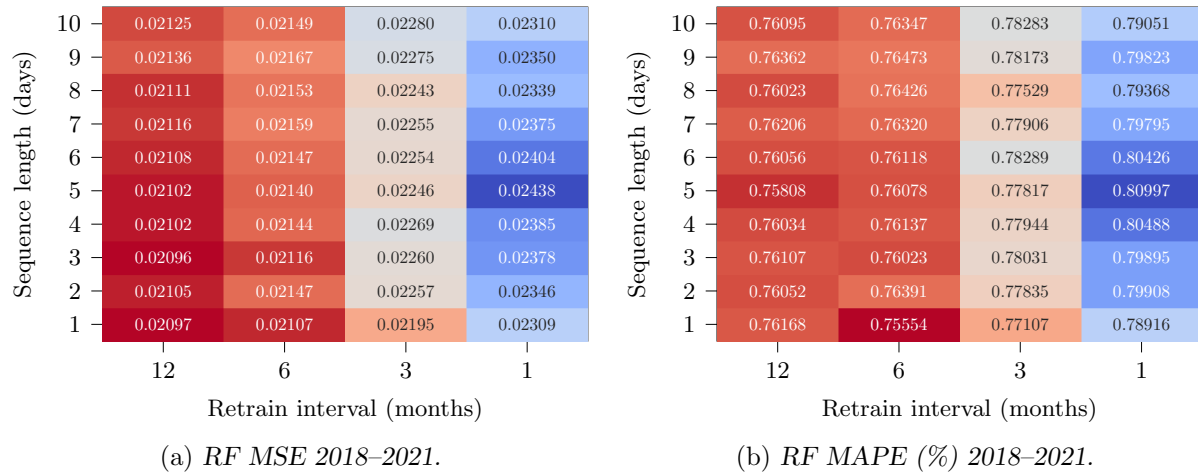


FIGURE 7.6: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' MSE (left) and MAPE (right), for the **2018–2021** test set.

Figure 7.7 contains the four lowest RF models concerning the MSE for the four different retraining intervals, RF(12, 3), RF(6, 1), RF(3, 1) and RF(1, 1). The RF(12, 3) and RF(6, 1)

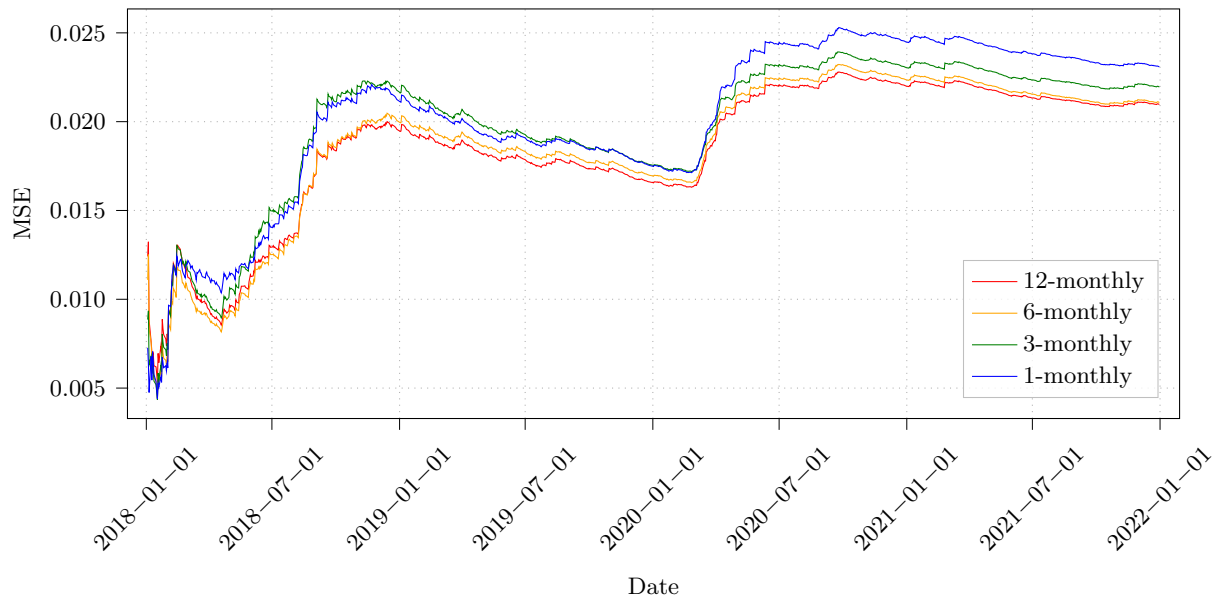


FIGURE 7.7: The MSE through time of the best performing **RF** models for each *retrain interval* on the **2018–2021** test set.

are consistently lower than the RF(3, 1) and RF(1, 1) throughout the entire testing period. In 2018, the RF(6, 1) had the lowest MSE values from 2018-03-01 to 2018-08-24 until the RF(12, 3) caught up and outperformed all other models for the remainder of the testing period. The more timesteps, the better the convergence of the 12-month model became, with the model less susceptible to price fluctuations as the MSE shows a smoother line through time. The RF(1, 1) is notably poor from 2020-03-14, when the USD/ZAR closing prices fell from the maximum of ZAR 19.09 per USD, showing a large gap from the RF models with lower intervals. The 12 and 6 month interval models converge to similar MSE through time towards the end of 2021, with the RF(12, 3) marginally lower.

RFs have an attribute called *feature importance*, where each model can rank the features. Each feature is given a total percentage contribution (importance) summing up to 100. Investigating the feature importance list for each model is not feasible, given that over 3 000 models were trained. However, a summarised feature importance ranking for the 12-month frequency for the 2020 testing set is given in Table 7.3. The importance of the features can show if the RF is using the increased feature set (more days of data) for the higher sequence lengths or if the models are more biased towards recent observations.

The average true range (ATR) is the most important feature for the RF models, accounting for seven different **sequence lengths**. This volatility indicator captures the average volatility over 14 days and suggests that the model weighs the magnitude of price movements more than any other indicator. As the number of features increases due to the **sequence length** increasing, the weight of the best and second-best features decreases, thus suggesting that the more features the RF has, the more it will incorporate more features, each at a lower importance. Thus, it uses more information at a given time. The higher **sequence lengths** also appear to use lagged features, taking advantage of the increased feature set. For example, the 9-day **sequence length** has the 9-day lagged stochastic oscillator value as the most important at 2.14%, and the 8-day lagged RSI value at 1.72% as the second most important.

The threshold accuracies summarised for all four years of testing given in Figure 7.8 confirm that

Seq len	Feature (1 st)	Lag ($t-i$)	Score (%)	Feature (2 nd)	Lag ($t-i$)	Score (%)
1	ADX	1	8.60	ATR	1	7.56
2	ATR	2	5.94	Open	1	4.64
3	ATR	3	4.80	ATR	2	4.22
4	ATR	2	3.68	Open	4	3.19
5	ATR	4	2.86	ADX	2	2.56
6	ATR	5	2.57	SMA 21	3	2.49
7	ATR	5	2.19	High	7	2.12
8	High	7	1.94	ATR	3	1.77
9	Stochastic oscillators	9	2.14	RSI	8	1.72
10	ATR	4	2.10	Stochastic oscillators	3	1.69

TABLE 7.3: The top two important features considered by the **RF** model for the 12-month **retraining frequency** across different **sequence lengths** for the **2020** test set.

the longer retraining intervals are more stable and perform better than shorter ones. However, the 6-monthly retraining interval manages to beat the 12-monthly on average. This suggests the 12-month interval cannot keep up with the changes in the market, and the RF(12, s) models begin to decay since the underlying data set, USD/ZAR, is changing. The 6-month interval thus offers a better balance between long-term stability and short-term fluctuations in the market for smaller threshold values of R0.025 and R0.05 in Figures 7.8(a). The bigger thresholds of R0.075 and R0.1 in Figures 7.8(c) and 7.8(d), respectively, show a closer performance between the 12 and 6-month **retrain intervals**. The **sequence length** effect is negligible again; no clear conclusion was derived from these results.

Figure 7.9 contains a summary of the results of the threshold accuracies in Figure 7.8, showing the histogram of the aggregated **retraining intervals** across the different threshold accuracy values. The aggregated accuracies appear similar, with the biggest disparity coming from the R0.075 threshold accuracy, where the 12-month outperformed the 1-month by 2.19%. The threshold accuracies increase incrementally by 0.025 following a ratio of 2, 3 and 4 for each threshold based on the initial 0.025 threshold. The aggregated increase for all the **retraining intervals** accuracies of the different thresholds follow a ratio of 2.05, 3.15 and 4.01, respectively. This indicates that the percentage increase in accuracy follows a near-linear relationship to the percentage increase in the threshold value.

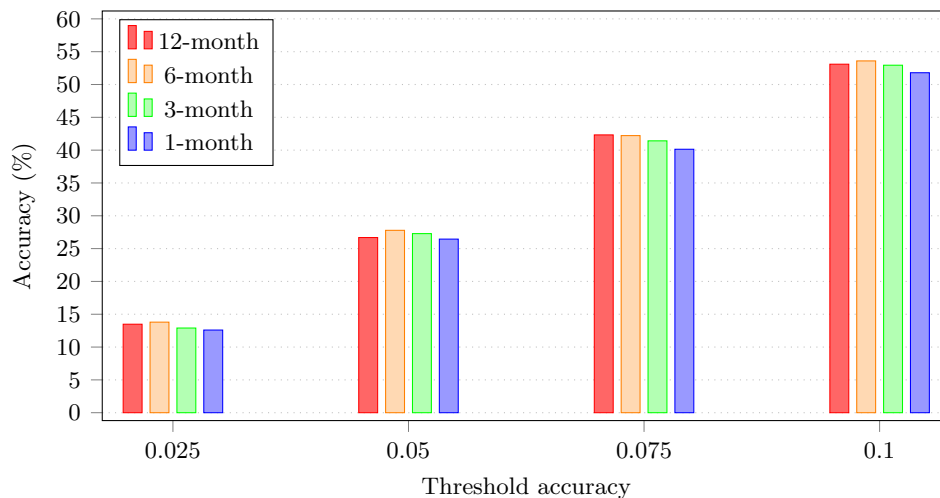


FIGURE 7.9: The comparison between the retraining intervals threshold accuracy for the **RF** models over the **2018–2021** test set.

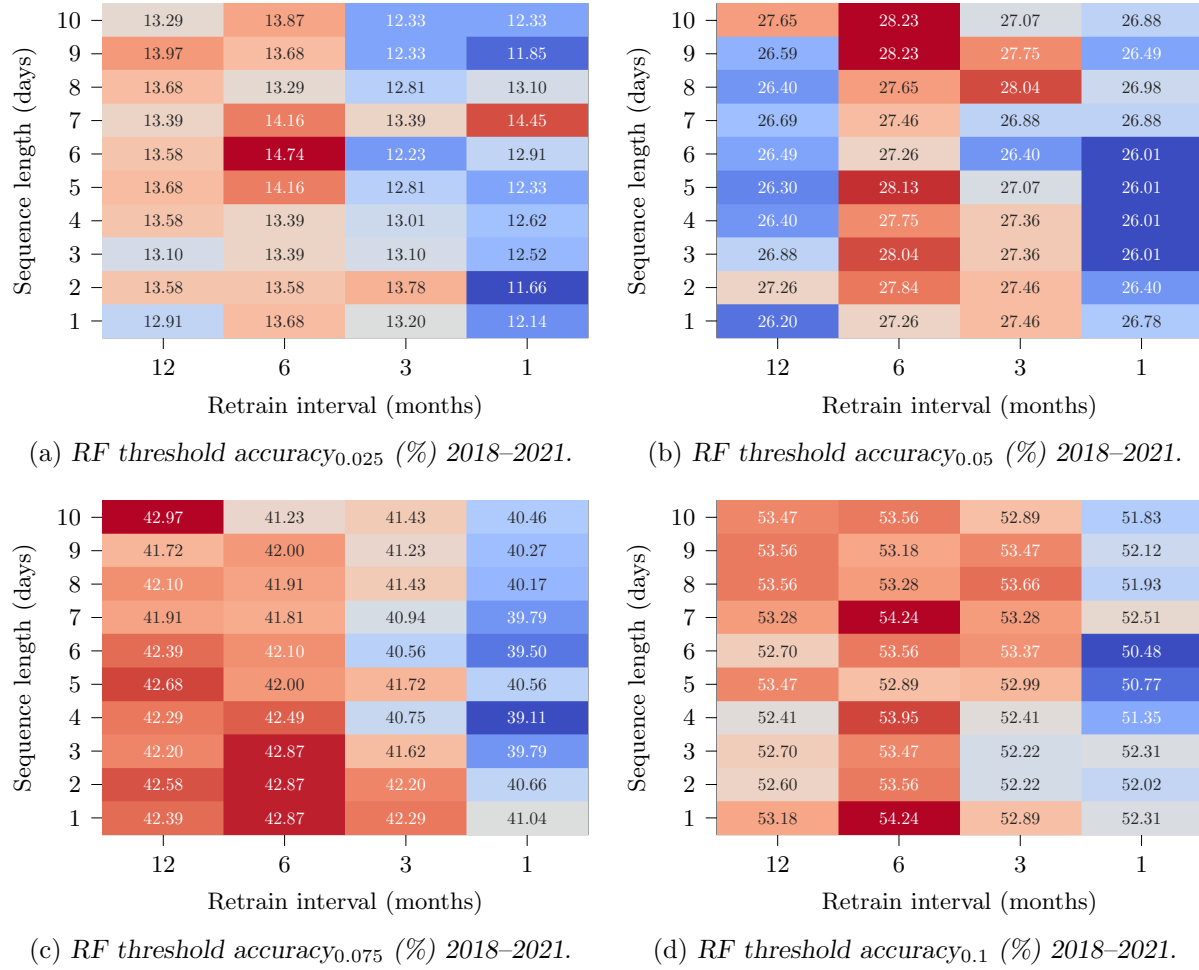


FIGURE 7.8: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' different threshold accuracies for the **2018–2021** test set.

The cumulative profit results in Figure 7.10(a) are not surprising as MSE and cumulative profit showed a -0.66 (all) and a -0.93 (bagging only) correlation from the hyperparameter calibration results in Figure 7.4. The RF models with higher MSE had the best cumulative profit, most of which came from the 12-month **retrain interval**. The highest cumulative profit came from the RF(12, 3), with a cumulative profit equating to 28.96%, although it is 5.47% lower than the LSTM models' best profit. However, the RF models are more consistent at making profitable returns for the 12-month **retrain interval**, resulting in an average cumulative profit across all sequence lengths of 19.8%, compared to the LSTM models' 11.26% average.

The maximum drawdown in Figure 7.10(b) illustrates the volatility of the RF models trading performance with larger drawdown notably for the 1-month intervals. The smallest drawdown is -11.27% from the RF(6, 3), which is 3.46% and 1.73% worse than the LSTM and ARIMA, respectively, but similar to the naïve of -10.77% . The average drawdown of the RF(1, s) and RF(3, s) models of -79.78% and -73.88% , respectively, serve as a reminder of the volatile nature of the foreign exchange market and the potential to lose large percentages of capital if basing trades off of poor predictive models.

The 6-month intervals had a higher percentage of profitable trades in Figures 7.11(a). Interestingly, no model obtained a profitable trades percentage above 50%, meaning there are more

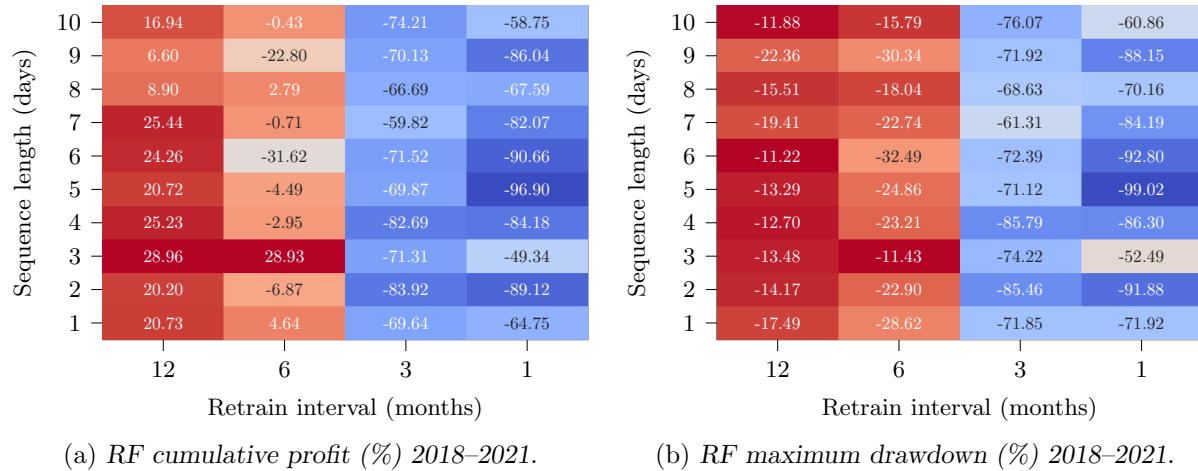


FIGURE 7.10: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' cumulative profit (left) and maximum drawdown (right) for the **2018–2021** testing set.

losing trades than winning trades for each model. However, the positive cumulative profits from the RF models in Figure 7.6(a) indicate that the models could negate the more frequent losing trades with profitable trades that carried more weight. The RF(6, 3) had the best profitable trades percentage of 49.63% but only the second-best profit. This model lost to the RF(12, 3), albeit only by 0.03%, but the RF(12, 3) only had a profitable trades percentage of 46.63%. Since the cumulative profits for these two models are similar, further investigation is needed to decipher which model had the better trading performance. The RF(6, 3) had a lower drawdown of 11.43% compared to the RF(12, 1) of -13.48% but had a lower average profit per trade of 0.0628% compared to the RF(12, 1)'s 0.075%.

The number of trades executed by the RF models, in Figure 7.11(b) are far higher than the LSTM model, with the fewest number of trades coming from the RF(12, 2), coinciding with the convention of longer retrain intervals making fewer predictions. This model executed 362 trees compared to the LSTM's lowest number of trades at 214. The RF model has a 37.01%, 42.23%, 59.31% and 71.98% of executing a trade (predicting a price change above the spread) at the 12, 6, 3, and 1 month **retrain intervals**, respectively.

Figure 7.12 contains the two most profitable RF models, RF(12, 2) and RF(6, 3), to analyse further their trading performance over the 2018 to 2021 test period. The RF(12, 3) ends with a final cumulative profit of 28.96%, achieving 12.69% higher than the cumulative profit of the naïve and 28.39% higher than the ARIMA. The cumulative profit of the RF(12, 2) is correlated to the USD/ZAR prices and seems to perform well on continuous trending periods. The cumulative profit performs worse when there are significant price reversals, for example, during 2020, when prices fell from the all-time high of ZAR 19.09 per USD.

The RF(6, 3) is included in the cumulative plot to showcase the increased frequency of trades from the 6-monthly model. The RF(6, 3) has a lower maximum drawdown compared to the RF(12, 3), which experienced this drawdown from 2019-08-20, losing 13.4% to 2020-03-26, where the RF(6, 3) drifted sideways. The RF(6, 3) cumulative plot, contradictory to having a smaller maximum drawdown, shows more frequent drawdowns than the RF(12, 3), suggesting it is riskier than the 12-month interval, notably for the 2021 period. Therefore, it is recommended to base a trading strategy on more than one risk measure to negate the possible biases introduced by a single indicator.

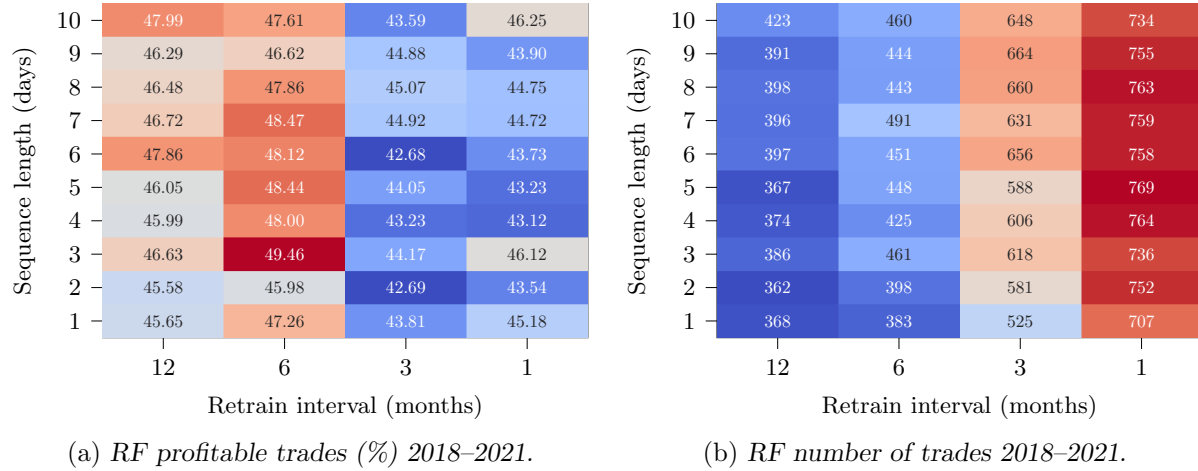


FIGURE 7.11: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' profitable trades percentage (left) and the number of trades (right) for the **2018–2021** test set.

Table 7.4 contains a summary of the different **retrain intervals** and **sequence lengths** tested and the averages across selected performance metrics. The 12-month interval outperformed all other parameter values tested regarding MSE, MAPE, cumulative profit, and maximum drawdown and achieved the second-best profitable trades percentage. The profit-related metrics in Table 7.4 are notably poor for the 3-month and 1-month intervals, suggesting these **retraining intervals** do not contain enough information to predict prices. This could be due to insufficient training samples in the data set to perform splits since the 3-month and 1-month data sets contain roughly 259 and 84 samples to train on, respectively. Therefore, the RF architecture from the hyperparameter calibration seemed to introduce a bias towards the more extended retrain intervals with 1037 and 520 samples for the RF(12, s) and RF(6 s) models, respectively. The result shows that the RF performs better with more data points from a highly regularised hyperparameter configuration.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0211	0.76091	19.80	−15.15	46.52	386
	6	0.02143	0.76187	−3.35	−23.04	47.78	440
	3	0.02253	0.77891	−71.98	−73.88	43.91	618
	1	0.02363	0.79867	−76.94	−79.78	44.45	750
Seq length	1	0.02177	0.76936	−27.25	−47.47	45.48	496
	2	0.02214	0.77546	−39.93	−53.60	44.45	523
	3	0.02212	0.77514	−15.69	−37.90	46.60	550
	4	0.02225	0.77651	−36.15	−52.00	45.09	542
	5	0.02232	0.77675	−37.64	−52.07	45.44	543
	6	0.02228	0.77722	−42.39	−52.23	45.60	566
	7	0.02226	0.77557	−29.29	−46.91	46.21	569
	8	0.02211	0.77337	−30.65	−43.09	46.04	566
	9	0.02232	0.77708	−43.09	−53.20	45.42	564
	10	0.02216	0.77444	−29.11	−41.15	46.36	566

TABLE 7.4: The **RF** models' aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

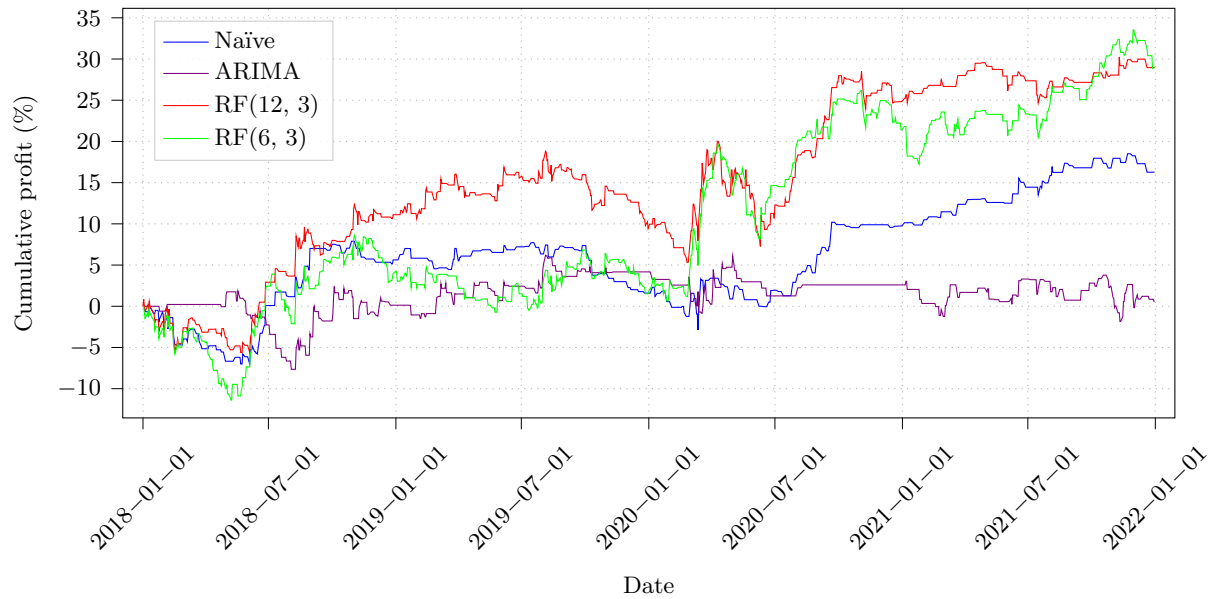


FIGURE 7.12: The results of the highest cumulative profit of the RF(12, 3) and second highest RF(6,3), compared to the naïve and ARIMA for the **2018–2021** testing set.

Test set 2018

The threshold accuracies observed at the R0.025 threshold in 2018, in Figure 7.13(a), showcased high accuracy levels. These levels were prominent on a subset of parameter combinations from the 1-month **retraining interval** with a sequence length of 6 to 8 days. The 2018 USD/ZAR prices drifted sideways for the year's first half, increasing volatility for the remainder of the year. The higher accuracy could suggest the 1-month models with higher sequence lengths adapted faster to the changing market patterns. The RF(1, 7) had the highest accuracy of 16.60%, which was 1.54% higher than naïve and 0.77% higher than ARIMA for the 2018 period.

Even though the 12-month interval RF models had lower threshold accuracies than the best RF(1, 6–8) models, the RF(12, s) exhibited positive returns for all the sequence lengths. The most profitable is the RF(12, 2) in Figure 7.13(b) at a cumulative profit in this period of 25.15%. This model could not maintain this cumulative profit and ended 2021 with only 20.20%, thus losing 4.95% of its total profits in the following three years.

Test set 2019

The 2019 period, with the most price reversals throughout the year, saw that the RF models retrained every quarter and had the lowest MSE in Figure 7.14(a) and the highest cumulative profit in Figure 7.14(b). The 3-month interval captured the different patterns in the market, suggesting that the 2019 period fluctuated between different market regimes every three months.

The performance across the RF(r , 1) models is investigated by plotting the MSE through time illustrated in Figure 7.15. The first 11 iterations have been clipped to an MSE of 0.016 to negate the influence of unstable MSE values due to a limited number of observations, thus keeping the scale of the figure smaller to distinguish between the different models. Interestingly, the RF(1, 1) outperformed the other retraining intervals from 2019-04-12 to 2019-06-12, coinciding with a change in trend in the USD/ZAR closing prices. The RF(3, 1), however, has the lowest MAPE

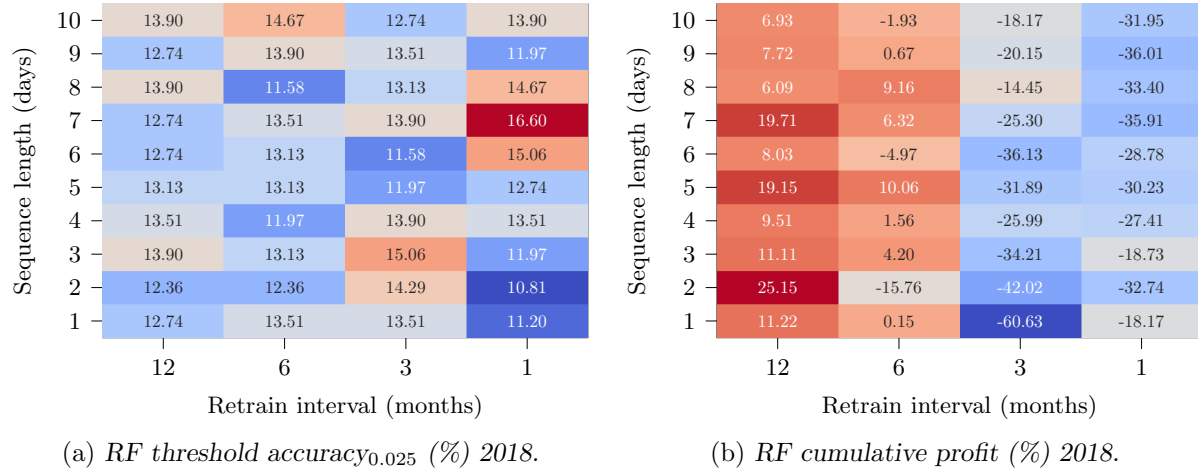


FIGURE 7.13: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' threshold accuracy (left) and the cumulative profit (right) for the **2018** test set.

for the majority of the 2019 testing period, with the RF(12, 1) marginally outperforming the RF(6, 1) and RF(1, 1). The MSE of all models follows the movement of the prices. Therefore, when prices increase, the MSE tend to increase since MSE is proportional to the scale of data.

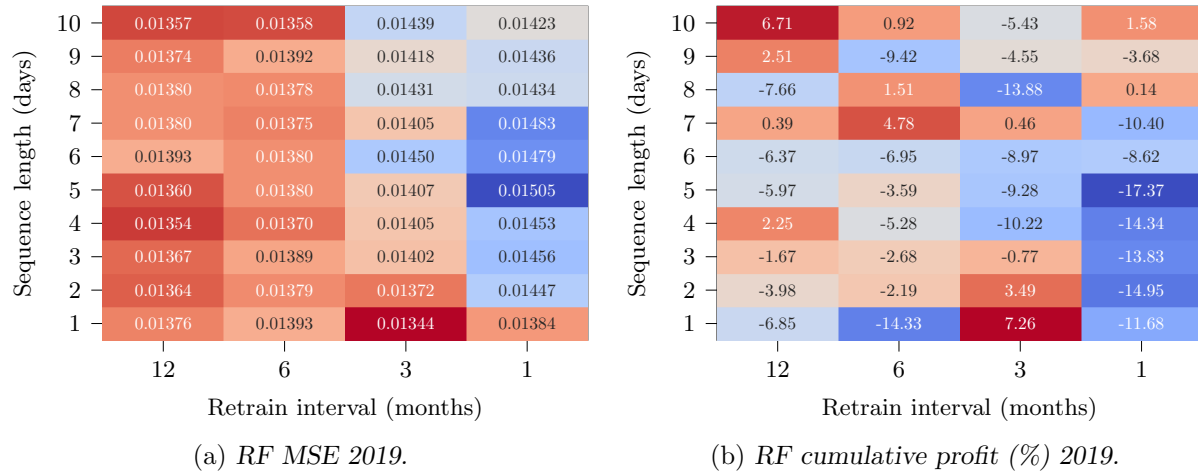


FIGURE 7.14: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **RF** models' MSE (left) and the cumulative profit (right) for the **2019** test set.

Test set 2021

The 2021 testing set saw the 6-month **retraining interval** outperforming all other retraining intervals, with the RF(6, 1) having the lowest MAPE of 0.70861% in Figure 7.16(a). This MAPE is 2.23% lower than the second-best model RF(12, 10) at 0.72483%. The profitable trades percentages in Figure 7.16(b) saw a surprising 60.87% of trades making a profit for the RF(6, 1) model. This value is 8.76% higher than that of the LSTM model, with the 12-month, 3-month and 1-month RF models (except RF(12, 3)) having profitable trade percentages all below 50%. The RF(6, 1) resulted in a cumulative profit of 9.01% and only executed 69 trades. These results indicate the RF(6, 1) model did not predict large price movements frequently

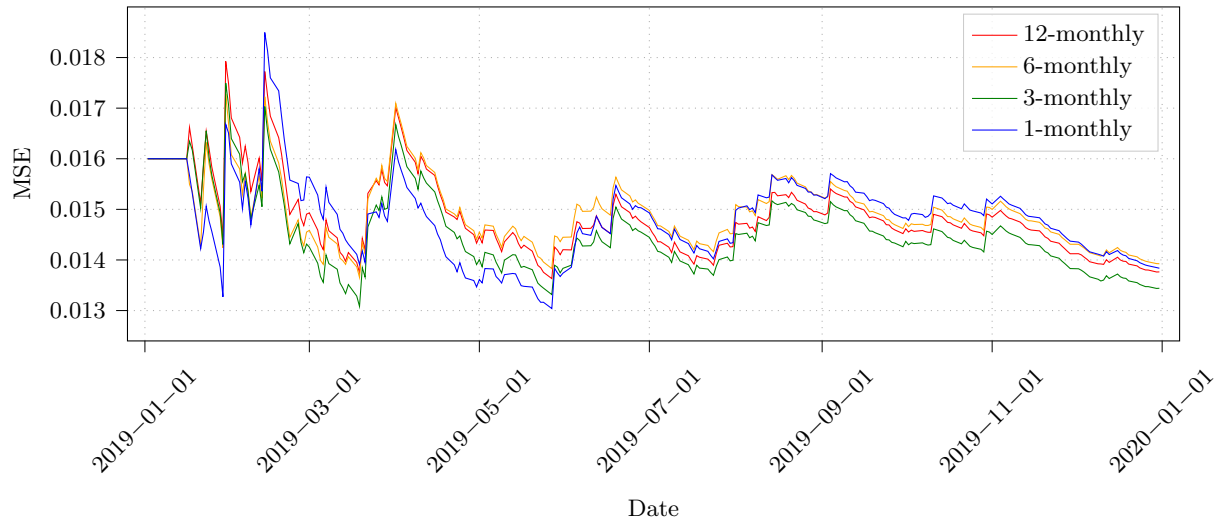


FIGURE 7.15: The MSE of **RF** models over time for the lowest MSE models for each of the different **retraining** intervals with a **sequence length** of 1 over the **2019** test set. The first 11 MSE values have been clipped to 0.016 to remove the instability and plot the graph on a comparable scale (results unaffected).

(above the spread), but when the model did, it did so with a higher accuracy. The entire results of the RF model for all years can be found in §B.2.

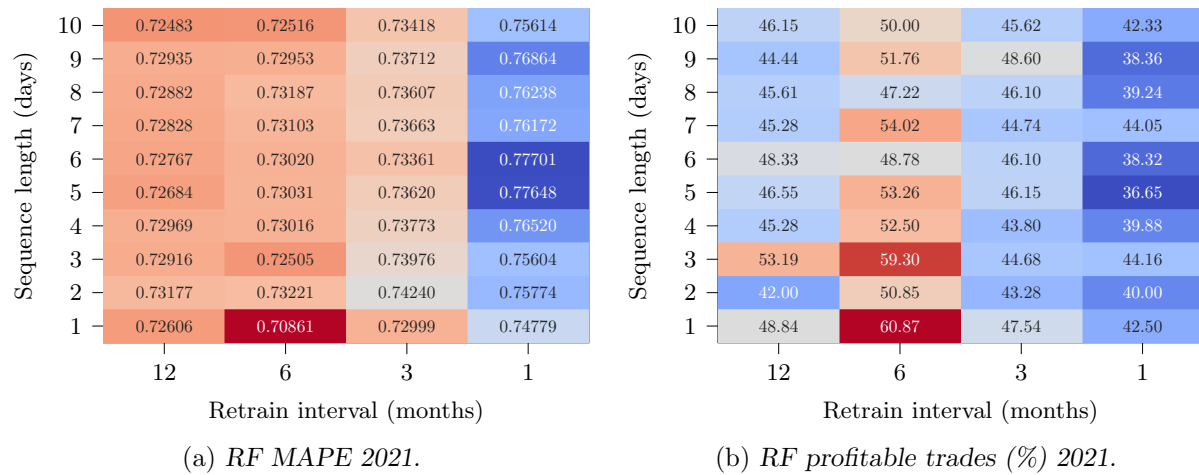


FIGURE 7.16: The heatmaps displaying the effects of **retraining** interval and **sequence length** on the **RF** models' MAPE (left) and the profitable trades percentage (right) for the **2021** test set.

7.6 Chapter summary

In this chapter, the RF model was introduced, accompanied by a discussion of fundamental concepts of DTs in §7.1 and ensemble learning in §7.2. Various hyperparameters concerning RF models and the results from calibrating each of these parameters using Bayesian optimization were illustrated in §7.3 and §7.4, respectively. Finally, the chapter concluded with an evaluation of the overall results obtained through four years of testing data and the subsequent effects of the testing parameters on the RF model's performance in §7.5.

CHAPTER 8

Support vector machines

Contents

8.1	Introduction	109
8.2	The kernel	110
8.3	Formulation	110
8.4	Hyperparameters	112
8.5	Hyperparameter calibration results	114
8.6	Results	117
8.7	Chapter summary	126

This chapter contains an overview of the implemented support vector machine (SVM) model. The chapter begins with an introduction to SVMs concerning the hyperplane and the non-linear mapping techniques. The architecture and training of SVM models, with the results of the hyperparameter calibration, are then discussed. The chapter concludes with the results of the effects of **retraining interval** and **sequence length** for the SVM model on the 2018 to 2021 USD/ZAR test set.

8.1 Introduction

Support vector machines (SVM) were initially developed by Vladimir Vapnik [9] as binary classifiers. The idea behind SVMs is explained through an example in Figure 8.1(a). The algorithm aims to separate classes by fitting a hyperplane, H_0 , through the data points. The objective is to minimise classification errors by maximising the margin width, determined by the distance between the support vectors. The support vectors, marked as dashed lines (H_1 and H_2), define the edges of this margin and are positioned to maintain the maximum separation between classes without any margin violations *i.e.*, data points within the support vector lines.

The technique employed in SVMs can be extrapolated to support regression tasks by reversing the objective function. Thus, instead of trying to fit the optimal hyperplane that maximises the margin width while limiting margin violations in the case of SVM classification. SVM regression aims to fit as many instances on the hyperplane while minimising margin violations [24]. In Figure 8.1(b), the application of SVM regression is illustrated. The hyperparameter, ϵ , pre-determines the width of the margin. The margin violations the algorithm aims to minimise are represented by ξ and ξ^* .

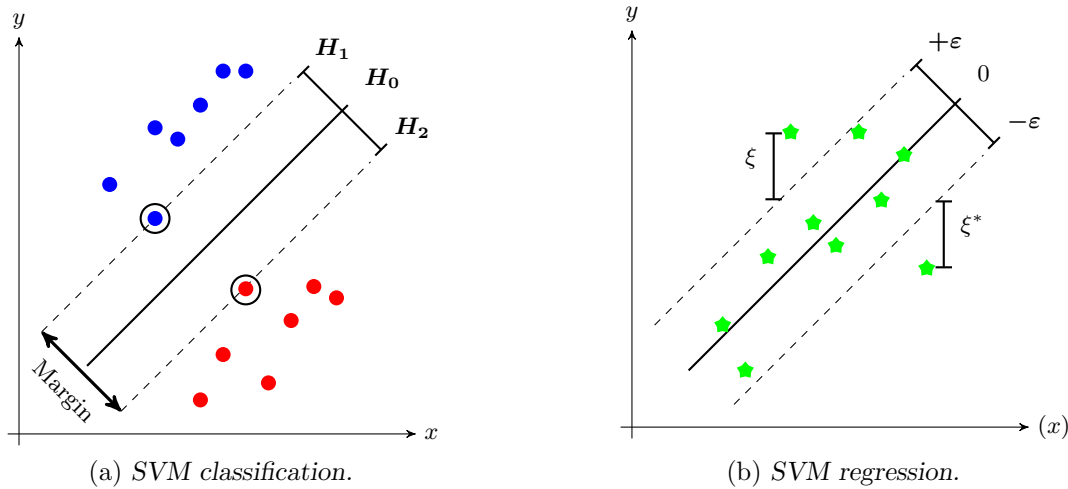


FIGURE 8.1: An example of the SVM algorithm used for classification (left) and regression (right).

8.2 The kernel

The examples in Figure 8.1 contain instances where the data points appear linear, and hyperplanes can easily be fit between the classes. However, this is seldom the case, notably for foreign exchange prices. One way of dealing with non-linear data is to add more features, such as polynomial features, that can describe the relationships between the data points [24]. The caveat is that more features increase the computational complexity of the model.

Vladimir Vapnik [9] proposed a technique called the ‘kernel trick’ that can mimic the result of adding a high degree of polynomial features without the increased computation. The approach involves non-linear mapping of input data into a higher dimensional space, whereby the data becomes linear, and an optimal hyperplane can be derived. Figure 8.2 contains an example of the kernel trick. In Figure 8.2(a), the input data is non-linear, which results in a curved hyperplane that is likely not to generalise well. The data is then transformed through the Φ kernel in Figure 8.2(b). The transformed data now appears linear in the higher dimensions, and a hyperplane with clearer boundaries that generalise well can be derived.

8.3 Formulation

This study predicts continuous values of the USD/ZAR prices. Therefore, the focus will be the formulation of an SVM regression model. The formulation of SVM regression is a quadratic optimisation problem, which, according to Kamruzzaman *et al.* [33], gives them a major advantage over neural networks.

Assume a data set $G = \{\mathbf{x}_i, y_i\}$, $i = 1, 2, \dots, N$, where N is the number of data points and each $\mathbf{x}_i$ is the features for a corresponding label, y_i . The function that represents the data, $g(\mathbf{x})$ is unknown, and the aim is to determine a function f that approximates $g(\mathbf{x})$ from the characteristic of G .

SVMs utilise the kernel trick by mapping $\mathbf{x}_i$ to a higher dimensional space through non-linear mapping, after which linear regression is performed in the higher dimension. The SVM thus

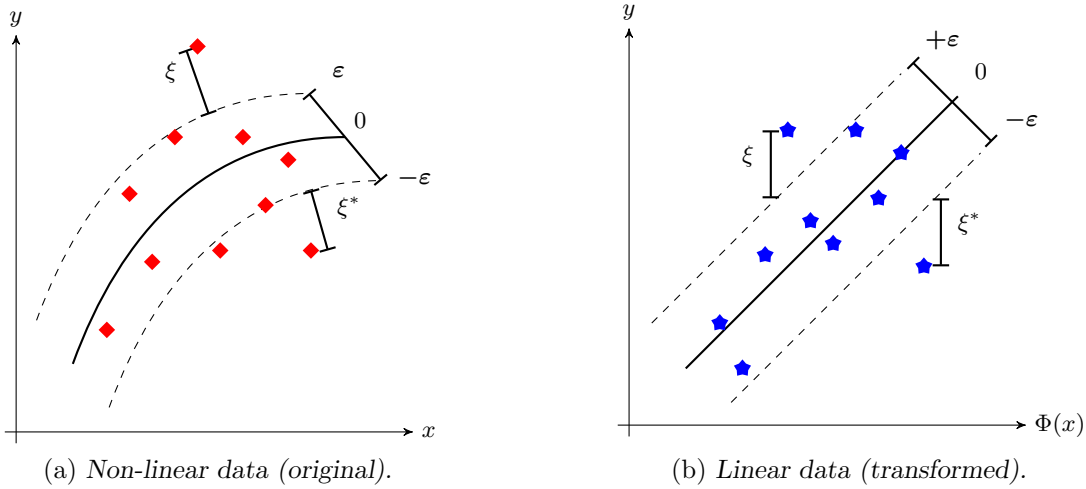


FIGURE 8.2: An example of how the SVM uses a kernel function, Φ , to transform non-linear data (left) to linear data (right).

approximates the function as,

$$f(\mathbf{x}) = \omega_i \Phi_i(\mathbf{x}) + b, \quad (8.1)$$

where $\Phi_i(\mathbf{x})$ are non-linearly mapped features, ω_i are the coefficients and b is the bias. Since the choice of Φ is pre-defined, the ω and b are determined by minimising the sum of the empirical risk defined as,

$$R = C \frac{1}{N} \sum_{i=1}^N |y_i - f(\mathbf{x}_i)|_\epsilon + \frac{1}{2} \|\omega\|^2, \quad (8.2)$$

where ϵ is a user-defined parameter for the margin width. The errors *i.e.*, margin violations are penalised if exceeding the margin width according to the error function defined as,

$$|y_i - f(\mathbf{x}_i)|_\epsilon = \begin{cases} 0 & \text{if } |y_i - f(\mathbf{x}_i)| < \epsilon, \text{ and} \\ |y_i - f(\mathbf{x}_i)| & \text{otherwise.} \end{cases} \quad (8.3)$$

The first term in equation (8.2) represents the ϵ -sensitive loss function, and the second term describes the function's flatness. Therefore, SVMs perform linear regression in the high dimensional feature space using the ϵ -sensitive loss function while simultaneously minimising $\|\omega\|^2$ to decrease the model's complexity. The constant, $C > 0$, is a regularisation value pre-defined in the SVM algorithm and is used to trade-off between training error and model flatness. The slack variables ξ and ξ^* represent the magnitude of margin violations, and by introducing them, the SVM regression model can now be formulated as an optimisation problem, defined as,

$$\text{minimise } \frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*), \quad (8.4)$$

subject to

$$\begin{aligned} f(\mathbf{x}_i) + b - y_i &\leq \epsilon + \xi & i = 1, 2, \dots, N, \\ y_i - f(\mathbf{x}_i) - b &\leq \epsilon + \xi^* & i = 1, 2, \dots, N, \\ \xi_i, \xi_i^* &\geq 0 & i = 1, 2, \dots, N. \end{aligned}$$

Equation (8.4) represents the primal quadratic programming problem of SVM regression and can be computationally inefficient to solve with the incorporation of the kernel trick. However, using a Lagrangian and transforming equation (8.4) into its dual problem can be used instead. The solution to the dual is defined by Boser *et al.* [9] as,

$$f(\mathbf{x}, \alpha, \alpha^*) = \sum_{i=1}^N (\alpha_i - \alpha_i^*) K(\mathbf{x}, \mathbf{x}_i) + b, \quad (8.5)$$

where Lagrange multipliers denoted as α_i and α_i^* are allocated to each data point $\mathbf{x}_i$ and have the following constraints with $0 \leq \alpha_i^*, \alpha_i \leq C$ and $\sum_{i=1}^N (\alpha_i - \alpha_i^*) = 0$. The data points containing non-zero Lagrange multipliers are the support vectors. The kernel function, K , is the inner dot product using the Φ kernel which satisfies the Mercer's conditions [9], given as,

$$K(\mathbf{x}, \mathbf{x}_i) = \Phi_i(\mathbf{x})\Phi_i(\mathbf{x}_i). \quad (8.6)$$

The coefficients in equation (8.4), α and α^* , can be determined by maximising the quadratic formula defined as,

$$R(\alpha, \alpha^*) = \frac{1}{2} \sum_{i,j=1}^N (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*)K(\mathbf{x}_i, \mathbf{x}_j) + \epsilon \sum_{i=1}^N (\alpha_i + \alpha_i^*) - \sum_{i=1}^N y_i(\alpha_i - \alpha_i^*), \quad (8.7)$$

subject to

$$\begin{aligned} \sum_{n=1}^N (\alpha_i - \alpha_i^*) &= 0 & i = 1, 2, \dots, N, \\ 0 \leq \alpha_i, \alpha_i^* &\leq C & i = 1, 2, \dots, N. \end{aligned}$$

With all the coefficients now calculated, the estimations for the SVM regression model can be determined through equation (8.5). The hyperparameters, C , ϵ and kernel function are all set before the training of the SVM regression model.

8.4 Hyperparameters

The goal of choosing the best hyperparameter configuration for the SVM regression model is to find a configuration that can first map the input data into the appropriate dimension and secondly derive a hyperplane that can describe the training data and generalise to the testing data. The different types of kernel functions available, along with the hyperparameters controlling the regularisation, are all discussed in this section.

Regularisation (C)

The regularisation parameter, C , determines the trade-off between maximising the margin and minimising the training error [33]. A smaller C establishes a wider margin, thus allowing more training errors (margin violations) and a model that generalises better with the risk of underfitting. Conversely, a large C value decreases the width of the margin. It fits the hyperplane closer to the data, thus leading to better accuracy on the training data with the risk of overfitting.

Epsilon (ϵ)

This parameter directly controls the width of the margin. The four main effects the epsilon hyperparameter has on the SVM model are:

1. **Wider margin:** A larger epsilon leads to a wider margin around the regression line. Consequently, more data points can fall into this broader margin, meaning more data points may become support vectors.
2. **Increased tolerance for errors:** A larger proportion of errors are tolerated with a higher epsilon value. Data points within the margin but outside the ϵ -tube are not considered errors. As it increases, more data points may be classified as non-errors; thus, more can become support vectors.
3. **Smoother model:** A larger epsilon may lead to a smoother regression model. Smoother models are preferred when noise or fluctuations in the data are insignificant. This smoothness can cause more data points to fall within the margin, increasing the number of support vectors.
4. **Risk of underfitting:** However, increasing epsilon too much can lead to underfitting. An overly wide margin might cause the model to ignore essential data points, resulting in a less accurate regression model with fewer support vectors.

Gamma (γ)

The gamma parameter defines how much influence a single instance in the training data has [17]. It affects the smoothness and flexibility of the decision boundary, with large gamma values making the SVM more sensitive to individual points, leading to complex and irregular boundary lines to fit the data points. A smaller gamma has the opposite, and the SVM model becomes less sensitive to individual points, resulting in smoother boundary lines.

Kernel function

The kernel function transforms the input features into a higher-dimensional space, where the data might become more linearly separable. The four most common kernels are the linear, polynomial, radial basis function (RBF) and sigmoidal functions, each with a specific use case dependent on the modelled data set. The linear kernel is used for linearly separable data and can be a good choice if many features seem linearly separable. The polynomial kernel is advantageous when the data exhibits a polynomial relationship. Additionally, the degree of the polynomial can be adjusted to fit the complexity of the data. The RBF kernel is the most commonly used and is suitable when there is no prior knowledge about the data distribution; thus, it generally works well. Lastly, the sigmoid kernel, which shares applications with neural networks, is used when the data has a clear sigmoidal relationship. These kernel function formulations are given below as,

$$\text{Linear: } K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j, \quad (8.8)$$

$$\text{Polynomial: } K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i \cdot \mathbf{x}_j + c)^d, \quad (8.9)$$

$$\text{RBF: } K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad (8.10)$$

$$\text{Sigmoidal: } K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i \cdot \mathbf{x}_j + c), \quad (8.11)$$

where γ is the hyperparameter used to scale the non-linear kernels. The constant c is used for both the polynomial and sigmoid functions, with the polynomial function containing another parameter d representing the degree of the function.

8.5 Hyperparameter calibration results

Following from §4.4, Bayesian optimisation with manual k -fold cross-validation was used to calibrate selected hyperparameters for the SVM model. SVM models contain the fewest parameters compared to the LSTM and RF models; therefore, a wider range of parameter values could be considered. Table 8.1 contains the search space used and the corresponding hyperparameter configuration with the lowest MSE value found through 150 iterations of Bayesian optimisation. The SVM calibration was the shortest of all the considered models, taking 3 hours and 33 minutes to complete.

Hyper-parameter	Abbreviation	Search space	best value
kernel	kernel	[rbf, sigmoid, poly]	rbf
gamma	gamma	[0.001, ..., 0.1]	0.0742
regulariser (C)	reg (C)	[0.001, ..., 100]	0.1176
epsilon	epsilon	[0.01, ..., 1]	0.9421

TABLE 8.1: *The SVM hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.*

The SVM had the second lowest MSE for the calibration, outperforming the RF model and slightly worse than the LSTM model. The top 5 iterations of the SVM model are given in Table 8.2. The average of the SVM models' top 5 MSE values is 0.02045, which is only 0.34% higher than that of the LSTM and 4.662% than the RF model. The lower MSE values from the LSTM and SVM models indicate that the RF models are not complex enough to model the USD/ZAR data compared to the former sophisticated algorithms. The results for all iterations of the SVM hyperparameter calibration can be found in §A.3 in Tables A.11, A.12, A.13.

N	SVM parameters				Test set		Train set		Time (mins)
	Kernel	gamma	reg (C)	epsilon	MSE	profit	MSE	Profit	
76	rbf	0.0742	0.1176	0.9421	0.02042	-1.60	0.01750	-1.89	1.4824
26	rbf	0.0009	0.0014	0.6400	0.02046	-0.94	0.01746	0.45	1.4341
85	rbf	0.0604	0.6567	0.9732	0.02046	-2.13	0.01825	-9.15	1.3812
58	sigmoid	0.0006	0.0093	0.6250	0.02046	-1.85	0.01746	-0.10	1.3189
133	poly	0.0001	0.2104	0.6847	0.02047	-1.19	0.01746	0.24	1.2802

TABLE 8.2: *The top five hyperparameter calibration iterations for the SVM model found through 150 iterations of Bayesian optimisation.*

The best-performing activation function for the LSTM model was the sigmoid function; however, it produced the worst results for the SVM models. Using the sigmoid kernel leads to convergence issues for specific iterations during Bayesian optimisation, as seen in Table 8.3. The combination of high **reg (C)** and the sigmoid kernel seem to have difficulty fitting the data during training and generalises poorly. These results are contrary to the effects of a large **reg (C)** parameter that, in theory, should penalise errors and reduce the size of the margin, thus leading to overfitting the training set and poor performance on the test set. This was not the case, as the training and test sets had similar MSE for the five worst iterations.

N	SVM parameters				Test set		Train set		Time (mins)
	Kernel	gamma	reg (C)	epsilon	MSE	profit	MSE	Profit	
66	sigmoid	0.0503	85.9400	0.6090	8.06E+136	-10.88	5.94E+138	-33.99	1.5739
29	sigmoid	0.0079	44.5131	0.8100	2.54E+35	-12.33	1.83E+23	-37.01	1.6029
14	sigmoid	0.0062	49.8304	0.8959	6.73E+29	-10.88	1.19E+18	-35.58	1.7120
139	sigmoid	0.0227	11.8972	0.4426	8.521E+11	-11.04	1.55E+15	-40.49	1.4914
50	sigmoid	0.0458	4.2636	0.8981	258173.4	-11.93	311876	-40.81	1.6054

TABLE 8.3: The worst five hyperparameter calibration iterations for the SVM model found through 150 iterations of Bayesian optimisation.

The numerous outliers from the sigmoid and polynomial kernels reduce the interpretability of the hyperparameter results. The instability was initially assumed to arise from the combinations with high `reg (C)` values. This assumption was investigated through the scatter plot in Figure 8.3, where the numerical hyperparameters are plotted against the MSE and cumulative profit metrics with colour-coded iterations based on the kernel function used. Additionally, all outliers were clipped to an MSE of 0.035 to have the iterations on a comparable scale. There are 40 outliers clipped at 0.035, of which 13 are from the sigmoid and 27 are from the polynomial function. The higher values of the `reg (C)` parameter did coincide with outliers. However, there were also outliers for the lower values, and the Pearson correlation coefficient only indicated a weak positive relationship between (`reg (C)`) and the MSE at 0.23 (no post-processing of outliers). Therefore, the assumption of instability caused by the higher values of `reg (C)` was dropped, and further investigation is necessary.

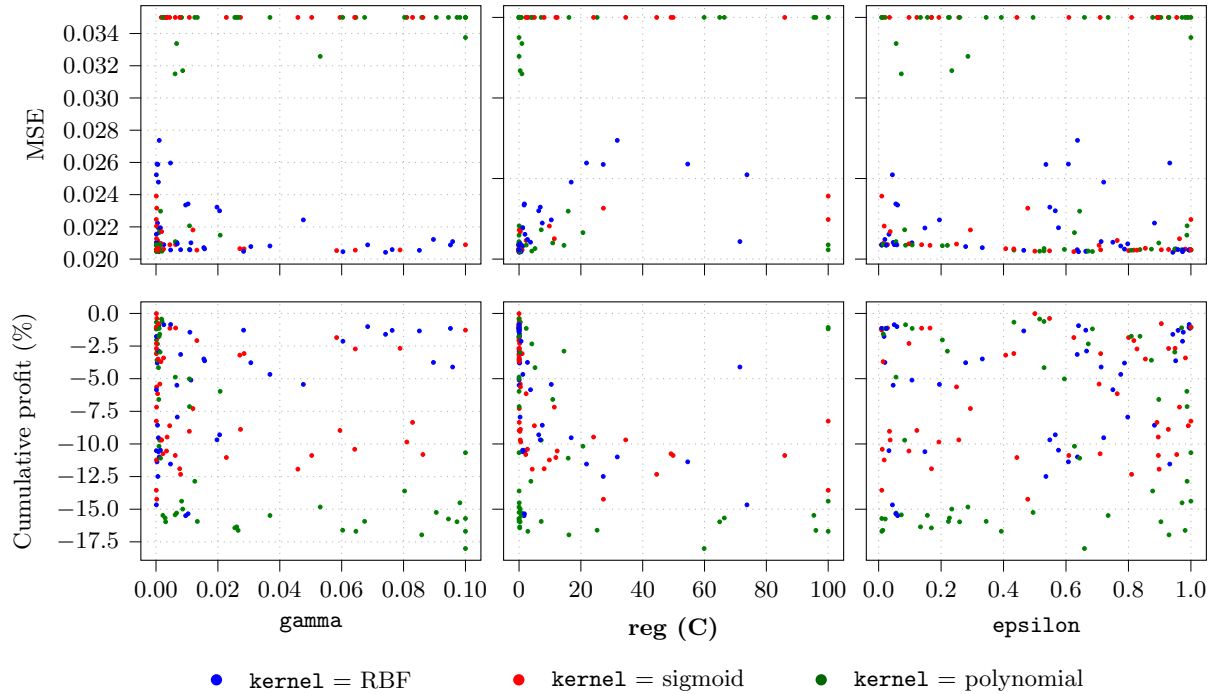


FIGURE 8.3: The scatter plots of the hyperparameters `gamma`, `reg (C)` and `epsilon`, against the MSE and cumulative profit from the SVM models' hyperparameter calibration. The iterations are coloured depending on the `kernel`, with RBF in blue, sigmoid in red and poly in green. The outliers have all been clipped to 0.035 to identify which kernel functions are responsible for the outliers and keep the graph on an interpretable scale between iterations.

Iterations containing the RBF kernel were more consistent than the sigmoid and polynomial kernels throughout the calibration. These iterations had no outliers, with its highest MSE coming from iteration 49 at 0.02737, the 46th worst MSE value. The sigmoid kernel function coincides with the 13 worst iterations of the highest MSE values. The box and whisker graphs of the three kernel functions with the outliers clipped at an MSE of 0.035 are given in Figure 8.4(a). The RBF has the lowest interquartile range, confirming the superior performance to the sigmoid and polynomial functions. The sigmoid and polynomial contain upper quartiles at 0.035 due to their outliers clipped to the same value. However, the sigmoid function is slightly better as it contains a lower median MSE of 0.02121 than that of the polynomial at 0.02207.

The assumption to clip the outliers at 0.035 is likely introducing a bias into the results; therefore, the box and whisker diagram of the kernel functions with the outliers removed instead of clipped is shown in Figure 8.4(b). The new diagram indicates a contradictory finding, with the sigmoid outperforming the RBF and polynomial functions with the outliers removed. The RBF function's median of 0.020919 is still within 0.39% and 0.24% of the sigmoid and polynomial medians, respectively. However, the RBF now has the widest interquartile range, indicating it is less consistent than the other kernels. These results indicate that the sigmoid and polynomial kernels can outperform the RBF function if hyperparameter configurations are carefully considered. Due to the high number of models to be tested in the USD/ZAR testing set, stability is preferred to get reliable results. Therefore, the RBF function was the best kernel function overall, corresponding to the lowest 3 MSE iterations found. The investigation of these kernels for further improved performance falls outside the scope, and the hyperparameter results are used without further exploration.

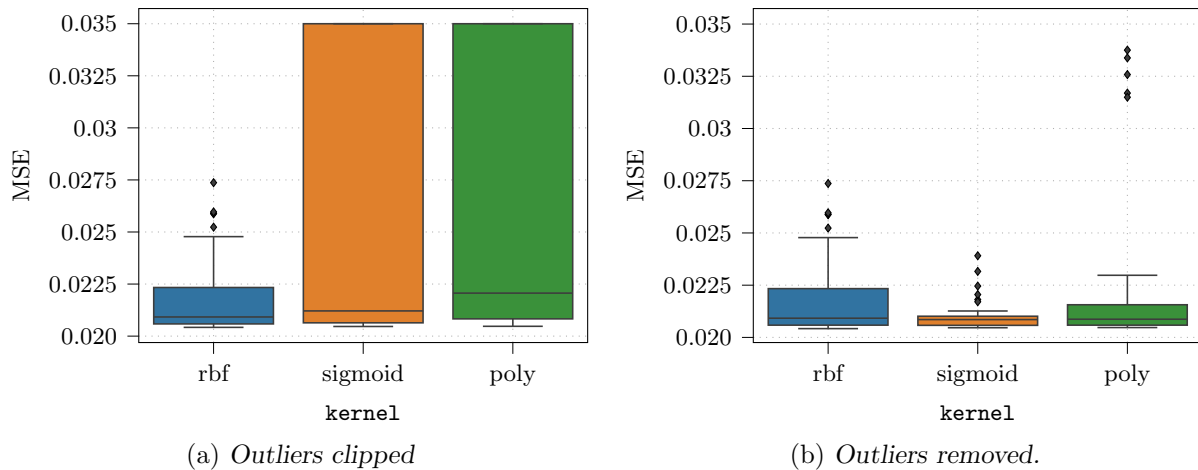


FIGURE 8.4: The box and whisker plots showing the distribution of the MSE for the SVM's **kernel** hyperparameters with the outliers clipped (left) and removed (right).

The finding in Figure 8.3 for the **gamma** and **epsilon** hyperparameters were inconclusive, and no clear correlation could be detected. In Figure 8.5, the Pearson correlation matrices are given to explore the numerical values' correlations further. The correlations are given for the two techniques employed to handle the outliers, namely clipping in Figure 8.5(a) and removal in Figure 8.5(b), both using MSE values of 0.035. The correlations are stronger in Figure 8.5(a) with the **gamma** having the strongest positive correlation of 0.4 with MSE. This value suggested that a higher **gamma** value, which leads to the SVM being more sensitive to individual data points, is causing the SVM model to overfit the training data, thus generalising poorly on the testing data.

The removal of outliers causes a noticeable effect on the correlations in Figure 8.5(b) as all correlations between the hyperparameters and the metrics decreased. The MSE and profit metrics are again closely correlated as -0.81 and -0.74 for the clipped and removal techniques, suggesting better accuracy leads to more profitable models.

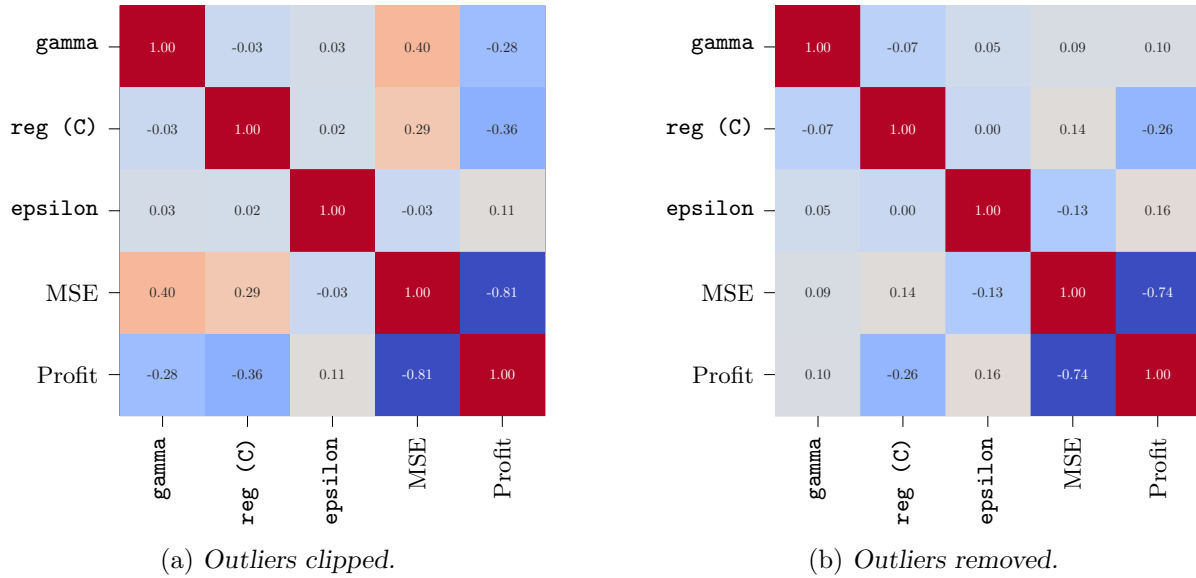


FIGURE 8.5: The Pearson correlations matrices containing the relationship between the hyperparameters and testing metrics for all iterations of Bayesian optimisation with outliers clipped at an MSE of 0.035 (left) and outliers with an MSE over 0.035 removed (right).

8.6 Results

The best SVM model in Table 8.1 took 8 minutes and 34 seconds to run through the 40 different combinations of **sequence length** and **retraining interval** for the four years of testing data. The notation used to describe the LSTM and RF models continues with SVM(r , s) representing an SVM model with **retrain interval** set to every r -months and uses a s -day **sequence length** for the features.

All test years combined

The SVM results differ from the LSTM and RF models, showing that less frequently updated models performed better than more frequently updated ones. In contrast, the SVM results display the number of lagged features that have a more prominent effect on the metrics used to evaluate the models, with negligible influence caused by updates.

The SVM models' MSE outperforms all other ML and statistical models. The 1-day **sequence length** stood out and achieved the lowest MSE values in its category, with all MSE values being within 0.29% of each other, seen in Figure 8.6(a). The highest MSE value was from the SVM(6, 1) at an impressive 0.02067, the lowest MSE found of all models. This MSE value is lower than the best RF model by 1.38%, the LSTM model by 1.48%, naïve by 1.94% and ARIMA by 2.08%.

Notably, the SVM model was the most consistent in MSE values compared to other ML models across all combinations of **sequence lengths** and **retrain interval**. The worst SVM model, SVM(3, 10), had the lowest MSE of 0.02117, which was only 0.91% and 1% higher than the best MSE values found for the LSTM and RF models. Comparing the SVM(3, 10)’s MSE to that of the worst-performing LSTM model of 0.02297 and RF model of 0.2438 indicates a decrease in MSE by 7.8% and 13.17%, respectively.

The MAPE results in Figure 8.6(b) contain similar correlations between the MSE and testing parameters, with the 1-day **sequence length** having the biggest disparity between combinations. However, the longer **retrain intervals**, 12-months and 6-months, had better MAPE for the 1-day **sequence length** compared to the 3-month and 1-month intervals. Interestingly, the best MAPE, 0.076017, from the SVM(12, 1) model was the worst out of all the three ML models. The LSTM and RF achieved lower MAPE values by 0.45% and 0.62%, respectively. The lower MSE and higher MAPE values could be due to the SVM model being more sensitive to outliers. Thus, the model likely overfit the training set to reduce larger errors caused by the outliers. The overfitting will lead to better MSE values overall but result in higher MAPE since the models struggle to generalise on more data points (‘non-outliers’).

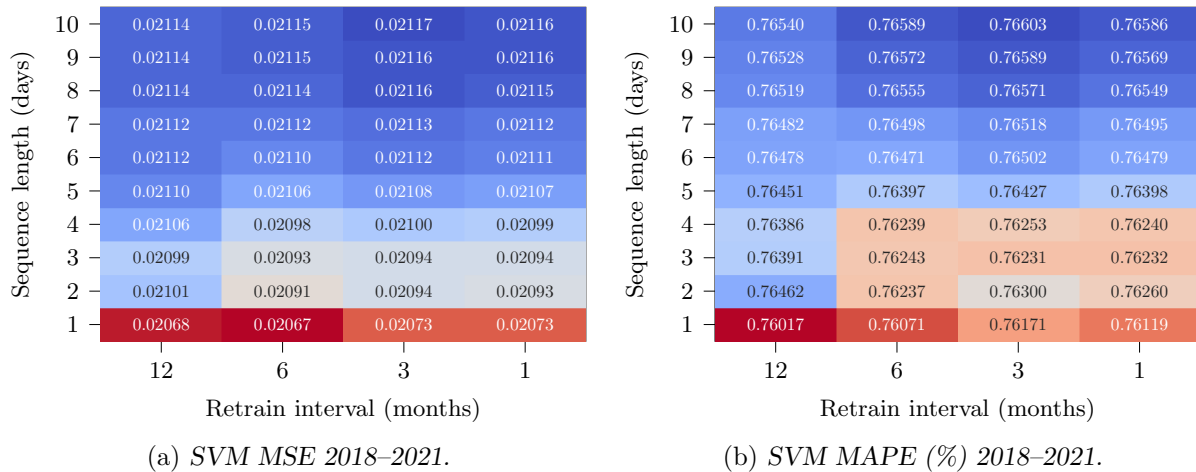


FIGURE 8.6: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models’ MSE (left) and MAPE (right), for the **2018–2021** test set.

The threshold accuracies summarised for all four years of testing in Figure 8.7 contain contradictory results for the SVM models. The lower threshold accuracies of R0.025 and R0.05 in Figure 8.7(a) and 8.7(b) confirm the improved performance of the lower **sequence lengths** with a length of 2 resulting in the highest average accuracy of 13.39% (R0.025) and 27.51% (R0.05). The higher threshold accuracies of R0.075 and R0.1 in Figures 8.7(c) and 8.7(d) show the polar opposite of the lower thresholds and the convention that lower **sequence lengths** improve accuracy. The higher **sequence lengths** are more accurate, with the 2-day **sequence lengths** now among the worst performing models. The **retrain interval** showed no clear correlation between the accuracy obtained.

Although the accuracy at the higher threshold for lower **sequence lengths** are relatively worse than the higher lengths, the ranges between the accuracy values are considered minimal at 1.93% for the R0.075 and 1.45% for the R0.1 thresholds.

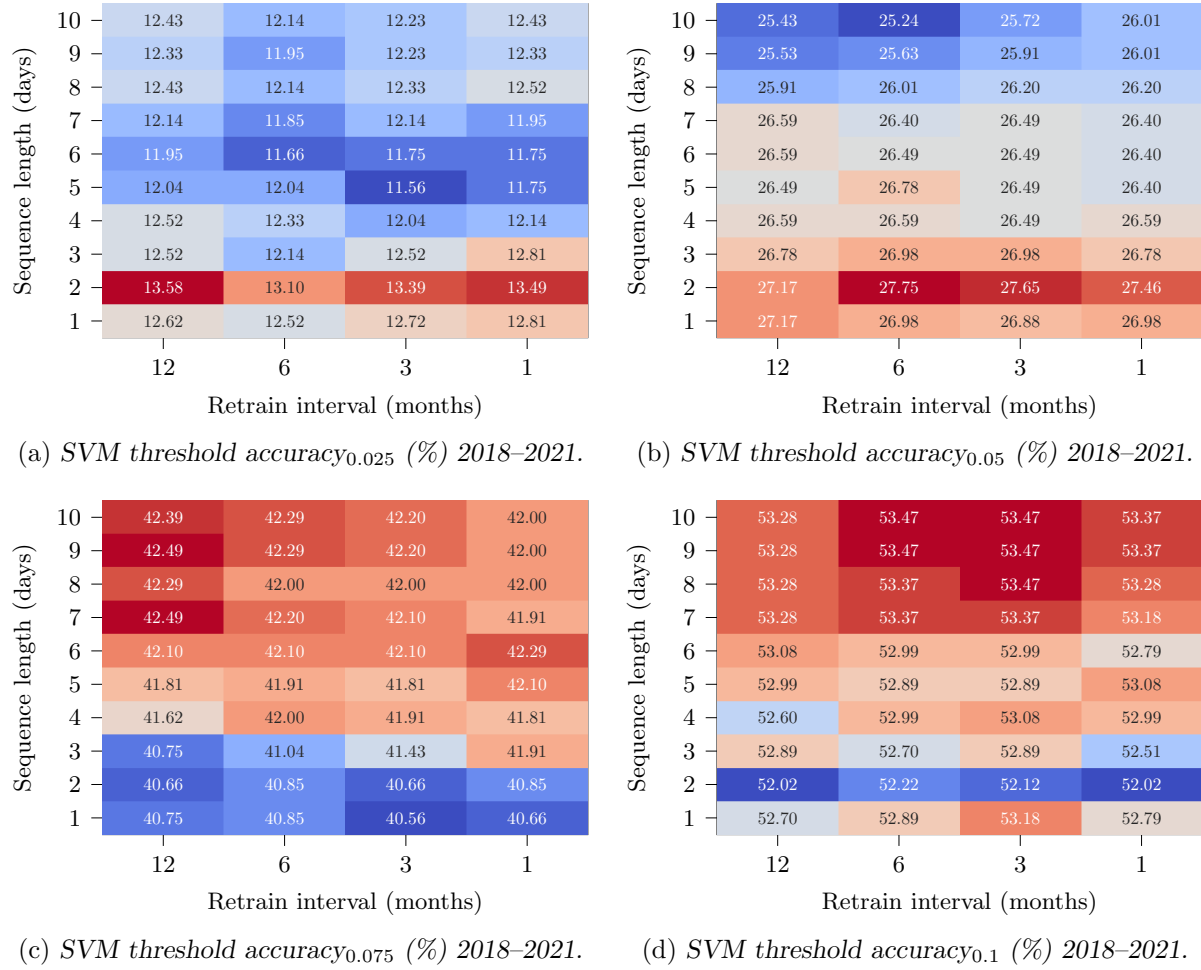


FIGURE 8.7: The heatmaps displaying the effects of *retraining interval* and *sequence length* on the **SVM** models' different threshold accuracies for the **2018–2021** test set.

The cumulative profit in Figure 8.8(a) is the metric that stands out for all the SVM models in the 2-D grid search. All 40 combinations of *retrain interval* and *sequence length* result in a profit at the end of the 2018–2021 testing period. The cumulative profit ranges from the lowest, SVM(1, 8) at 15.65%, to the highest, SVM(6, 1), at 50.22%. The most profitable SVM(6, 1) model had a cumulative profit of 21.26% higher than that of the second-highest ML model's profit of 28.96% from the RF(12, 3) model. The highest profit for each of the different *retraining intervals* came from the 1-day *sequence length*.

The results are unsurprising since the MSE and cumulative profit metrics were highly correlated in Figure 8.5 and throughout the study. The SVM models all had relatively low MSE values, thus resulting in a profit. Similar findings did not occur for the maximum drawdown in Figure 8.8(b). The SVM models with higher cumulative profits resulted in high absolute drawdown values, indicating that the SVM models with lower *sequence lengths* utilise a riskier trading strategy with large gains and losses. The smallest maximum drawdown of -31.86% coincided with the most profitable model, SVM(6, 1). The 1-month *retrain interval* appears to have the smallest deviation of drawdown values, and the *sequence length* does not have a sizeable effect on the drawdowns for these values. The drawdowns are still lower than the RF and LSTM models, with the worst maximum drawdowns of -99.02% and -67.90% , respectively.

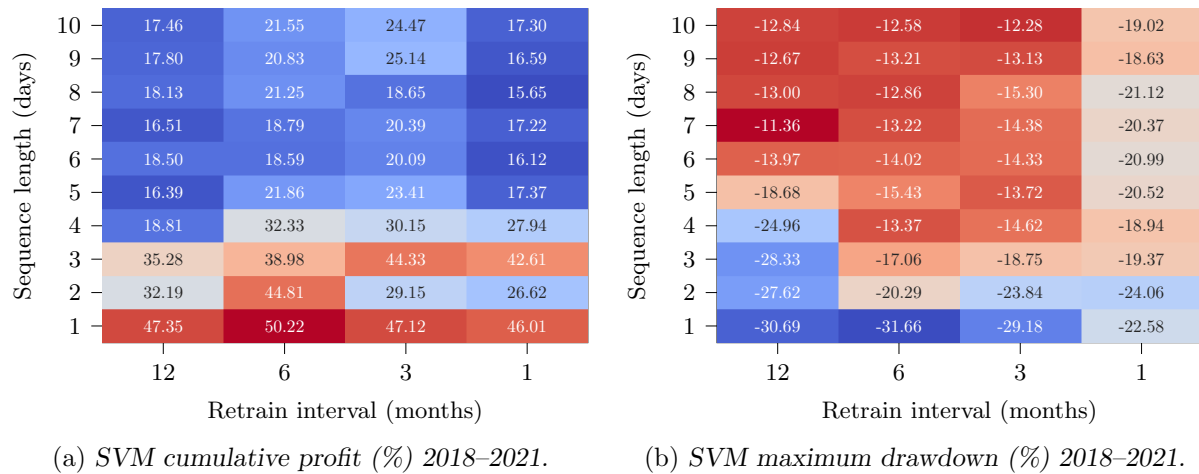


FIGURE 8.8: The heatmaps displaying the effects of *retraining interval* and *sequence length* on the **SVM** models' cumulative profit (left) and maximum drawdown (right) for the **2018–2021** testing set.

Figure 8.9 contains the SVM(6, 1) models cumulative profit, with the inclusion of the SVM(1, 1) since it had similar profits of only 4.21% less than the SVM(6, 1) at 50.22%, however had a 9.08% better maximum drawdown. Both SVM models were close to breaking even at the end of the 2018 period, with the SVM(6, 1) slightly higher at 3.48% compared to the SVM(1, 1) at 2.92%, with drawdowns of -8.61% and -11.49% , respectively. The 2019 testing year, in which all the ML models struggled to achieve consistent profits, saw the SVM(6, 1) lose 20.8% in cumulative profit with a drawdown of the same value. The SVM(1, 1) was able to negate some of the losing trades the SVM(6, 1) incurred but still ended the year losing 14.59%, again with this same value for the maximum drawdown.

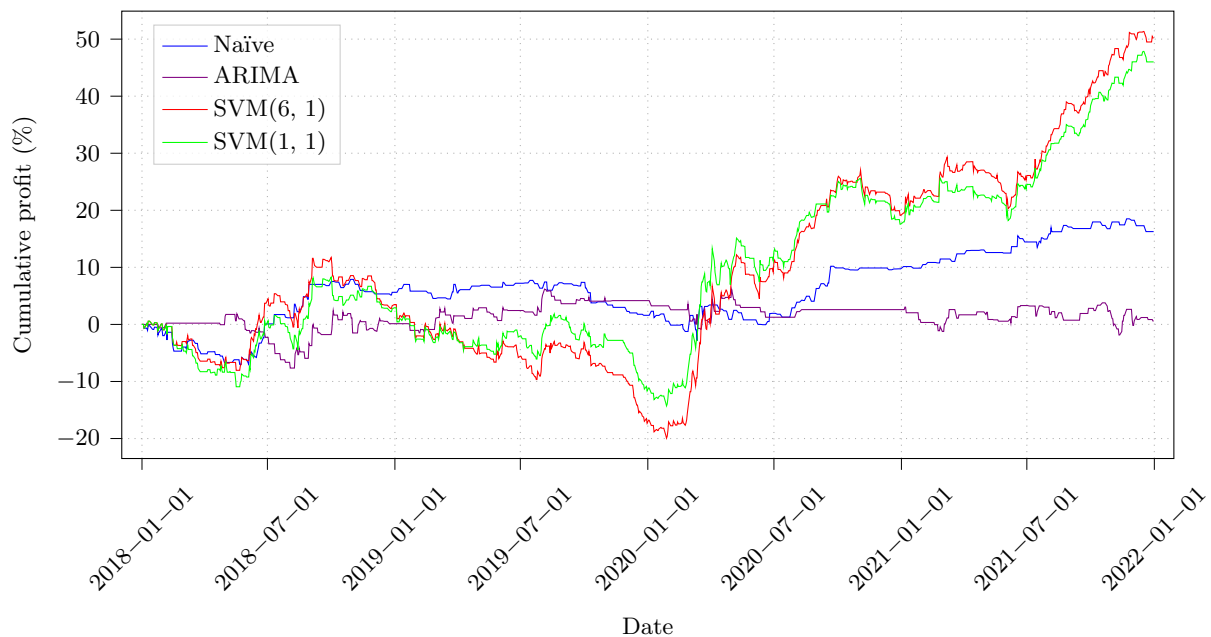


FIGURE 8.9: The results of the highest cumulative profit of the SVM(12, 3) and second highest SVM(6,3), compared to the naïve and ARIMA for the **2018–2021** testing set.

The 2020 and 2021 years contained a bounce back in profit for the SVM models. In 2020, the SVM(6, 1) experienced an impressive 36.45% cumulative profit, and the SVM(1, 1) closely followed with a cumulative profit of 29.24%. The cumulative profit at the end of 2020 was almost equal despite the SVM(6, 1) obtaining more profits since it had a lower maximum drawdown in 2019. The 2021 year experienced a similar high profit for the SVM(6, 1) at 31.09% and SVM(1, 1) at 28.44%. The last two years resulted in 67.54% and 58.38% cumulative profit, thus indicating the SVM models with the best hyperparameter configurations can be considered for real-world trading applications.

The profitable trades percentage for the entire testing set in Figure 8.10(a) confirms the better performance of the shorter **sequence lengths** with minimal effect from the **retraining interval**. Investigating the profitable trades percentage and the correlation between the number of trades show vastly different results compared to the LSTM and RF models. The RF and LSTM model showed models with a higher profitable trades percentage correlated with fewer total trades and *vice versa*.. In contrast, the SVM models show the opposite findings, with correlations with more profitable models associated with a higher number of trades in Figure 8.10(b). The profitable trades percentage does, however, fluctuate between the different testing years and should be further explored.

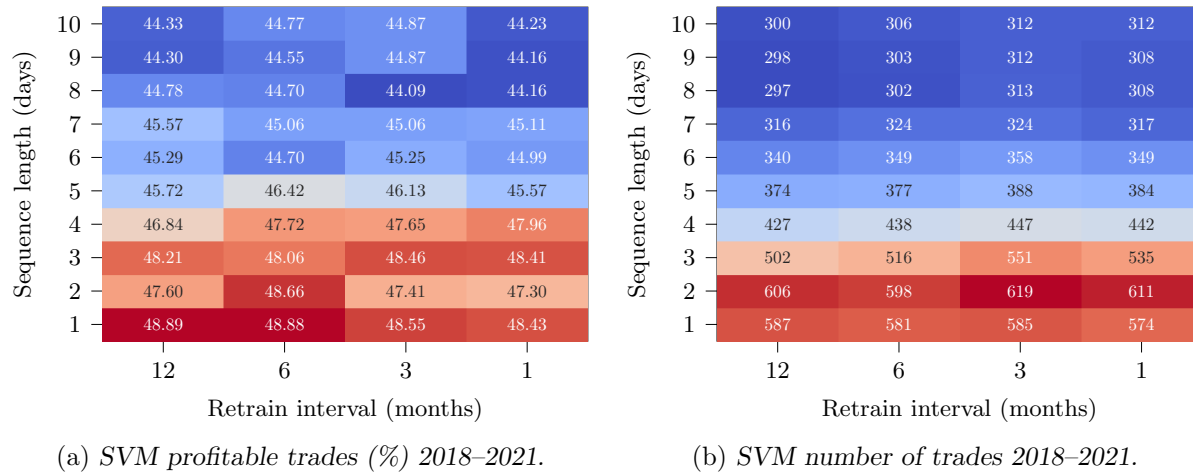


FIGURE 8.10: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models' profitable trades percentage (left) and the number of trades (right) for the **2018–2021** test set.

Figure 8.11 displays the average profitable trades percentage across all **retrain intervals** for the SVM models with selected **sequence lengths**. The **sequence lengths** chosen were based on the top 2 average profitable trades percentage SVM models (1 and 3 days) and the worst 2 average profitable trades SVM models (8 and 9 days). The **sequence lengths** also correlated to the models with the most trades, 3 days and 1 day, respectively and the least frequent trading models, 8 and 9 days.

Across the years, there is fluctuation in the percentage of profitable trades for each **sequence length**. This variation highlights the model's sensitivity to different market conditions over time. The largest disparity was in 2021, with the higher **sequence lengths**, 1 and 3 days, having over 58% profitability while the 8 and 9 days were both below 48%. Regarding the influence of **sequence length** on model performance, shorter lengths (1 and 3 days) consistently had the highest profitability in 2018, 2020, and 2021. However, longer lengths (8 and 9 days) only

perform better in 2019. All models performed the worst in 2019, having the lowest profitable trades percentage out of the four periods, indicating increased complexity in market conditions and the lack of repeating patterns for this period.

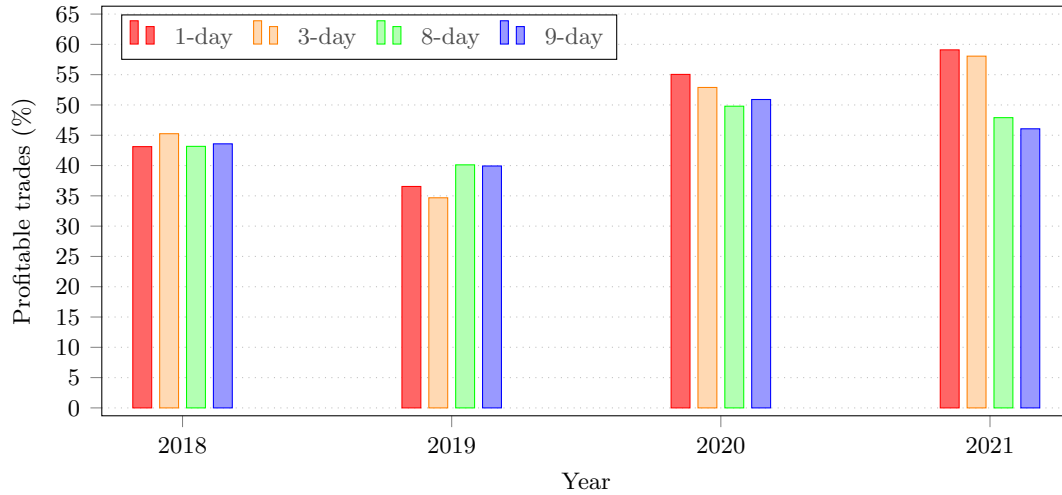


FIGURE 8.11: The comparison between the retraining intervals profitable trades percentage for the **SVM** models over the **2018–2021** test set.

Table 8.4 contains a summary of the average selected metrics for the different **retrain intervals** and **sequence lengths** tested. The averages for the **retrain interval** are similar across all the metrics, confirming the findings of negligible impact on the SVM models. The **sequence length** of 1 day had the best MSE, MAPE, cumulative profit and profitable trades percentage; however, it contained the worst maximum drawdown with the second-highest number of trades. The Pearson correlation coefficient between the **sequence length** and MSE is 0.88 with a p -value of 0.0008, indicating a statistically significant high positive correlation at a 95% confidence level.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.02105	0.76425	23.84	−19.41	46.15	405
	6	0.02102	0.76387	28.92	− 16.37	46.35	409
	3	0.02104	0.76416	28.29	−16.95	46.23	421
	1	0.02104	0.76393	24.34	−20.56	46.03	414
Seq length	1	0.0207	0.76094	47.67	−28.53	48.69	582
	2	0.02095	0.76315	33.19	−23.95	47.74	608
	3	0.02095	0.76274	40.30	−20.88	48.28	526
	4	0.02101	0.7628	27.31	−17.98	47.54	438
	5	0.02108	0.76418	19.76	−17.09	45.96	381
	6	0.02111	0.76482	18.33	−15.83	45.06	349
	7	0.02112	0.76498	18.22	−14.83	45.20	320
	8	0.02115	0.76548	18.42	−15.57	44.43	305
	9	0.02115	0.76564	20.09	−14.41	44.47	305
	10	0.02115	0.7658	20.20	− 14.18	44.55	308

TABLE 8.4: The **SVM** models' aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

The SVM model thus performs better with fewer features and is more biased towards recent

observations in the data set. Adding more features through the **sequence length** parameter likely adds noise to the data set and produces poorer hyperplanes that generalise worse. The variation in **retraining intervals** had minimal effect on the SVM model. Additionally, this finding indicates that the size of the training set does not influence the SVM model, with the 12-month, 6-month, 3-month and 1-month intervals containing approximately 1037, 520, 259 and 84 instances, which resulted in subtle variation in the performance.

Test set 2018

The 2018 testing period contained different results compared to the summarised findings. The SVM models with a 1-day **sequence length** were the worst performing models, notably the SVM(6, 1), SVM(3, 1) and SVM(1, 1), which contained the highest MAPE in Figure 8.12(a). The poor prediction accuracy also leads to cumulative profits for these three models in Figure 8.12(b). The 12-month **retrain interval** had the best MAPE values across all **sequence lengths** in Figure 8.12(b) and had the highest average profit of 13.34%, which was 3%, 2.79% and 5.19% higher than the 6-month, 3-month and 1-month intervals respectively.

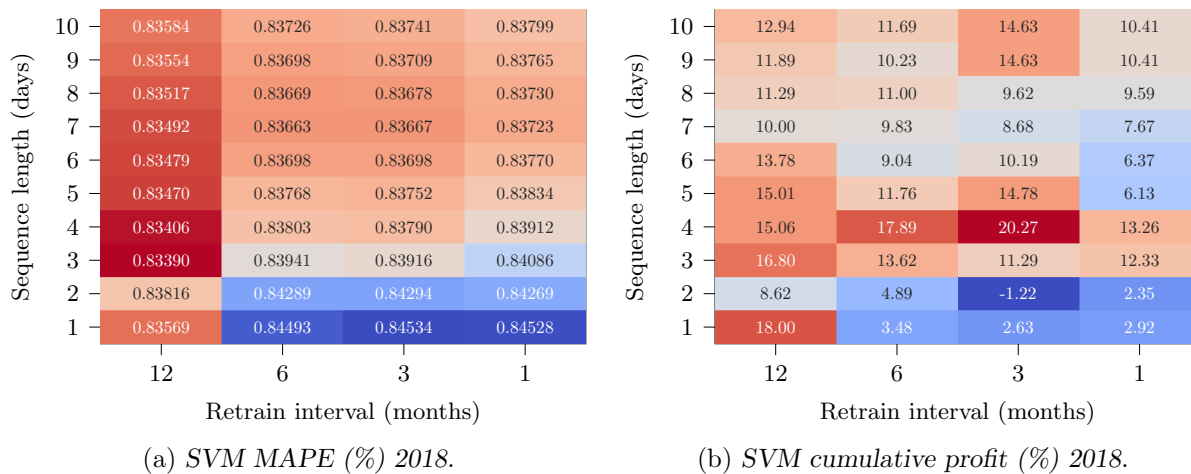


FIGURE 8.12: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models' MAPE (left) and cumulative profit (right) for the **2018** testing set.

Test set 2019

In 2019, the SVM could not capture the market patterns with limited features. Therefore, higher **sequence lengths** were favoured, resulting in better models. The MSE seen in Figure 8.13(a) contained a low of 0.01386 obtained by 7 different combinations of the testing parameters, all of which come from **sequence lengths** above 6 days. This period also had the lowest profitable trades percentage seen in Figure 8.13(b), ranging from SVM(12, 3) at 32.82% to an unimpressive 41.46% from the SVM(6, 8). A longer retrain interval and fewer days of input led to poor MSE and profitable trade percentages, indicating that these combinations of SVM models began to decay, and insufficient information was available to capture the market conditions.

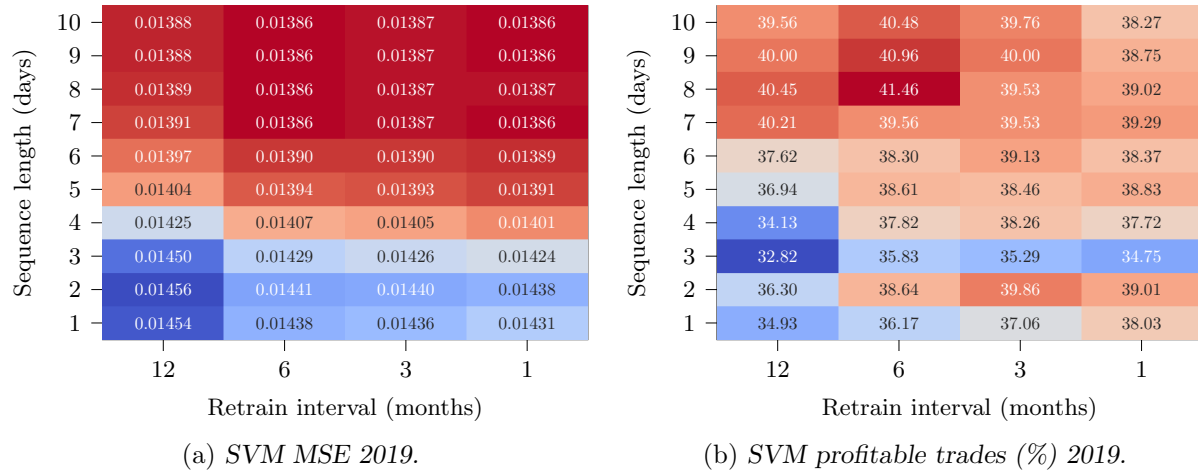


FIGURE 8.13: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models' MSE (left) and profitable trades (right) for the **2019** testing set.

Figure 8.14 contains the MSE through time of the SVM(12, 1) and SVM(12, 9) models to investigate the performance difference. The SVM(12, 9) has consistently lower MSE values for the entirety of the 2019 testing set than the SVM(12, 1). The gap between the two MSE values appears constant; thus, more information benefits the SVM(12, 9) model, with the SVM(12, 1) not able to fit the data set well.

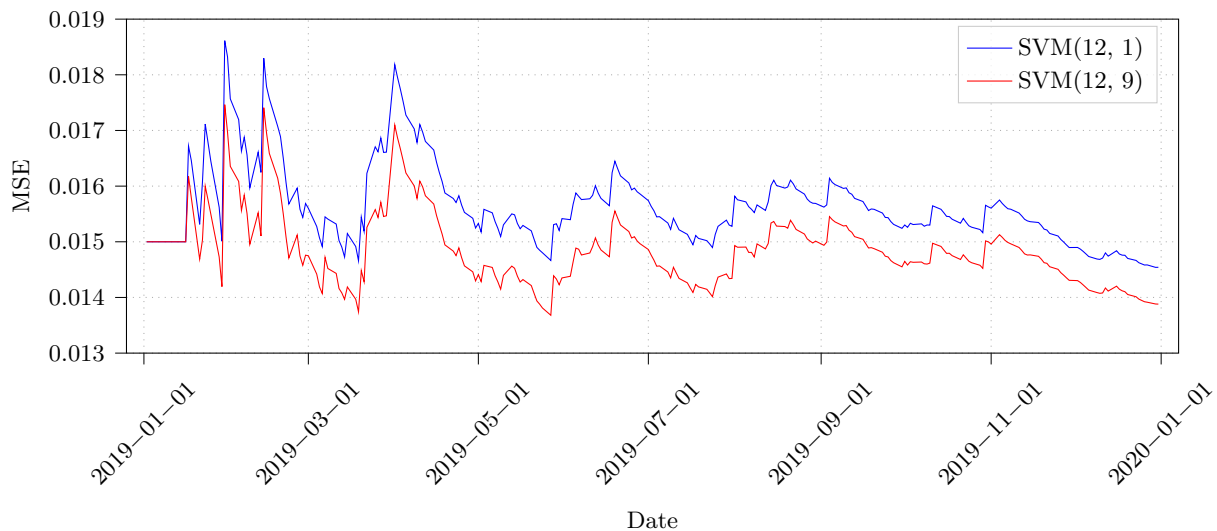


FIGURE 8.14: The MSE of the SVM(12, 1) and SVM(12, 9) models over the **2019** test set. The first 11 MSE values have been clipped to 0.015 to remove the instability and plot the graph on a comparable scale (results unaffected).

Test set 2020–2021

The results for the 2020 and 2021 testing periods were consistent with the summarised findings. However, due to the superior performance of using only 1 previous day as input, the summaries are likely skewed towards these testing periods, as the results for 2018 and 2019 contradict the

findings. The performance regarding the MSE results of the different combinations of testing parameters are given in Figure 8.15(a) and 8.15(b) for 2020 and 2021, respectively. The best MSE for 2020 came from the SVM(6, 1) model at 0.03168, which was 5.26% lower than the worst MSE by SVM(3, 9) at 0.03344. 2021 had similar disparities between the **sequence lengths** with the lowest MSE, SVM(3, 1) at 0.01681 and highest of SVM(12, 9) at 0.01777, having a 5.4% difference. The **retraining intervals** did not impact the MSE values for both periods.

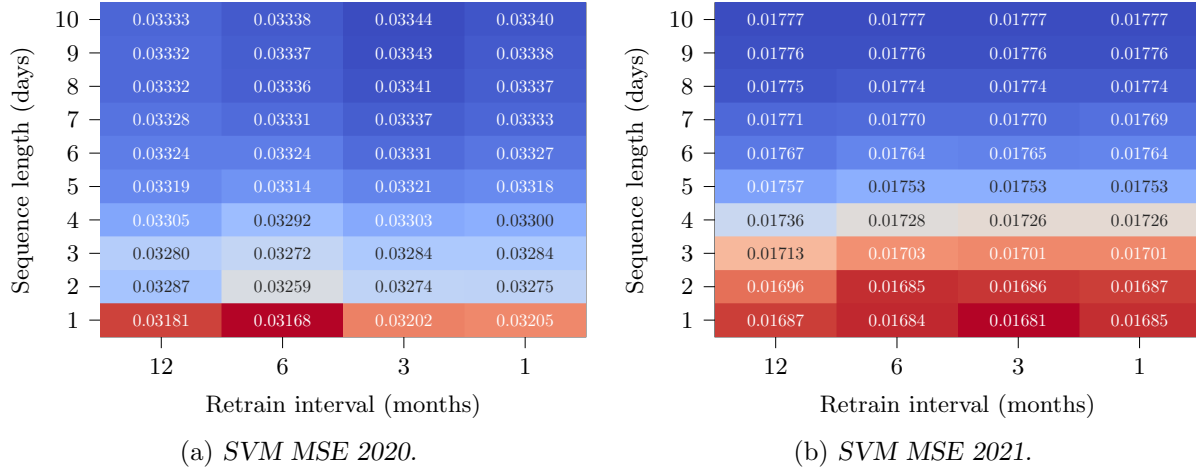


FIGURE 8.15: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models' MSE for the **2020** testing set (left) and **2021** testing set (right).

The low MSE values for 2020 and 2021 led to the highest profits obtained by any model in a single year, and the cumulative profits for these periods are given in Figures 8.16(a) and 8.16(b). All 40 combinations of testing parameters resulted in a profit, but the lower **sequence length** contained the higher cumulative profits. The entire results of the SVM model for all years can be found in §B.3.

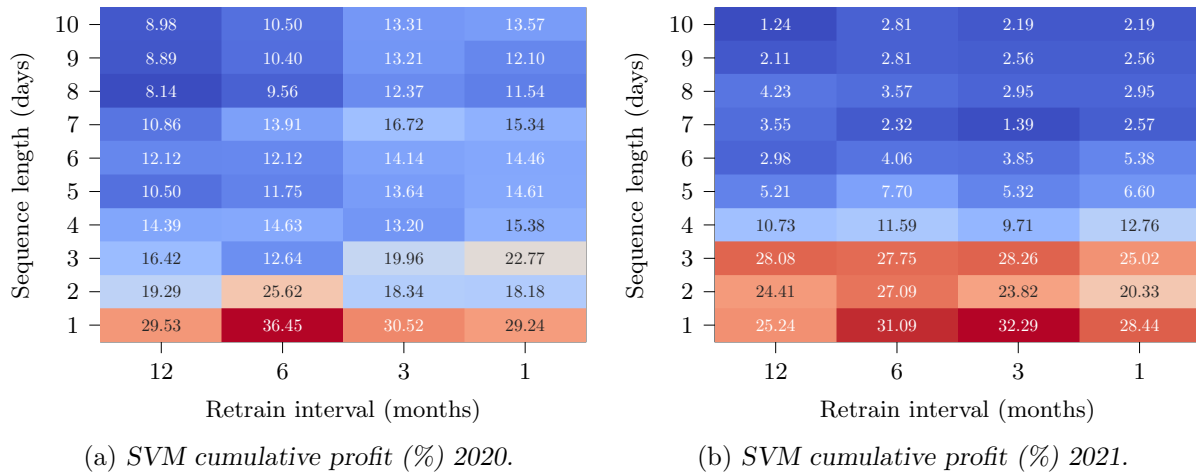


FIGURE 8.16: The heatmaps displaying the effects of **retraining interval** and **sequence length** on the **SVM** models' cumulative profit for the **2020** testing set (left) and **2021** testing set (right).

8.7 Chapter summary

In this chapter, the SVM model was introduced with a discussion on separating data via a hyperplane in §8.1 and on non-linear transformations via a kernel function in §8.2. The SVM formulation regarding the derivation of the hyperplane was provided in §8.3. Various hyperparameters concerning SVM models and the results from calibrating each of these parameters using Bayesian optimization were illustrated in §8.4 and §8.5, respectively. Finally, the chapter concluded with an evaluation of the overall results obtained through the four years of testing data and the subsequent effects of the testing parameters on the SVM model's performance in §8.6.

 CHAPTER 9

Model Comparison

Contents

9.1 Accuracy metrics	127
9.2 Profit metrics	129
9.3 Summary	131

In this chapter, a comparison of the implemented models from Chapters 5, 6, 7 and 8 is given. The aim is to analyse each testing period's results and compare model performance for accuracy and profit. Only the best-performing ML models regarding their hyperparameter configurations for the parameters **retraining interval** and **sequence lengths** are considered. This ensures each model's best performance is assessed and limits the comparisons of the ML models to a single model instead of 40 different models. The chapter is divided into accuracy-related and profit-related metrics that compare the five models: naïve, ARIMA, LSTM, RF and SVM.

9.1 Accuracy metrics

Each model's lowest MSE and corresponding configuration regarding **retraining interval** and **sequence length** are given in Table 9.1 for the 2018–2021 test period. The MSE across all models in 2018 had the closest range, with all models being within 1.84% of each other. The LSTM had the best MSE at 0.01926 from the LSTM(12, 2). The statistical model ARIMA performed the best in 2019 with an MSE of 0.01319, indicating that the more complex ML models struggled to capture this period's market conditions. The biggest disparities, equating to 6.57% and 5.88% between MSE values, came in 2020 and 2021. The superior performance of the SVM model caused these wide ranges and resulted in the lowest MSE, which were 3.39% and 2.72% lower than the 2nd best models, RF, for 2020 and 2021, respectively. The SVM also had the lowest overall MSE for the entire testing period at 0.2067.

The RF model consistently had the second-lowest MSE value for all testing periods, thus highlighting the reliability and robustness of the model over varying market conditions. The LSTM, aside from being the most accurate in 2018, had the 3rd lowest MSE overall, but only 0.095% higher than the RF model. The statistical models had the highest MSE values overall, with the ARIMA outperforming naïve. Compared to the ML models, the simplicity of the statistical models hindered their ability to minimise prediction errors.

Model	2018		2019		2020		2021		All	
	MSE	(<i>r</i> , <i>s</i>)	MSE	(<i>r</i> , <i>s</i>)	MSE	(<i>r</i> , <i>s</i>)	MSE	(<i>r</i> , <i>s</i>)	MSE	(<i>r</i> , <i>s</i>)
Naïve	0.01961	-	0.01375	-	0.03329	-	0.01777	-	0.02111	-
ARIMA	0.01933	-	0.01319	-	0.03391	-	0.01786	-	0.02108	-
LSTM	0.01926	(12, 2)	0.01367	(12, 1)	0.03302	(1, 4)	0.01759	(12, 1)	0.02098	(12, 2)
RF	0.01928	(12, 5)	0.01344	(3, 1)	0.03279	(12, 3)	0.01728	(6, 1)	0.02096	(12, 3)
SVM	0.01946	(12, 1)	0.01386	(12, 7)	0.03168	(6, 1)	0.01681	(3, 1)	0.02067	(6, 1)

TABLE 9.1: The lowest MSE value obtained by all models and their respective hyperparameter configurations (*retrain interval*, *sequence length*) from the USD/ZAR 2018–2021 testing period.

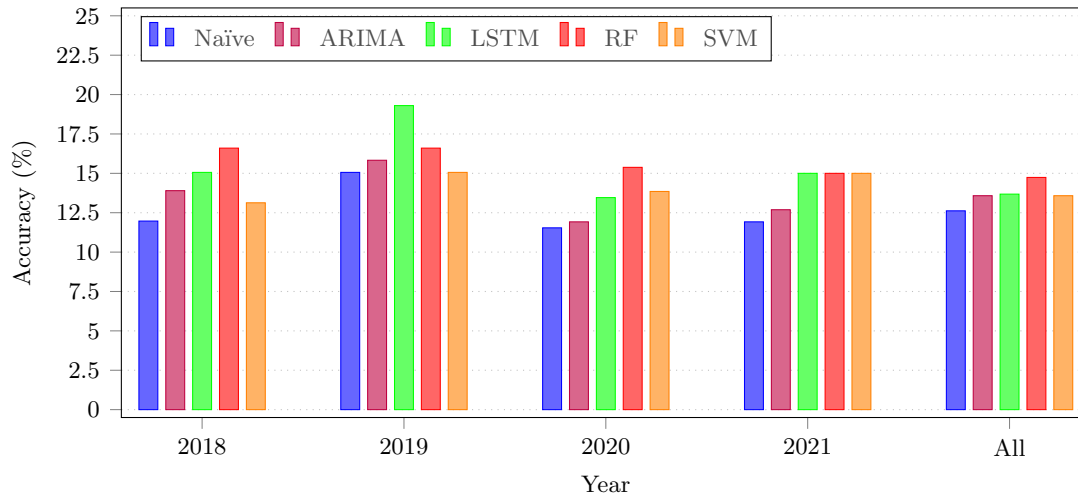
The results for the MAPE are given in Figure 9.2 and exhibited similar trends to the MSE results. The main deviation came from the SVM model, which had the lowest MSE but had the 2nd worst MAPE overall of 0.76017. Thus, the SVM was outperformed by the lowest MAPE model, ARIMA, by 0.638% and had a marginally better MAPE than the naïve by 0.12%. The ARIMA surprisingly outperformed all 3 ML models, suggesting that the scale-independent accuracy was better using the ARIMA model. The ARIMA is likely susceptible to outliers but can achieve closer predictions than any other model.

Model	2018		2019		2020		2021		All	
	MAPE	(<i>r</i> , <i>s</i>)	MAPE	(<i>r</i> , <i>s</i>)	MAPE	(<i>r</i> , <i>s</i>)	MAPE	(<i>r</i> , <i>s</i>)	MAPE	(<i>r</i> , <i>s</i>)
Naïve	0.83258	-	0.63770	-	0.84475	-	0.72904	-	0.76107	-
ARIMA	0.82270	-	0.62907	-	0.84237	-	0.72693	-	0.75532	-
LSTM	0.81480	(12, 2)	0.63613	(6, 6)	0.84000	(12, 2)	0.71832	(12, 1)	0.75640	(12, 1)
RF	0.81906	(12, 5)	0.63243	(12, 2)	0.83845	(12, 6)	0.70861	(6, 1)	0.75554	(6, 1)
SVM	0.83390	(12, 3)	0.64414	(12, 10)	0.82411	(6, 1)	0.70720	(3, 1)	0.76017	(12, 1)

TABLE 9.2: The lowest MAPE (%) value obtained by all models and their respective hyperparameter configurations (*retrain interval*, *sequence length*) from the USD/ZAR 2018–2021 testing period.

The best R0.025 threshold accuracies for each model are displayed in Figure 9.1 during the different testing periods. The RF models had the highest accuracy overall and were only outperformed by the LSTM models in 2019. The LSTM achieved an anomalous accuracy of 19.30% accuracy in 2019, being 2.7% higher than that of the 2nd best model from the RF at 16.60%. The naïve model has the worst threshold accuracy for each period, exposing its simplicity and the notation that prices deviate from their previous day’s value.

The aggregated R0.025 threshold accuracies of all models for the entire testing period (All) contain the smallest range from the best model of RF at 14.74% to the lowest of naïve at 12.62%. The smaller range comes from aggregating results from a single model rather than averaging each model’s highest accuracy. For example, the LSTM(12, 2) had the highest overall accuracy from 2018–2021, but the LSTM(12, 3) was higher in 2018. The LSTM(12, 2) is chosen to represent the ‘All’ category in Figure 9.1 to allow a fairer comparison between the statistical models whose parameters remain constant. The lower accuracy during the entire testing period indicates that different hyperparameter configurations (*retrain interval* and *sequence length*) performed better in different years. Thus, no combination of *retrain interval* and *sequence length* was consistently the best throughout all the testing periods. These results highlight the evolution of market conditions and suggest that models should constantly be adapted to keep up with these changes.

FIGURE 9.1: The threshold accuracies_{0.025} for all models over the 2018–2021 test set.

9.2 Profit metrics

All models were trained on minimising MSE and incorporating a trading strategy is *ad hoc* to model optimisation. However, after investigating each model's result section, the models with higher accuracy appear more profitable. Therefore, a strong correlation between the MSE and cumulative profit exists. Table 9.3 contains the highest cumulative profit for each model throughout the different testing periods and the corresponding maximum drawdown to measure the risk of the trading performance.

The RF model achieved higher profits than the statistical models for every year in the testing period and had the highest cumulative profits in 2018 and 2019 at 25.15% and 7.26%, respectively. The SVM models in 2020 and 2021 saw impressive 36.45% and 32.29% cumulative profits; they also had the worst maximum drawdown for both periods. The high profits and low drawdowns suggest that the SVM models are more volatile than any other model and are susceptible to periods of continuous losses. The LSTM model had the 2nd best overall profit at 34.43% with the best drawdown of -7.76% . Therefore, the LSTM model would be preferred by risk-averse traders who prefer consistent profits, albeit smaller than the SVM, with fewer periods of consecutive losses.

	2018		2019		2020		2021		All	
Model	Profit	Draw	Profit	Draw	Profit	Draw	Profit	Draw	Profit	Draw
Naïve	5.6	-7	-4.03	-6.12	8.15	-5.19	6.53	-2.24	16.27	-10.77
ARIMA	0.13	-9.49	4.04	-2.56	-1.58	-5.02	-2	-5.02	0.6	-9.49
LSTM	20.84	-7.76	2.8	-9.19	17.36	-4.46	6.84	-2.92	34.43	-7.76
RF	25.15	-3.9	7.26	-5.69	21.4	-7.34	9.01	-5.58	28.96	-13.48
SVM	20.27	-7.6	-2.62	-7.06	36.45	-7.9	32.29	-8.15	50.22	-31.66

TABLE 9.3: The highest cumulative profit (%) (Profit) and its corresponding maximum drawdown (%) (Draw) values obtained by all models from the USD/ZAR 2018–2021 testing period.

The highest cumulative profits obtained from each model are shown in Figure 9.2. All ML models follow similar cumulative profits to the naïve up until 2018-10-12, after which all ML models deviate. The LSTM model's low volatility is observed throughout the testing period compared to the RF and SVM. The SVM model loses -31.66% of its cumulative profit and experiences a

trough on 2020-02-5 at -19.94% . After the trough, the SVM model capitalises on the volatility and large price movements in 2020 and manages to negate all the losing trades, ending the year with a cumulative profit of 9.87% . In 2021, the SVM was again able to capitalise on market conditions and achieve a further 32.29% , ending the testing period at the highest cumulative profit out of all of the models at 50.22% . The ARIMA model's cumulative profit was close to breakeven at 0.6% , displaying no large deviations from 0. The naïve, similar to the SVM, was able to make the majority of its profit during the 2020 and 2021 periods; however, it had smaller drawdowns and consistent periods of profit.

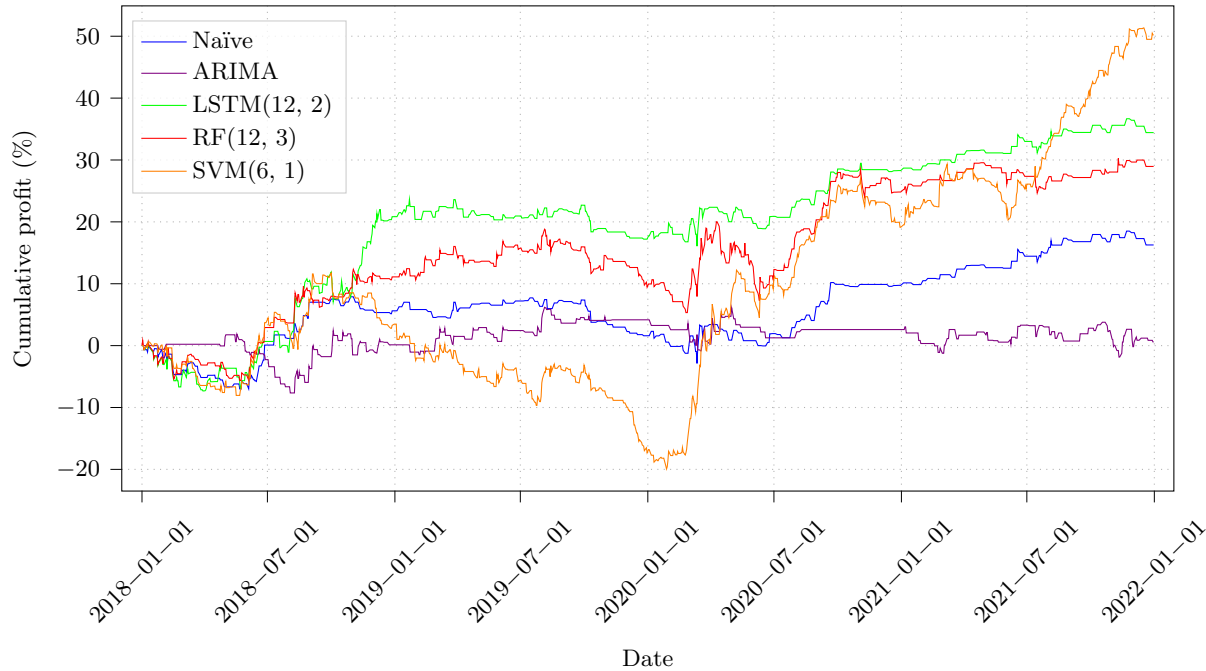


FIGURE 9.2: The highest cumulative profit (%) obtained by all models from 2018–2021 testing period.

The trading performance is further analysed through the percentage of profitable trades and the maximum number of trades in Table 9.4. The ML models dominate the statistical models regarding the profitable trades percentage metric. The LSTM model made the most profitable trades out of all the models, achieving the highest percentage in 2018 and 2020. This suggests the LSTM could correctly predict the direction of the USD/ZAR closing prices when the movements exceeded the spread 51.01% of the time. The RF model was 2nd best at just under 50% , with the highest profitable trades percentage observed in 2021. The SVM had the worst profitable trades percentage out of the 3 ML models, although still higher than the statistical models. This is a surprise, as the SVM had the highest cumulative profits from all the models. Speculating on these results, the SVM did execute fewer trades; therefore, the profitable trades made by the SVM model had a greater magnitude in profit than the subsequent models with higher profitable trade percentages.

These findings show that the profitable trades percentage does not necessarily indicate that a model will make a profit or the size of the profit, as models below 50% have more losing than winning trades. Therefore, picking the right winning trades that result in higher profits is more important, which can offset the smaller, albeit more frequent, losing trades.

The cumulative profits achieved by the ML models were higher than those of the statistical models, and the number of trades metric gives more insight into the value predicted for each

subset of models. The trading strategy is only executed if the predicted price movement exceeds the spread (0.1%). Therefore, given the low number of total trades, 201 for naïve and 203 for ARIMA, the statistical models do not predict large price movements above 0.1% often compared to the ML models. The ML models, on average, based on the ‘All’ period in Table 9.4, predict a movement above the spread (0.1%), 71.76%, 73.80% and 59.40% of the time for the LSTM, RF and SVM models, respectively. Comparing the ML model’s number of trade percentages to that of the statistical models of around 19.39%, with the addition of improved accuracy (MSE) for the ML models, it is unsurprising that the ML models had better trading performances.

Model	2018		2019		2020		2021		All	
	P trades	Num	P trades	Num	P trades	Num	P trades	Num	P trades	Num
Naïve	42.86	70	42.86	42	46.15	52	48.65	37	44.78	201
ARIMA	37.29	59	43.40	53	48.28	29	46.47	62	43.35	203
LSTM	55.10	211	46.32	203	56.59	191	52.11	198	51.01	748
RF	53.68	211	49.43	190	53.04	211	60.87	179	49.46	769
SVM	48.15	148	41.46	147	56.52	175	60.47	161	48.88	619

TABLE 9.4: The highest profitable trades (%) (P trades) and the highest number of trades (Num) values obtained by all models from the USD/ZAR 2018–2021 testing period.

9.3 Summary

Table 9.5 contains a summary of the best model for each metric in every testing period. No single model achieved the best results in every category, with the results appearing sporadic for each year and metric tested. This highlights the complexity of the USD/ZAR data set; with constantly changing market conditions, different models can perform better throughout the years. The summary indicates that the LSTM model is the best choice if a conservative trading approach is preferred, as the model has the best risk-related trading metrics. The SVM was the most accurate model in 2020 and 2021, having the best accuracy-related metrics, MSE and MAPE, and the highest cumulative profit, but with more volatility. The RF model had the best threshold accuracies, with it having the highest threshold accuracy in at least 2 of the 4 threshold accuracies each year. In 2018, the RF model had the highest accuracy for all threshold values. Therefore, if getting predictions within a specific margin is valuable, then the RF model is a suitable choice.

Metric	Period				
	2018	2019	2020	2021	All
MSE	LSTM	ARIMA	SVM	SVM	SVM
MAPE	LSTM	ARIMA	SVM	SVM	ARIMA
Threshold of R0.025 (%)	RF	LSTM	RF	LSTM, RF, SVM	RF
Threshold of R0.05 (%)	RF	LSTM	LSTM	SVM	LSTM
Threshold of R0.075 (%)	RF	RF	RF	SVM	RF
Threshold of R0.1 (%)	LSTM, RF	RF	LSTM	RF	ARIMA, RF
Cumulative profit (%)	RF	RF	SVM	SVM	SVM
Maximum drawdown (%)	RF	ARIMA	LSTM	Naïve	LSTM
Average profit per trade (%)	RF	ARIMA	SVM	SVM	LSTM
Profitable trades (%)	LSTM	RF	SVM	SVM	LSTM

TABLE 9.5: The best models for each performance metric over the 2018–2021 USD/ZAR testing set.

CHAPTER 10

Conclusion

Contents

10.1 Summary of findings	133
10.2 Recommendations	136
10.3 Achievement of objectives	136
10.4 Ideas for future work	137

The aim of this study was to investigate the impact of the ML model's retraining intervals and the number of lagged data points to use as features. The effects were measured on the predictive performance of three ML models on foreign exchange rate forecasting. Performance metrics associated with accuracy and profit enabled measuring the subsequent effects from the testing parameters and allowed comparisons between models. Additionally, Bayesian optimisation was used to find the best hyperparameter configuration for each ML model. This chapter contains a summary of the key findings from each ML model's results and recommendations from the aggregated findings. Finally, the achievement of objectives and areas to expand the study are discussed.

10.1 Summary of findings

This study considered three ML models, LSTM, RF and SVM, to investigate the effects of varying the retraining interval and sequence lengths. Additionally, two statistical forecasting models, naïve and ARIMA, were implemented as a benchmark to test the comparative performance between the more complex ML models and less sophisticated statistical models.

The data set used was the foreign exchange pair of the USD/ZAR and their OHLC prices, with the addition of popular technical indicators that showcased strong performance in literature and among traders. Initially, the non-stationary feature set would have contained mismatched scales between the training and testing sets. The data was transformed using log percentage conversions and standardisation to improve interpretability and avoid common ML pitfalls.

The architecture and hyperparameters of the three ML models were calibrated using Bayesian optimisation, which incorporated previous iterations into the algorithm's search for the best configurations. The findings for each model's hyperparameter calibration results can be summarised as follows:

1. **LSTM:** Convergence issues occurred with iterations using a combination of the ReLu activation function and a network depth above nine layers. During Bayesian optimisation, the LSTM model's best hyperparameter configuration had the lowest MSE, 0.02026, of all three ML models. The different activation functions had the biggest effect on the LSTM model, with the sigmoid function achieving the best accuracy overall. The LSTM model had the longest run times with an average of 76 minutes, compared to the 1.5 and 2.45 minutes of the SVM and RF models, respectively.
2. **RF:** The RF model had the most consistent MSE values during Bayesian optimisation, containing no outliers. However, the RF model's best hyperparameter configuration had the worst MSE of all three ML models at 0.02143, which was 5.77% and 4.95% higher than the LSTM and SVM, respectively. The method used to train the RF *i.e.*, sampling with or without replacement (bagging or pasting), had the most notable impact on the RF models, with the bagging method outperforming pasting on average.
3. **SVM:** Similar convergence issues were seen in the SVM hyperparameter calibration caused by the sigmoid and polynomial kernel functions that transform the data into a higher dimension. The SVM had the 2nd lowest MSE of 0.02042 from its best hyperparameter configuration. The kernel function had the largest impact on the SVM model, with the RBF function obtaining the lowest and most consistent MSE values.

The best hyperparameter configurations were used for each ML model during the 2018–2021 testing period of the USD/ZAR data set. The effects of retraining interval and sequence length were tested over 40 combinations of the two hyperparameters, with the search spaces given in Table 10.1.

Parameter	Search space	Timeframe
Retrain interval	[12, 6, 3, 1]	Months
Sequence length	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]	Days

TABLE 10.1: *The search spaces for retraining interval and sequence length.*

The results for all hyperparameter combinations were annualised for different testing years. Each of the ML model's results are summarised as follows:

1. **LSTM:** The retraining intervals had a more prominent effect than the sequence length. The two longer intervals (12 and 6) performed better overall, with a noticeable decline in accuracy from the two shorter intervals (3 and 1). The best average MSE was from the 12-month interval at 0.0212, which was an improvement of 0.51%, 3.06% and 5.53% over the 6, 3 and 1-month intervals, respectively. The LSTM models also performed better with higher sequence lengths above three days, with the lowest MSE coming from a 9-day sequence length of 0.02157. The 1–3 day sequence lengths had the worst MSEs, which were 1.76%, 1.44% and 1.34% higher than the lowest MSE, respectively. These results indicate that the LSTM models prefer less frequent updates and more features, thus enabling them to remain stable with good performance for longer. The LSTM models are, therefore, less susceptible to model decay over 12 months. The more frequent updates cause instability in the LSTM models, making them biased towards recent observations and unable to capture long-term dependencies in the data set.
2. **RF:** The effects of different retraining intervals showed similar results to the LSTM models. However, the disparity between the MSE values for the different intervals was exaggerated,

with a sharper decline in performance for the 3-month and 1-month intervals. The 12-month MSE was again the lowest at 0.0211, 0.47% lower than the LSTM model. The 6, 3, and 1-month intervals were 1.54%, 6.35% and 10.71% worse than the 12-month MSE. Sequence lengths had a minimal effect on the RF model's accuracies. However, the 1-day sequence length had the best MSE overall of 0.02177 and 1.54% lower than the 2nd lowest sequence length. The sequence lengths of 2–10 days had similar performance with all these MSE values within 0.85% of each other. The RF model's results suggest that retraining intervals affect model performance and the generalisation of predictions, with longer intervals being more stable than shorter ones. Additionally, the number of features has a smaller effect on performance, indicating that a single time step of the chosen features is adequate to characterise the USD/ZAR data set.

3. **SVM:** The SVM models showed opposite findings to the LSTM and RF models. The sequence lengths had a larger impact than the retraining intervals. Interestingly, the preferred sequence lengths depended on market conditions during that year. The 2018–2019 testing periods had better accuracies at a higher sequence length (6–10), with worse accuracies coming from the lower sequence lengths (1–5). The 2020–2021 testing period performed best from the 1-day sequence length, with the SVM models with a 1-day sequence length, regardless of the retraining interval, dominating all other sequence lengths. These two years saw exceptionally high cumulative profits of 36.45% and 32.29% for 2020 and 2021, respectively. The MSE values from the different retraining intervals were within 0.14%, indicating minimal effects from this hyperparameter. Overall, the SVM had consistently low MSE values with the smallest difference between the best and worst-performing SVM models.

The best hyperparameter configurations regarding each ML model's retraining interval and sequence length based on the MSE are given in Table 10.2. The results confirm that longer retrain intervals and fewer sequences are preferred. However, there are exceptions, with shorter retraining intervals having better accuracy in at least one of the testing years for each model. Notably, 2019 saw the 3-month interval have the best MSE for the RF models. The LSTM models in 2020 performed better for the 1-month retrain intervals, and retraining the SVM models every three months in 2021 led to better performance.

	(retrain interval, sequence length)				
Model	2018	2019	2020	2021	All
LSTM	(12, 2)	(12, 1)	(1, 4)	(12, 1)	(12, 2)
RF	(12, 5)	(3, 1)	(12, 3)	(6, 1)	(12, 3)
SVM	(12, 1)	(12, 7)	(6, 1)	(3, 1)	(6, 1)

TABLE 10.2: The best hyperparameter configuration of retraining interval and sequence length for each ML model through the 2018–2021 testing period based on MSE.

The comparative results between all five models: LSTM, RF, SVM, naïve, and ARIMA showed that the SVM models had the lowest MSE and highest cumulative profits due to their superior performance in 2020 and 2021. Surprisingly, the ARIMA model had the best MAPE but 4th worst MSE, indicating this model had the closest predictions but was more susceptible to outliers *i.e.*, larger price movements. The RF models were the most consistent across all metrics and usually had the 2nd best score, thus highlighting the reliability of these models. The LSTM models were average across all the metrics, thus never performing the worst but seldomly outperforming the other models. The LSTM had the most risk-averse trading performance with the 2nd highest cumulative profit with the best maximum drawdown. The naïve model performed

the worst since expecting prices to remain at their previous day's value is unrealistic. However, the naïve's performance was not far off of the other models and outperformed specific models during certain periods in the testing set. This indicates that foreign exchange prices deviate daily, but the movements are considerably small on average.

10.2 Recommendations

No single retraining interval and sequence length configuration outperformed all others through the four-year testing period. This emphasises the difficulty of static models to consistently have the best accuracy and stay profitable for an evolving data set. Even for the best LSTM(12, 2), RF(12, 3) and SVM(6, 1) models, there were winning and losing years, with different configurations outperforming them. Therefore, there is a need to test and constantly evolve models throughout the years to improve performance to match the conditions. As market conditions change, so does the model's performance. The LSTM and RF models show that the retraining interval is an important hyperparameter impacting performance. The number of lagged data points *i.e.*, sequence length is also worth consideration, as shown by the results of the SVM models.

10.3 Achievement of objectives

The objectives pursued in this study were identified in Chapter 1 as:

1. Provide a background to foreign exchange markets, ML, and the problem addressed.
2. Discuss existing literature on forecasting financial time series data with ML models.
3. Review, clean, validate and describe the data set used.
4. Based on technical analysis and literature performance, generate features for ML models.
5. Preprocess the data set into a viable format for all models.
6. Discuss the modelling process used in this study.
7. Select and describe appropriate ML and statistical models for time series forecasting.
8. Perform hyperparameter calibration and interpret the results on all applicable models.
9. Evaluate the performance of statistical models.
10. Investigate the ML model's forecasting performance regarding the different retraining intervals and sequence lengths.
11. Discuss the comparative performance between the different models.
12. Make recommendations based on results, summarise the study and discuss ideas for future research.

Objective 1 was achieved in Chapter 1 by providing a background to ML and financial markets and addressing the context of this study's research question. In Chapter 2, relevant literature regarding ML models used for financial times series was discussed, thus achieving Objective 2.

Objectives 3 and 4 were accomplished in Chapter 3, where an analysis of the data set and a description of the technical indicators were given. Chapter 4 contains an overview of the modelling process used and a detailed description of the methods incorporated in the study, therefore attaining Objective 6. In Chapters 5, 6, 7 and 8, the respective models implemented are discussed concerning the model's background, architecture, hyperparameter configuration, hyperparameter calibration results, and results for the different retraining intervals and sequence lengths. Therefore achieving Objectives 7, 8, 9 and 10. In Chapter 9, Objective 11 is achieved by comparing and evaluating the performance between the different models. Finally, the study concludes with accomplishing Objective 12 in Chapter 10 by summarising the results and giving potential areas for future work.

10.4 Ideas for future work

The scope of this study was limited to a fixed time series and feature set, focusing on the model's training frequency and the number of lagged features used. The study can be further expanded to incorporate more sophisticated feature selection and feature extraction techniques such as principle component and linear discriminant analysis, thus deepening the study. The training and test splits were kept at a constant 80/20 percentage split, resulting in the lower retraining intervals having fewer data points to train on. Another approach to deepen the study involves keeping the training and testing sets constant for each retraining interval and investigating if the effects caused by the number of data points used in training have a greater effect than the retraining interval.

The ML model's hyperparameter calibration incorporated the different retraining intervals and sequence length combinations. Therefore, the ML model's hyperparameters were set up to generalise to all combinations. Instead, hyperparameter calibration for each of the 40 combinations could be performed to optimise model hyperparameters for each retraining interval and sequence length. However, this approach requires careful examination of the experimental setup and whether the effects on performance arise from the hyperparameters or testing parameters.

The study only focuses on a single currency pair in the foreign exchange market. An approach to widening the study would be to test additional data sets, either different currency pairs or explore different financial markets such as the stock and commodities markets. Adding data sets will test whether different markets react differently to retraining intervals and sequence lengths. The conclusions could find that the retraining intervals differ between markets.

Another approach to widen the study is to include recent advances in DNNs, particularly the emergence of Gated Recurrent Unit (GRU) networks. While the study's focus has been on LSTM models, it is imperative to recognise the potential benefits that GRU networks offer. GRU networks, characterised by their simplified architecture and efficient training process, have demonstrated promising results in various applications [24]. Future work could explore the comparative performance of GRU networks against LSTM networks in the context of retraining intervals and sequence lengths, considering factors such as training times, predictive accuracy and profitability.

APPENDIX A

Hyperparameter calibration results

This chapter contains all the results from the hyperparameter calibration performed on the LSTM, RF, SVM and ARIMA models. The ML models, LSTM, RF and SVM use Bayesian optimisation, while the ARIMA model uses a grid search as its hyperparameter calibration technique.

A.1 LSTM

Table A.1 contains the different parameters, with their corresponding abbreviation, search space bounds and best values found through 150 iterations of Bayesian optimisation. The entire list of iterations with their respective hyperparameter configuration, performance on selected metrics for the test and training set and run times can be seen in Tables A.2, A.3, A.4 and A.5.

Hyperparameter	Abbreviation	Search space	Best value
number of layers	n_l	[1, 2, 3, ..., 10]	3
number of nodes	n_n	[16, 17, 18, ..., 512]	95
activation function	act_f	[ReLU, sigmoid, tanh]	sigmoid
initialiser	init	[glorot_uniform, he_normal, lecun_normal]	glorot_uniform
dropout rate	drop	[0, ..., 0.5]	0.005
batch size	b_s	[16, 17, 18, ..., 128]	83
learning rate	l_rate	[0.0001, ..., 0.1]	0.0664

TABLE A.1: The LSTM hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
1	3	44	tanh	he_norm	0.366	122	0.0157	0.02131	-5.63	0.01931	-0.18	100.76
2	10	121	sigm	glo_unif	0.354	83	0.0970	0.02054	-3.22	0.01891	2.63	37.86
3	5	319	ReLU	he_norm	0.152	36	0.0433	0.02726	-3.14	0.01895	0.96	97.74
4	4	271	ReLU	he_norm	0.183	48	0.0785	0.04714	-6.64	0.02130	0.18	32.01

TABLE A.2: The hyperparameter calibration results for the LSTM model (a).

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
5	11	494	tanh	glo_unif	0.304	21	0.0066	0.02059	-1.25	0.01889	1.53	38.87
6	4	261	sigm	lec_norm	0.049	50	0.0441	0.02069	-3.22	0.01895	2.63	49.61
7	7	287	ReLu	he_norm	0.129	117	0.0312	0.04665	-3.24	0.01874	3.91	51.48
8	8	473	ReLu	lec_norm	0.388	124	0.0895	3.90259	-6.00	2435.68	-9.29	48.60
9	5	427	ReLu	glo_unif	0.023	37	0.0389	0.02096	-1.22	0.01883	3.07	84.66
10	3	505	tanh	glo_unif	0.271	47	0.0802	0.02335	-7.63	0.01934	0.79	79.29
11	9	398	sigm	lec_norm	0.003	38	0.0707	0.02045	-2.80	0.01899	0.68	37.76
12	5	47	ReLu	lec_norm	0.058	56	0.0624	0.02080	-3.47	0.01890	3.30	103.36
13	7	75	ReLu	he_norm	0.365	52	0.0887	0.02162	-3.29	0.01897	3.12	34.60
14	7	228	sigm	lec_norm	0.281	101	0.0494	0.02050	-2.84	0.01893	2.64	42.70
15	7	466	ReLu	he_norm	0.016	28	0.0315	0.10835	-5.28	0.01935	0.80	112.78
16	5	95	ReLu	glo_unif	0.378	61	0.0078	0.02396	-3.43	0.01866	4.69	57.69
17	4	458	sigm	lec_norm	0.317	106	0.0804	0.02055	-2.84	0.01892	2.64	46.07
18	5	227	tanh	glo_unif	0.448	106	0.0111	0.02062	-1.78	0.01879	4.97	108.26
19	4	75	sigm	he_norm	0.003	112	0.0418	0.02058	-1.55	0.01887	2.69	104.40
20	6	498	tanh	he_norm	0.162	121	0.0703	0.02077	-1.77	0.01932	1.93	30.85
21	3	318	sigm	glo_unif	0.249	44	0.0286	0.02057	-1.26	0.01896	2.07	52.44
22	4	258	tanh	lec_norm	0.139	21	0.0240	0.02138	-2.57	0.01863	2.75	54.74
23	9	198	sigm	lec_norm	0.336	43	0.0238	0.02067	-1.57	0.01900	2.00	96.67
24	5	108	tanh	glo_unif	0.268	86	0.0835	0.02103	-2.66	0.01882	3.80	85.30
25	5	336	ReLu	glo_unif	0.339	82	0.0513	0.02118	-5.35	0.01900	2.70	110.92
26	6	72	ReLu	lec_norm	0.193	93	0.0138	0.02117	-3.76	0.01859	3.53	84.24
27	7	278	sigm	he_norm	0.129	114	0.0817	0.03081	-3.39	0.02747	2.38	43.00
28	6	189	ReLu	lec_norm	0.449	26	0.0633	0.02081	-3.16	0.01897	2.04	48.94
29	3	96	sigm	lec_norm	0.444	116	0.0642	0.02049	-2.70	0.01885	3.45	56.70
30	3	95	sigm	glo_unif	0.005	83	0.0664	0.02026	-0.97	0.01896	2.20	95.31
31	5	177	tanh	glo_unif	0.326	93	0.0712	0.02289	-4.45	0.01912	2.99	93.00
32	3	198	sigm	he_norm	0.425	88	0.0569	0.02060	-1.57	0.01891	2.00	84.05
33	8	410	ReLu	he_norm	0.487	43	0.0892	2.8E+51	-24.59	4.4E+12	-27.20	59.45
34	5	28	tanh	glo_unif	0.246	80	0.0723	0.02040	-2.24	0.01916	-0.01	30.70
35	6	23	tanh	lec_norm	0.470	35	0.0915	0.02063	-2.46	0.01862	3.73	79.25
36	5	207	sigm	lec_norm	0.483	63	0.0853	0.02056	-1.57	0.01892	2.00	64.14
37	9	298	sigm	he_norm	0.085	51	0.0936	0.02045	-2.83	0.01895	0.71	31.47
38	10	383	ReLu	glo_unif	0.495	84	0.0519	0.02060	-3.22	0.01896	2.63	114.77
39	10	446	sigm	glo_unif	0.180	94	0.0810	0.02062	-3.22	0.01892	2.63	96.74
40	9	410	sigm	lec_norm	0.251	73	0.0650	0.02100	-3.22	0.01899	2.63	110.06
41	3	246	sigm	glo_unif	0.188	53	0.0579	0.02068	-2.91	0.01890	2.79	107.40
42	10	194	tanh	glo_unif	0.295	48	0.0038	0.02078	-1.14	0.01911	0.72	54.87
43	3	41	ReLu	glo_unif	0.385	74	0.0623	0.02100	-3.20	0.01872	1.03	93.38
44	7	176	tanh	lec_norm	0.319	76	0.0976	0.02105	-4.11	0.01894	2.83	61.47
45	11	430	sigm	glo_unif	0.219	46	0.0026	0.02048	-2.91	0.01894	2.70	64.27
46	7	370	sigm	glo_unif	0.087	61	0.0251	0.02069	-1.22	0.01903	1.48	74.24
47	8	224	tanh	lec_norm	0.477	47	0.0555	0.02069	-1.50	0.01893	2.07	106.55
48	3	36	ReLu	glo_unif	0.379	55	0.0117	0.02482	-2.57	0.01884	4.00	71.61
49	7	101	sigm	glo_unif	0.237	94	0.0492	0.02079	-3.42	0.01911	1.76	31.73
50	6	326	tanh	he_norm	0.308	60	0.0046	0.02188	-3.28	0.01878	3.38	30.83
51	6	229	sigm	he_norm	0.242	102	0.0258	0.02055	-2.55	0.01895	2.32	134.77
52	7	389	tanh	le_norm	0.451	35	0.0766	0.02109	-4.92	0.01884	0.71	65.54
53	12	396	sigm	le_norm	0	26	0.0001	0.02051	-1.29	0.01900	1.97	100.51
54	4	440	ReLu	le_norm	0.222	36	0.0436	0.02531	-3.55	0.01921	3.30	62.12
55	8	87	ReLu	le_norm	0.467	100	0.0385	0.02081	-2.63	0.01921	0.92	141.28
56	10	105	sigm	he_norm	0.394	72	0.0944	0.02054	-2.94	0.01891	2.60	68.86
57	8	432	sigm	he_norm	0.402	24	0.0675	0.02059	-3.22	0.01892	2.63	55.75
58	7	192	tanh	le_norm	0.011	73	0.0233	0.02096	-3.59	0.01884	2.29	75.70
59	8	38	tanh	le_norm	0.116	41	0.1	0.02063	-1.22	0.01900	1.03	50.66
60	12	84	ReLu	le_norm	0.5	85	0.0115	0.02066	-0.62	0.01918	-0.08	31.22

TABLE A.3: The hyperparameter calibration results for the LSTM model (b).

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
61	4	84	ReLU	le_norm	0.454	71	0.0545	0.02081	-0.63	0.01882	2.40	60.43
62	12	210	sigm	glo_unif	0	51	0.0001	0.02083	-0.91	0.01920	-0.04	61.46
63	12	387	sigm	le_norm	0	46	0.0001	0.02060	-1.03	0.01906	0.62	109.85
64	6	443	sigm	glo_unif	0.294	50	0.0996	0.02050	-3.22	0.01893	2.63	100.28
65	10	221	sigm	glo_unif	0	62	0.0001	0.02070	-2.32	0.01910	0.87	50.17
66	11	206	tanh	le_norm	0.339	79	0.0567	0.02058	-1.40	0.01894	2.38	88.55
67	9	262	ReLU	glo_unif	0.5	35	0.1	2.11864	-9.14	3.01897	-12.62	29.93
68	7	247	ReLU	le_norm	0.163	37	0.0561	0.02389	-5.81	0.01994	-0.16	54.63
69	10	70	ReLU	le_norm	0.304	76	0.0865	0.02043	-0.04	0.01900	0.52	103.48
70	10	63	ReLU	he_norm	0.244	63	0.0636	0.03229	-1.48	0.01880	3.64	111.71
71	3	57	sigm	le_norm	0	74	0.0001	0.02063	-0.85	0.01901	1.98	105.23
72	9	60	tanh	he_norm	0.448	47	0.0897	0.02077	-1.17	0.01909	0.88	83.38
73	4	310	ReLU	glo_unif	0.196	60	0.0522	0.02204	-1.80	0.01898	2.15	82.14
74	8	304	ReLU	glo_unif	0.012	38	0.0792	1004777	-6.55	59.852	-1.70	55.06
75	3	218	sigm	glo_unif	0.5	75	0.0116	0.02044	-3.19	0.01887	2.47	82.15
76	3	213	sigm	he_norm	0.347	92	0.0270	0.02092	-2.42	0.01873	3.03	98.81
77	12	58	ReLU	glo_unif	0.5	87	0.1	0.02068	-1.34	0.01904	0.55	85.98
78	7	44	sigm	le_norm	0.455	89	0.0805	0.02046	-2.05	0.01900	1.53	29.82
79	3	57	sigm	le_norm	0.123	99	0.0072	0.02125	-2.35	0.01855	4.99	83.52
80	6	59	ReLU	glo_unif	0.343	115	0.0775	0.02059	-1.65	0.01890	3.22	106.24
81	8	46	ReLU	he_norm	0.011	105	0.0347	0.02137	-2.63	0.01884	2.29	99.45
82	3	320	tanh	glo_unif	0.418	74	0.0971	0.02259	-4.75	0.02020	-0.47	41.76
83	11	25	sigm	he_norm	0.391	51	0.0218	0.02080	-0.67	0.01916	0.24	111.15
84	6	25	ReLU	he_norm	0.123	66	0.0355	0.02129	-1.95	0.01888	3.19	62.10
85	3	456	tanh	he_norm	0.431	40	0.0160	0.02104	-4.91	0.01909	0.66	92.13
86	10	450	ReLU	glo_unif	0.421	27	0.0671	9.02398	-24.27	11.48146	-22.56	79.93
87	7	183	sigm	glo_unif	0.220	60	0.0599	0.02056	-3.22	0.01890	2.63	53.01
88	3	183	sigm	le_norm	0.433	41	0.1	0.02049	-3.22	0.01893	2.63	89.92
89	7	30	sigm	glo_unif	0.302	97	0.0938	0.02058	-4.12	0.01904	-1.63	30.00
90	7	32	ReLU	le_norm	0.232	113	0.0287	0.02083	-0.94	0.01894	3.04	98.70
91	5	287	ReLU	he_norm	0.141	42	0.0310	0.02103	-0.88	0.01877	4.16	116.40
92	3	275	sigm	le_norm	0	32	0.0001	0.02080	-2.18	0.01912	-0.09	95.78
93	3	284	sigm	he_norm	0	57	0.0736	0.02048	-1.54	0.01900	0.32	76.63
94	7	296	sigm	le_norm	0.308	67	0.0072	0.02045	-1.26	0.01892	2.07	34.17
95	11	117	sigm	he_norm	0.262	98	0.0139	0.02056	-1.19	0.01898	2.01	30.34
96	12	224	ReLU	le_norm	0	85	0.0001	0.02059	-0.39	0.01920	0.21	79.06
97	5	233	sigm	glo_unif	0.237	72	0.0500	0.02070	-1.57	0.01896	2.00	97.15
98	5	110	sigm	glo_unif	0.408	110	0.0737	0.02087	-3.03	0.01883	2.87	67.47
99	5	291	tanh	he_norm	0.183	27	0.0910	0.02160	-2.68	0.01869	3.10	33.23
100	3	235	sigm	glo_unif	0.5	87	0.1	0.02066	-2.12	0.01896	1.75	101.59
101	11	276	ReLU	he_norm	0.075	20	0.0416	0.66245	-9.45	0.57720	-6.70	64.30
102	3	269	ReLU	le_norm	0.348	64	0.0323	0.02320	-1.16	0.01834	4.06	92.09
103	11	256	sigm	he_norm	0.235	63	0.0102	0.02062	-1.57	0.01893	2.00	55.04
104	11	236	sigm	glo_unif	0.249	57	0.0758	0.02057	-3.85	0.01893	0.98	89.81
105	4	248	ReLU	glo_unif	0.368	76	0.0699	0.02152	-3.23	0.01847	3.30	104.71
106	5	264	tanh	glo_unif	0.073	77	0.0497	0.02091	-2.43	0.01864	3.48	102.19
107	8	281	sigm	he_norm	0.490	71	0.0646	0.02054	-3.22	0.01892	2.63	83.38
108	12	308	ReLU	le_norm	0	74	0.0001	0.02059	-0.39	0.01920	0.21	114.50
109	6	251	sigm	glo_unif	0.037	90	0.0535	0.02069	-2.04	0.01927	1.14	77.83
110	8	321	ReLU	glo_unif	0.401	89	0.0125	0.02071	-0.91	0.01916	-0.04	107.27
111	5	293	tanh	glo_unif	0.304	82	0.0177	0.02101	-3.84	0.01865	3.46	78.62
112	6	307	tanh	le_norm	0.142	90	0.0765	0.02118	-5.44	0.01897	2.52	30.57
113	8	277	tanh	he_norm	0.376	88	0.0458	0.02110	-5.89	0.01859	4.05	108.93
114	9	286	ReLU	le_norm	0.018	101	0.0848	0.02082	-4.86	0.01976	-1.11	53.31
115	10	270	sigm	he_norm	0.047	101	0.0893	0.02047	-2.84	0.01891	2.64	64.10
116	12	330	ReLU	le_norm	0	46	0.1	25.11	-11.96	3678.8	-10.82	104.04

TABLE A.4: The hyperparameter calibration results for the LSTM model (c).

N	LSTM parameters							Test set		Train set		Time (mins)
	n_l	n_n	act_f	init	drop	b_s	l_rate	MSE	Profit	MSE	Profit	
117	7	212	ReLu	le_norm	0.050	35	0.0110	0.02065	-1.64	0.01896	1.70	90.97
118	4	300	sigm	le_norm	0.238	105	0.0847	0.02051	-1.29	0.01892	1.97	75.53
119	4	307	sigm	glo_unif	0.495	24	0.0519	0.02058	-4.01	0.01901	1.77	31.03
120	3	316	tanh	he_norm	0.128	103	0.0879	0.02238	-3.94	0.01994	-1.87	99.96
121	3	331	tanh	le_norm	0.364	98	0.0046	0.02237	-4.14	0.01917	1.77	106.51
122	4	232	sigm	glo_unif	0.254	33	0.0404	0.02054	-0.71	0.01898	0.83	89.33
123	4	243	tanh	le_norm	0.318	20	0.0640	0.02114	-5.07	0.01904	-0.23	54.57
124	11	203	sigm	glo_unif	0.284	23	0.0970	0.02060	-3.78	0.01917	-1.05	31.74
125	4	219	ReLu	glo_unif	0.182	22	0.0528	0.02067	-2.05	0.01883	2.08	42.47
126	8	399	ReLu	glo_unif	0.139	83	0.0273	0.02061	-0.67	0.01909	0.24	110.96
127	11	186	sigm	he_norm	0.265	85	0.0288	0.02071	-1.97	0.01906	-1.10	108.39
128	9	189	tanh	he_norm	0.135	102	0.0625	0.02083	-2.18	0.01897	1.93	107.94
129	6	204	tanh	le_norm	0.394	105	0.0576	0.02066	-3.06	0.01882	3.78	76.11
130	8	384	sigm	glo_unif	0.220	69	0.0554	0.02058	-3.85	0.01893	0.98	37.45
131	3	372	tanh	he_norm	0.259	76	0.0258	0.02271	-3.62	0.01922	-0.67	35.69
132	11	339	sigm	le_norm	0.100	68	0.0023	0.02059	-1.57	0.01892	2.00	70.41
133	9	357	sigm	he_norm	0.417	72	0.0256	0.02055	-1.57	0.01892	2.00	33.97
134	6	341	sigm	he_norm	0.265	55	0.0392	0.02061	-1.57	0.01892	2.00	97.47
135	12	348	sigm	le_norm	0	84	0.0001	0.02069	-1.89	0.01905	1.79	71.30
136	11	366	tanh	le_norm	0.348	88	0.0049	0.02074	-4.11	0.01882	3.34	77.13
137	5	355	ReLu	le_norm	0.431	56	0.0766	0.56975	-2.63	0.03041	-2.62	113.60
138	6	348	ReLu	glo_unif	0.396	98	0.0526	0.02186	-5.92	0.01880	1.02	86.54
139	10	346	ReLu	glo_unif	0.374	41	0.0231	0.02063	-0.67	0.01911	0.24	120.77
140	7	333	sigm	he_norm	0.000	31	0.0127	0.02039	0.27	0.01897	0.59	86.17
141	10	369	sigm	le_norm	0.111	46	0.0204	0.02056	-2.55	0.01893	2.32	60.88
142	4	322	sigm	glo_unif	0.114	20	0.0178	0.02078	-1.31	0.01905	1.04	121.45
143	11	231	ReLu	le_norm	0.046	18	0.0130	0.02061	-0.39	0.01920	0.21	113.35
144	4	503	ReLu	le_norm	0.154	32	0.0861	4396.02	-6.57	0.10527	-7.41	93.88
145	11	491	sigm	glo_unif	0.094	38	0.0465	0.02060	-3.22	0.01891	2.63	114.99
146	8	481	tanh	le_norm	0.463	25	0.0409	0.02073	-4.72	0.01880	2.83	108.58
147	4	477	tanh	glo_unif	0.219	40	0.0168	0.02126	-3.76	0.01840	4.35	98.73
148	4	489	sigm	le_norm	0.208	51	0.0214	0.02054	-3.22	0.01894	2.63	87.69
149	3	52	ReLu	le_norm	0	36	0.0942	0.02124	-3.43	0.01883	3.18	101.63
150	3	69	ReLu	le_norm	0	37	0.0653	0.02085	-0.52	0.01887	0.98	77.78

TABLE A.5: The hyperparameter calibration results for the LSTM model (d).

A.2 RF

Table A.6 contains the different RF hyperparameters, with their corresponding abbreviation, search space bounds and best values found through 150 iterations of Bayesian optimisation. Tables A.7, A.8 and A.9 contain the entire list of iterations with their respective hyperparameter configuration, performance on selected metrics for the test and training set and run times.

Hyperparameter	Abbreviation	Search space	Best value
number of trees	n_tree	[1, 2, 3, ..., 1000]	808
maximum features	m_feat	[0.1, ..., 0.99]	0.1579
maximum depth	m_d	[1, 2, 3, ..., 200]	34
minimum number of samples per split	m_s.s	[2, 3, 4, ..., 50]	48
minimum number of samples per leaf	m.s.l	[2, 3, 4, ..., 50]	47
bootstrap (with or without replacement)	boot	[true, false]	true

TABLE A.6: The RF hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.

N	Random forest parameters						Test set		Train set		Time (mins)
	n_tree	m_feat	m_d	m_s_s	m_s_l	boot	MSE	Profit	MSE	Profit	
1	156	0.7515	190	9	30	false	0.02329	-8.58	0.01927	-10.37	1.867
2	969	0.6350	173	2	35	false	0.02263	-6.87	0.01887	-5.64	3.515
3	525	0.2618	43	16	10	true	0.02266	-9.02	0.01909	-9.78	2.621
4	366	0.6445	58	16	8	false	0.02606	-11.84	0.02373	-30.09	2.568
5	47	0.2777	157	30	26	false	0.02248	-7.89	0.01868	-6.13	1.490
6	160	0.8861	2	17	49	false	0.02207	-5.26	0.01856	2.00	1.841
7	495	0.7090	20	7	23	false	0.02356	-9.35	0.01954	-13.70	2.635
8	520	0.3303	181	16	33	false	0.02228	-7.45	0.01853	-2.97	2.334
9	894	0.9629	37	47	39	true	0.02162	-4.67	0.01795	10.03	3.403
10	326	0.1788	184	4	11	true	0.02244	-7.72	0.01876	-5.21	2.122
11	543	0.8376	54	15	19	false	0.02442	-9.14	0.02035	-16.98	2.934
12	199	0.1664	160	39	49	false	0.02165	-5.37	0.01800	8.85	1.693
13	74	0.7291	163	39	36	false	0.02273	-6.39	0.01898	-5.94	1.588
14	64	0.8682	24	17	31	false	0.02359	-8.83	0.01951	-11.18	1.594
15	472	0.7493	65	44	32	false	0.02304	-7.65	0.01914	-7.46	2.510
16	494	0.7771	142	39	28	false	0.02338	-9.06	0.01946	-13.12	2.649
17	636	0.1226	86	3	7	true	0.02271	-8.03	0.01944	-11.91	2.872
18	755	0.9077	102	21	13	false	0.02700	-13.10	0.02256	-26.34	3.912
19	808	0.3579	16	46	9	false	0.02370	-9.99	0.01991	-15.69	3.258
20	539	0.8153	174	44	10	true	0.02262	-8.51	0.01857	-3.62	2.837
21	427	0.3830	179	12	7	true	0.02312	-9.16	0.02005	-17.10	2.482
22	222	0.1062	172	22	26	true	0.02174	-4.22	0.01794	14.07	1.868
23	703	0.9392	68	26	17	false	0.02607	-11.70	0.02157	-23.79	3.635
24	301	0.9566	194	25	14	false	0.02761	-13.19	0.02277	-27.67	2.490
25	279	0.6425	8	4	26	false	0.02310	-9.27	0.01914	-7.94	2.095
26	242	0.2290	48	49	25	true	0.02192	-5.61	0.01808	8.92	1.959
27	632	0.3115	152	19	36	true	0.02159	-4.30	0.01791	12.73	2.697
28	187	0.1804	107	17	42	true	0.02149	-3.67	0.01787	16.84	1.754
29	227	0.7030	118	26	2	false	0.02820	-12.83	0.02479	-31.60	2.337
30	138	0.7149	35	46	20	true	0.02228	-7.53	0.01839	1.56	1.788
31	660	0.9230	23	14	44	false	0.02287	-7.08	0.01901	-6.10	3.070
32	897	0.5714	111	6	13	true	0.02259	-8.51	0.01885	-7.12	3.660
33	897	0.4017	126	36	18	true	0.02225	-6.92	0.01837	0.54	3.443
34	898	0.6714	156	9	6	true	0.02336	-9.27	0.02074	-20.31	3.930
35	161	0.1903	2	2	33	true	0.02145	-3.55	0.01790	15.35	1.672
36	238	0.6802	138	36	12	true	0.02253	-7.96	0.01866	-4.34	2.015
37	568	0.6782	149	33	42	false	0.02252	-6.90	0.01879	-2.33	2.616
38	393	0.3360	74	48	13	false	0.02318	-9.33	0.01950	-13.33	2.231
39	493	0.8074	126	29	26	true	0.02203	-6.06	0.01824	5.06	2.582
40	177	0.3499	144	32	3	false	0.02480	-11.19	0.02224	-26.11	1.889
41	928	0.9142	190	2	19	true	0.02235	-7.60	0.01848	-1.05	3.785
42	385	0.9576	193	16	42	false	0.02335	-8.05	0.01944	-13.59	2.372
43	696	0.2508	64	46	28	true	0.02176	-5.05	0.01798	10.01	2.876
44	518	0.6474	20	8	49	true	0.02153	-4.60	0.01793	13.91	2.478
45	294	0.7203	148	19	35	true	0.02166	-4.80	0.01796	9.64	2.045
46	502	0.8717	162	26	45	true	0.02155	-4.66	0.01793	12.64	2.518
47	338	0.7248	130	44	40	true	0.02158	-4.69	0.01792	12.25	2.160
48	543	0.6147	19	24	3	false	0.02688	-11.66	0.02416	-28.35	3.244
49	360	0.1271	118	41	3	false	0.02330	-9.42	0.02032	-18.16	2.159
50	86	0.7853	104	31	12	false	0.02532	-12.83	0.02148	-23.36	1.800

TABLE A.7: The hyperparameter calibration results for the random forest model (a).

N	Random forest parameters						Test set		Train set		Time (mins)
	n_tree	m_feat	m_d	m_s_s	m_s_l	boot	MSE	Profit	MSE	Profit	
51	541	0.2907	173	49	11	false	0.02318	-9.04	0.01949	-12.91	2.567
52	885	0.7592	142	32	9	false	0.02558	-11.20	0.02185	-25.67	4.332
53	185	0.6548	112	19	42	false	0.02249	-6.86	0.01876	-2.20	1.823
54	808	0.1706	37	45	38	true	0.02150	-4.03	0.01788	16.45	3.090
55	190	0.6902	100	7	42	false	0.02255	-6.66	0.01880	-2.97	1.858
56	501	0.2659	166	16	36	false	0.02215	-6.78	0.01840	0.45	2.340
57	821	0.6525	40	41	46	true	0.02153	-4.83	0.01791	13.85	3.239
58	812	0.9342	26	34	46	true	0.02154	-4.60	0.01793	12.27	3.288
59	213	0.6831	165	28	38	true	0.02162	-4.64	0.01792	11.52	1.899
60	183	0.5703	111	20	44	true	0.02155	-4.47	0.01789	13.70	1.805
61	188	0.3531	99	22	49	false	0.02180	-5.85	0.01819	3.42	1.790
62	172	0.5979	103	10	39	false	0.02253	-7.05	0.01877	-4.99	1.861
63	176	0.6037	108	30	37	false	0.02257	-6.83	0.01878	-3.71	1.884
64	820	0.7077	27	39	39	false	0.02270	-6.67	0.01891	-6.10	3.299
65	199	0.2694	169	26	43	false	0.02199	-6.48	0.01829	2.28	1.757
66	800	0.5222	37	36	43	false	0.02223	-6.59	0.01855	-0.18	3.011
67	208	0.2132	173	29	26	true	0.02186	-5.12	0.01803	9.32	1.916
68	203	0.3658	165	44	32	true	0.02167	-5.10	0.01793	11.20	1.948
69	176	0.8182	102	23	49	true	0.02152	-4.23	0.01790	14.28	1.825
70	220	0.8491	162	41	27	true	0.02196	-6.11	0.01819	5.34	1.999
71	217	0.3667	180	45	36	true	0.02162	-4.96	0.01790	12.05	1.901
72	222	0.5637	175	28	43	false	0.02230	-6.60	0.01859	-0.35	1.915
73	213	0.7491	162	47	41	false	0.02277	-7.30	0.01890	-5.45	1.960
74	151	0.4566	1	14	32	false	0.02169	-1.56	0.01843	5.45	1.608
75	204	0.3702	181	41	47	false	0.02182	-5.88	0.01819	2.64	1.865
76	810	0.3749	54	47	46	true	0.02151	-4.30	0.01790	15.55	3.541
77	213	0.4702	182	48	18	false	0.02302	-8.27	0.01940	-12.38	2.612
78	170	0.3582	6	15	35	false	0.02226	-6.79	0.01852	2.99	1.912
79	820	0.8221	54	39	36	true	0.02164	-4.71	0.01796	9.80	3.927
80	150	0.2496	10	3	46	false	0.02172	-6.08	0.01809	4.85	1.731
81	820	0.4417	46	23	48	false	0.02187	-6.02	0.01826	1.94	3.029
82	145	0.6230	12	9	24	false	0.02333	-9.39	0.01932	-11.03	1.857
83	821	0.6288	66	33	49	false	0.02207	-6.22	0.01845	-1.34	3.172
84	325	0.2716	133	31	33	false	0.02221	-7.54	0.01844	-1.47	2.081
85	518	0.3563	160	22	47	false	0.02181	-6.17	0.01817	3.01	2.403
86	182	0.8530	172	39	41	false	0.02305	-7.61	0.01910	-7.13	2.045
87	514	0.9445	170	35	41	false	0.02343	-8.18	0.01938	-12.66	2.927
88	512	0.6626	149	13	41	true	0.02159	-4.92	0.01792	12.20	2.718
89	331	0.2309	113	41	42	false	0.02198	-6.80	0.01826	3.72	2.172
90	808	0.1579	34	48	47	true	0.02143	-3.42	0.01789	18.09	2.986
91	502	0.4320	136	9	48	true	0.02152	-4.25	0.01793	14.52	2.815
92	487	0.6685	141	17	45	false	0.02204	-4.99	0.01838	0.62	2.679
93	506	0.4886	133	10	28	true	0.02184	-5.56	0.01807	7.60	2.796
94	516	0.6781	126	24	34	false	0.02275	-6.89	0.01897	-6.41	2.882
95	352	0.3359	125	35	49	false	0.02177	-5.87	0.01818	2.95	2.205
96	494	0.6816	115	11	37	true	0.02163	-4.75	0.01794	11.06	2.794
97	484	0.3497	137	5	24	false	0.02273	-8.53	0.01888	-8.26	2.642
98	481	0.6419	116	21	49	false	0.02209	-6.39	0.01846	-1.48	2.662
99	635	0.7420	146	29	21	true	0.02225	-6.81	0.01838	1.52	3.385
100	490	0.1816	123	3	49	true	0.02145	-3.47	0.01791	17.35	2.775

TABLE A.8: The hyperparameter calibration results for the random forest model (b).

N	Random forest parameters						Test set		Train set		Time (mins)
	n_tree	m_feat	m_d	m_s_s	m_s_l	boot	MSE	Profit	MSE	Profit	
101	805	0.4276	55	47	23	true	0.02208	-6.19	0.01822	4.61	3.761
102	348	0.2666	146	37	48	true	0.02148	-3.59	0.01791	16.78	2.377
103	351	0.1881	136	32	32	false	0.02213	-7.64	0.01829	1.64	2.263
104	511	0.9523	121	4	49	true	0.02153	-4.50	0.01794	13.43	2.874
105	340	0.1464	129	21	47	true	0.02144	-3.47	0.01790	18.23	2.352
106	349	0.2817	145	14	42	true	0.02152	-4.09	0.01790	15.20	2.378
107	357	0.2223	126	12	47	true	0.02148	-3.84	0.01791	16.59	2.388
108	326	0.3321	152	17	49	false	0.02177	-5.78	0.01817	3.05	2.213
109	341	0.8679	128	2	44	true	0.02154	-4.43	0.01792	13.12	2.436
110	344	0.7762	110	15	37	true	0.02163	-4.77	0.01795	10.69	2.450
111	318	0.7854	127	7	46	false	0.02235	-5.78	0.01853	-2.59	2.390
112	362	0.8457	132	4	30	true	0.02177	-5.48	0.01804	7.24	2.568
113	339	0.9287	135	6	25	true	0.02208	-6.20	0.01830	3.85	2.560
114	499	0.1748	97	7	46	false	0.02167	-5.97	0.01801	7.92	2.440
115	336	0.2166	161	37	40	true	0.02152	-4.43	0.01789	15.49	2.274
116	356	0.9052	169	37	40	false	0.02320	-7.75	0.01932	-10.75	2.556
117	330	0.6496	147	44	47	true	0.02152	-4.30	0.01791	13.89	2.296
118	680	0.5910	58	49	41	false	0.02244	-7.30	0.01867	-2.51	3.152
119	305	0.2469	152	34	45	true	0.02150	-4.15	0.01790	15.58	2.182
120	297	0.4263	131	29	45	true	0.02154	-4.50	0.01792	14.10	2.176
121	284	0.7898	145	38	44	true	0.02155	-4.63	0.01791	12.34	2.196
122	296	0.6295	136	49	32	true	0.02169	-4.85	0.01798	8.79	2.248
123	283	0.2313	137	34	20	false	0.02268	-9.08	0.01894	-9.20	2.119
124	367	0.8821	109	3	42	false	0.02303	-7.41	0.01918	-7.53	2.525
125	307	0.3863	118	45	47	true	0.02151	-4.13	0.01792	14.76	2.200
126	312	0.8532	140	48	48	true	0.02152	-4.47	0.01792	13.39	2.260
127	280	0.6156	151	18	49	true	0.02151	-3.95	0.01792	13.97	2.159
128	277	0.3546	131	24	46	false	0.02180	-5.84	0.01817	2.62	2.041
129	620	0.9477	166	27	47	false	0.02294	-8.34	0.01895	-8.68	3.185
130	649	0.2070	149	6	38	false	0.02204	-6.91	0.01826	3.17	2.794
131	263	0.4584	156	35	43	true	0.02154	-4.30	0.01790	13.66	2.114
132	283	0.7186	119	42	48	true	0.02152	-4.38	0.01792	13.80	2.175
133	265	0.9587	134	48	49	true	0.02153	-4.42	0.01792	13.74	2.156
134	44	0.8479	62	41	26	false	0.02399	-10.77	0.01977	-15.54	1.677
135	254	0.7334	148	48	29	false	0.02309	-8.81	0.01927	-11.25	2.243
136	257	0.6492	148	13	45	true	0.02155	-4.52	0.01791	13.18	2.107
137	253	0.6829	165	13	33	false	0.02275	-7.19	0.01899	-7.09	2.185
138	246	0.7275	136	23	49	true	0.02152	-4.27	0.01793	13.79	2.080
139	237	0.9096	80	28	45	true	0.02154	-4.23	0.01791	12.43	2.100
140	256	0.4455	72	35	39	true	0.02158	-4.40	0.01791	12.79	2.096
141	260	0.1334	103	32	45	true	0.02146	-3.61	0.01790	17.13	2.079
142	181	0.8731	120	25	45	false	0.02242	-4.90	0.01868	-2.59	2.014
143	273	0.2064	99	48	40	true	0.02151	-4.21	0.01788	15.47	2.131
144	291	0.3491	100	28	39	true	0.02156	-4.38	0.01790	13.57	2.185
145	794	0.4883	189	33	25	false	0.02281	-8.77	0.01897	-8.96	3.464
146	274	0.2980	89	28	32	false	0.02229	-7.71	0.01848	-1.97	2.083
147	405	0.4216	114	28	9	true	0.02274	-9.05	0.01898	-8.34	2.631
148	258	0.9638	115	44	48	false	0.02299	-8.12	0.01903	-9.85	2.237
149	682	0.4302	123	21	35	true	0.02163	-5.15	0.01793	10.99	3.145
150	252	0.1212	87	48	42	true	0.02147	-3.40	0.01789	16.87	2.056

TABLE A.9: The hyperparameter calibration results for the random forest model (c).

A.3 SVM

Table A.10 contains the different SVM hyperparameters, with their corresponding abbreviation, search space bounds and best values found through 150 iterations of Bayesian optimisation. Tables A.11, A.12 and A.13 contain the entire list of iterations with their respective hyperparameter configuration, performance on selected metrics for the test and training set and run times.

Hyperparameter	Abbreviation	Search space	Best value
kernel	kernel	[RBF, sigmoid, poly]	RBF
gamma	gamma	[0.001, ..., 0.1]	0.0742
regulariser (C)	reg (C)	[0.001, ..., 100]	0.1176
epsilon	epsilon	[0.01, ..., 1]	0.9421

TABLE A.10: *The SVM hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 150 iterations of Bayesian optimisation.*

N	SVM parameters				Test set		Train set		Time (mins)
	Kernel	gamma	reg (C)	epsilon	MSE	profit	MSE	Profit	
1	RBF	0.0157	0.0746	0.9512	0.02063	-3.63	0.01731	4.49	1.4864
2	sigmoid	0.0001	0.0060	0.1644	0.02085	-1.12	0.01753	-0.27	1.3790
3	sigmoid	0.0001	1.0129	0.7110	0.02058	-3.08	0.01733	0.75	1.3376
4	poly	0.0004	14.5282	0.2202	0.02085	-2.89	0.01753	-1.71	1.3059
5	poly	0.0020	0.0332	0.5295	0.02051	-0.61	0.01745	1.10	1.3371
6	RBF	0.0008	1.1462	0.1481	0.02193	-10.59	0.01758	-18.69	1.4579
7	RBF	0.0004	0.1907	0.7873	0.02062	-3.80	0.01730	2.36	1.3211
8	poly	0.0066	0.9164	0.0560	0.03338	-15.29	0.01955	-20.72	1.4884
9	sigmoid	0.0789	0.0021	0.9494	0.02057	-2.67	0.01740	2.64	1.3216
10	RBF	0.0113	0.0333	0.1067	0.02101	-5.11	0.01756	-0.23	1.4838
11	sigmoid	0.0001	0.0041	0.5002	0.02049	-0.01	0.01745	1.20	1.3552
12	RBF	0.0009	0.0197	0.6659	0.02050	-2.88	0.01742	-0.75	1.4072
13	sigmoid	0.0811	0.5414	0.1930	6.37248	-9.85	5.44736	-41.84	1.6042
14	sigmoid	0.0062	49.8304	0.8959	6.73E+29	-10.88	1.19E+18	-35.58	1.7120
15	poly	0.0001	0.0028	0.2040	0.02083	-2.03	0.01754	-2.31	1.3902
16	RBF	0.0306	0.0878	0.2786	0.02078	-3.78	0.01750	-3.01	1.3757
17	sigmoid	0.0003	0.0254	0.5473	0.02052	-0.38	0.01744	1.10	1.3962
18	poly	0.0207	0.0024	0.9870	0.02148	-5.97	0.01729	4.32	1.3309
19	sigmoid	0.0132	0.0011	0.8173	0.02056	-2.08	0.01744	0.29	1.4189
20	poly	0.0012	7.1838	0.0833	0.02182	-9.71	0.01768	-10.14	1.3441
21	poly	0.0010	20.6784	0.6271	0.02164	-10.18	0.01740	-5.77	1.4068
22	RBF	0.0154	0.0359	0.3319	0.02071	-3.49	0.01740	0.22	1.3654
23	sigmoid	0.0002	27.2938	0.4775	0.02316	-14.24	0.01798	-21.85	1.5786
24	RBF	0.0206	6.3669	0.5657	0.02300	-9.30	0.02317	-26.49	1.5620
25	poly	0.0001	0.4108	0.4333	0.02056	-0.67	0.01746	1.11	1.3570
26	RBF	0.0009	0.0014	0.6400	0.02046	-0.94	0.01746	0.45	1.4341
27	sigmoid	0.0017	34.5011	0.2568	0.85372	-9.69	0.09895	-30.61	1.6579
28	poly	0.0007	0.0139	0.0862	0.02086	-0.86	0.01754	-0.48	1.4481
29	sigmoid	0.0079	44.5131	0.8100	2.54E+35	-12.33	1.83E+23	-37.01	1.6029
30	RBF	0.0476	10.4318	0.1947	0.02243	-5.44	0.02898	-33.79	1.5516
31	poly	0.0009	10.8943	0.8971	0.02100	-6.59	0.01737	-0.20	1.3330
32	sigmoid	0.0284	0.0138	0.4328	0.02063	-3.07	0.01741	2.33	1.3535
33	poly	0.0018	0.0011	0.5156	0.02049	-0.43	0.01745	0.77	1.4498
34	poly	0.0674	0.0040	0.3442	0.04944	-15.93	0.02093	-22.88	1.4364
35	sigmoid	0.0012	0.3926	0.7060	0.02085	-5.41	0.01724	2.67	1.3615

TABLE A.11: *The hyperparameter calibration results for the SVM (a).*

N	SVM parameters				Test set		Train set		Time (mins)
	Kernel	gamma	reg (C)	epsilon	MSE	profit	MSE	Profit	
36	poly	0.0031	64.8988	0.2593	0.06019	-15.97	0.02216	-23.88	1.4601
37	RBF	0.0067	0.0266	0.0465	0.02103	-5.50	0.01753	0.17	1.3723
38	poly	0.0531	0.0018	0.2859	0.03258	-14.83	0.01892	-15.18	1.3846
39	poly	0.0906	0.0053	0.4946	0.06320	-15.25	0.02193	-22.16	1.4543
40	sigmoid	0.0005	2.2944	0.7640	0.02116	-6.15	0.01724	-0.22	1.4180
41	RBF	0.0080	0.0690	0.6360	0.02058	-3.14	0.01730	4.85	1.3998
42	poly	0.0009	0.0028	0.8369	0.02059	-1.75	0.01745	0.35	1.3708
43	poly	0.0108	0.0016	0.5950	0.02060	-5.01	0.01743	-1.72	1.3792
44	poly	0.0086	0.3635	0.2342	0.03169	-14.99	0.01888	-15.52	1.4026
45	poly	0.0646	2.8490	0.3929	0.25734	-16.69	0.02752	-36.00	1.7066
46	sigmoid	0.0594	0.0507	0.1223	0.05154	-8.97	0.04119	-34.03	1.6383
47	RBF	0.0283	0.0195	0.6634	0.02048	-1.28	0.01743	0.21	1.3674
48	sigmoid	0.0002	0.4449	0.2494	0.02093	-5.63	0.01749	-1.13	1.4104
49	RBF	0.0010	31.7753	0.6368	0.02737	-11.00	0.02020	-21.44	1.5288
50	sigmoid	0.0458	4.2636	0.8981	258173.4	-11.93	311876	-40.81	1.6054
51	poly	0.0005	4.1269	0.6714	0.02050	-2.33	0.01745	-0.21	1.3185
52	sigmoid	0.0063	0.0018	0.0283	0.02090	-1.11	0.01753	-0.47	1.3514
53	RBF	0.0025	0.0011	0.0500	0.02090	-0.86	0.01755	-0.48	1.5555
54	RBF	0.0959	71.4466	0.7125	0.02109	-4.11	0.02236	-23.23	1.4667
55	poly	0.0255	0.2278	0.1689	0.08042	-16.43	0.02398	-28.04	1.4695
56	sigmoid	0.0022	49.1551	0.7093	7.28023	-10.75	1.94376	-32.15	1.6050
57	RBF	0.0005	7.5149	0.8833	0.02224	-8.57	0.01770	-3.81	1.4987
58	sigmoid	0.0006	0.0093	0.6250	0.02046	-1.85	0.01746	-0.10	1.3189
59	RBF	0.0851	0.0011	0.4647	0.02055	-1.34	0.01746	0.96	1.3372
60	poly	0.0008	5.1641	0.5298	0.02065	-4.15	0.01743	-0.05	1.2654
61	RBF	0.0006	27.2350	0.5357	0.02588	-12.49	0.01911	-19.33	1.4337
62	poly	0.0001	0.0107	0.9702	0.02058	-1.10	0.01746	-0.82	1.3405
63	sigmoid	0.0001	0.1745	0.0970	0.02095	-2.30	0.01755	-0.04	1.3678
64	sigmoid	0.0584	0.0026	0.8004	0.02053	-1.85	0.01739	-0.15	1.2903
65	sigmoid	0.0119	0.0665	0.2939	0.02181	-7.29	0.01786	-13.06	1.3936
66	sigmoid	0.0503	85.9400	0.6090	8.06E+136	-10.88	5.94E+138	-33.99	1.5739
67	RBF	0.0001	3.7116	0.7501	0.02105	-5.84	0.01720	2.28	1.4222
68	RBF	0.0008	16.8655	0.7207	0.02478	-9.52	0.01888	-13.16	1.4347
69	poly	0.1	59.881	0.659	1.05583	-18.01	0.02594	-31.30	1.6401
70	sigmoid	0.0044	4.966	0.992	4.37246	-8.60	0.44078	-27.09	1.4674
71	RBF	0.0896	2.722	0.020	0.02122	-3.75	0.03147	-43.88	1.5469
72	sigmoid	0.0045	0.002	0.138	0.02089	-1.14	0.01754	-0.34	1.3579
73	RBF	0.0003	54.561	0.608	0.02590	-11.37	0.01897	-16.65	1.4403
74	poly	0.0029	66.395	0.227	0.05878	-15.67	0.02209	-23.84	1.6144
75	poly	0.0001	100	1	0.02057	-1.08	0.01745	1.02	1.3531
76	RBF	0.0742	0.1176	0.9421	0.02042	-1.60	0.01750	-1.89	1.4824
77	sigmoid	0.0273	0.4475	0.9277	1.38484	-8.88	0.96917	-40.86	1.5512
78	sigmoid	0.0001	100	0.01	0.02391	-13.55	0.01859	-31.92	1.4292
79	sigmoid	0.0271	0.0142	0.4069	0.02065	-3.20	0.01742	1.07	1.3126
80	poly	0.0001	0.001	0.01	0.02088	-1.15	0.01753	-0.49	1.3391
81	poly	0.0603	25.2449	0.0133	0.85252	-16.61	0.03308	-45.02	1.8794
82	poly	0.0108	0.0312	0.9878	0.02207	-7.14	0.01724	5.85	1.3001
83	poly	0.0062	0.9328	0.0726	0.03150	-15.43	0.01923	-19.53	1.4086
84	sigmoid	0.0002	9.8069	0.0177	0.02205	-11.23	0.01774	-23.53	1.4086
85	RBF	0.0604	0.6567	0.9732	0.02046	-2.13	0.01825	-9.15	1.3812
86	RBF	0.0368	1.2279	0.7752	0.02081	-4.67	0.01975	-19.39	1.4326
87	sigmoid	0.0024	0.0323	0.9825	0.02062	-3.41	0.01735	6.81	1.3164
88	poly	0.0083	100	1	0.07623	-14.38	0.02127	-15.40	1.4339
89	RBF	0.0001	0.0693	0.0173	0.02092	-1.76	0.01752	-0.26	1.4201
90	poly	0.0010	0.0132	0.8093	0.02055	-1.75	0.01745	0.35	1.3199
91	poly	0.0003	0.0038	0.8735	0.02062	-3.59	0.01745	-0.99	1.3540
92	poly	0.0013	0.2758	0.9483	0.02062	-2.96	0.01745	-0.35	1.2933
93	poly	0.0001	100	0.01	0.02088	-1.18	0.01753	-0.48	1.3393

TABLE A.12: The hyperparameter calibration results for the SVM (b).

N	SVM parameters				Test set		Train set		Time (mins)
	Kernel	gamma	reg (C)	epsilon	MSE	profit	MSE	Profit	
94	sigmoid	0.0001	100	1	0.02246	-8.25	0.01784	-5.56	1.4300
95	sigmoid	0.1	0.001	0.01	0.02089	-1.28	0.01752	-0.68	1.3727
96	poly	0.1	0.001	1	0.03375	-10.67	0.01843	-4.97	1.3543
97	poly	0.0261	0.2226	0.1339	0.08397	-16.35	0.02433	-28.95	1.4628
98	poly	0.0125	3.8328	0.9878	0.05260	-12.86	0.02007	-12.86	1.4207
99	poly	0.0859	16.1366	0.9298	0.21499	-16.97	0.02346	-23.90	1.6647
100	sigmoid	0.0008	0.0012	0.9054	0.02058	-0.78	0.01745	0.28	1.3426
101	poly	0.0945	0.0274	0.0212	0.15762	-15.75	0.02854	-37.02	1.5433
102	sigmoid	0.0016	0.0228	0.0143	0.02097	-3.69	0.01752	-0.12	1.3932
103	sigmoid	0.0644	0.0026	0.8273	0.02056	-2.72	0.01740	-2.06	1.2999
104	sigmoid	0.0001	0.001	0.01	0.02088	-1.15	0.01753	-0.49	1.4543
105	RBF	0.0001	0.001	0.01	0.02088	-1.15	0.01753	-0.49	1.4541
106	poly	0.097	0.338	0.904	0.11549	-15.96	0.02267	-22.70	1.4851
107	RBF	0.010	1.628	0.061	0.02336	-15.49	0.02153	-32.72	1.5023
108	RBF	0.000	0.002	0.994	0.02057	-1.06	0.01745	0.98	1.2992
109	RBF	0.001	1.171	0.576	0.02195	-10.48	0.01771	-13.92	1.3523
110	RBF	0.005	21.824	0.932	0.02596	-11.54	0.02060	-21.98	1.4110
111	poly	0.000	0.039	0.107	0.02088	-1.15	0.01754	-0.49	1.3021
112	poly	0.002	95.370	0.156	0.05158	-15.47	0.02152	-22.47	1.3593
113	poly	0.013	7.173	0.224	0.11873	-15.94	0.02606	-32.87	1.4544
114	poly	0.1	0.001	0.01	0.05424	-15.71	0.02219	-25.72	1.3694
115	RBF	0.0001	1.9135	0.0330	0.02154	-10.52	0.01762	-16.47	1.3449
116	RBF	0.0001	0.0294	0.9965	0.02057	-1.14	0.01744	1.63	1.2669
117	poly	0.0062	0.0096	0.0555	0.02108	-4.88	0.01756	-2.22	1.2516
118	sigmoid	0.0001	0.0965	0.9286	0.02060	-2.69	0.01744	-0.04	1.2778
119	poly	0.0012	0.1839	0.0150	0.02090	-1.59	0.01752	-0.59	1.2938
120	sigmoid	0.0036	24.1065	0.8954	106.900	-9.47	21.166	-34.54	1.4697
121	sigmoid	0.0018	0.4033	0.0369	0.02171	-9.70	0.01766	-20.07	1.3242
122	RBF	0.0069	0.3320	0.7977	0.02095	-7.94	0.01732	-6.01	1.3354
123	poly	0.0001	0.001	1	0.02057	-1.06	0.01745	0.98	1.2667
124	poly	0.0982	0.0563	0.9716	0.07749	-14.51	0.02141	-16.60	1.3816
125	sigmoid	0.0829	0.0695	0.8912	0.09824	-8.35	0.07886	-34.12	1.5120
126	sigmoid	0.0034	12.3362	0.0970	3.67540	-10.54	0.81690	-32.29	1.4828
127	sigmoid	0.0108	0.2547	0.0347	0.07321	-9.03	0.04050	-24.58	1.4541
128	RBF	0.0001	73.7189	0.0441	0.02523	-14.67	0.01912	-37.21	1.4773
129	poly	0.0001	2.3374	0.0225	0.02087	-1.15	0.01753	-0.49	1.3389
130	sigmoid	0.0002	11.4049	0.9635	0.02126	-7.18	0.01719	-1.32	1.3568
131	poly	0.0015	15.8974	0.6437	0.02297	-11.09	0.01743	-6.11	1.2956
132	RBF	0.0047	0.0054	0.9944	0.02057	-0.85	0.01742	4.26	1.2764
133	poly	0.0001	0.2104	0.6847	0.02047	-1.19	0.01746	0.24	1.2802
134	sigmoid	0.0001	0.001	1	0.02057	-1.06	0.01745	0.98	1.2706
135	poly	0.0803	0.0155	0.8775	0.05739	-13.60	0.02068	-18.36	1.3569
136	RBF	0.0684	0.0263	0.0585	0.02088	-1.01	0.01756	-0.12	1.3295
137	sigmoid	0.0862	2.1708	0.9545	424.328	-10.81	422.143	-41.09	1.4949
138	poly	0.0010	0.0023	0.0245	0.02087	-1.15	0.01753	-0.49	1.3047
139	sigmoid	0.0227	11.8972	0.4426	8.52E+11	-11.04	1.55E+15	-40.49	1.4914
140	sigmoid	0.0077	8.1212	0.1692	64885.68	-11.90	241.07088	-35.78	1.4723
141	poly	0.0368	0.7719	0.7350	0.08990	-15.48	0.02279	-24.90	1.3869
142	sigmoid	0.0007	0.0928	0.8541	0.02067	-3.49	0.01738	0.75	1.2400
143	sigmoid	0.0642	2.7141	0.0115	1140.66	-10.41	1095.65	-39.51	1.4983
144	RBF	0.0763	0.0824	0.9588	0.02059	-1.30	0.01745	2.51	1.2375
145	RBF	0.0110	0.0023	0.9756	0.02057	-1.44	0.01744	1.64	1.2753
146	poly	0.0265	96.0230	0.9814	0.13958	-16.62	0.02271	-20.88	1.3965
147	RBF	0.0197	6.8701	0.5482	0.02322	-9.69	0.02337	-26.75	1.4171
148	RBF	0.0951	0.0050	0.0251	0.02087	-1.15	0.01753	-0.51	1.3496
149	RBF	0.0104	1.7327	0.0550	0.02343	-15.35	0.02208	-33.40	1.4616
150	poly	0.1	100	0.01	2.60012	-16.70	0.03372	-44.88	2.5295

TABLE A.13: The hyperparameter calibration results for the SVM (c).

A.4 ARIMA

Table A.14 contains the different ARIMA hyperparameters, with their corresponding search space bounds and best values found through 216 iterations of the grid search. Tables A.15, A.16 and A.17 contain the entire list of iterations with their respective hyperparameter configuration, performance on the test set and run times.

Hyperparameter	Search space	Best value
p	[0, 1, 2, 3, 4, 5]	4
d	[0, 1, 2, 3, 4, 5]	0
q	[0, 1, 2, 3, 4, 5]	3

TABLE A.14: The ARIMA hyperparameter calibration parameters and their corresponding abbreviation, search spaces and best values found through 216 iterations of grid search.

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
1	0	0	0	0.01755	0.00	0.879
2	0	0	1	0.01757	-4.74	0.770
3	0	0	2	0.01758	-5.10	1.065
4	0	0	3	0.01757	-5.39	1.325
5	0	0	4	0.01755	-8.27	1.685
6	0	0	5	0.01759	-12.60	1.711
7	0	1	0	0.03475	-30.46	0.272
8	0	1	1	0.01757	0.00	1.091
9	0	1	2	0.01890	-14.72	0.889
10	0	1	3	4.54237	0.00	16.667
11	0	1	4	0.01763	-4.91	2.657
12	0	1	5	0.10606	-48.36	0.258
13	0	2	0	0.03480	-30.46	1.225
14	0	2	1	0.01898	-16.79	1.702
15	0	2	2	0.01776	-4.45	2.374
16	0	2	3	0.01781	-4.64	2.959
17	0	2	4	0.01892	-11.25	3.223
18	0	2	5	0.35597	-48.00	0.356
19	0	3	0	0.10617	-48.36	1.494
20	0	3	1	0.05172	-23.98	2.238
21	0	3	2	0.01802	-1.00	3.074
22	0	3	3	0.01821	-10.30	3.964
23	0	3	4	0.01807	3.04	4.514
24	0	3	5	1.24345	-30.35	0.294
25	0	4	0	0.35652	-48.00	1.332
26	0	4	1	0.10750	-48.57	1.827
27	0	4	2	0.03547	-29.55	2.764
28	0	4	3	0.02025	-21.25	4.735
29	0	4	4	0.02165	-24.18	5.503
30	0	4	5	4.44876	-15.63	0.259
31	0	5	0	1.24631	-30.35	1.496
32	0	5	1	0.35971	-48.00	1.723
33	0	5	2	0.10865	-53.66	2.658
34	0	5	3	0.03716	-29.60	5.141
35	0	5	4	0.02849	-12.09	6.659

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
36	0	5	5	0.01757	-4.74	0.672
37	1	0	0	0.01758	-5.76	0.945
38	1	0	1	0.01789	1.81	1.231
39	1	0	2	0.01757	-4.71	1.524
40	1	0	3	0.01756	-8.66	1.875
41	1	0	4	0.01760	-9.83	2.126
42	1	0	5	0.02527	-1.38	0.314
43	1	1	0	0.01759	-6.43	1.395
44	1	1	1	0.01745	3.53	2.025
45	1	1	2	0.01747	-5.11	2.241
46	1	1	3	0.01747	-5.11	2.241
47	1	1	4	0.01764	5.74	3.227
48	1	1	5	0.05696	-14.27	0.351
49	1	2	0	0.02530	-1.38	1.548
50	1	2	1	0.01931	-25.42	1.846
51	1	2	2	0.01802	-12.51	2.860
52	1	2	3	0.01758	-6.90	3.423
53	1	2	4	0.01765	-3.56	3.547
54	1	2	5	0.15416	-44.18	0.382
55	1	3	0	0.05710	-14.14	1.991
56	1	3	1	0.03316	-22.10	1.969
57	1	3	2	0.03185	4.21	2.901
58	1	3	3	0.01864	-22.67	4.477
59	1	3	4	0.01770	5.63	6.792
60	1	3	5	0.45537	-49.27	0.351
61	1	4	0	0.15438	-44.11	1.920
62	1	4	1	0.06723	-21.71	2.042
63	1	4	2	0.03391	-22.94	3.703
64	1	4	3	0.03514	-33.38	4.497
65	1	4	4	0.03514	-33.38	4.497
66	1	4	5	1.41420	-42.69	0.423
67	1	5	0	0.45627	-49.49	2.106
68	1	5	1	0.16344	-43.40	2.432
69	1	5	2	0.07133	-24.12	3.997
70	1	5	3	0.13728	-47.90	4.864

TABLE A.15: The hyperparameter calibration results for the ARIMA model (a).

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
71	1	5	4	0.03600	-30.29	9.850
72	1	5	5	0.01758	-5.10	0.983
73	2	0	0	0.01768	-5.50	1.055
74	2	0	1	0.01767	-7.80	1.785
75	2	0	2	0.01756	-5.39	1.585
76	2	0	3	0.01759	-11.78	1.998
77	2	0	4	0.01761	-8.68	2.143
78	2	0	5	0.02231	-14.35	0.439
79	2	1	0	0.01762	-0.11	1.760
80	2	1	1	0.01749	-0.41	2.560
81	2	1	2	0.01763	-0.55	3.361
82	2	1	3	0.01761	-4.45	3.620
83	2	1	4	0.01747	0.74	4.211
84	2	1	5	0.04016	-10.59	0.460
85	2	2	0	0.02239	-16.03	1.793
86	2	2	1	0.02480	2.09	2.226
87	2	2	2	0.01906	-3.52	2.582
88	2	2	3	0.01866	-14.00	3.318
89	2	2	4	0.01973	-32.51	3.806
90	2	2	5	0.09010	-26.47	0.786
91	2	3	0	0.04027	-9.84	2.068
92	2	3	1	0.02588	-4.93	2.427
93	2	3	2	0.02680	-12.54	3.427
94	2	3	3	0.03532	-23.27	3.435
95	2	3	4	0.02048	3.11	4.494
96	2	3	5	0.23098	-38.39	0.654
97	2	4	0	0.09032	-28.01	2.437
98	2	4	1	0.04439	-13.62	3.318
99	2	4	2	0.04560	-6.42	3.929
100	2	4	3	0.03011	-15.43	6.473
101	2	4	4	0.03756	-26.97	6.456
102	2	4	5	0.63994	-39.60	0.732
103	2	5	0	0.23199	-38.39	2.852
104	2	5	1	0.23199	-38.39	2.852
105	2	5	2	0.09341	-26.64	4.477
106	2	5	3	0.06557	-15.91	7.791
107	2	5	4	0.11198	-46.56	6.577
108	2	5	5	0.01757	-5.65	1.152
109	3	0	0	0.01757	-5.65	1.326
110	3	0	1	0.01761	3.92	2.479
111	3	0	2	0.01758	-3.17	2.268
112	3	0	3	0.01751	0.15	3.568
113	3	0	4	0.01763	-7.89	2.549
114	3	0	5	0.02181	-19.12	0.548
115	3	1	0	0.01798	-25.32	1.987
116	3	1	1	0.01766	-0.95	2.623
117	3	1	2	0.01766	-0.95	2.623
118	3	1	3	0.01764	-10.96	4.176
119	3	1	4	0.01776	-12.89	4.859
120	3	1	5	0.03513	-12.75	0.539
121	3	2	0	0.02216	-20.97	2.137
122	3	2	1	0.02262	-12.20	2.640
123	3	2	2	0.02382	-5.70	3.105
124	3	2	3	0.01808	-7.67	4.014
125	3	2	4	0.01801	-3.62	4.528

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
126	3	2	5	0.06744	-1.03	0.822
127	3	3	0	0.03568	-0.31	2.186
128	3	3	1	0.03923	-7.55	2.752
129	3	3	2	0.02425	-8.59	3.371
130	3	3	3	0.02943	-10.04	4.366
131	3	3	4	0.02978	14.35	4.865
132	3	3	5	0.14993	-40.50	0.868
133	3	4	0	0.06883	2.12	2.527
134	3	4	1	0.04662	-27.65	3.101
135	3	4	2	0.04642	5.97	4.622
136	3	4	3	0.04207	16.36	5.244
137	3	4	4	0.03260	-22.13	5.766
138	3	4	5	0.37162	-44.30	0.954
139	3	5	0	0.16403	-40.68	3.171
140	3	5	1	0.07613	0.30	3.451
141	3	5	2	0.07613	0.30	3.451
142	3	5	3	0.05286	2.56	6.350
143	3	5	4	0.07243	-31.42	9.964
144	3	5	5	0.01754	-6.48	1.530
145	4	0	0	0.01754	-7.17	1.762
146	4	0	1	0.01768	-8.47	2.872
147	4	0	2	0.01769	-4.24	3.115
148	4	0	3	0.01741	1.73	4.269
149	4	0	4	0.01764	-3.60	2.921
150	4	0	5	0.02107	-24.39	0.594
151	4	1	0	0.01755	-9.15	2.514
152	4	1	1	0.01842	-10.03	2.465
153	4	1	2	0.01760	-4.69	2.902
154	4	1	3	0.01764	-14.02	4.105
155	4	1	4	0.01799	-13.62	4.981
156	4	1	5	0.03248	-16.43	0.683
157	4	2	0	0.02179	-23.77	2.447
158	4	2	1	0.02182	-18.42	2.959
159	4	2	2	0.02243	-12.44	3.186
160	4	2	3	0.02338	-13.82	3.941
161	4	2	4	0.02000	-5.12	4.878
162	4	2	5	0.05692	-5.79	0.777
163	4	3	0	0.03326	-13.91	2.769
164	4	3	1	0.03545	-7.43	3.496
165	4	3	2	0.03604	-12.07	3.912
166	4	3	3	0.02394	-5.88	4.455
167	4	3	4	0.02536	-6.77	5.019
168	4	3	5	0.11169	-14.50	1.108
169	4	4	0	0.05787	-7.89	3.305
170	4	4	1	0.06690	-9.99	3.372
171	4	4	2	0.04667	-15.41	4.000
172	4	4	3	0.04497	-14.11	4.200
173	4	4	4	0.04383	10.22	5.374
174	4	4	5	0.24436	-25.88	1.053
175	4	5	0	0.11350	-15.46	3.643
176	4	5	1	0.14632	-44.52	3.956
177	4	5	2	0.07577	-8.40	4.920
178	4	5	3	0.05678	-23.54	8.256
179	4	5	4	0.05814	-21.31	7.427
180	4	5	5	0.01758	-6.74	1.716

TABLE A.16: The hyperparameter calibration results for the ARIMA model (b).

N	ARIMA			Test set		Time (mins)
	p	d	q	MSE	Profit	
181	5	0	0	0.01758	-7.08	2.133
182	5	0	1	0.01742	-4.41	3.075
183	5	0	2	0.01763	-8.43	3.695
184	5	0	3	0.01779	-2.42	5.206
185	5	0	4	0.01763	-6.87	3.067
186	5	0	5	0.02020	-29.31	0.794
187	5	1	0	0.01843	-4.39	2.631
188	5	1	1	0.01856	-27.97	2.966
189	5	1	2	0.01773	0.31	3.488
190	5	1	3	0.01754	4.70	4.471
191	5	1	4	0.01794	-12.45	4.945
192	5	1	5	0.03040	-27.63	0.802
193	5	2	0	0.02214	-30.68	2.836
194	5	2	1	0.02223	-31.21	3.126
195	5	2	2	0.02339	-25.34	3.524
196	5	2	3	0.02286	-14.34	3.995
197	5	2	4	0.02286	-14.34	3.995
198	5	2	5	0.05224	-24.49	0.863
199	5	3	0	0.03447	-32.60	3.527
200	5	3	1	0.03400	-15.34	3.231
201	5	3	2	0.03636	-14.71	4.034
202	5	3	3	0.03231	-18.01	4.200
203	5	3	4	0.02392	5.32	4.622
204	5	3	5	0.09663	-16.39	1.160
205	5	4	0	0.05894	-18.40	3.912
206	5	4	1	0.05660	-8.09	3.884
207	5	4	2	0.06822	-11.89	4.334
208	5	4	3	0.04025	-7.63	4.879
209	5	4	4	0.04203	1.12	5.338
210	5	4	5	0.19405	-23.79	1.249
211	5	5	0	0.10800	-5.00	4.484
212	5	5	1	0.10800	-5.00	4.484
213	5	5	2	0.06427	-15.18	4.423
214	5	5	3	0.07019	-6.87	5.412
215	5	5	4	0.07019	-6.87	5.412
216	5	5	5	0.02021	-3.17	2.596

TABLE A.17: The hyperparameter calibration results for the ARIMA model (c).

APPENDIX B

Model results

This chapter contains the ML models' results respective to the testing year and the two parameters, **retraining interval** and **sequence length**. The results are displayed in heatmaps corresponding to the 2-D grid search applied to each test set.

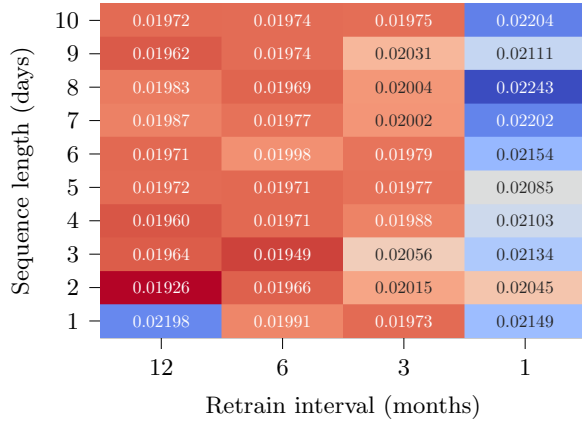
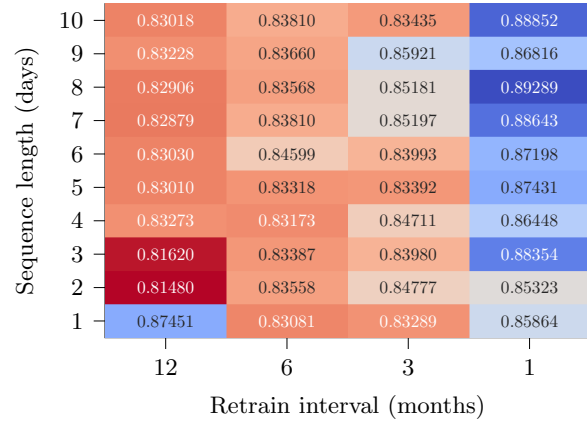
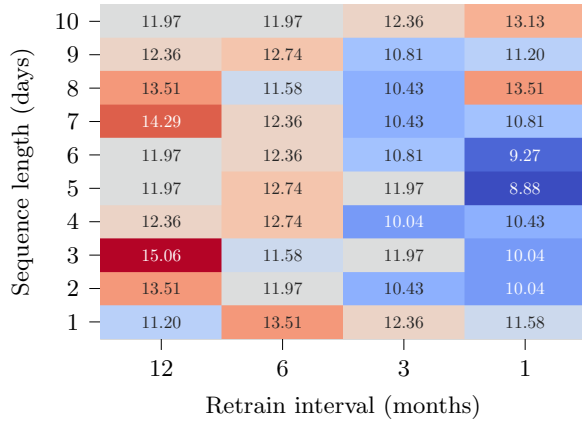
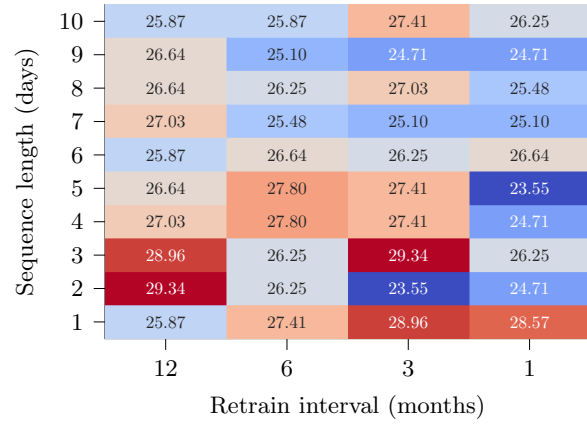
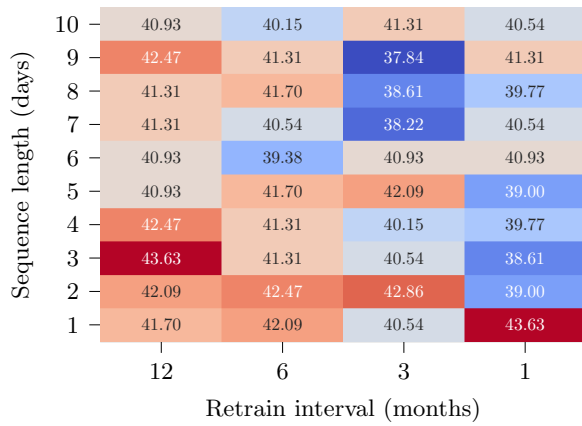
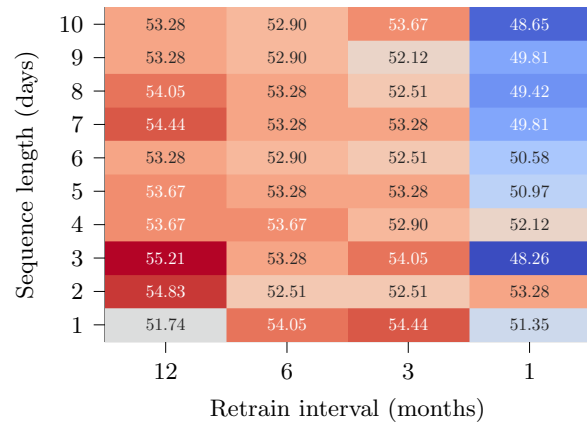
B.1 LSTM

This section contains the LSTM model results, with a breakdown by testing year and a summary of the entire testing period.

B.1.1 Test set 2018

The results obtained for the 2018 testing period of the LSTM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.1(a) and Figure B.1(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.2(a), Figure B.2(b), Figure B.2(c), and Figure B.2(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.3(a) and Figure B.3(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.4(a), Figure B.4(b) and Figure B.4(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.1 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2018 testing period.
- Figure B.5 shows the cumulative profit of the most profitable LSTM model, LSTM(12, 2), compared to the naïve and ARIMA models.

(a) *LSTM MSE 2018.*(b) *LSTM MAPE (%) 2018.*FIGURE B.1: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's MSE (left) and MAPE (right) for the **2018** test set.(a) *LSTM threshold accuracy_{0.025} (%) 2018.*(b) *LSTM threshold accuracy_{0.05} (%) 2018.*(c) *LSTM threshold accuracy_{0.075} (%) 2018.*(d) *LSTM threshold accuracy_{0.1} (%) 2018.*FIGURE B.2: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's different threshold accuracies for the **2018** test set.

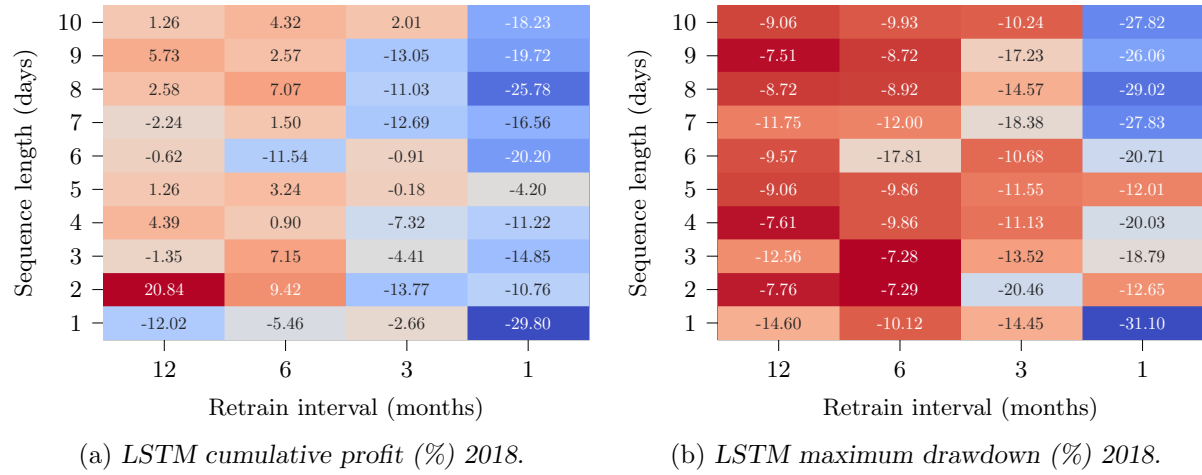


FIGURE B.3: The heatmaps displaying the effect of *retraining interval* and *sequence length* on the **LSTM** model's cumulative profit (left) and maximum drawdown (right) for the **2018** test set.

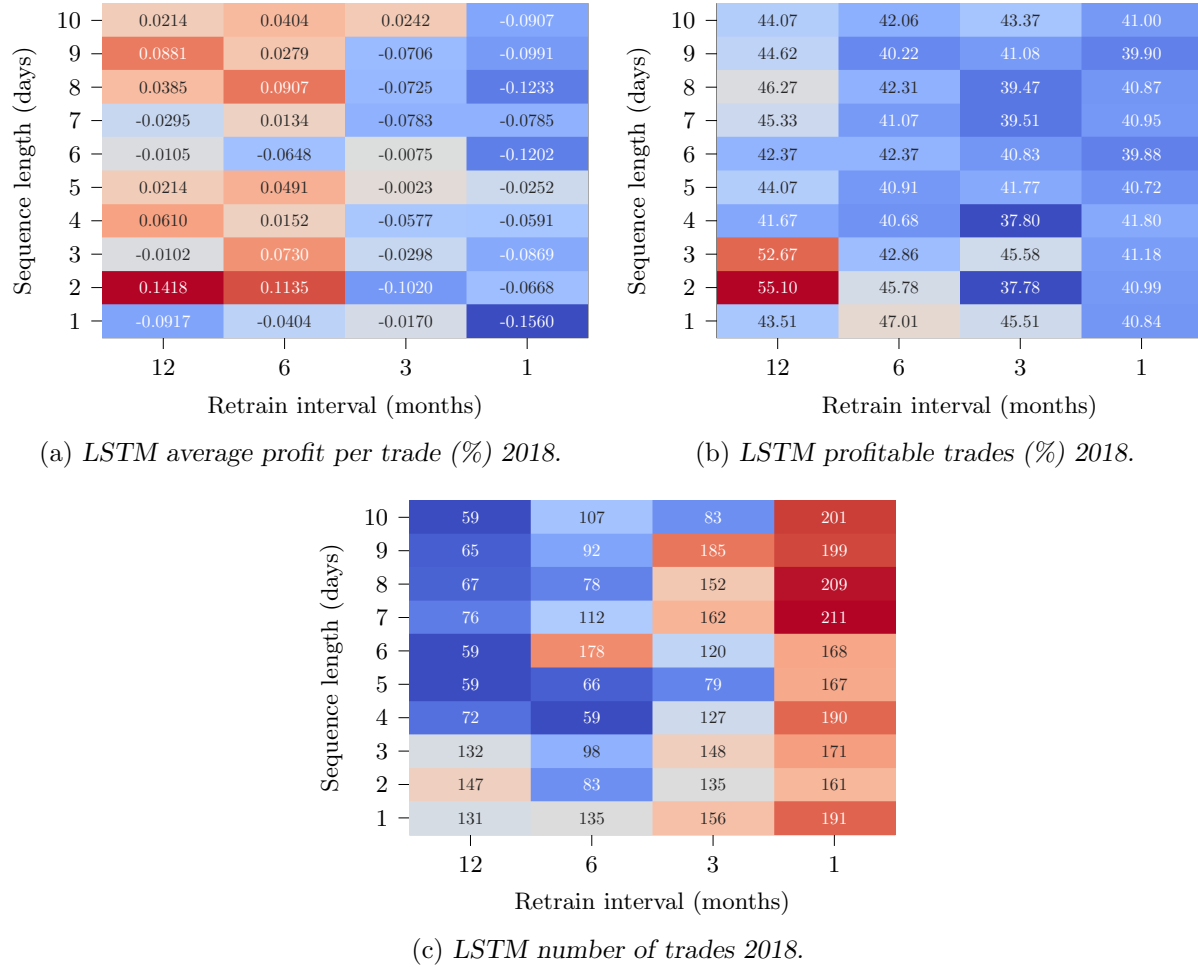


FIGURE B.4: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0199	0.8319	1.98	−9.82	45.97	87
	6	0.01974	0.83596	1.92	−10.18	42.53	101
	3	0.02	0.84388	−6.4	−14.22	41.27	135
	1	0.02143	0.87422	−17.13	−22.60	40.81	187
Seq length	1	0.02078	0.84921	−12.48	−17.57	44.22	153
	2	0.01988	0.83784	1.43	−12.04	44.91	132
	3	0.02026	0.84335	−3.36	−13.04	45.57	137
	4	0.02005	0.84401	−3.31	−12.16	40.48	112
	5	0.02001	0.84288	0.03	−10.62	41.87	93
	6	0.02026	0.84705	−8.32	−14.69	41.37	131
	7	0.02042	0.85132	−7.50	−17.49	41.72	140
	8	0.0205	0.85236	−6.79	−15.31	42.23	126
	9	0.0202	0.84906	−6.12	−14.88	41.45	135
	10	0.02031	0.84779	−2.66	−14.26	42.62	112

TABLE B.1: The **LSTM** model's aggregated results for each tested parameter value on selected performance metrics for the **2018** test set.

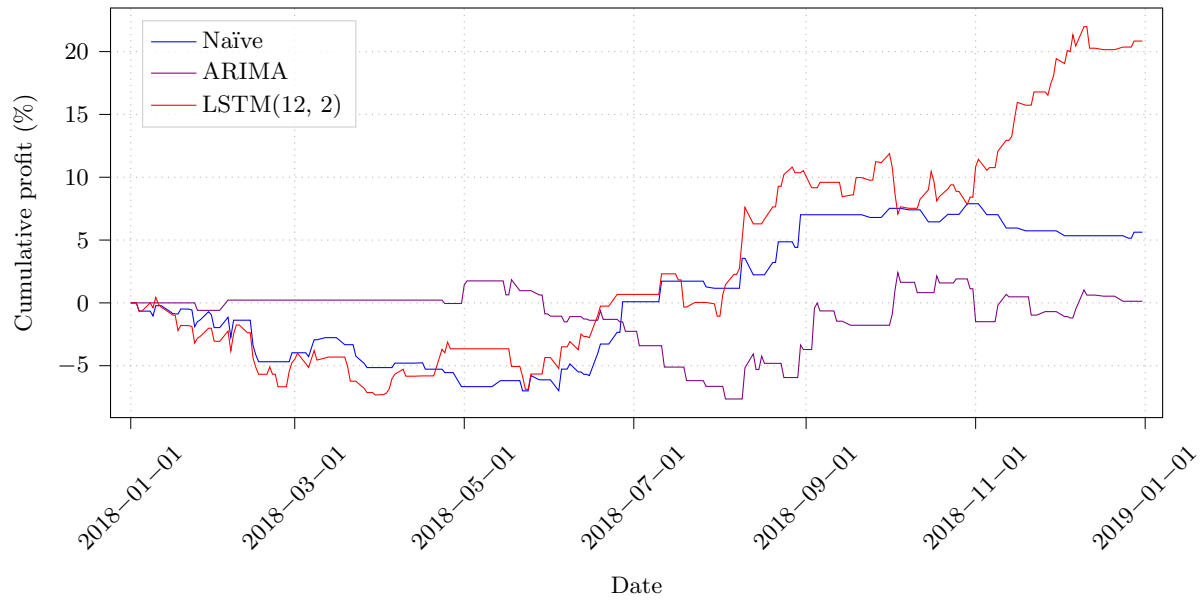


FIGURE B.5: The **LSTM(12, 2)** model's (most profitable **LSTM** model) cumulative profit for the **2018** test set compared to the naïve and ARIMA models.

B.1.2 Test set 2019

The results obtained for the 2019 testing period of the LSTM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.6(a) and Figure B.6(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.7(a), Figure B.7(b), Figure B.7(c), and Figure B.7(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.8(a) and Figure B.8(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.9(a), Figure B.9(b) and Figure B.9(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.2 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2019 testing period.
- Figure B.10 shows the cumulative profit of the most profitable LSTM model, LSTM(6, 3), compared to the naïve and ARIMA models.

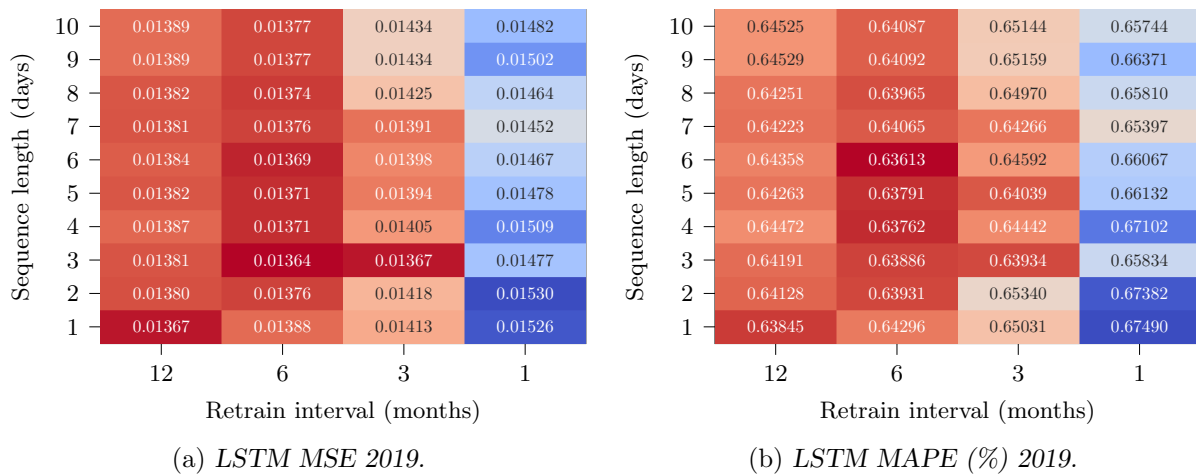


FIGURE B.6: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's MSE (left) and MAPE (right) for the **2019** test set.

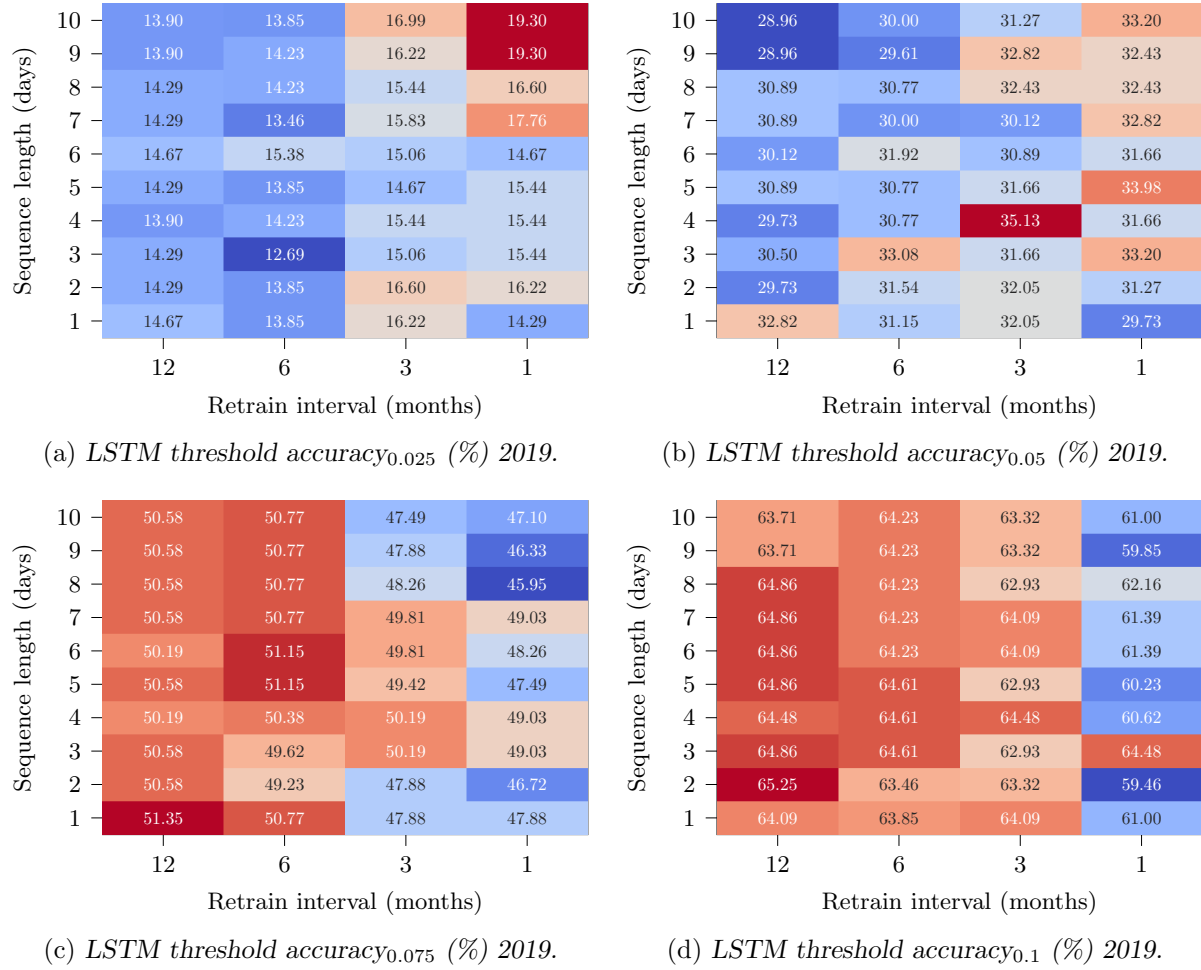


FIGURE B.7: The heatmaps displaying the effect of *retraining interval* and *sequence length* on the **LSTM** model's different threshold accuracies for the **2019** test set.

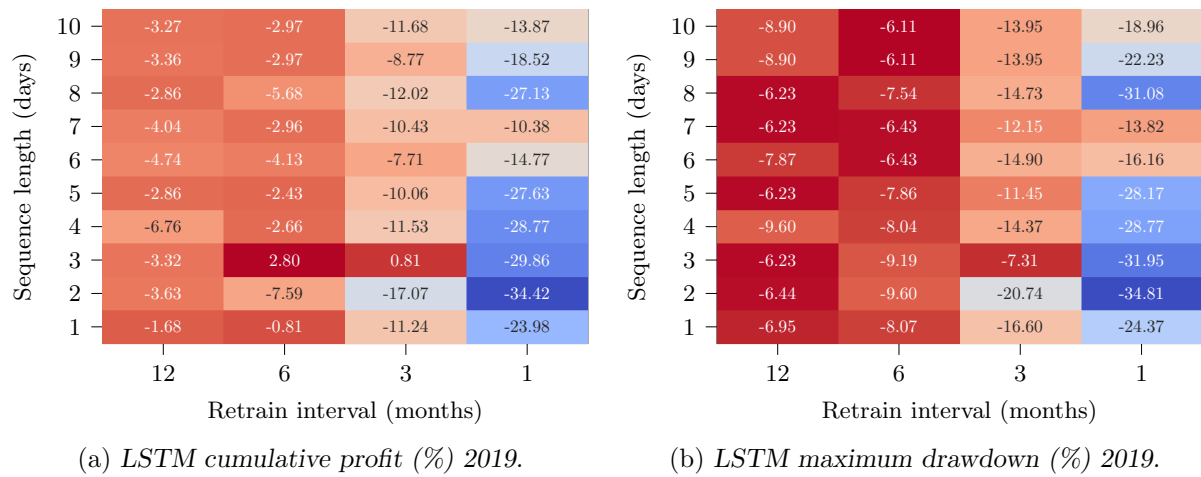


FIGURE B.8: The heatmaps displaying the effect of *retraining interval* and *sequence length* on the **LSTM** model's cumulative profit (left) and maximum drawdown (right) for the **2019** test set.

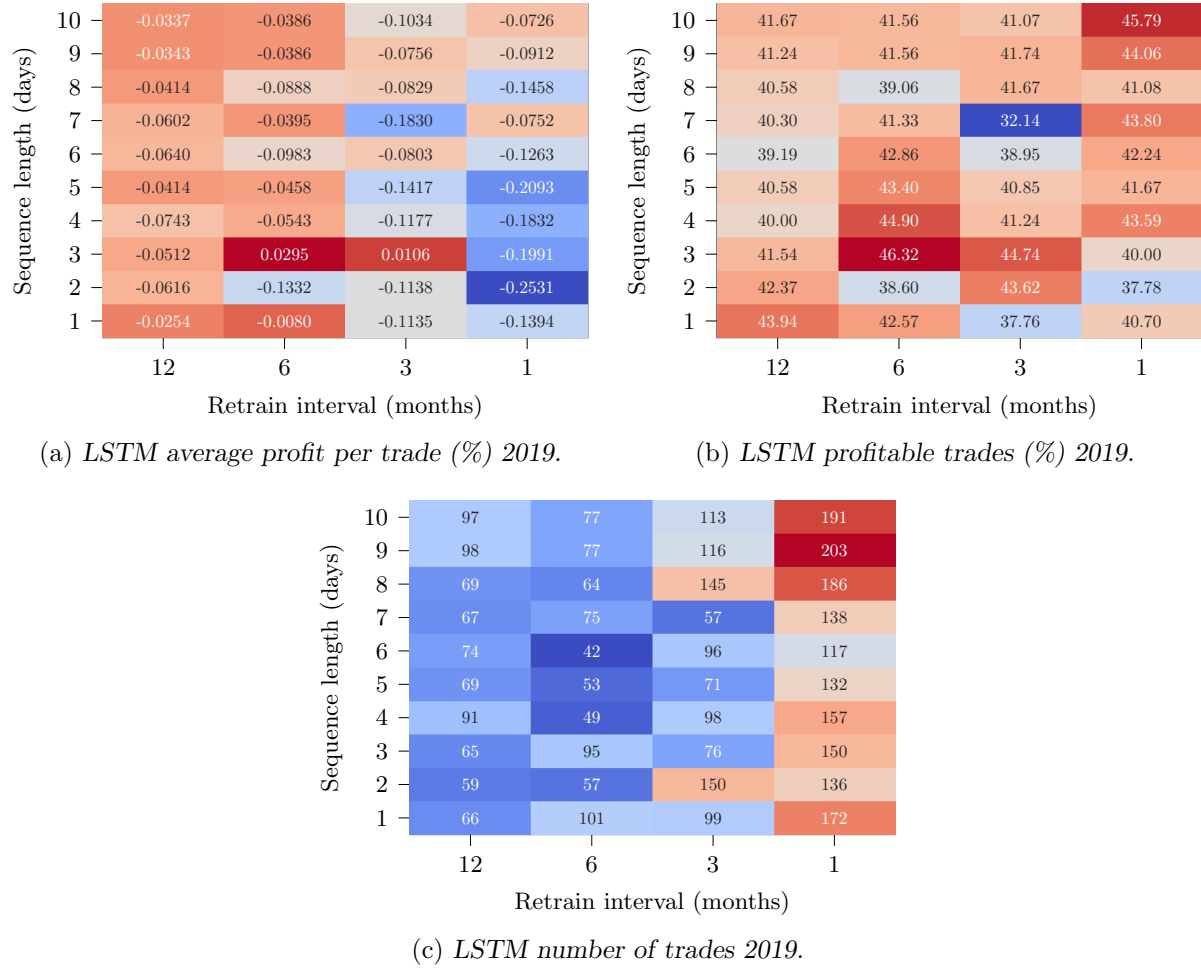


FIGURE B.9: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2019** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01382	0.64279	−3.65	−7.36	41.14	76
	6	0.01374	0.63949	−2.94	−7.54	42.22	69
	3	0.01408	0.64692	−9.97	−14.01	40.38	102
	1	0.01489	0.66333	−22.93	−25.03	42.07	158
Seq length	1	0.01423	0.65166	−9.43	−14.0	41.24	110
	2	0.01426	0.65195	−15.68	−17.9	40.59	100
	3	0.01397	0.64461	−7.39	−13.67	43.15	96
	4	0.01418	0.64945	−12.43	−15.2	42.43	99
	5	0.01406	0.64556	−10.74	−13.43	41.62	81
	6	0.01404	0.64658	−7.84	−11.34	40.81	82
	7	0.014	0.64488	−6.95	−9.66	39.39	84
	8	0.01411	0.64749	−11.92	−14.9	40.6	116
	9	0.01425	0.65038	−8.41	−12.8	42.15	124
	10	0.0142	0.64875	−7.95	−11.98	42.52	120

TABLE B.2: The **LSTM** model's aggregated results for each tested parameter value on selected performance metrics for the **2019** test set.

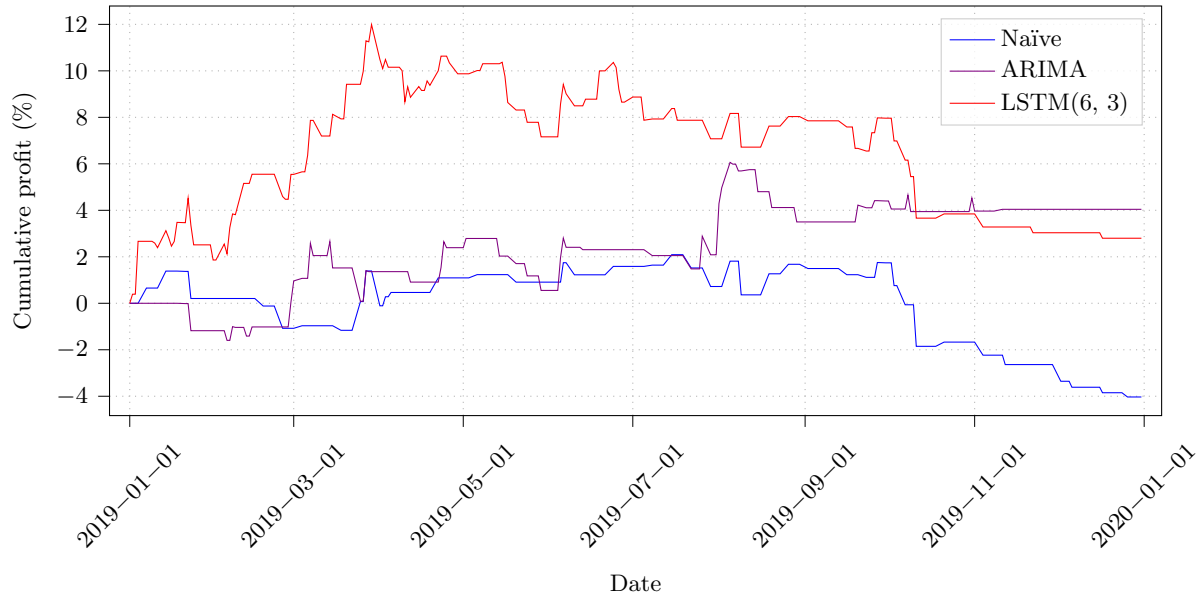


FIGURE B.10: The **LSTM(6, 3)** model's (most profitable **LSTM** model) cumulative profit for the **2019** test set compared to the naïve and ARIMA models.

B.1.3 Test set 2020

The results obtained for the 2020 testing period of the LSTM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.11(a) and Figure B.11(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.12(a), Figure B.12(b), Figure B.12(c), and Figure B.12(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.13(a) and Figure B.13(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.14(a), Figure B.14(b) and Figure B.14(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.3 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2020 testing period.
- Figure B.15 shows the cumulative profit of the most profitable LSTM model, LSTM(1, 9), compared to the naïve and ARIMA models.

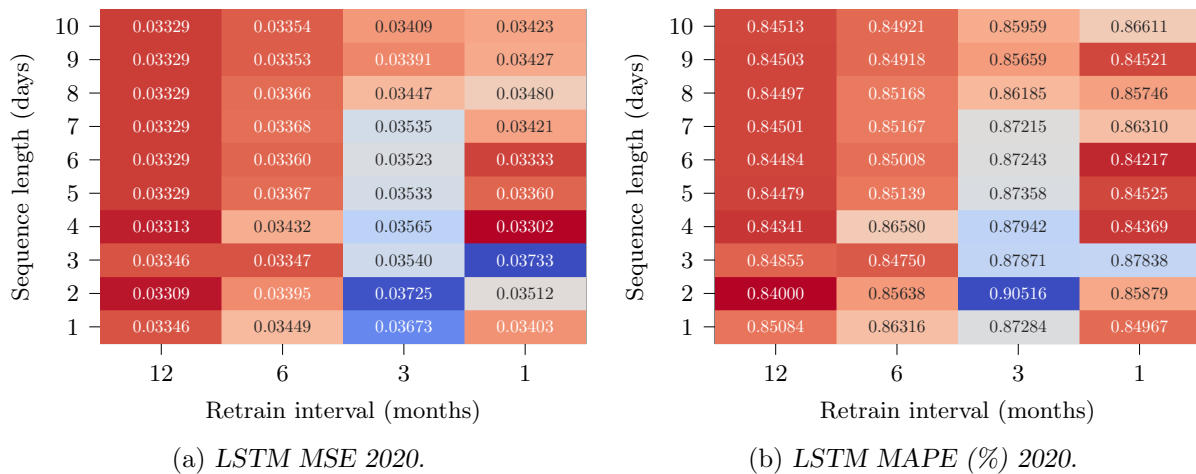


FIGURE B.11: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's MSE (left) and MAPE (right) for the **2020** test set.

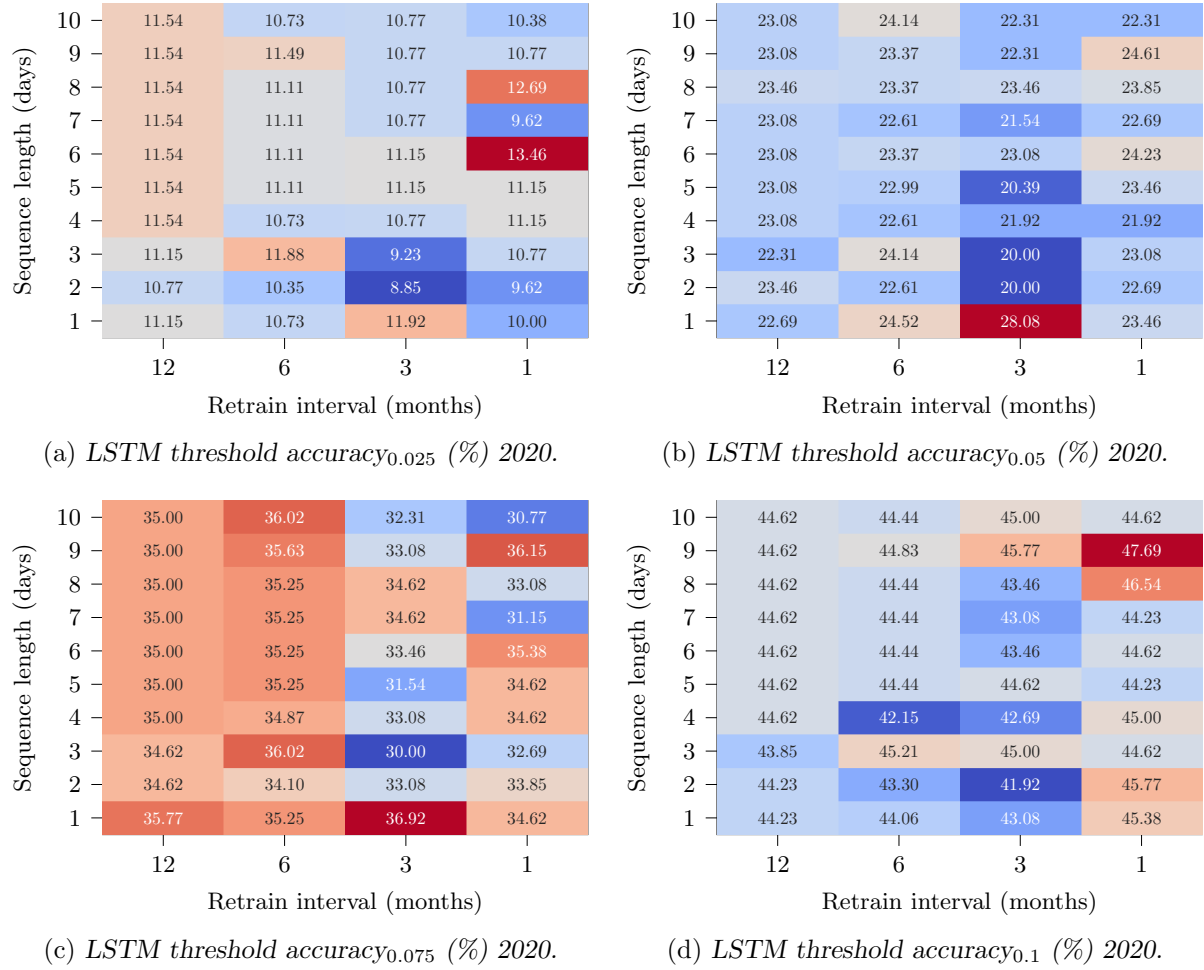


FIGURE B.12: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's different threshold accuracies for the **2020** test set.

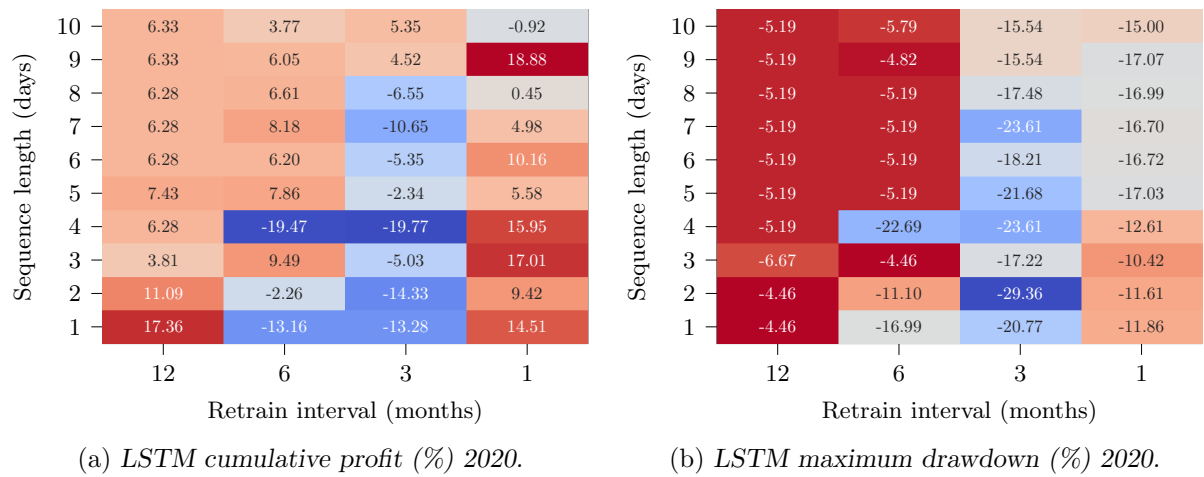


FIGURE B.13: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's cumulative profit (left) and maximum drawdown (right) for the **2020** test set.

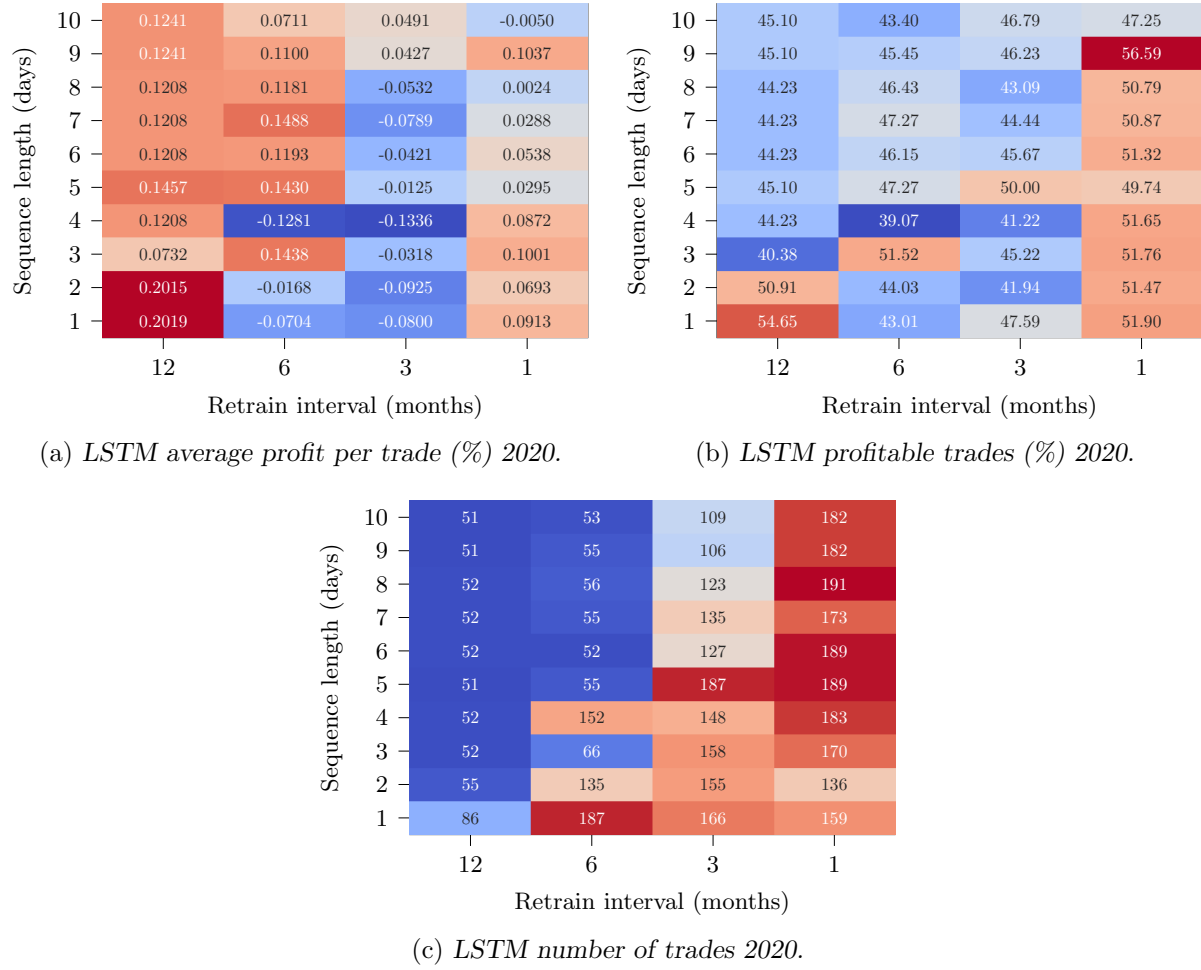


FIGURE B.14: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2020** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.03329	0.84526	7.75	-5.19	45.82	55
	6	0.03379	0.8536	1.33	-8.66	45.36	87
	3	0.03534	0.87323	-6.74	-20.3	45.22	141
	1	0.03439	0.85498	9.6	-14.6	51.33	175
Seq length	1	0.03468	0.85913	1.36	-13.52	49.29	150
	2	0.03485	0.86508	0.98	-14.13	47.09	120
	3	0.03492	0.86328	6.32	-9.69	47.22	112
	4	0.03403	0.85808	-4.25	-16.03	44.04	134
	5	0.03397	0.85375	4.63	-12.27	48.03	120
	6	0.03386	0.85238	4.32	-11.32	46.84	105
	7	0.03413	0.85798	2.2	-12.67	46.7	104
	8	0.03406	0.85399	1.7	-11.21	46.13	106
	9	0.03375	0.849	8.95	-10.65	48.34	98
	10	0.03379	0.85501	3.63	-10.38	45.63	99

TABLE B.3: The **LSTM** model's aggregated results for each tested parameter value on selected performance metrics for the **2020** test set.

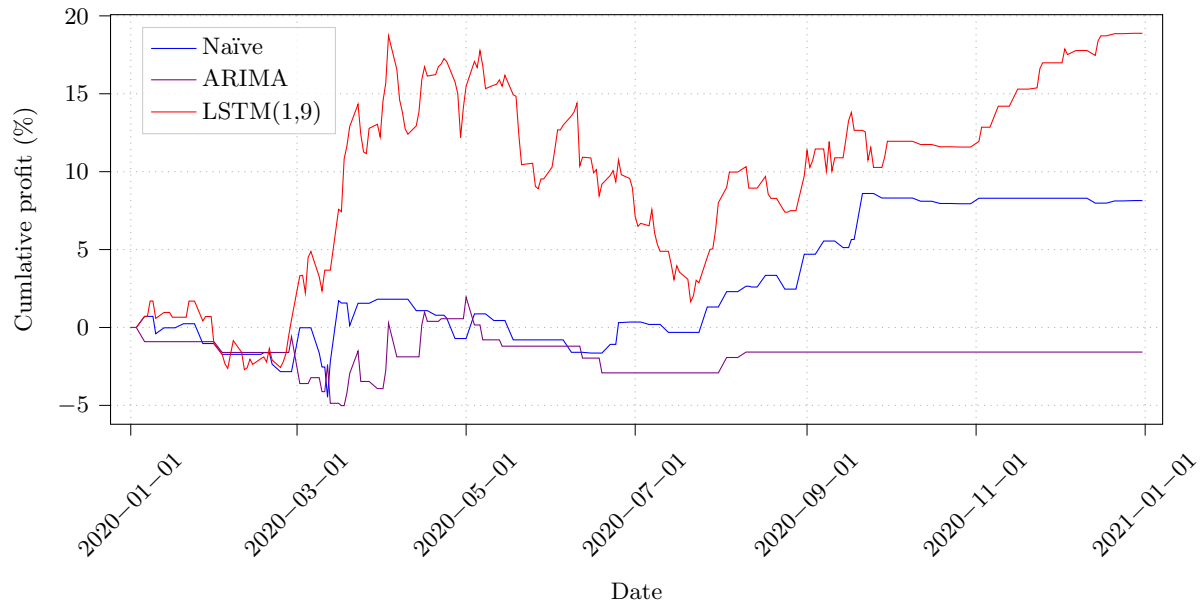


FIGURE B.15: The **LSTM**(1, 9) model's (most profitable **LSTM** model) cumulative profit for the **2020** test set compared to the naïve and ARIMA models.

B.1.4 Test set 2021

The results obtained for the 2021 testing period of the LSTM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.16(a) and Figure B.16(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.17(a), Figure B.17(b), Figure B.17(c), and Figure B.17(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.18(a) and Figure B.18(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.19(a), Figure B.19(b) and Figure B.19(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.4 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2021 testing period.
- Figure B.20 shows the cumulative profit of the most profitable LSTM model, LSTM(12, 4), compared to the naïve and ARIMA models.

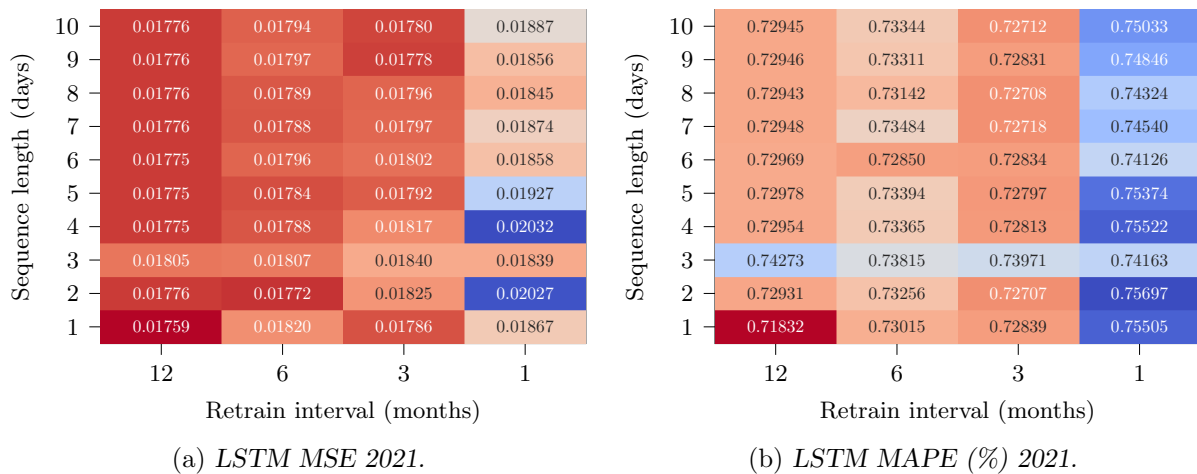


FIGURE B.16: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's MSE (left) and MAPE (right) for the **2021** test set.

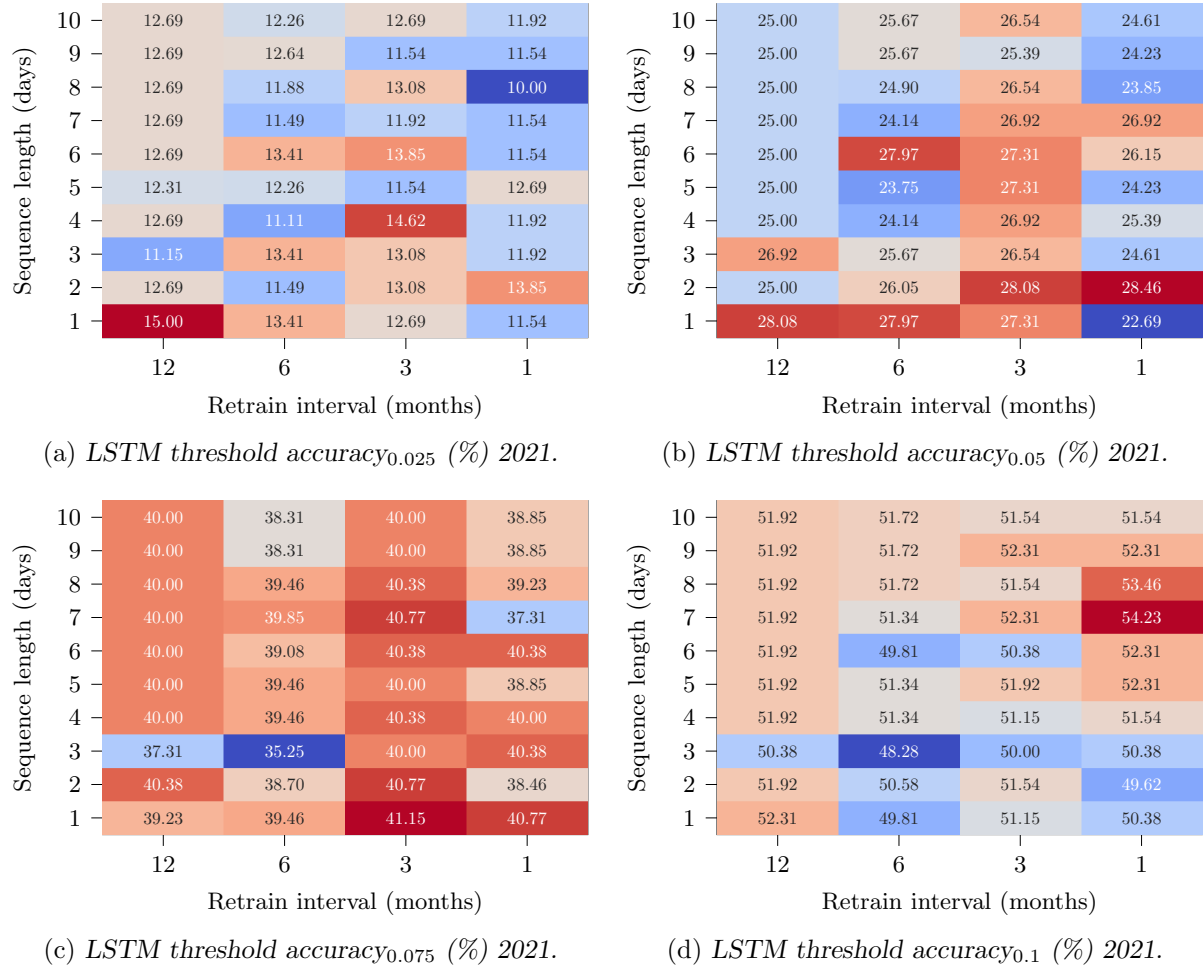


FIGURE B.17: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's different threshold accuracies for the **2021** test set.

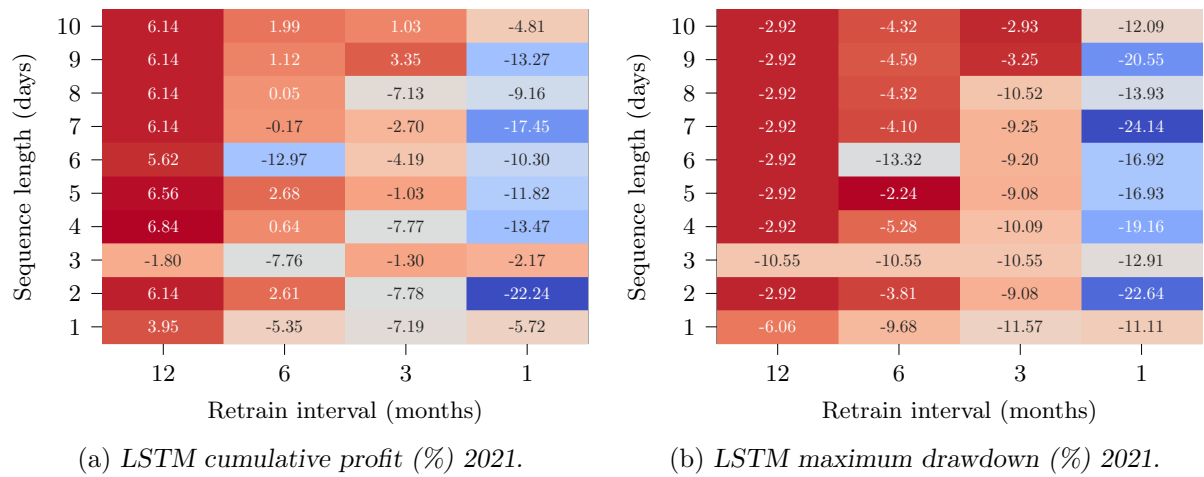


FIGURE B.18: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's cumulative profit (left) and maximum drawdown (right) for the **2021** test set.

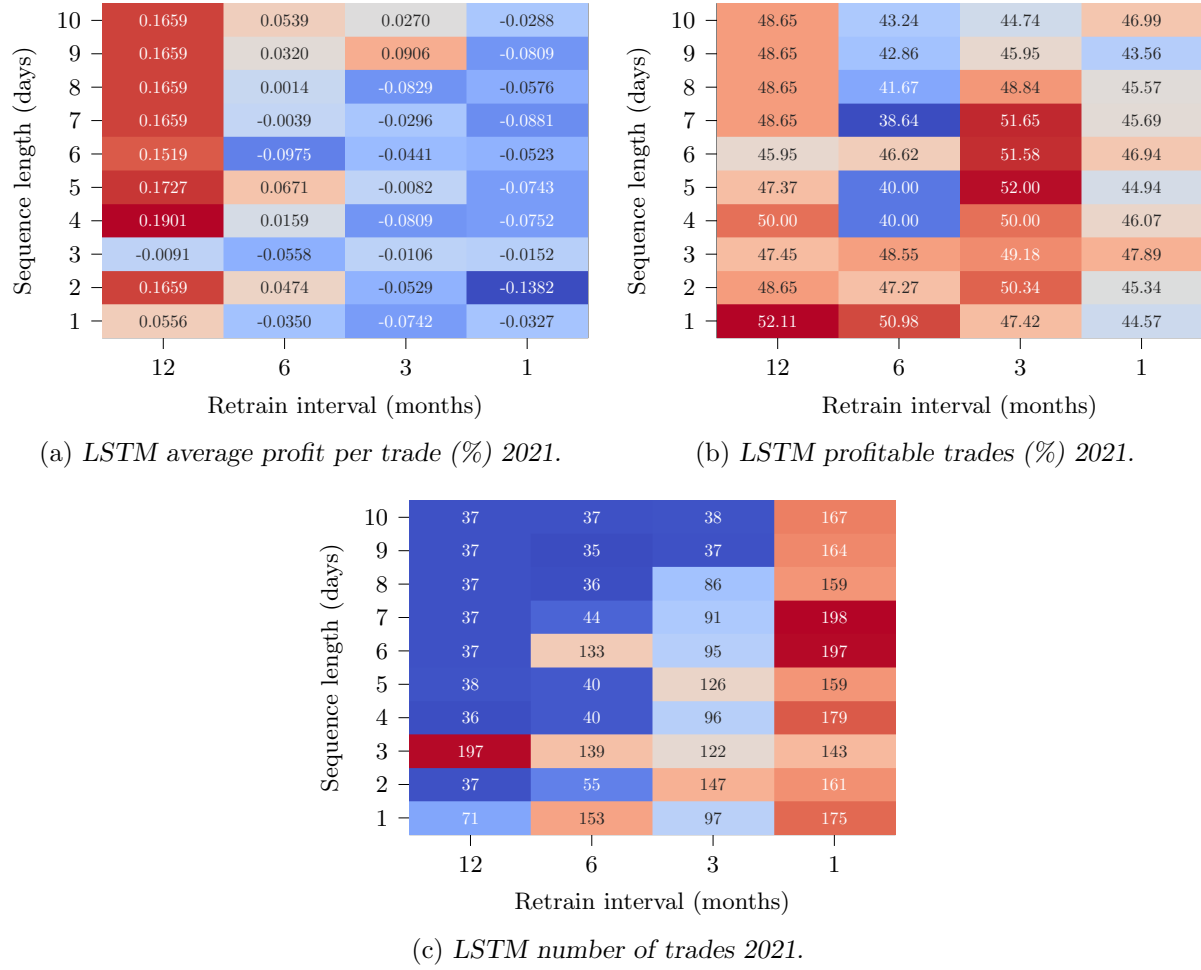


FIGURE B.19: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01777	0.72972	5.19	−4.0	48.61	56
	6	0.01794	0.73298	−1.72	−6.22	43.98	71
	3	0.01801	0.72893	−3.47	−8.55	49.17	94
	1	0.01901	0.74913	−11.04	−17.04	45.75	170
Seq length	1	0.01808	0.73298	−3.58	−9.6	48.77	124
	2	0.0185	0.73648	−5.32	−9.61	47.9	100
	3	0.01823	0.74056	−3.26	−11.14	48.27	150
	4	0.01853	0.73664	−3.44	−9.36	46.52	88
	5	0.0182	0.73636	−0.9	−7.79	46.08	91
	6	0.01808	0.73195	−5.46	−10.59	47.77	116
	7	0.01809	0.73422	−3.55	−10.1	46.15	92
	8	0.01802	0.73279	−2.53	−7.92	46.18	80
	9	0.01802	0.73484	−0.66	−7.83	45.25	68
	10	0.01809	0.73508	1.09	−5.57	45.9	70

TABLE B.4: The **LSTM** model's aggregated results for each tested parameter value on selected performance metrics for the **2021** test set.

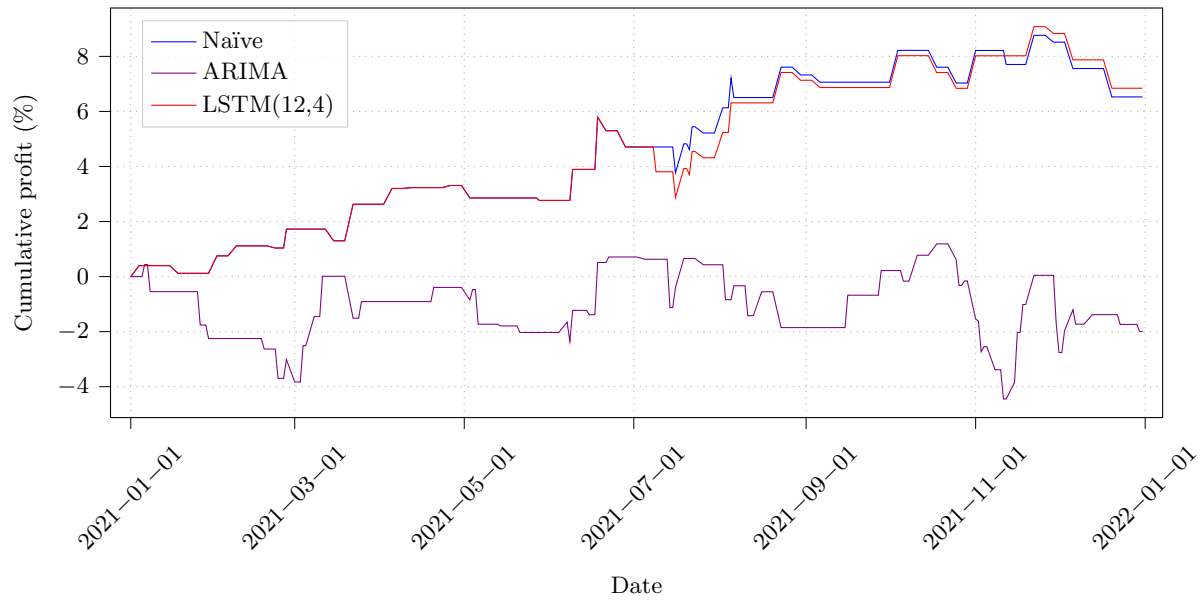


FIGURE B.20: The **LSTM**(12, 4) model's (most profitable **LSTM** model) cumulative profit for the **2021** test set compared to the naïve and Arima models.

B.1.5 Test set 2018–2021

The results obtained for the 2018–2021 testing period of the LSTM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.21(a) and Figure B.21(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.22(a), Figure B.22(b), Figure B.22(c), and Figure B.22(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.23(a) and Figure B.23(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.24(a), Figure B.24(b) and Figure B.24(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.5 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the entire testing period 2018–2021.
- Figure B.25 shows the cumulative profit of the most profitable LSTM model, LSTM(12, 2), compared to the naïve and ARIMA models.

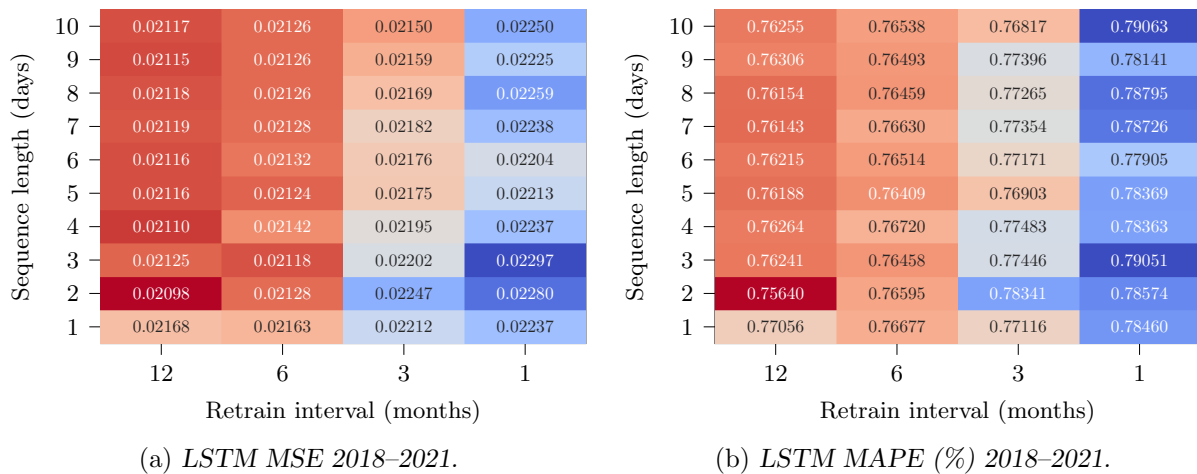


FIGURE B.21: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's MSE (left) and MAPE (right) for the **2018–2021** test set.

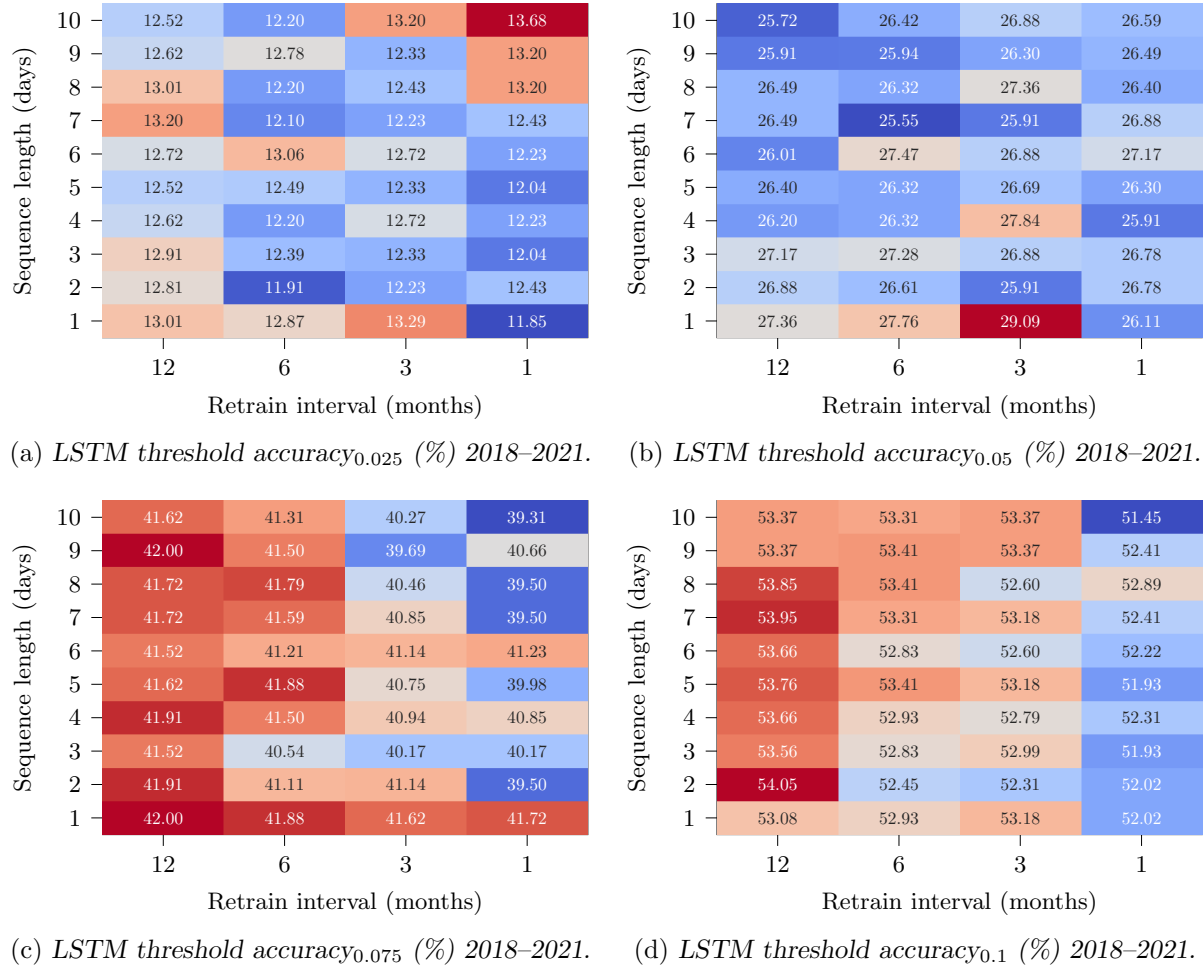


FIGURE B.22: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's different threshold accuracies for the **2018–2021** test set.

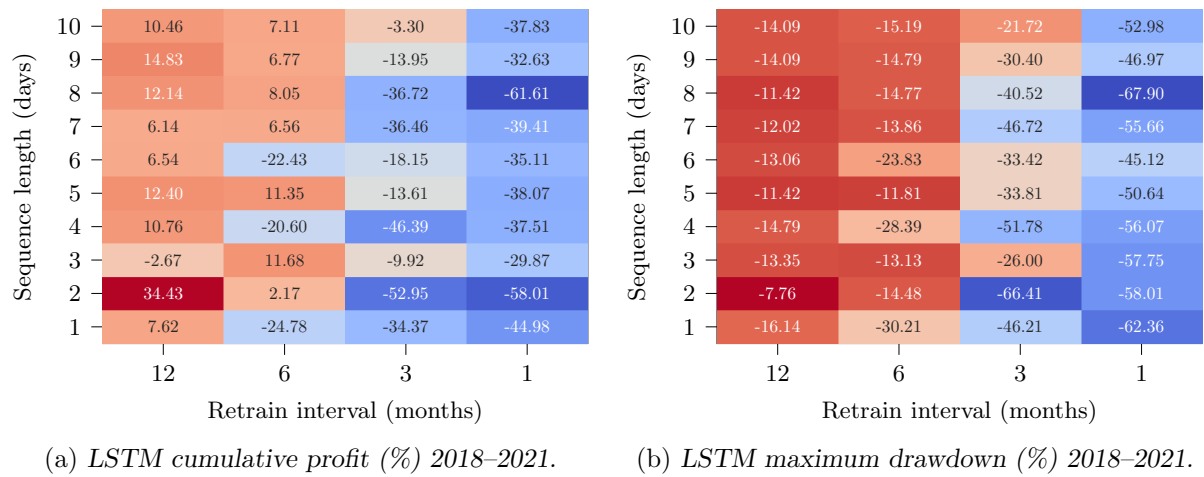


FIGURE B.23: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's cumulative profit (left) and maximum drawdown (right) for the **2018–2021** test set.

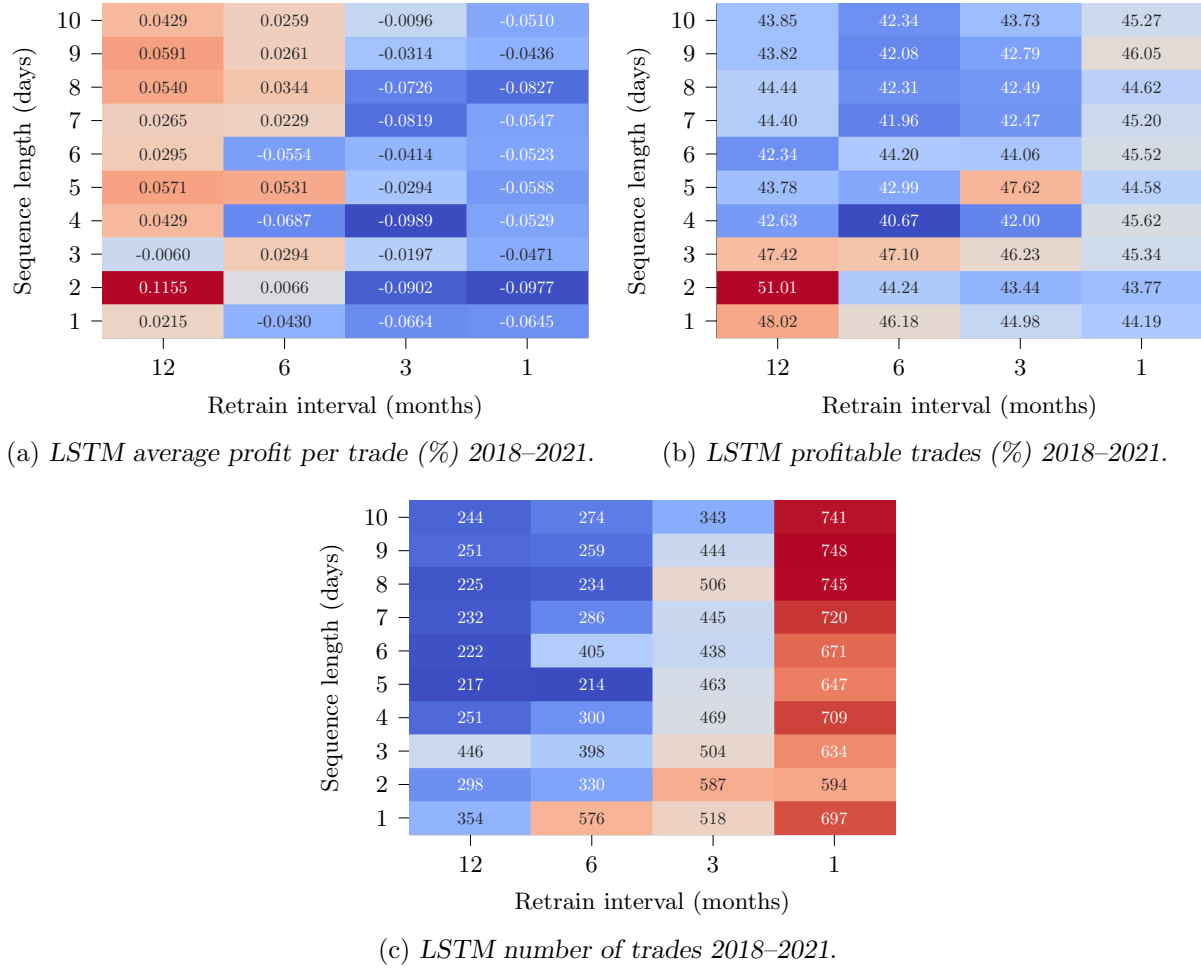


FIGURE B.24: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **LSTM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018–2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0212	0.76246	11.26	−12.81	45.17	274
	6	0.02131	0.76549	−1.41	−18.05	43.41	328
	3	0.02187	0.77329	−26.58	−39.7	43.98	472
	1	0.02244	0.78545	−41.5	−55.35	45.02	691
Seq length	1	0.02195	0.77327	−24.13	−38.73	45.84	536
	2	0.02188	0.77288	−18.59	−36.67	45.62	452
	3	0.02186	0.77299	−7.69	−27.56	46.52	496
	4	0.02171	0.77208	−23.44	−37.76	42.73	432
	5	0.02157	0.76967	−6.98	−26.92	44.74	385
	6	0.02157	0.76951	−17.29	−28.86	44.03	434
	7	0.02167	0.77213	−15.79	−32.07	43.51	421
	8	0.02168	0.77168	−19.53	−33.65	43.47	428
	9	0.02156	0.77084	−6.24	−26.56	43.69	426
	10	0.02161	0.77168	−5.89	−25.99	43.8	400

TABLE B.5: The **LSTM** model's aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

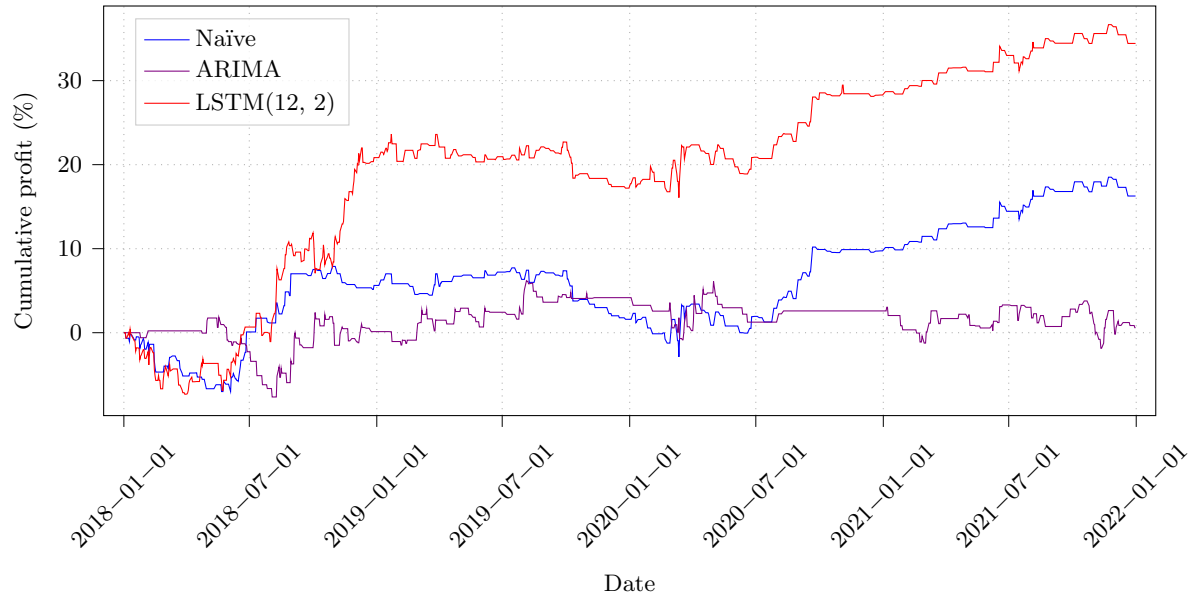


FIGURE B.25: The **LSTM(12, 2)** model's (most profitable **LSTM** model) cumulative profit for the **2018–2021** test set compared to the naïve and ARIMA models.

B.2 RF

This section contains the RF model results, with a breakdown by testing year and a summary of the entire testing period.

B.2.1 Test set 2018

The results obtained for the 2018 testing period of the RF model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.26(a) and Figure B.26(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.27(a), Figure B.27(b), Figure B.27(c), and Figure B.27(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.28(a) and Figure B.28(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.29(a), Figure B.29(b) and Figure B.29(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.6 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2018 testing period.
- Figure B.30 shows the cumulative profit of the most profitable RF model, RF(12, 2), compared to the naïve and ARIMA models.

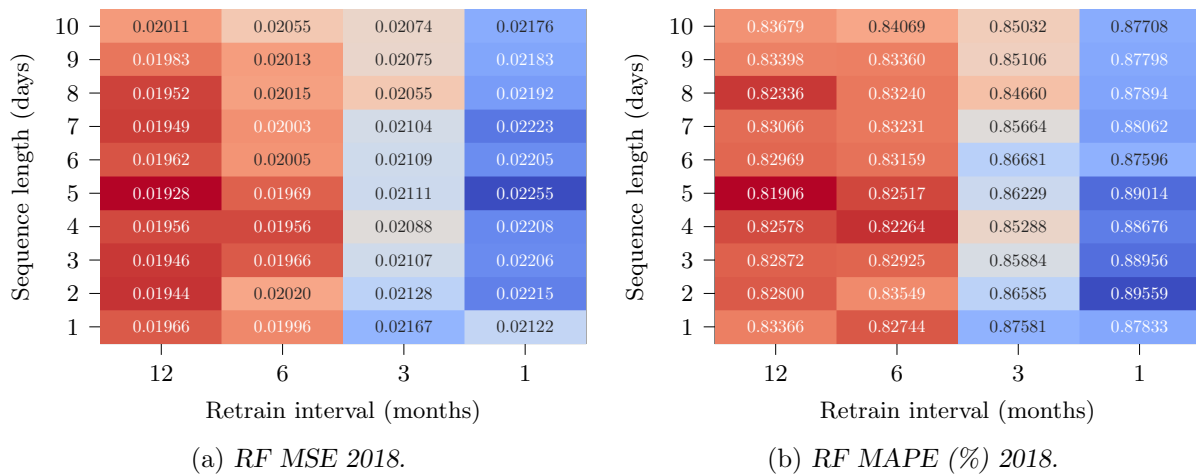


FIGURE B.26: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's MSE (left) and MAPE (right) for the **2018** test set.

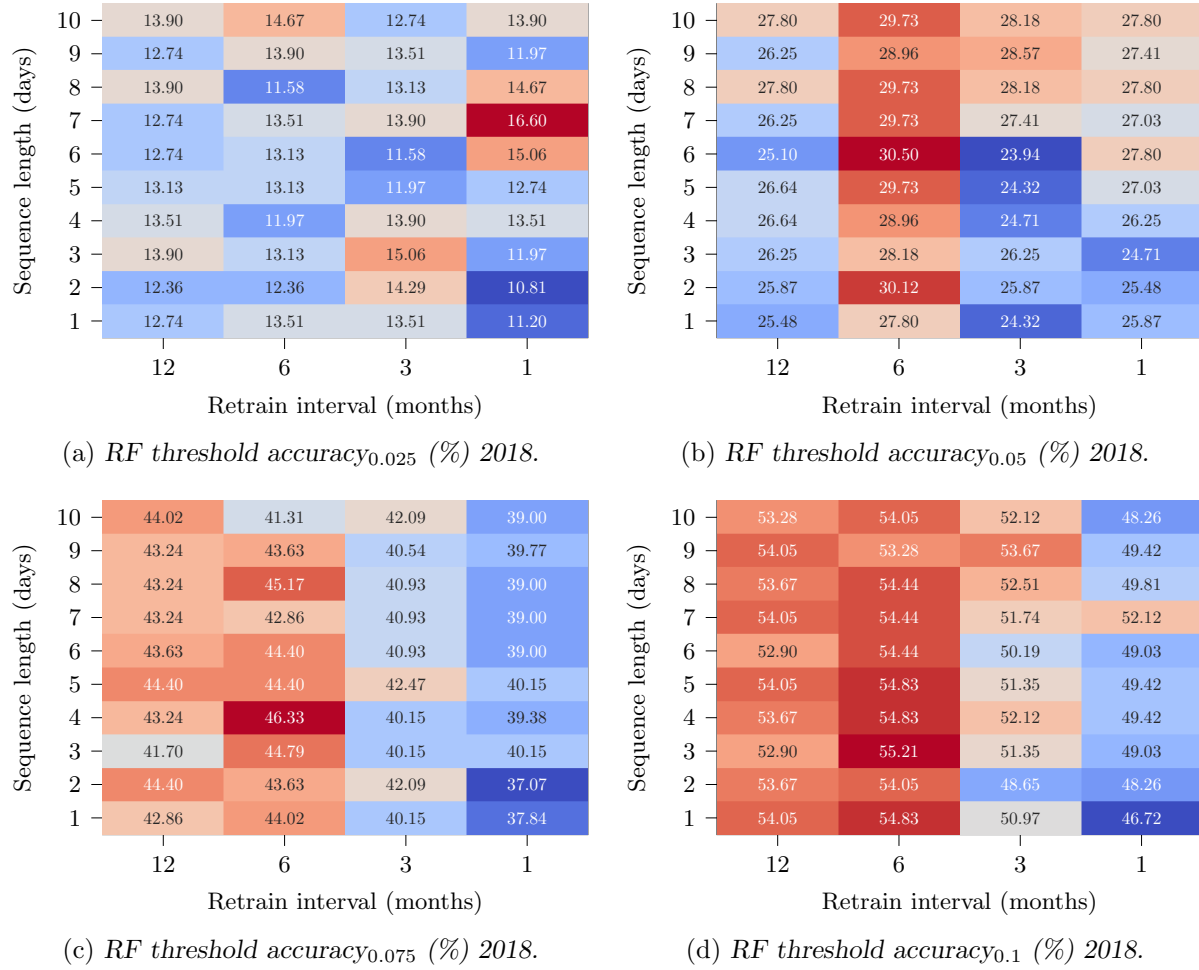


FIGURE B.27: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's different threshold accuracies for the **2018** test set.

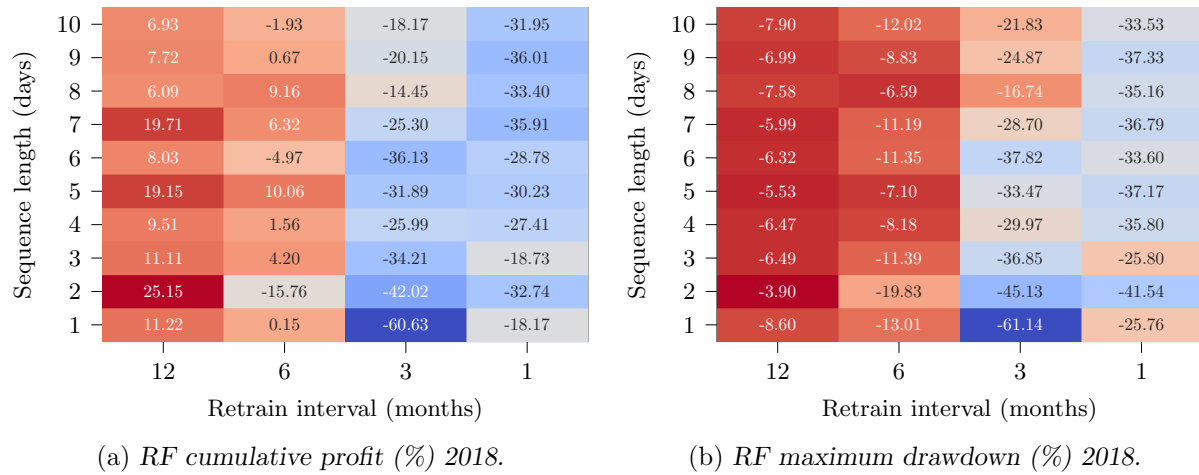


FIGURE B.28: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's cumulative profit (left) and maximum drawdown (right) for the **2018** test set.

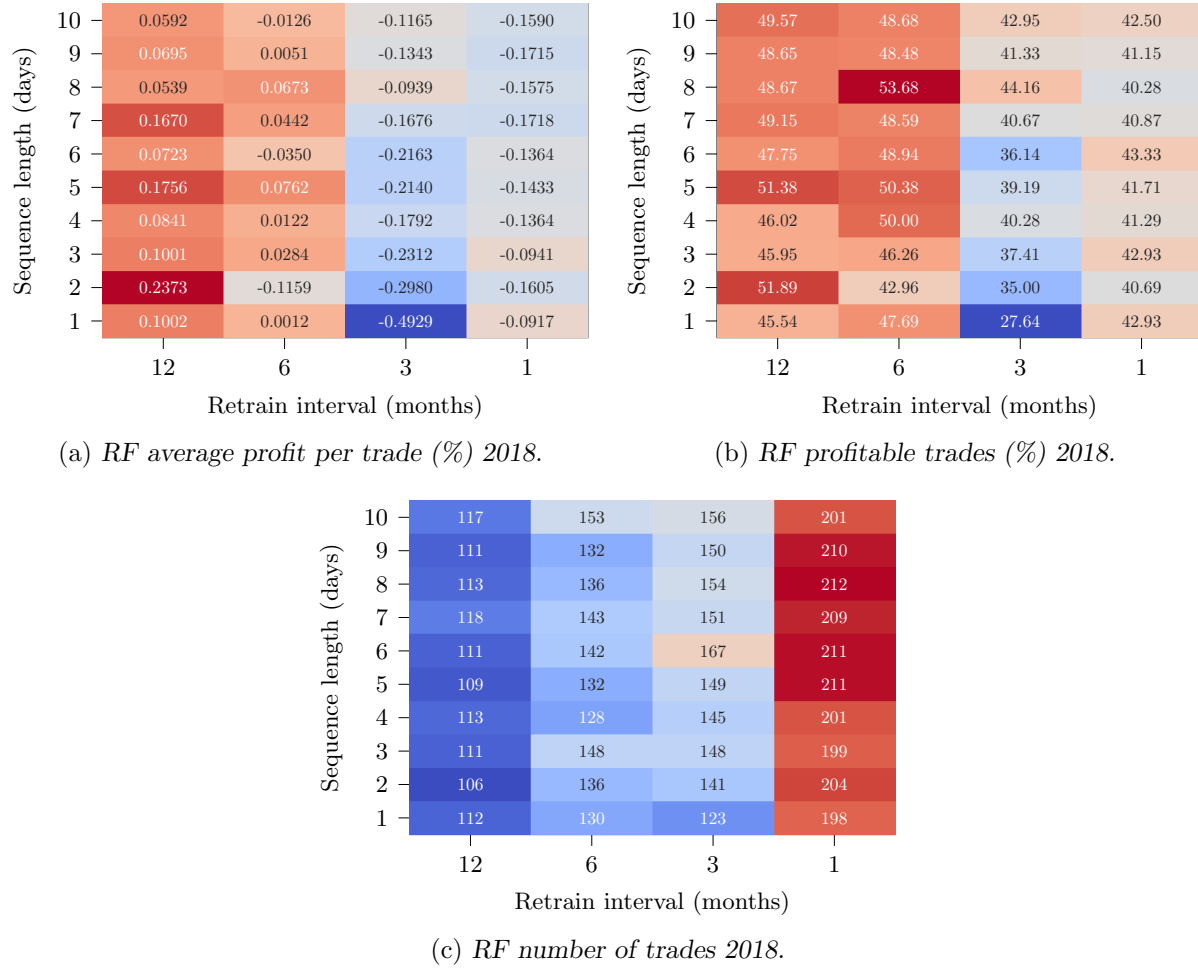


FIGURE B.29: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0196	0.82897	12.46	−6.58	48.46	112
	6	0.02	0.83106	0.95	−10.95	48.57	138
	3	0.02102	0.85871	−30.89	−33.65	38.48	148
	1	0.02198	0.8831	−29.33	−34.25	41.77	206
Seq length	1	0.02063	0.85381	−16.85	−27.13	40.95	141
	2	0.02077	0.85623	−16.34	−27.6	42.63	147
	3	0.02056	0.85159	−9.41	−20.13	43.14	152
	4	0.02052	0.84702	−10.58	−20.11	44.4	147
	5	0.02066	0.84916	−8.23	−20.82	45.66	150
	6	0.0207	0.85101	−15.46	−22.27	44.04	158
	7	0.0207	0.85006	−8.8	−20.67	44.82	155
	8	0.02053	0.84532	−8.15	−16.52	46.7	154
	9	0.02064	0.84916	−11.94	−19.51	44.9	151
	10	0.02079	0.85122	−11.28	−18.82	45.93	157

TABLE B.6: The **RF** model's aggregated results for each tested parameter value on selected performance metrics for the **2018** test set.

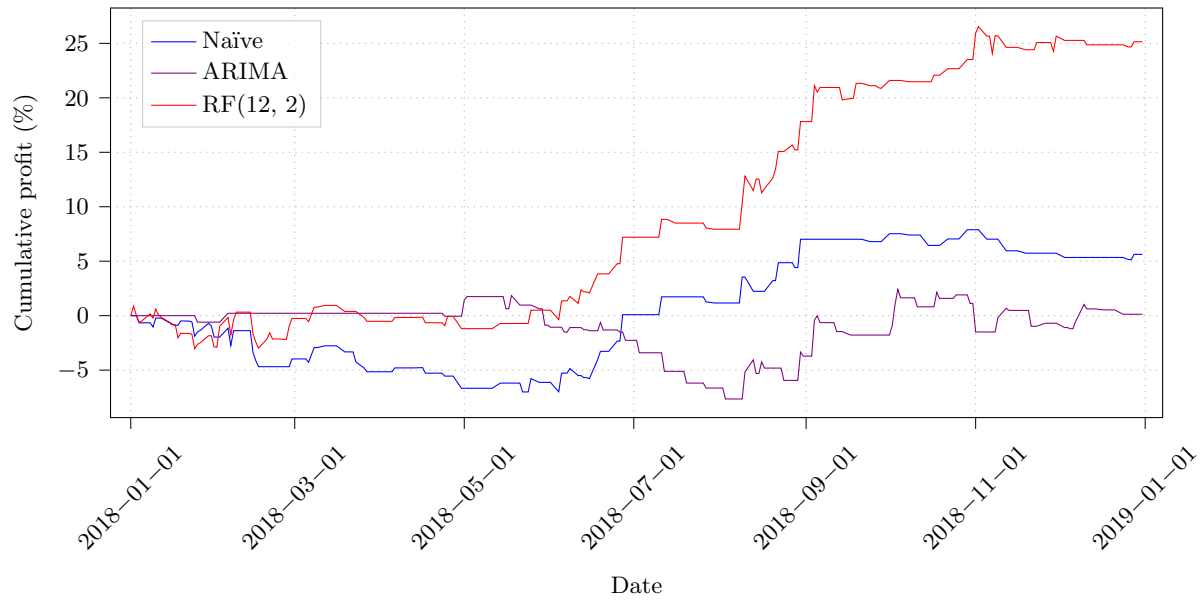


FIGURE B.30: The **RF(12, 2)** model's (most profitable **RF** model) cumulative profit for the **2018** test set compared to the naïve and **ARIMA** models.

B.2.2 Test set 2019

The results obtained for the 2019 testing period of the RF model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.31(a) and Figure B.31(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.32(a), Figure B.32(b), Figure B.32(c), and Figure B.32(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.33(a) and Figure B.33(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.34(a), Figure B.34(b) and Figure B.34(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.7 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2019 testing period.
- Figure B.35 shows the cumulative profit of the most profitable RF model, RF(3, 1), compared to the naïve and ARIMA models.

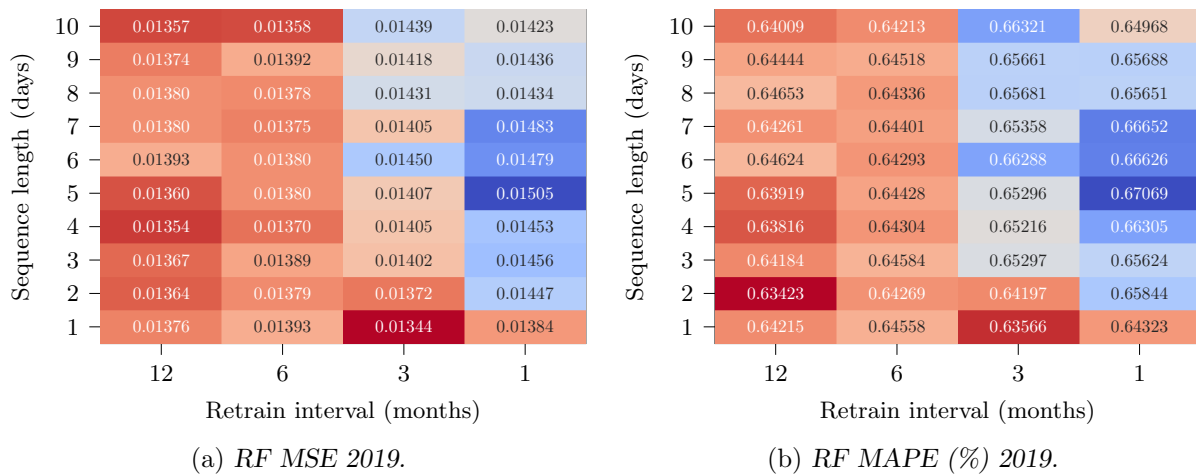


FIGURE B.31: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's MSE (left) and MAPE (right) for the **2019** test set.

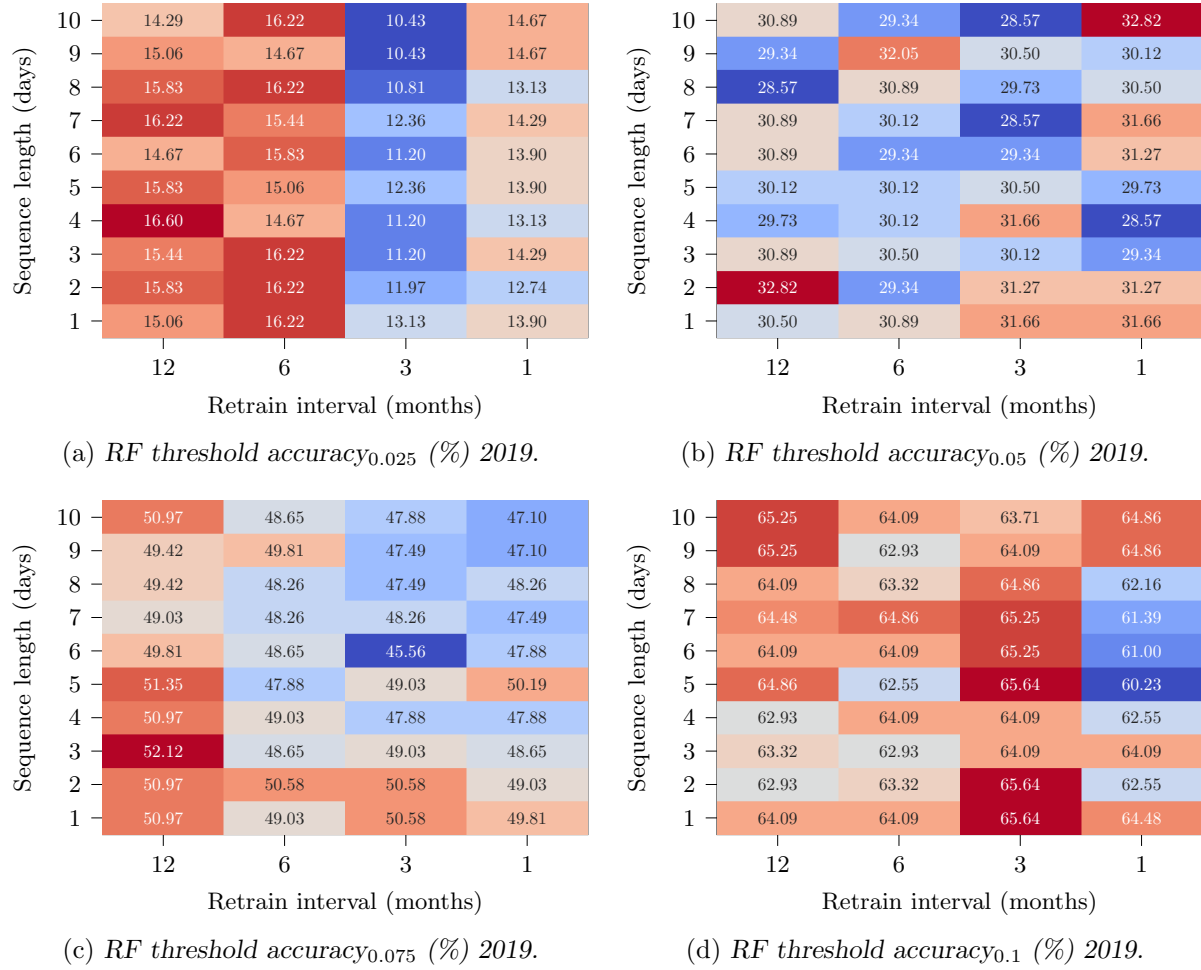


FIGURE B.32: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's different threshold accuracies for the **2019** test set.

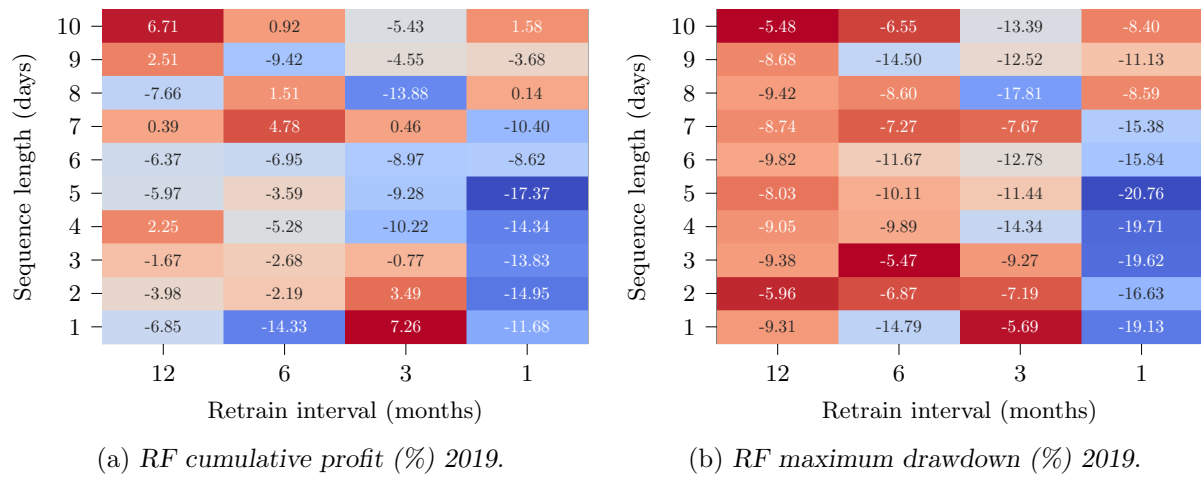


FIGURE B.33: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's cumulative profit (left) and maximum drawdown (right) for the **2019** test set.

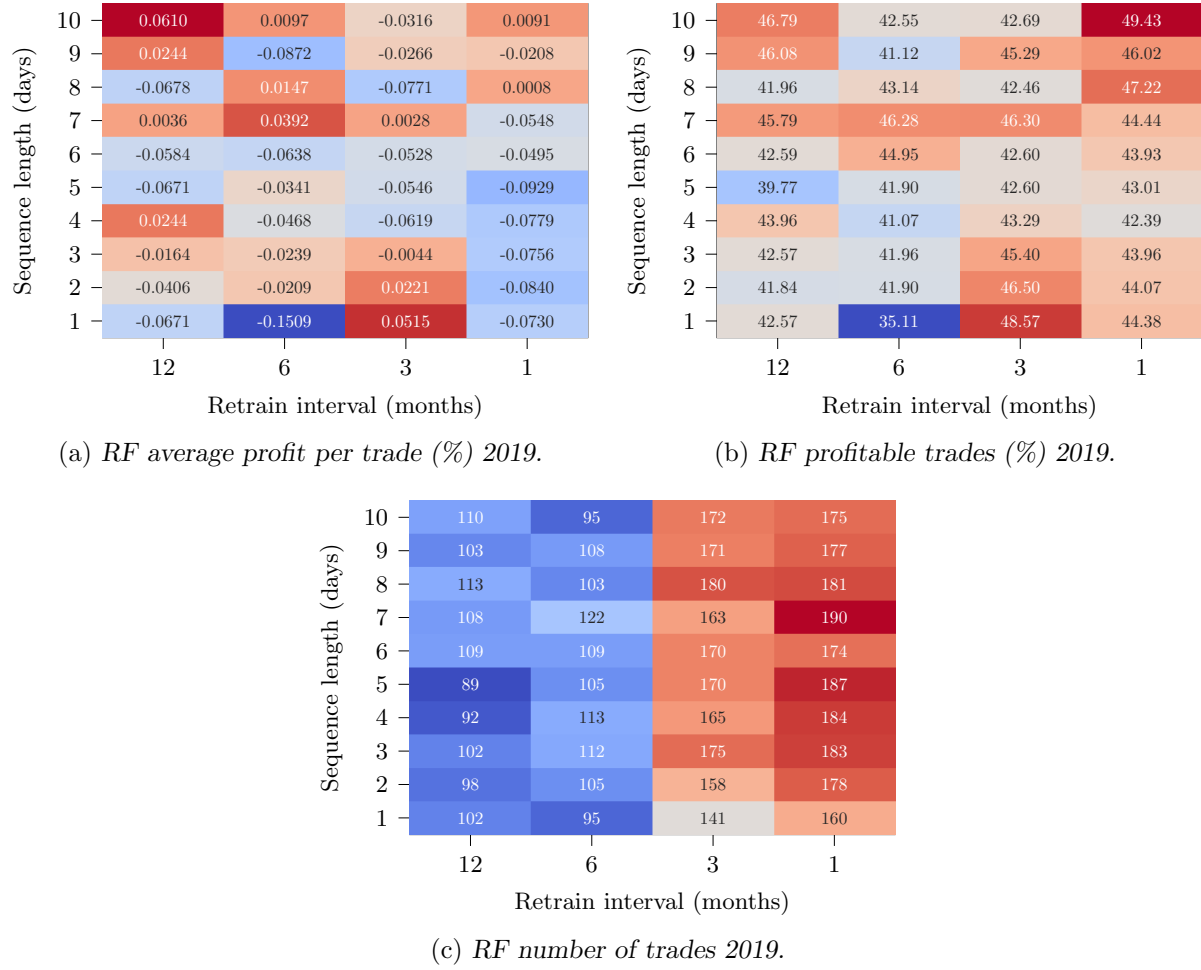


FIGURE B.34: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2019** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0137	0.64155	−2.06	−8.39	43.39	103
	6	0.01379	0.6439	−3.72	−9.57	42.0	107
	3	0.01407	0.65288	−4.19	−11.21	44.57	166
	1	0.0145	0.65875	−9.31	−15.52	44.88	179
Seq length	1	0.01374	0.64166	−6.4	−12.23	42.66	124
	2	0.0139	0.64433	−4.41	−9.16	43.58	135
	3	0.01404	0.64922	−4.74	−10.94	43.47	143
	4	0.01396	0.6491	−6.9	−13.25	42.68	138
	5	0.01413	0.65178	−9.05	−12.59	41.82	138
	6	0.01425	0.65458	−7.73	−12.53	43.52	140
	7	0.01411	0.65168	−1.19	−9.77	45.7	146
	8	0.01406	0.6508	−4.97	−11.11	43.7	144
	9	0.01405	0.65078	−3.79	−11.71	44.63	140
	10	0.01394	0.64878	0.95	−8.46	45.36	138

TABLE B.7: The **RF** model's aggregated results for each tested parameter value on selected performance metrics for the **2019** test set.

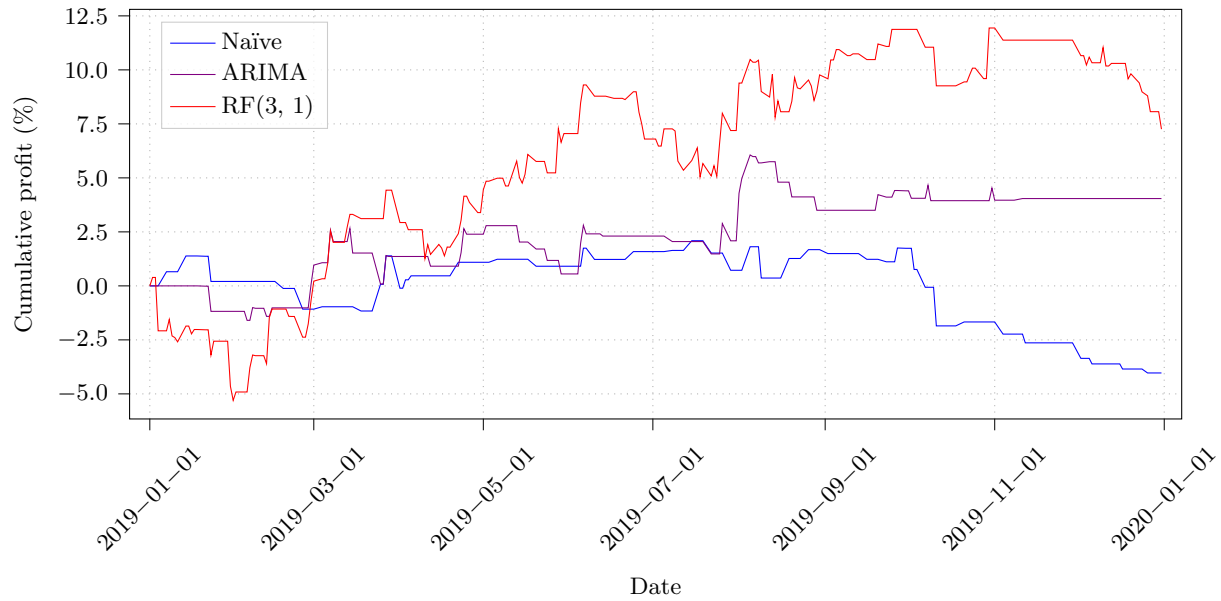


FIGURE B.35: The **RF(3, 1)** model's (most profitable **RF** model) cumulative profit for the **2019** test set compared to the naïve and ARIMA models.

B.2.3 Test set 2020

The results obtained for the 2020 testing period of the RF model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.36(a) and Figure B.36(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.37(a), Figure B.37(b), Figure B.37(c), and Figure B.37(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.38(a) and Figure B.38(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.39(a), Figure B.39(b) and Figure B.39(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.8 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2020 testing period.
- Figure B.40 shows the cumulative profit of the most profitable RF model, RF(12, 6), compared to the naïve and ARIMA models.

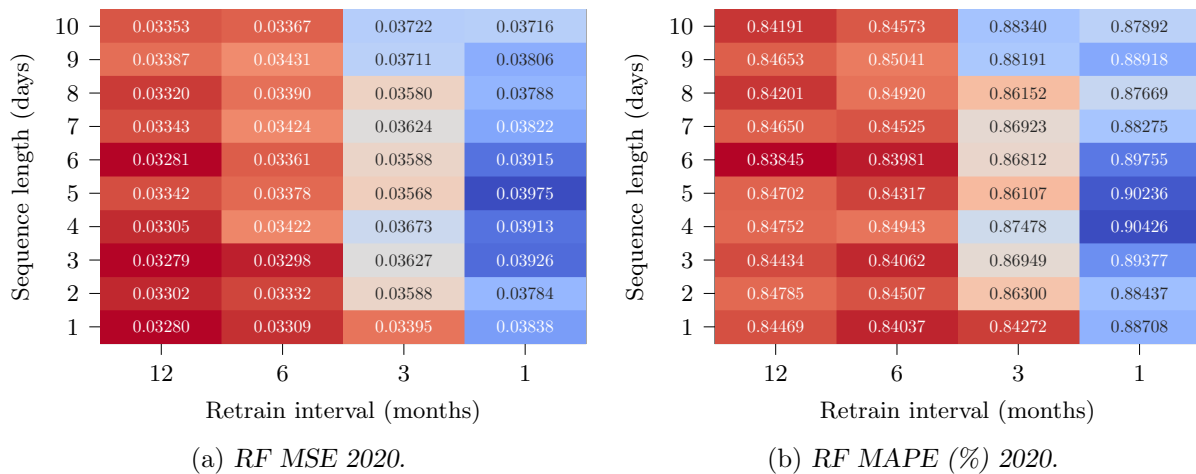


FIGURE B.36: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's MSE (left) and MAPE (right) for the **2020** test set.

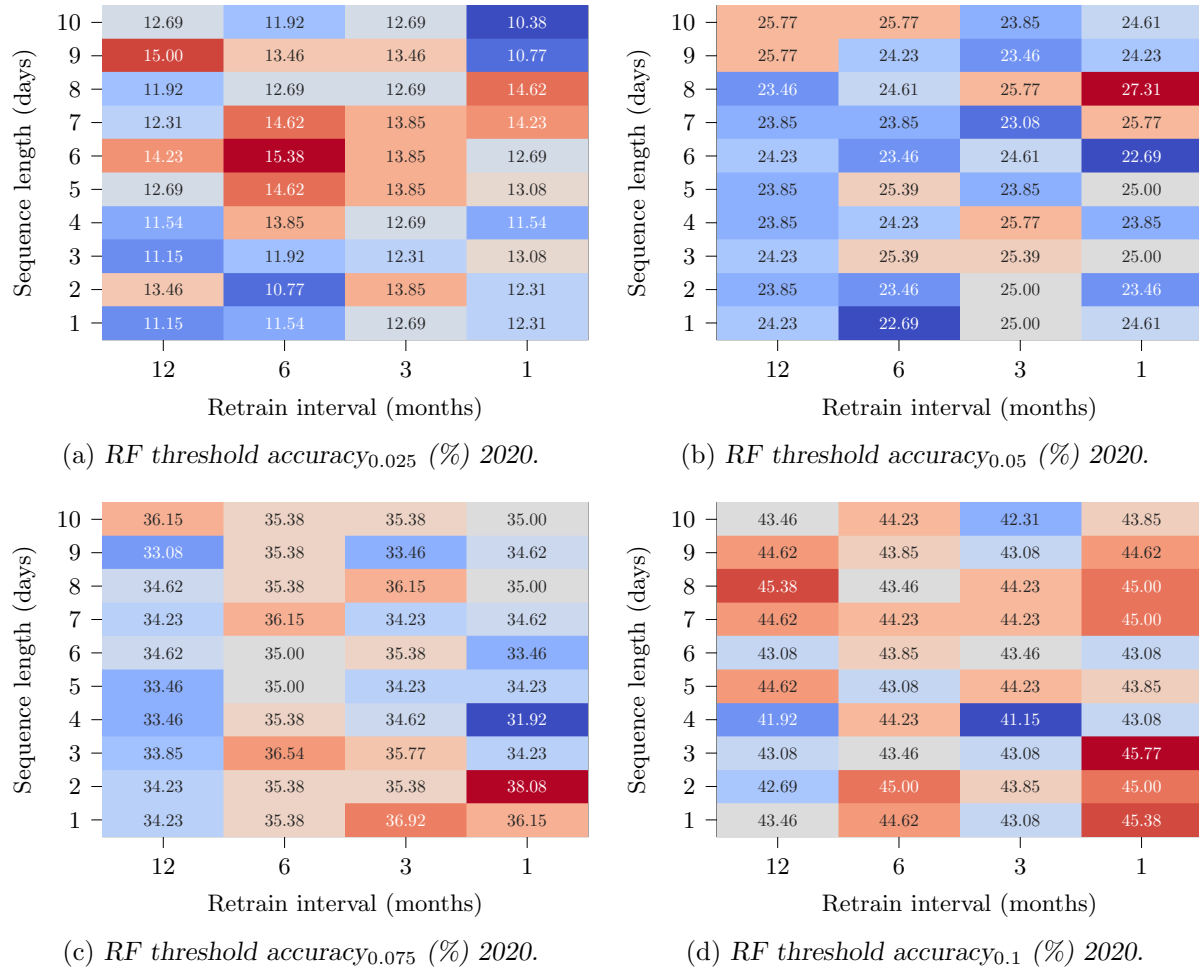


FIGURE B.37: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's different threshold accuracies for the **2020** test set.

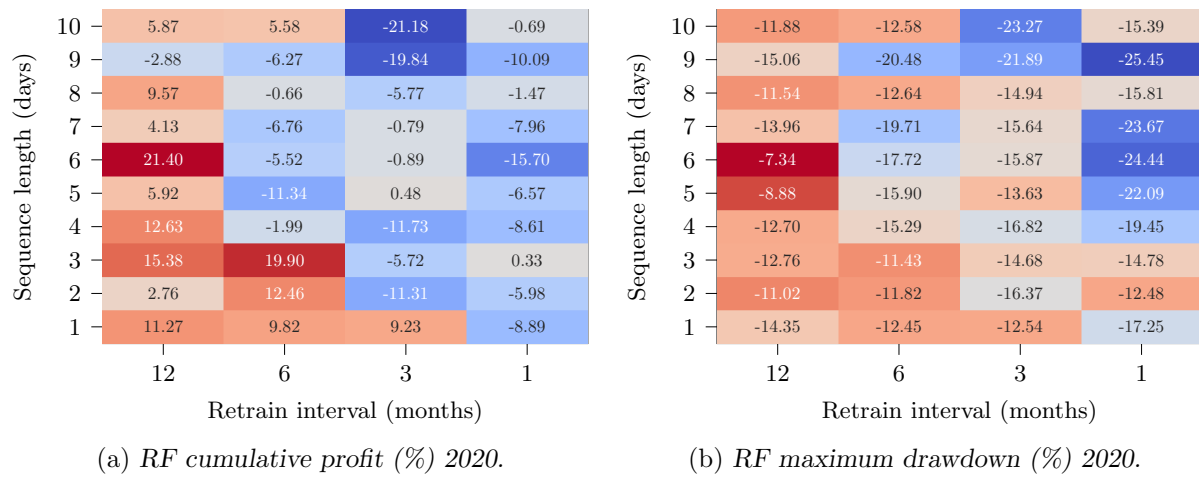


FIGURE B.38: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's cumulative profit (left) and maximum drawdown (right) for the **2020** test set.

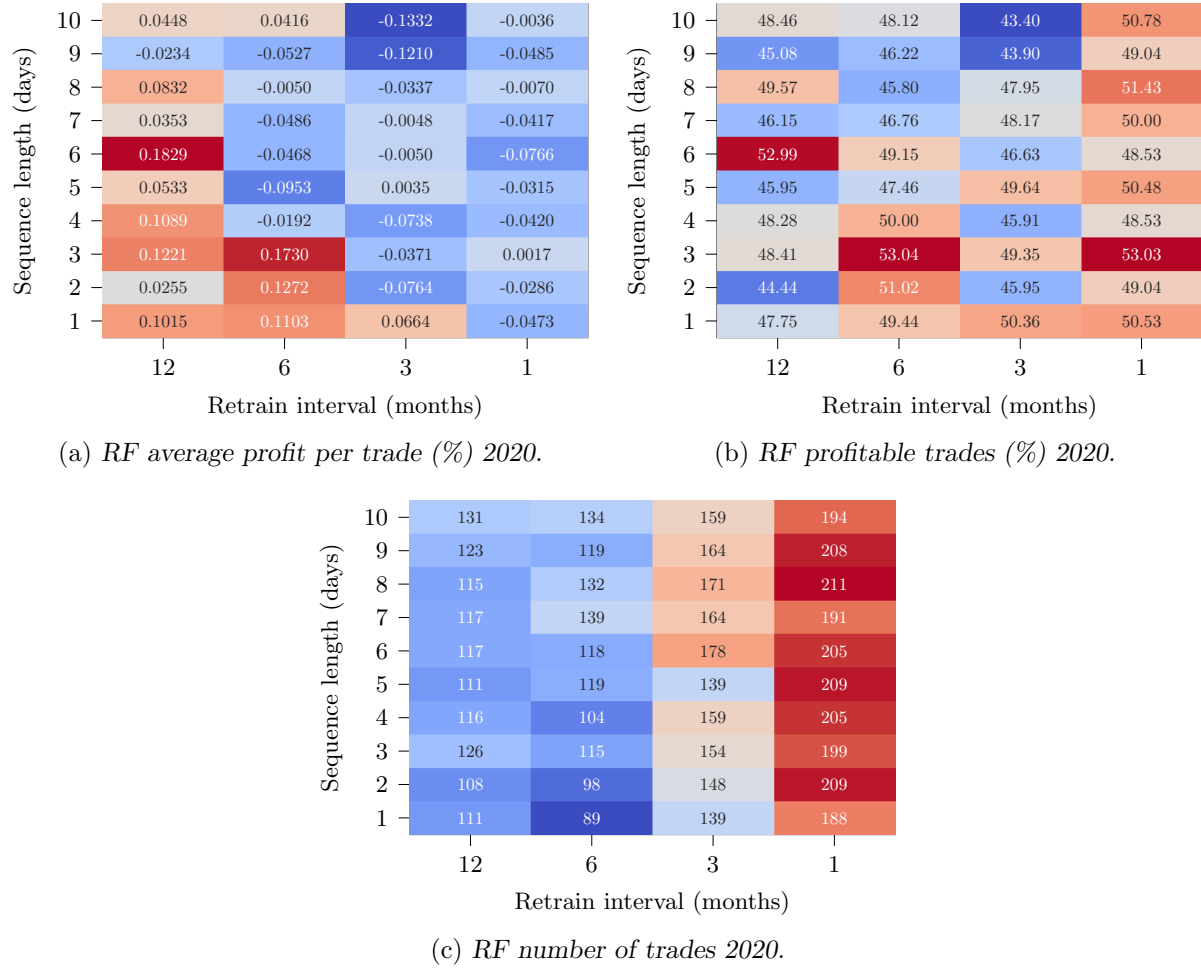


FIGURE B.39: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2020** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.03319	0.84468	8.61	−11.95	47.71	118
	6	0.03371	0.84491	1.52	−15.0	48.7	117
	3	0.03608	0.86752	−6.75	−16.57	47.13	158
	1	0.03848	0.88969	−6.56	−19.08	50.14	202
Seq length	1	0.03456	0.85372	5.36	−14.15	49.52	132
	2	0.03502	0.86007	−0.52	−12.92	47.61	141
	3	0.03532	0.86206	7.47	−13.41	50.96	148
	4	0.03578	0.869	−2.43	−16.06	48.18	146
	5	0.03566	0.8634	−2.88	−15.12	48.38	144
	6	0.03536	0.86098	−0.18	−16.34	49.33	154
	7	0.03553	0.86093	−2.84	−18.25	47.77	153
	8	0.03519	0.85736	0.42	−13.73	48.69	157
	9	0.03584	0.86701	−9.77	−20.72	46.06	154
	10	0.03539	0.86249	−2.61	−15.78	47.69	154

TABLE B.8: The **RF** model's aggregated results for each tested parameter value on selected performance metrics for the **2020** test set.

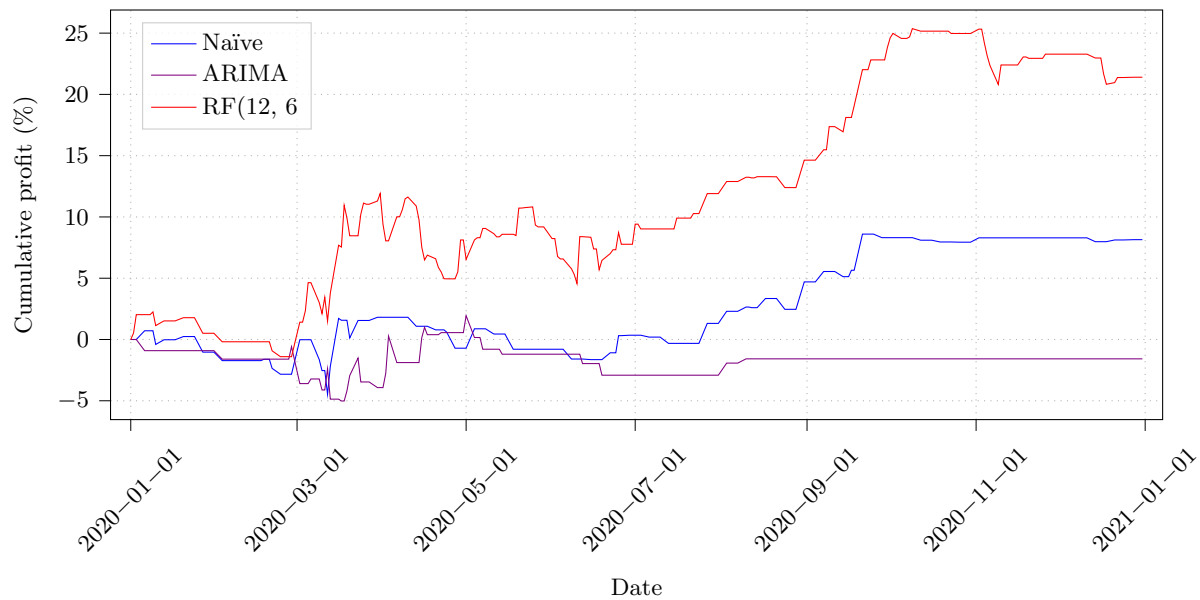


FIGURE B.40: The **RF(12, 6)** model's (most profitable **RF** model) cumulative profit for the **2020** test set compared to the naïve and **ARIMA** models.

B.2.4 Test set 2021

The results obtained for the 2021 testing period of the RF model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.41(a) and Figure B.41(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.42(a), Figure B.42(b), Figure B.42(c), and Figure B.42(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.43(a) and Figure B.43(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.44(a), Figure B.44(b) and Figure B.44(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.9 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2021 testing period.
- Figure B.45 shows the cumulative profit of the most profitable RF model, RF(6, 1), compared to the naïve and ARIMA models.

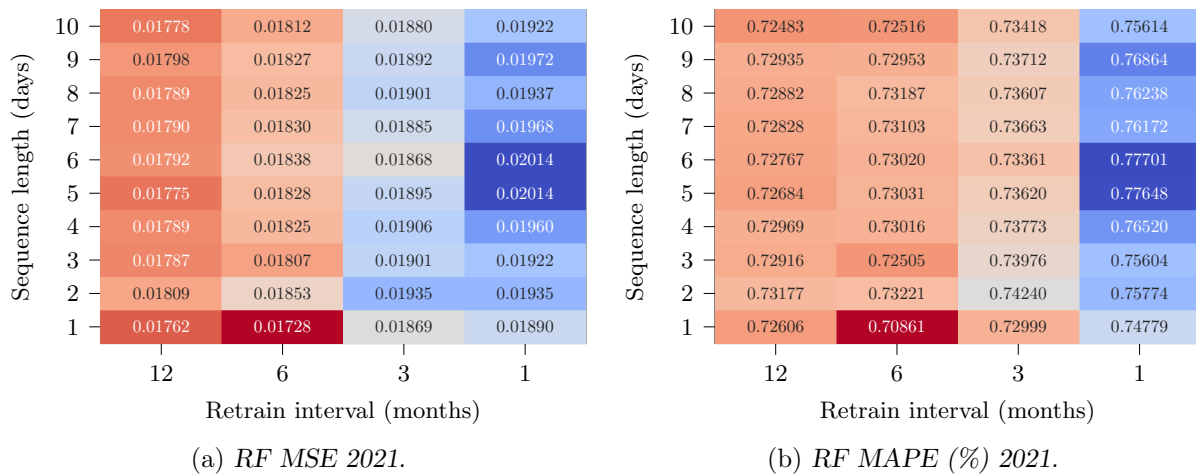


FIGURE B.41: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's MSE (left) and MAPE (right) for the **2021** test set.

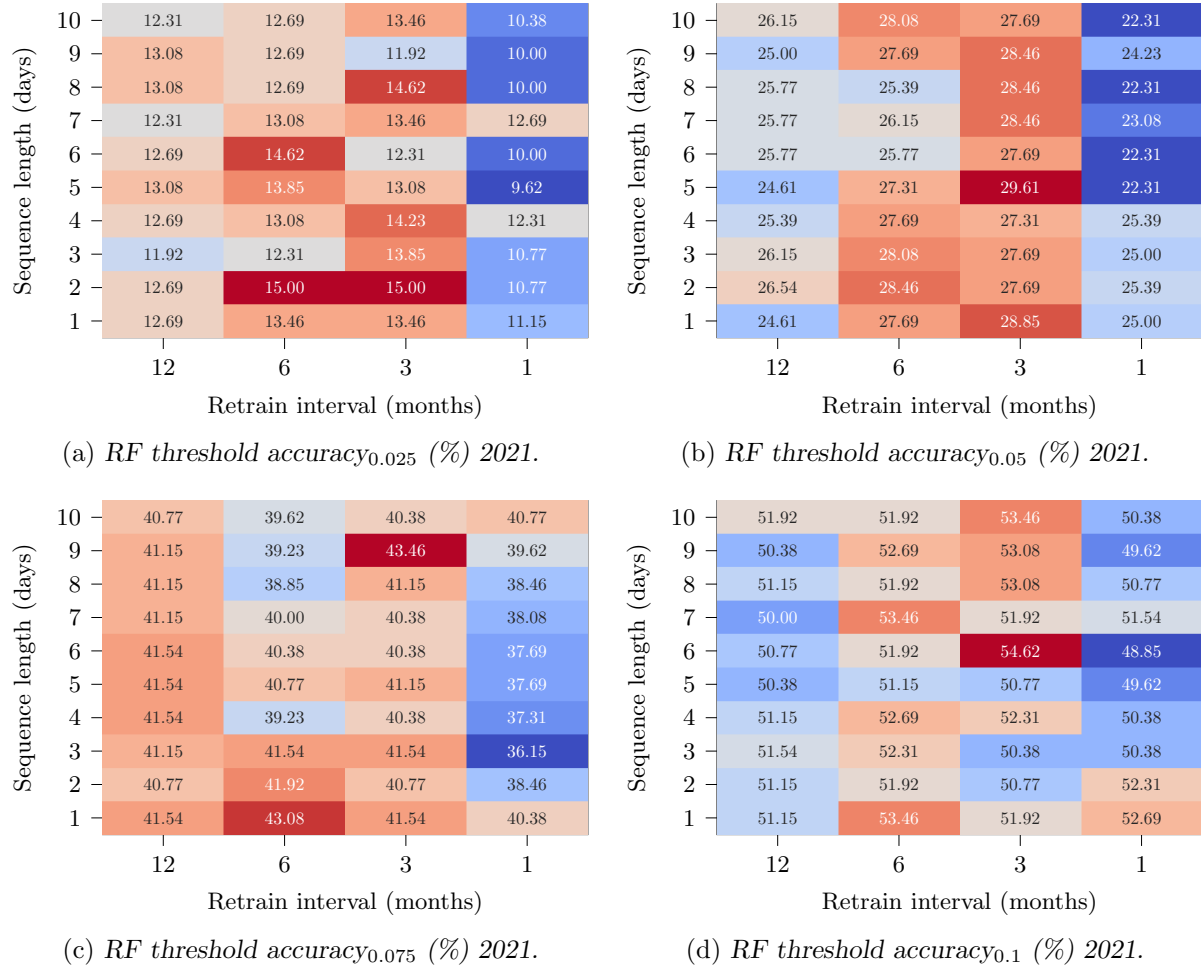


FIGURE B.42: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's different threshold accuracies for the **2021** test set.

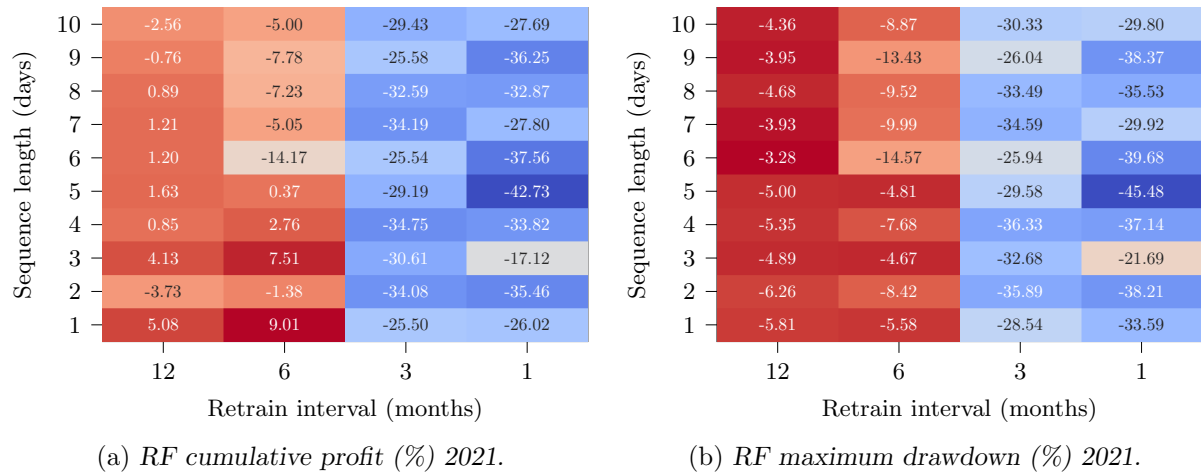


FIGURE B.43: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's cumulative profit (left) and maximum drawdown (right) for the **2021** test set.

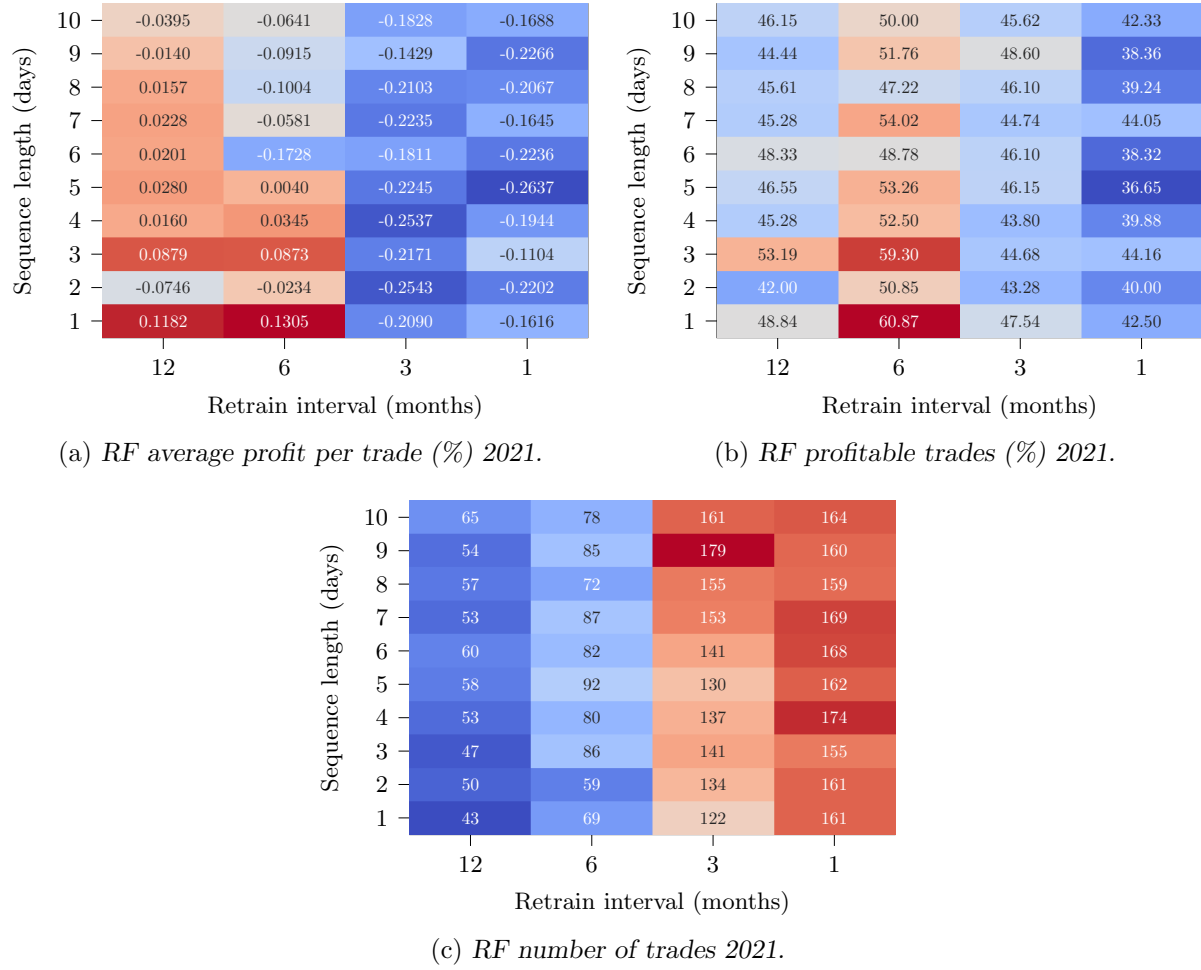


FIGURE B.44: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01787	0.72825	0.79	−4.75	46.57	54
	6	0.01817	0.72741	−2.1	−8.75	52.86	79
	3	0.01893	0.73637	−30.15	−31.34	45.66	145
	1	0.01953	0.76291	−31.73	−34.94	40.55	163
Seq length	1	0.01812	0.72811	−9.36	−18.38	49.94	99
	2	0.01883	0.74103	−18.66	−22.2	44.03	101
	3	0.01854	0.7375	−9.02	−15.98	50.33	107
	4	0.0187	0.7407	−16.24	−21.63	45.37	111
	5	0.01878	0.74246	−17.48	−21.22	45.65	110
	6	0.01878	0.74212	−19.02	−20.87	45.38	113
	7	0.01868	0.73942	−16.46	−19.61	47.02	116
	8	0.01863	0.73978	−17.95	−20.8	44.55	111
	9	0.01872	0.74116	−17.59	−20.45	45.79	120
	10	0.01848	0.73508	−16.17	−18.34	46.03	117

TABLE B.9: The **RF** model's aggregated results for each tested parameter value on selected performance metrics for the **2021** test set.

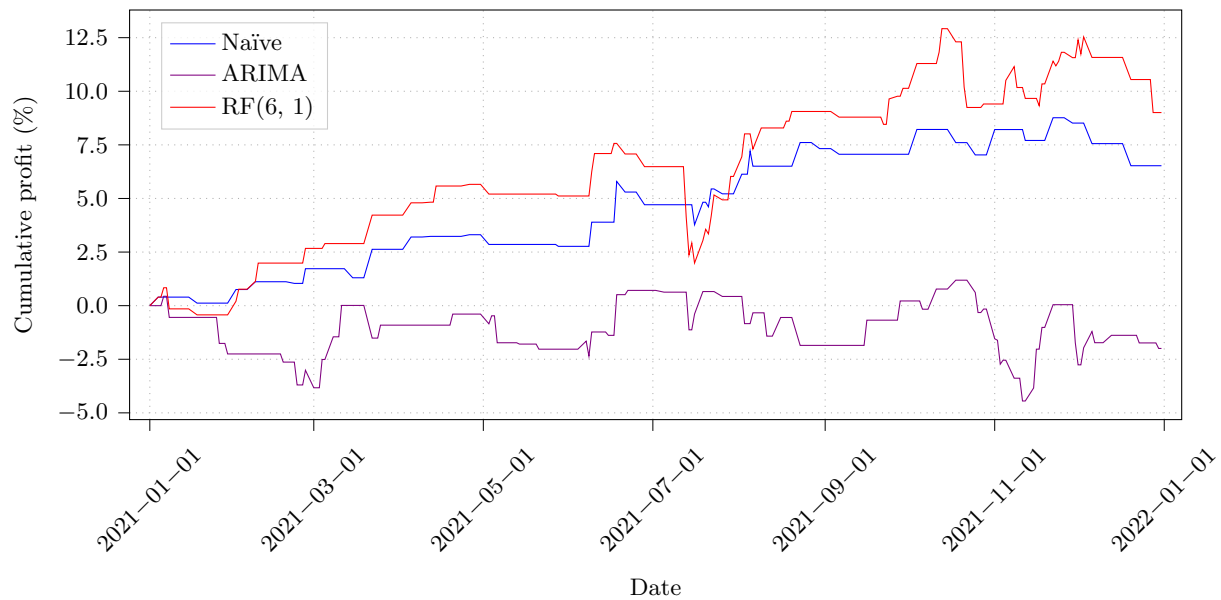


FIGURE B.45: The **RF**(12, 4) model's (most profitable **RF** model) cumulative profit for the **2021** test set compared to the naïve and Arima models.

B.2.5 Test set 2018–2021

The results obtained for the 2018–2021 testing period of the RF model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.46(a) and Figure B.46(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.47(a), Figure B.47(b), Figure B.47(c), and Figure B.47(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.48(a) and Figure B.48(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.49(a), Figure B.49(b) and Figure B.49(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.10 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the entire testing period 2018–2021.
- Figure B.50 shows the cumulative profit of the most profitable RF model, RF(12, 3), compared to the naïve and ARIMA models.

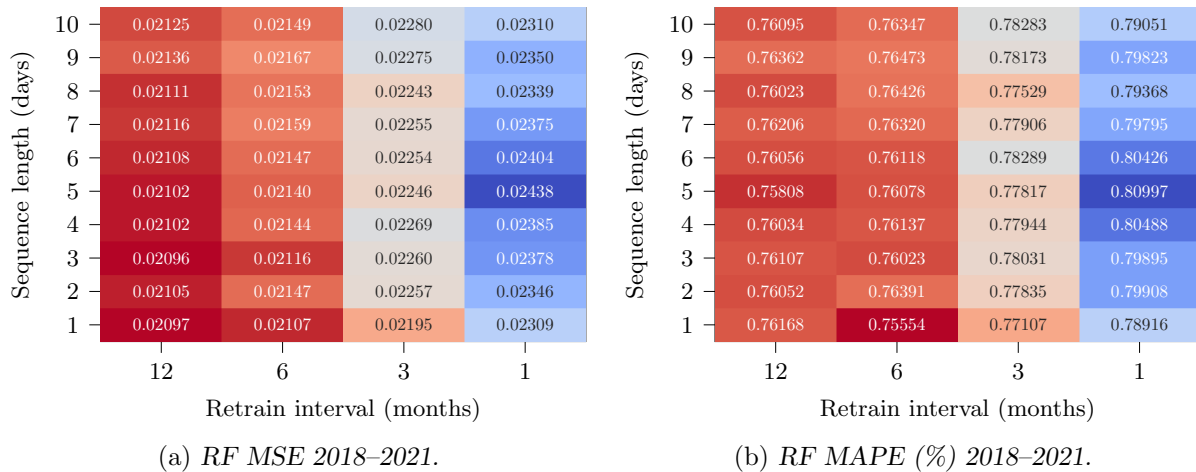


FIGURE B.46: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's MSE (left) and MAPE (right) for the **2018–2021** test set.

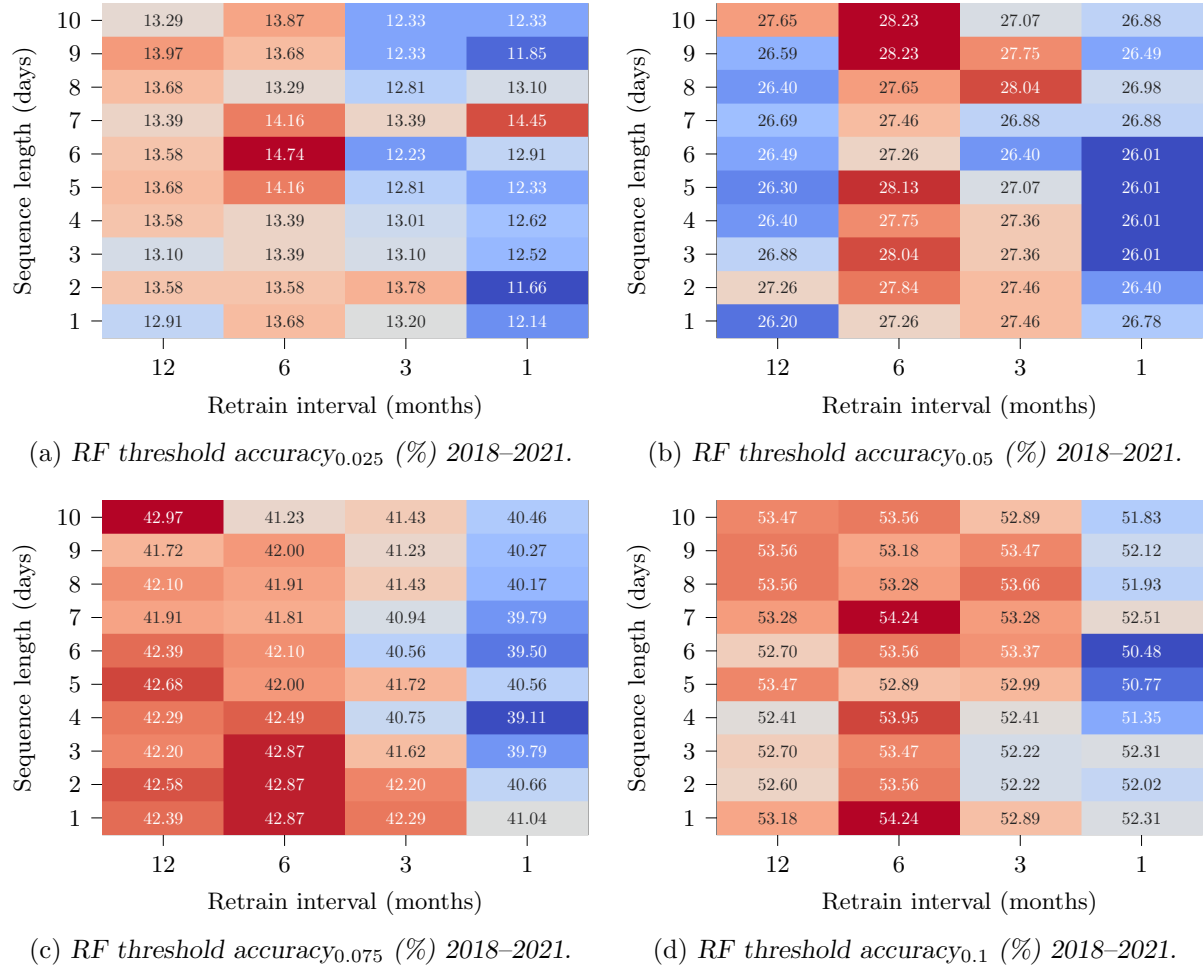


FIGURE B.47: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's different threshold accuracies for the **2018–2021** test set.

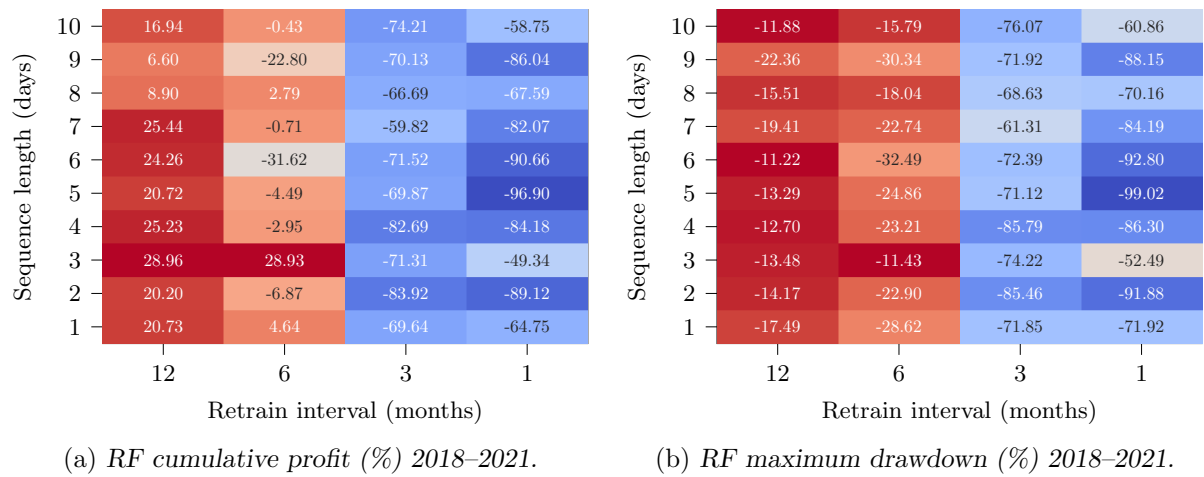


FIGURE B.48: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's cumulative profit (left) and maximum drawdown (right) for the **2018–2021** test set.

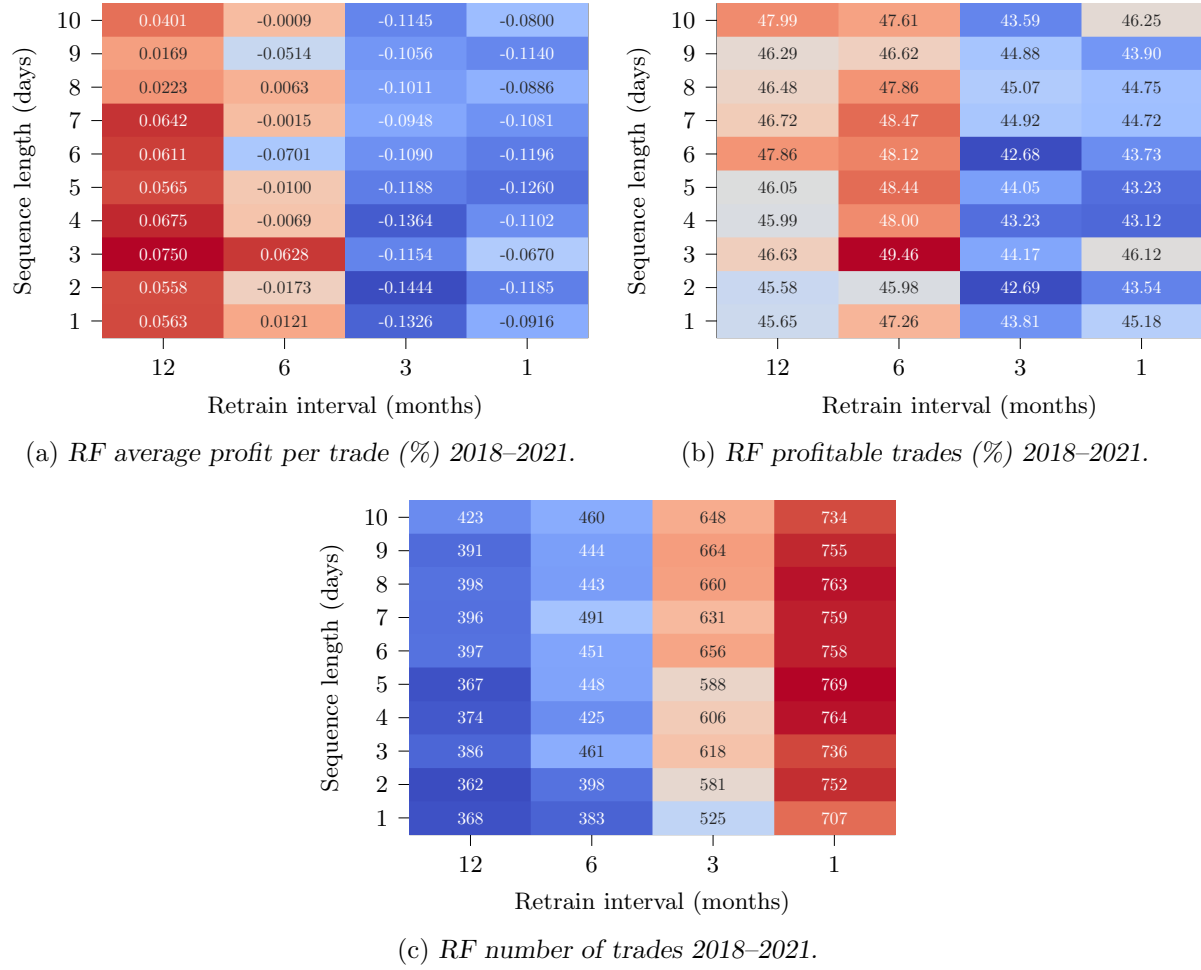


FIGURE B.49: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **RF** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018–2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.0211	0.76091	19.8	−15.15	46.52	386
	6	0.02143	0.76187	−3.35	−23.04	47.78	440
	3	0.02253	0.77891	−71.98	−73.88	43.91	618
	1	0.02363	0.79867	−76.94	−79.78	44.45	750
Seq length	1	0.02177	0.76936	−27.25	−47.47	45.48	496
	2	0.02214	0.77546	−39.93	−53.6	44.45	523
	3	0.02212	0.77514	−15.69	−37.9	46.6	550
	4	0.02225	0.77651	−36.15	−52.0	45.09	542
	5	0.02232	0.77675	−37.64	−52.07	45.44	543
	6	0.02228	0.77722	−42.39	−52.23	45.6	566
	7	0.02226	0.77557	−29.29	−46.91	46.21	569
	8	0.02211	0.77337	−30.65	−43.09	46.04	566
	9	0.02232	0.77708	−43.09	−53.2	45.42	564
	10	0.02216	0.77444	−29.11	−41.15	46.36	566

TABLE B.10: The **RF** model's aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

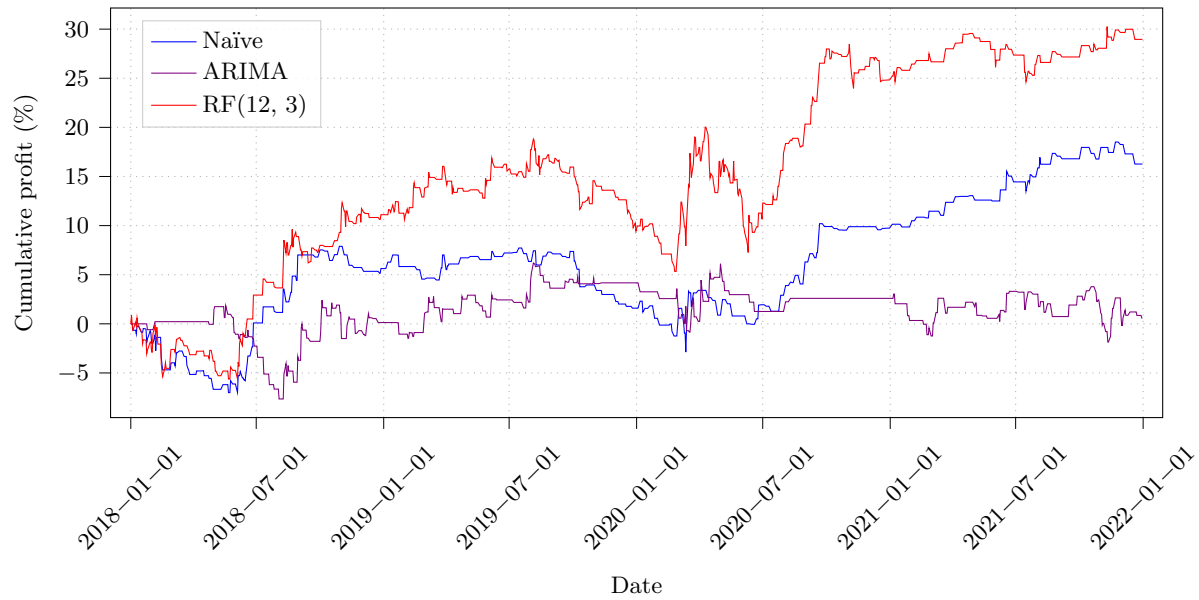


FIGURE B.50: The **RF(12, 3)** model's (most profitable **RF** model) cumulative profit for the **2018–2021** test set compared to the naïve and ARIMA models.

B.3 SVM

This section contains the SVM model results, with a breakdown by testing year and a summary of the entire testing period.

B.3.1 Test set 2018

The results obtained for the 2018 testing period of the SVM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.51(a) and Figure B.51(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.52(a), Figure B.52(b), Figure B.52(c), and Figure B.52(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.53(a) and Figure B.53(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.54(a), Figure B.54(b) and Figure B.54(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.11 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2018 testing period.
- Figure B.55 shows the cumulative profit of the most profitable SVM model, SVM(3, 4), compared to the naïve and ARIMA models.

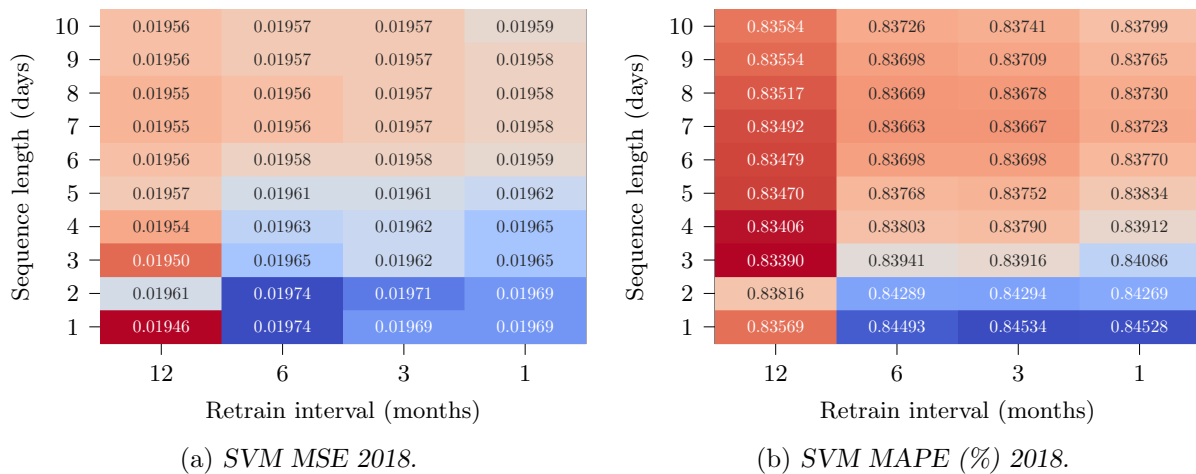


FIGURE B.51: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's MSE (left) and MAPE (right) for the **2018** test set.

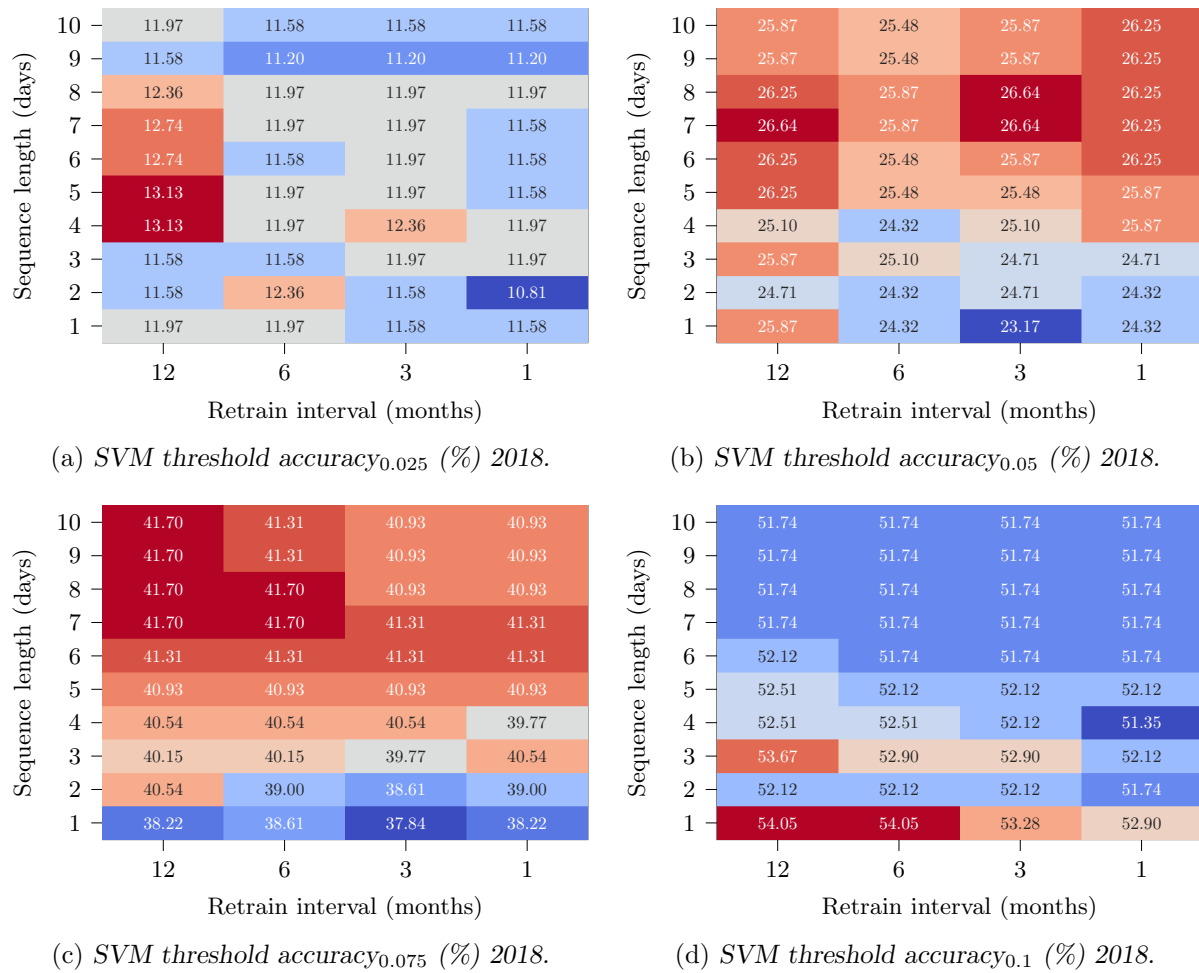


FIGURE B.52: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's different threshold accuracies for the **2018** test set.

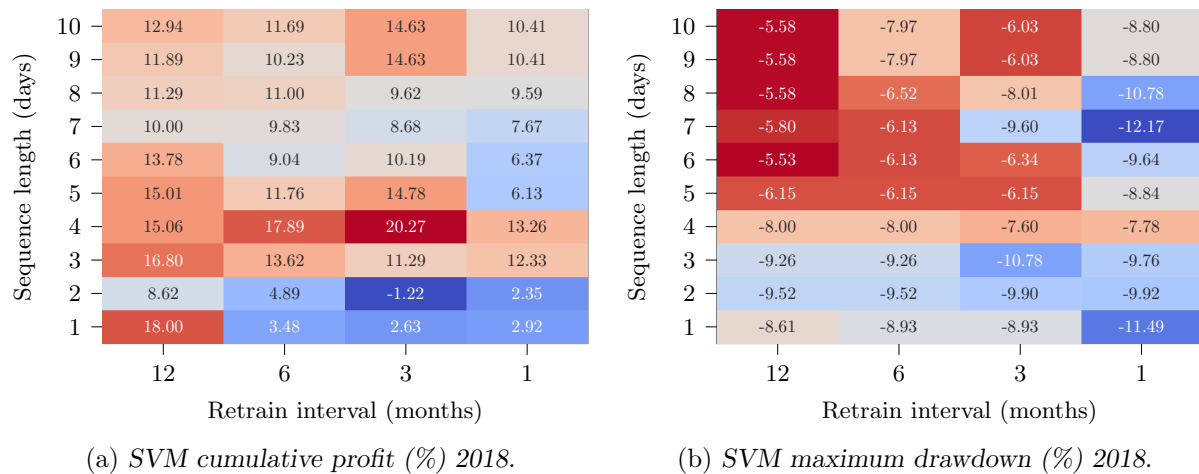


FIGURE B.53: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's cumulative profit (left) and maximum drawdown (right) for the **2018** test set.

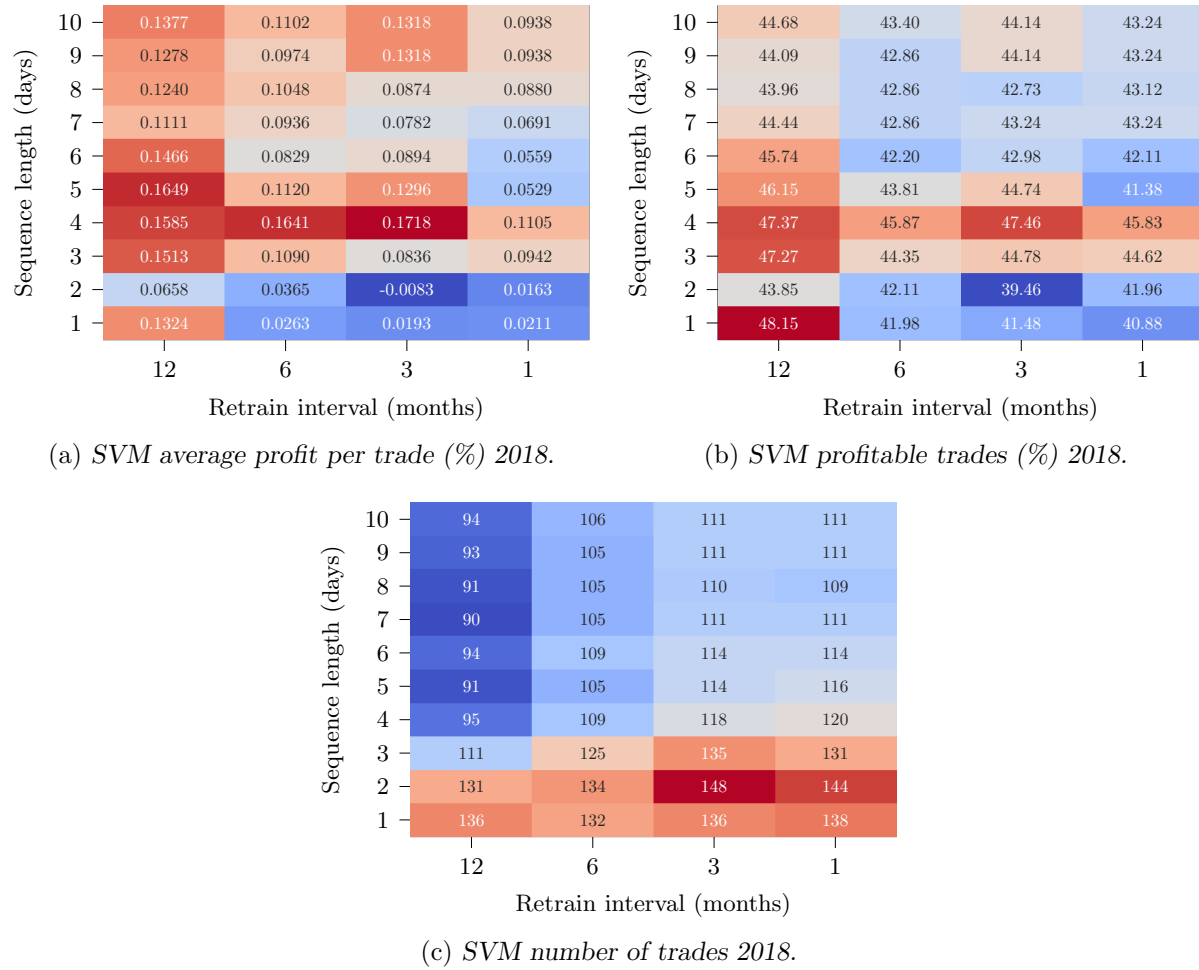


FIGURE B.54: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01955	0.83528	13.34	-6.96	45.57	103
	6	0.01962	0.83875	10.34	-7.66	43.23	114
	3	0.01961	0.83878	10.55	-7.94	43.51	121
	1	0.01962	0.83942	8.15	-9.8	42.96	120
Seq length	1	0.01964	0.84281	6.76	-9.49	43.12	136
	2	0.01969	0.84167	3.66	-9.72	41.84	139
	3	0.0196	0.83833	13.51	-9.77	45.25	126
	4	0.01961	0.83728	16.62	-7.84	46.63	110
	5	0.0196	0.83706	11.92	-6.83	44.02	106
	6	0.01958	0.83661	9.85	-6.91	43.26	108
	7	0.01957	0.83636	9.05	-8.42	43.45	104
	8	0.01957	0.83648	10.37	-7.72	43.16	104
	9	0.01957	0.83682	11.79	-7.1	43.58	105
	10	0.01957	0.83712	12.42	-7.1	43.87	106

TABLE B.11: The **SVM** model's aggregated results for each tested parameter value on selected performance metrics for the **2018** test set.

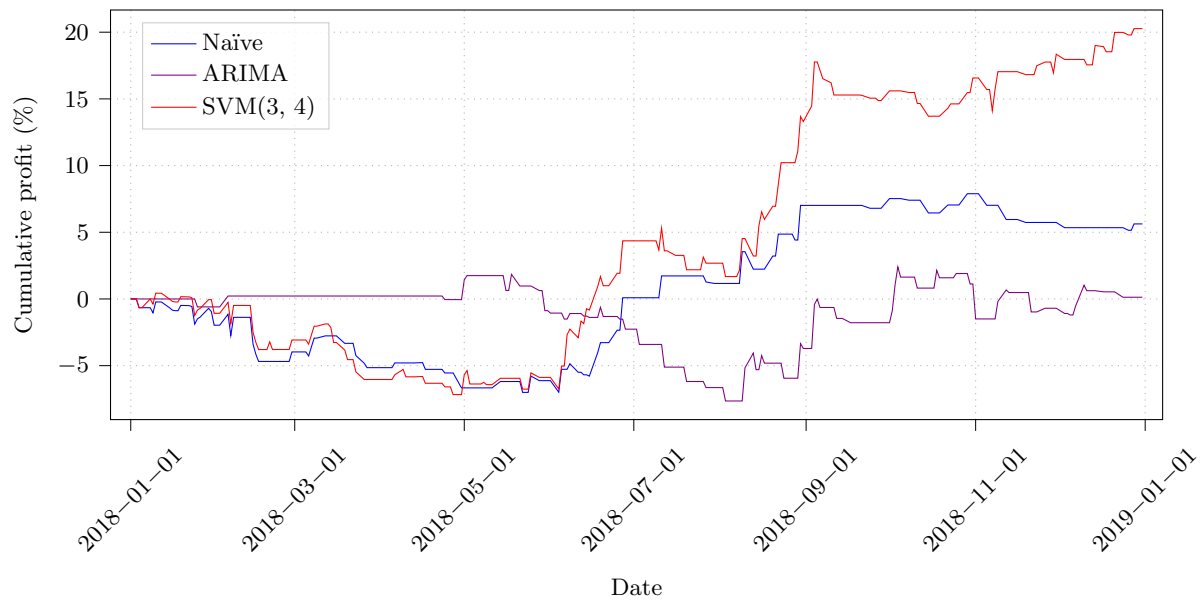


FIGURE B.55: The **SVM(3, 4)** model's (most profitable **SVM** model) cumulative profit for the **2018** test set compared to the naïve and ARIMA models.

B.3.2 Test set 2019

The results obtained for the 2019 testing period of the SVM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.56(a) and Figure B.56(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.57(a), Figure B.57(b), Figure B.57(c), and Figure B.57(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.58(a) and Figure B.58(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.59(a), Figure B.59(b) and Figure B.59(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.12 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2019 testing period.
- Figure B.60 shows the cumulative profit of the most profitable SVM model, SVM(6, 9), compared to the naïve and ARIMA models.

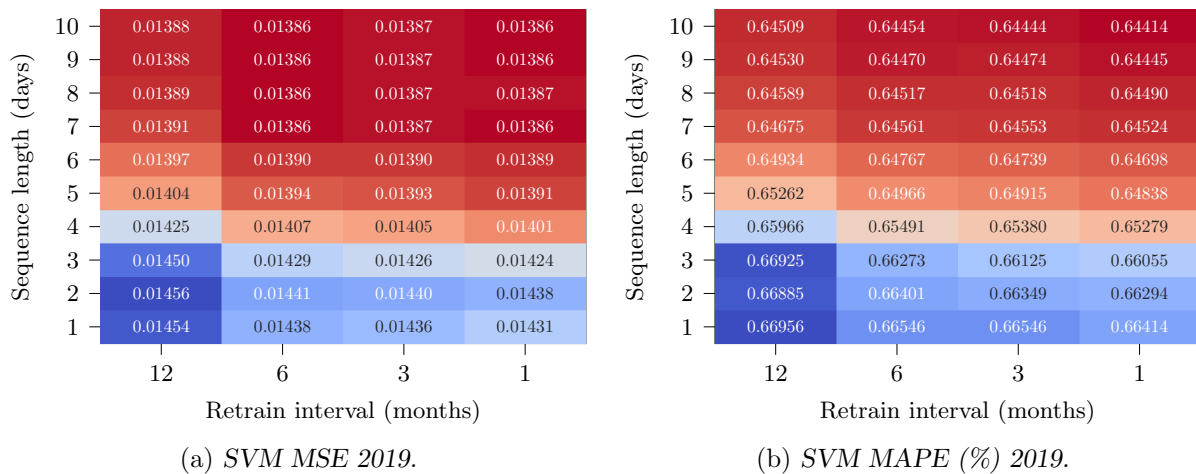


FIGURE B.56: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's MSE (left) and MAPE (right) for the **2019** test set.

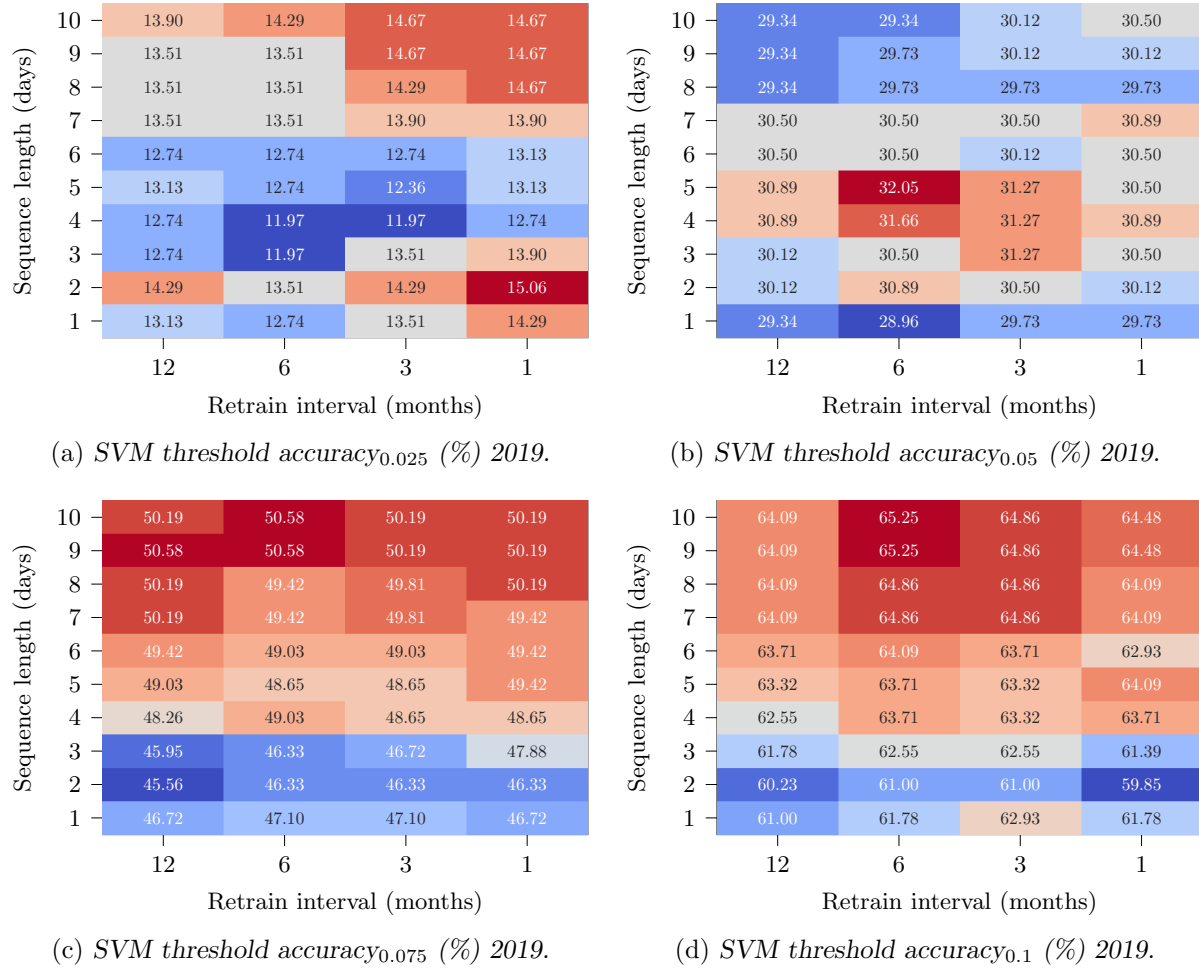


FIGURE B.57: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's different threshold accuracies for the **2019** test set.

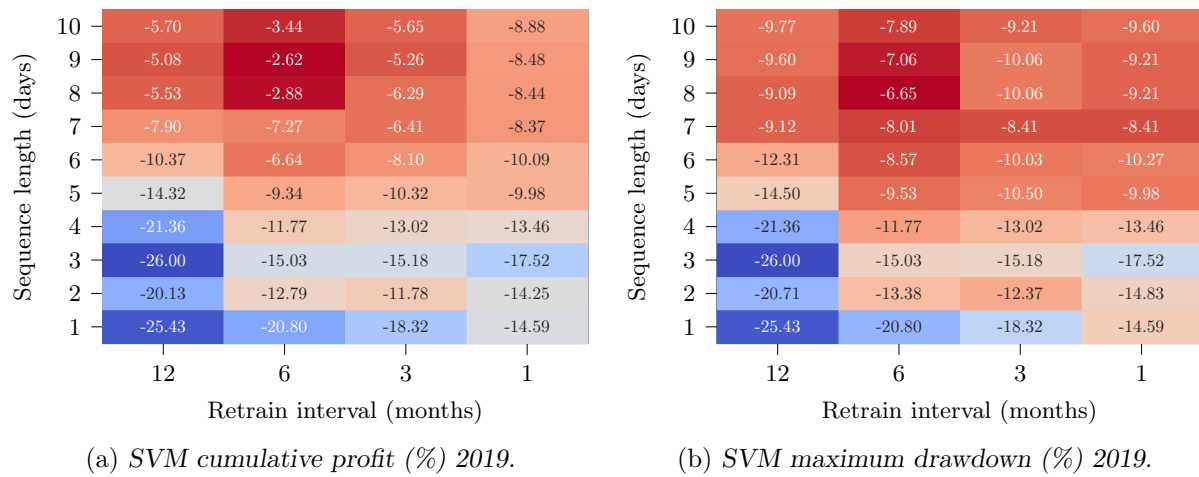


FIGURE B.58: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's cumulative profit (left) and maximum drawdown (right) for the **2019** test set.

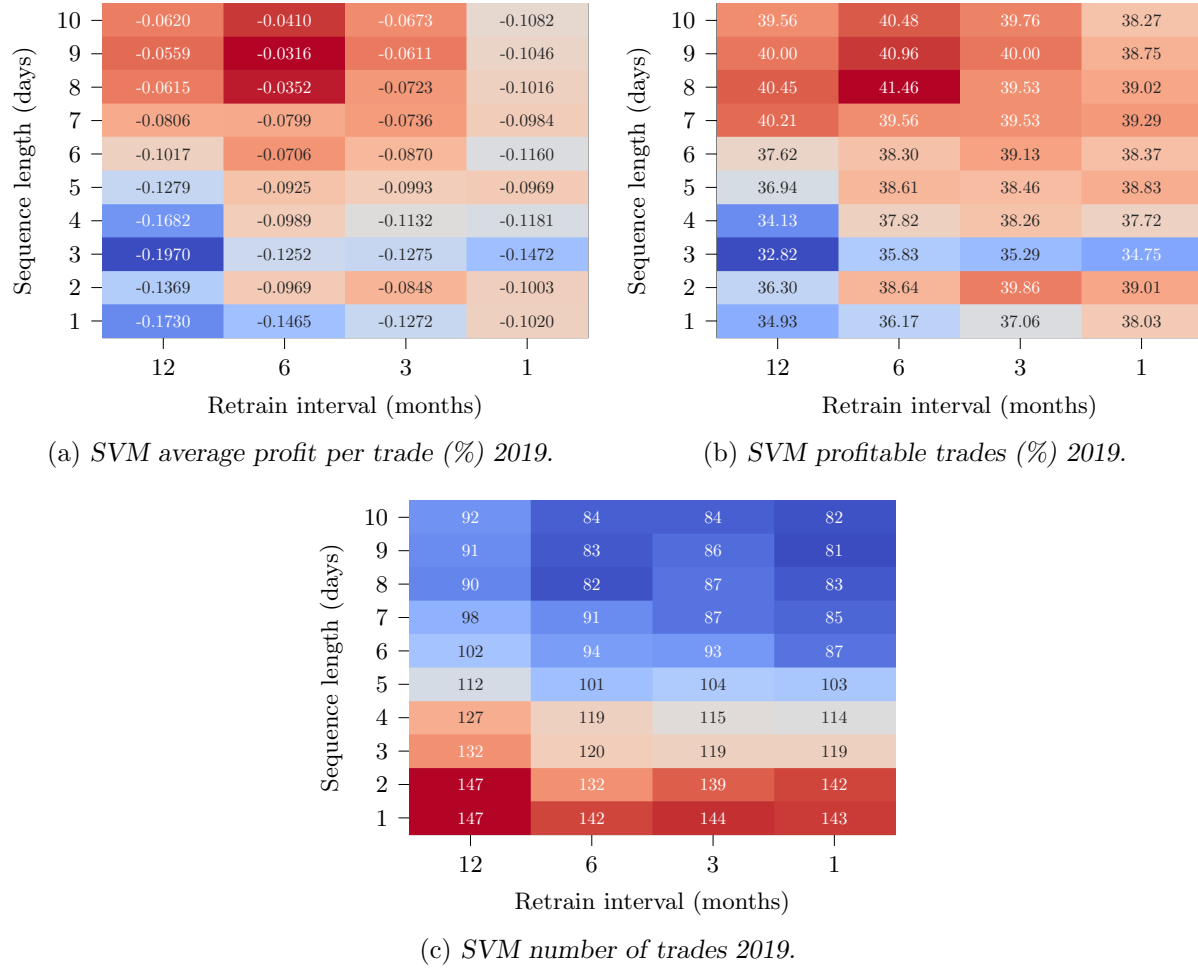


FIGURE B.59: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2019** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01414	0.65523	-14.18	-15.79	37.3	114
	6	0.01404	0.65245	-9.26	-10.87	38.78	105
	3	0.01404	0.65204	-10.03	-11.72	38.69	106
	1	0.01402	0.65145	-11.4	-11.71	38.2	104
Seq length	1	0.0144	0.66616	-19.78	-19.78	36.55	144
	2	0.01444	0.66482	-14.74	-15.32	38.45	140
	3	0.01432	0.66344	-18.43	-18.43	34.67	122
	4	0.0141	0.65529	-14.9	-14.9	36.98	119
	5	0.01395	0.64995	-10.99	-11.13	38.21	105
	6	0.01392	0.64784	-8.8	-10.29	38.36	94
	7	0.01388	0.64578	-7.49	-8.49	39.65	90
	8	0.01387	0.64528	-5.79	-8.75	40.12	86
	9	0.01387	0.6448	-5.36	-8.98	39.93	85
	10	0.01387	0.64455	-5.92	-9.12	39.52	86

TABLE B.12: The **SVM** model's aggregated results for each tested parameter value on selected performance metrics for the **2019** test set.

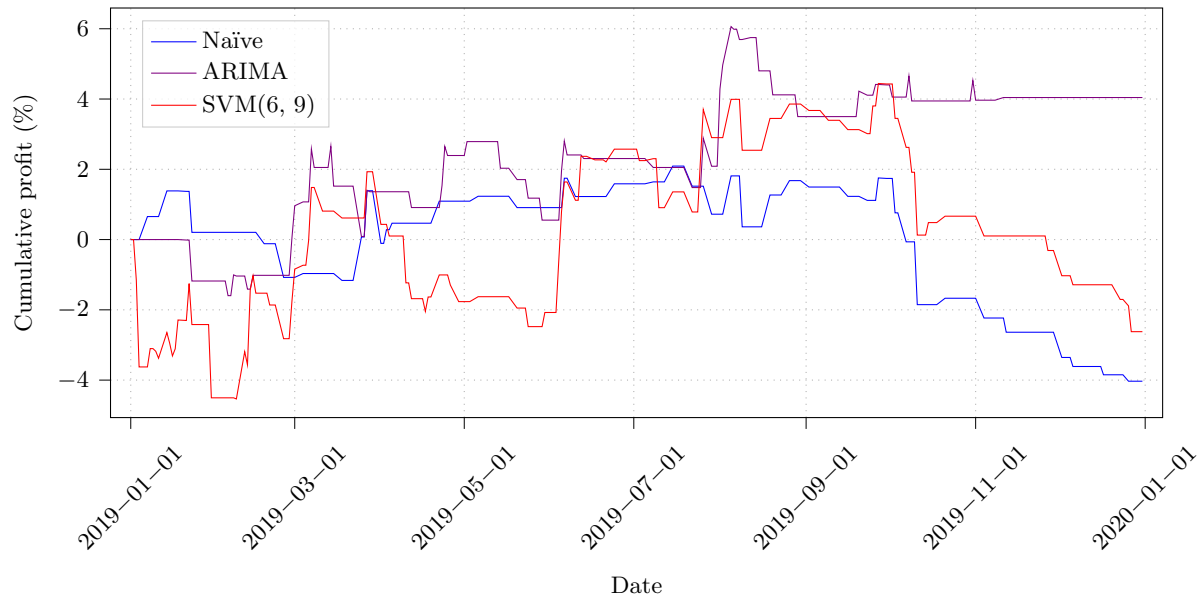


FIGURE B.60: The **SVM**(6, 9) model's (most profitable **SVM** model) cumulative profit for the **2019** test set compared to the naïve and ARIMA models.

B.3.3 Test set 2020

The results obtained for the 2020 testing period of the SVM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.61(a) and Figure B.61(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.62(a), Figure B.62(b), Figure B.62(c), and Figure B.62(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.63(a) and Figure B.63(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.64(a), Figure B.64(b) and Figure B.64(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.13 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2020 testing period.
- Figure B.65 shows the cumulative profit of the most profitable SVM model, SVM(3, 1), compared to the naïve and ARIMA models.

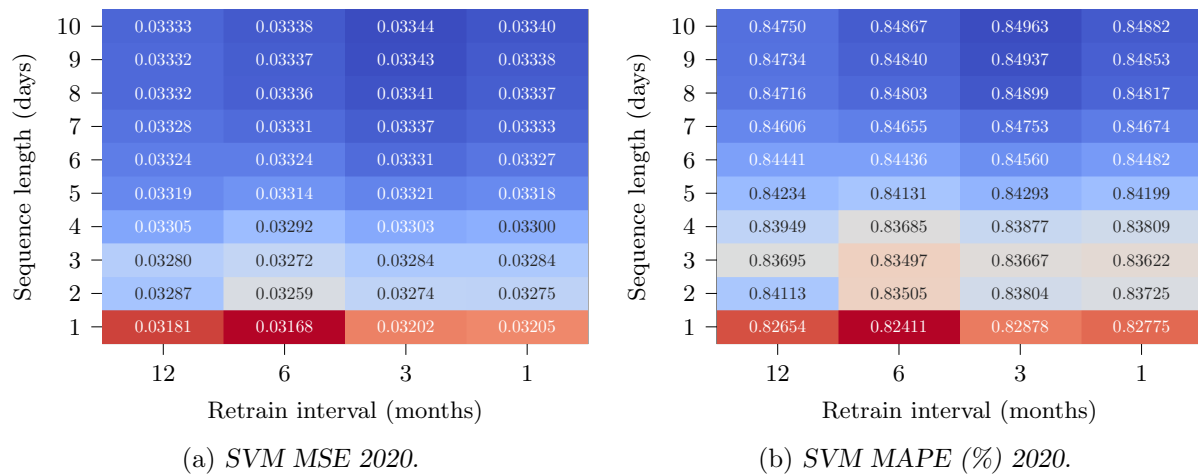


FIGURE B.61: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's MSE (left) and MAPE (right) for the **2020** test set.

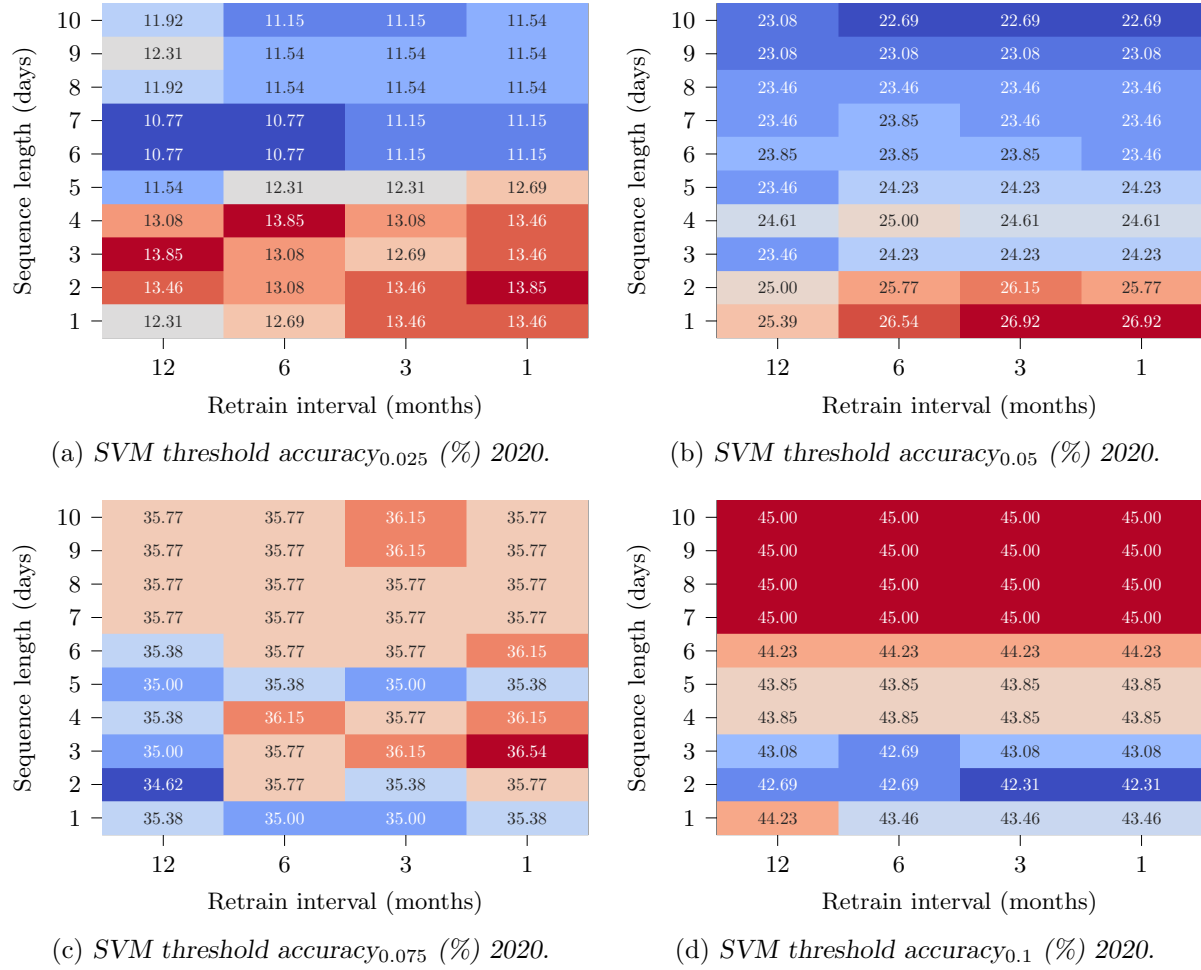


FIGURE B.62: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's different threshold accuracies for the **2020** test set.

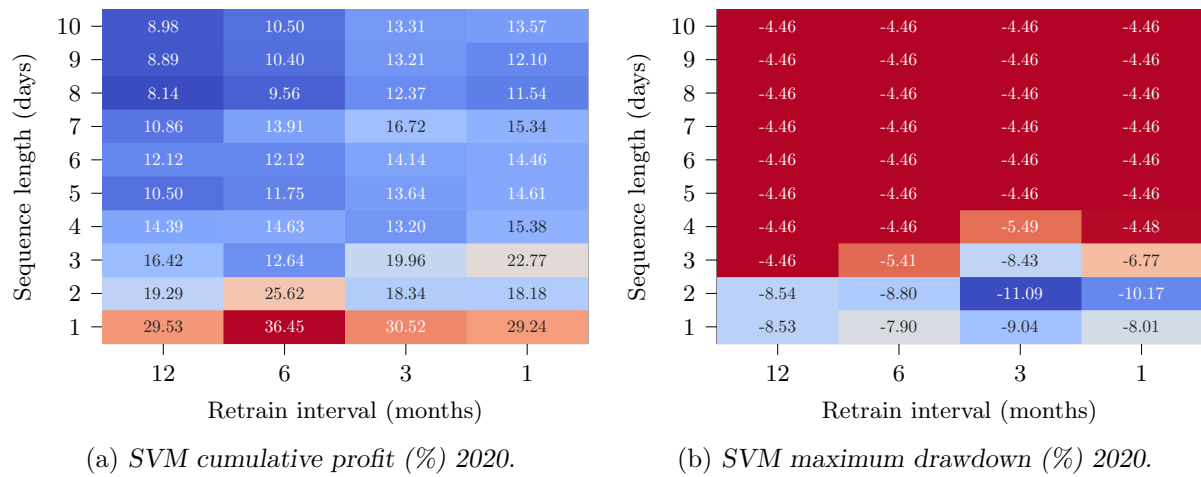


FIGURE B.63: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's cumulative profit (left) and maximum drawdown (right) for the **2020** test set.

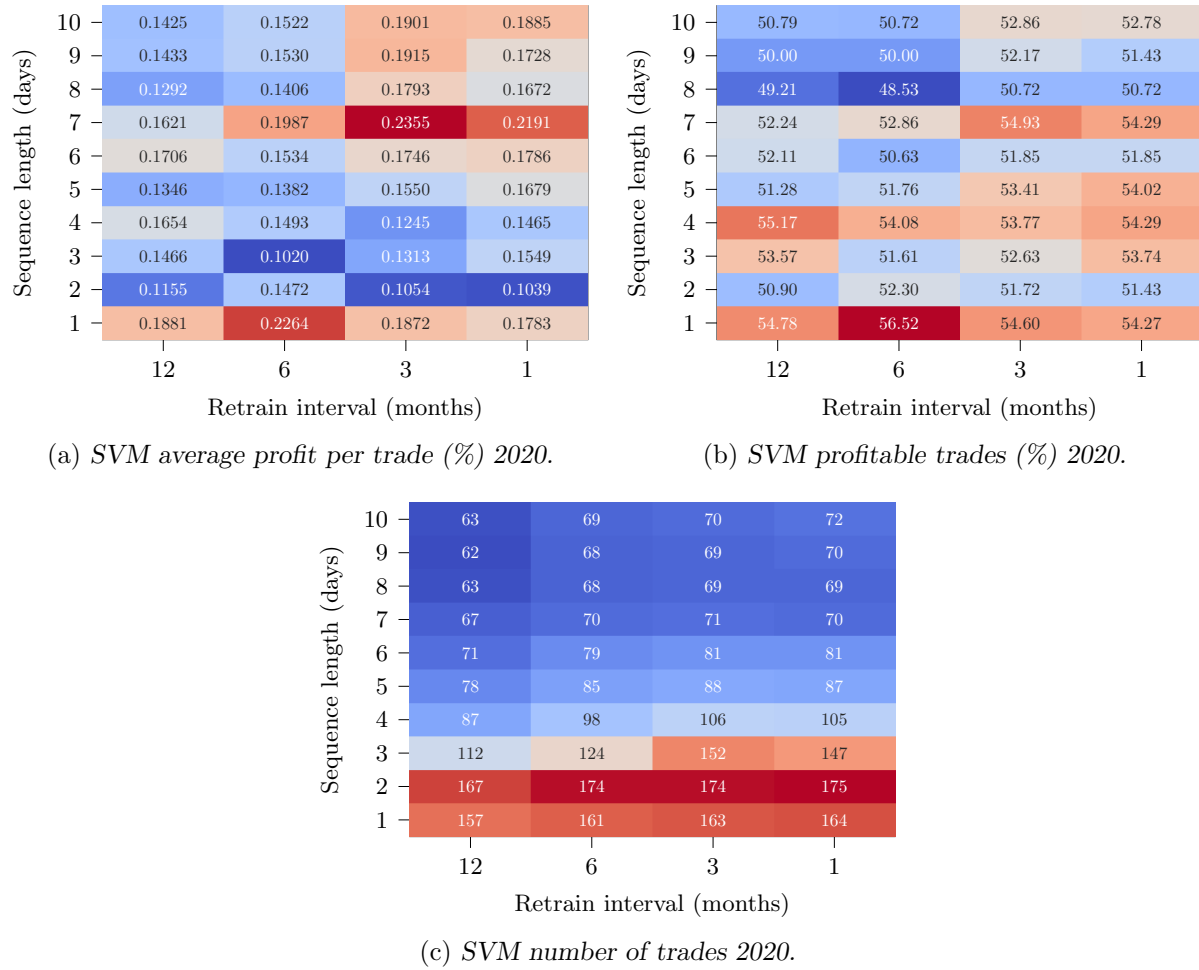


FIGURE B.64: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2020** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.03302	0.84189	13.91	-5.27	52.01	93
	6	0.03297	0.84083	15.76	-5.33	51.9	100
	3	0.03308	0.84263	16.54	-6.08	52.87	104
	1	0.03306	0.84184	16.72	-5.62	52.88	104
Seq length	1	0.03189	0.8268	31.43	-8.37	55.04	161
	2	0.03274	0.83787	20.36	-9.65	51.59	172
	3	0.0328	0.8362	17.95	-6.27	52.89	134
	4	0.033	0.8383	14.4	-4.72	54.33	99
	5	0.03318	0.84214	12.63	-4.46	52.62	84
	6	0.03326	0.8448	13.21	-4.46	51.61	78
	7	0.03332	0.84672	14.21	-4.46	53.58	70
	8	0.03337	0.84809	10.4	-4.46	49.8	67
	9	0.03338	0.84841	11.15	-4.46	50.9	67
	10	0.03339	0.84866	11.59	-4.46	51.79	68

TABLE B.13: The **SVM** model's aggregated results for each tested parameter value on selected performance metrics for the **2020** test set.

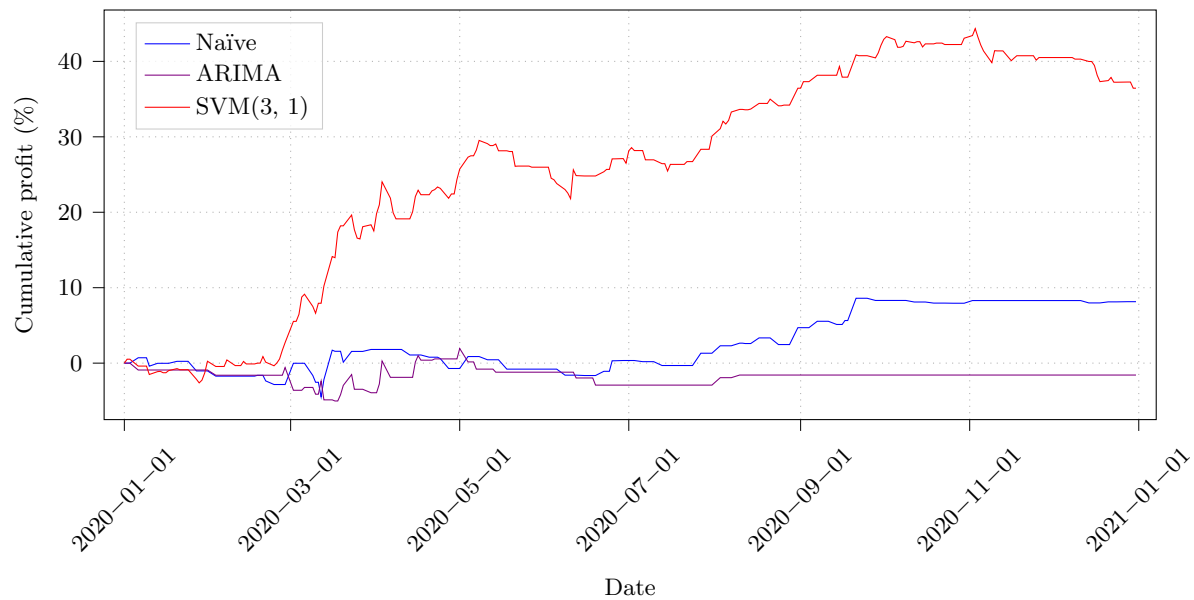


FIGURE B.65: The **SVM(3, 1)** model's (most profitable **SVM** model) cumulative profit for the **2020** test set compared to the naïve and ARIMA models.

B.3.4 Test set 2021

The results obtained for the 2021 testing period of the SVM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.66(a) and Figure B.66(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.67(a), Figure B.67(b), Figure B.67(c), and Figure B.67(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.68(a) and Figure B.68(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.69(a), Figure B.69(b) and Figure B.69(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.14 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the 2021 testing period.
- Figure B.70 shows the cumulative profit of the most profitable SVM model, SVM(3, 1), compared to the naïve and ARIMA models.

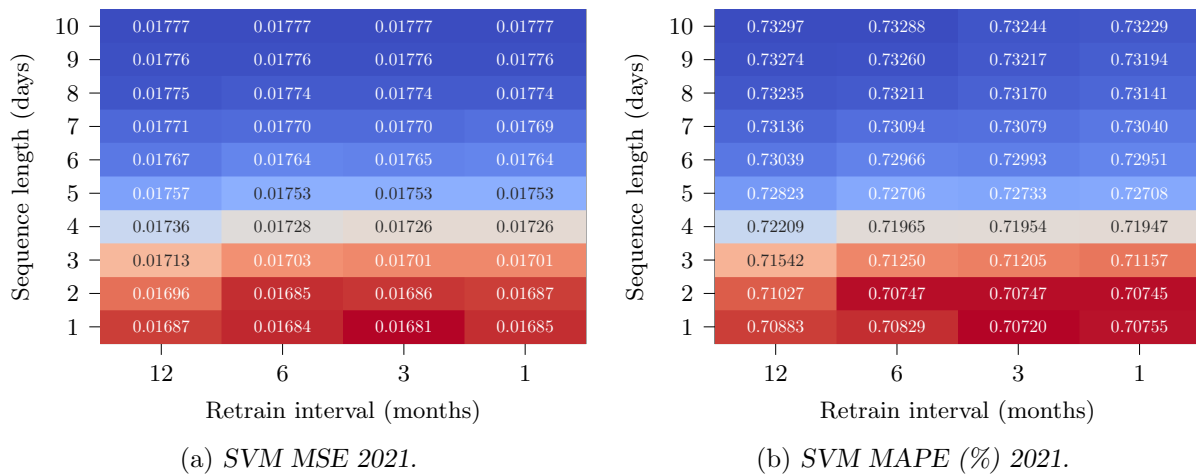


FIGURE B.66: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's MSE (left) and MAPE (right) for the **2021** test set.

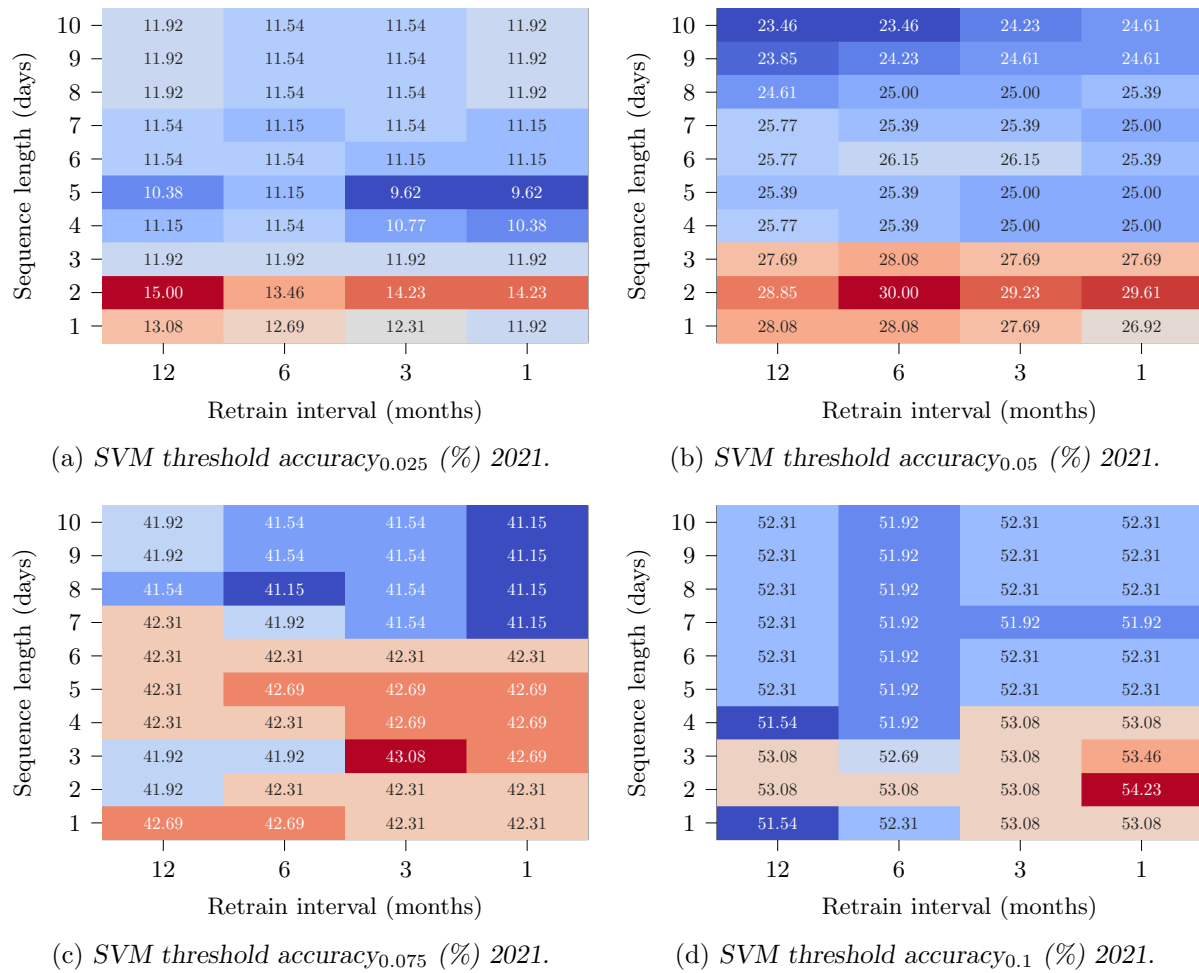


FIGURE B.67: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's different threshold accuracies for the **2021** test set.

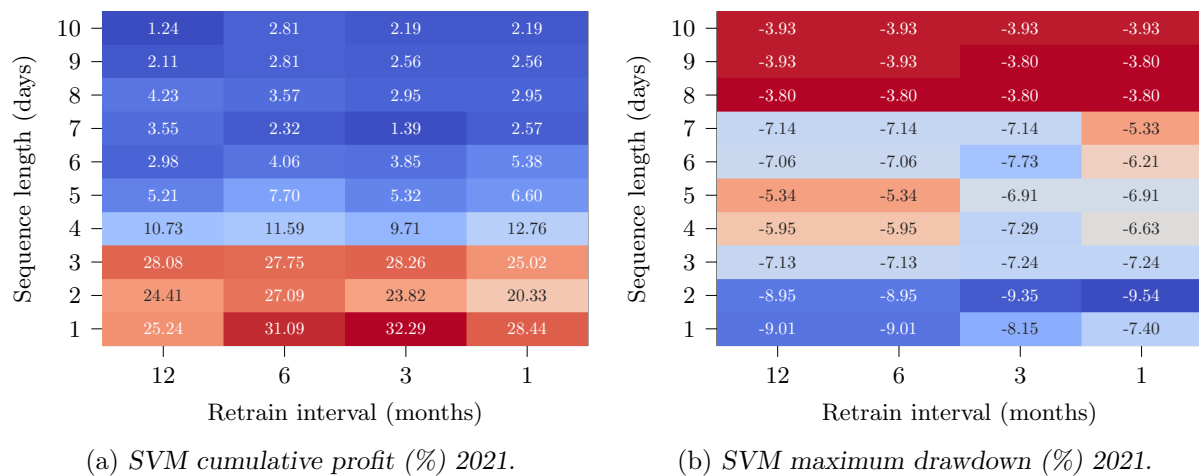


FIGURE B.68: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's cumulative profit (left) and maximum drawdown (right) for the **2021** test set.

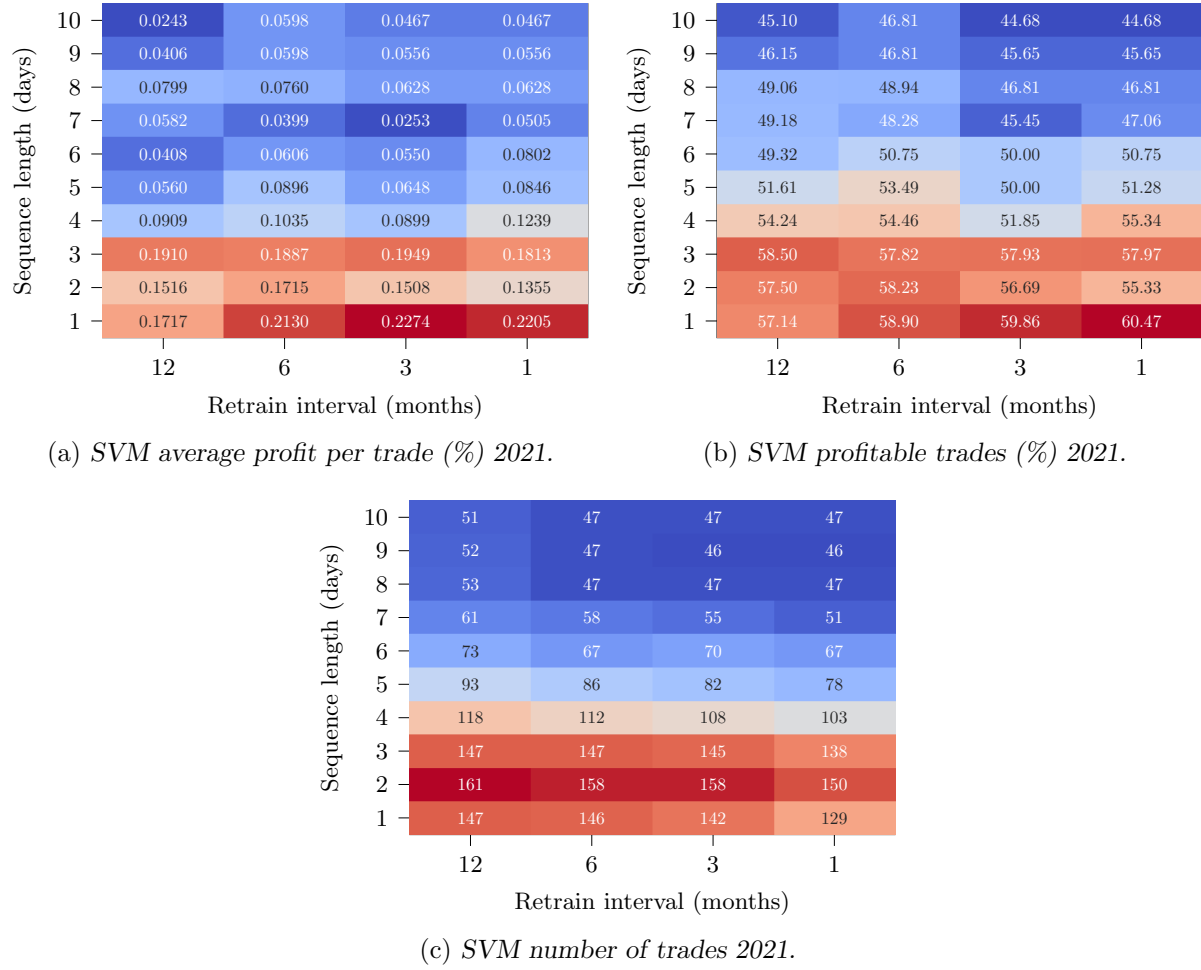


FIGURE B.69: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.01746	0.72446	10.78	-6.22	51.78	96
	6	0.01741	0.72332	12.08	-6.22	52.45	92
	3	0.01741	0.72306	11.23	-6.53	50.89	90
	1	0.01741	0.72287	10.88	-6.08	51.53	86
Seq length	1	0.01684	0.70797	29.27	-8.39	59.09	141
	2	0.01688	0.70816	23.91	-9.2	56.94	157
	3	0.01704	0.71288	27.28	-7.19	58.06	144
	4	0.01729	0.72019	11.2	-6.46	53.97	110
	5	0.01754	0.72742	6.21	-6.12	51.6	85
	6	0.01765	0.72987	4.07	-7.01	50.2	69
	7	0.0177	0.73087	2.46	-6.69	47.49	56
	8	0.01774	0.73189	3.43	-3.8	47.9	48
	9	0.01776	0.73236	2.51	-3.86	46.07	48
	10	0.01777	0.73264	2.11	-3.93	45.32	48

TABLE B.14: The **SVM** model's aggregated results for each tested parameter value on selected performance metrics for the **2021** test set.

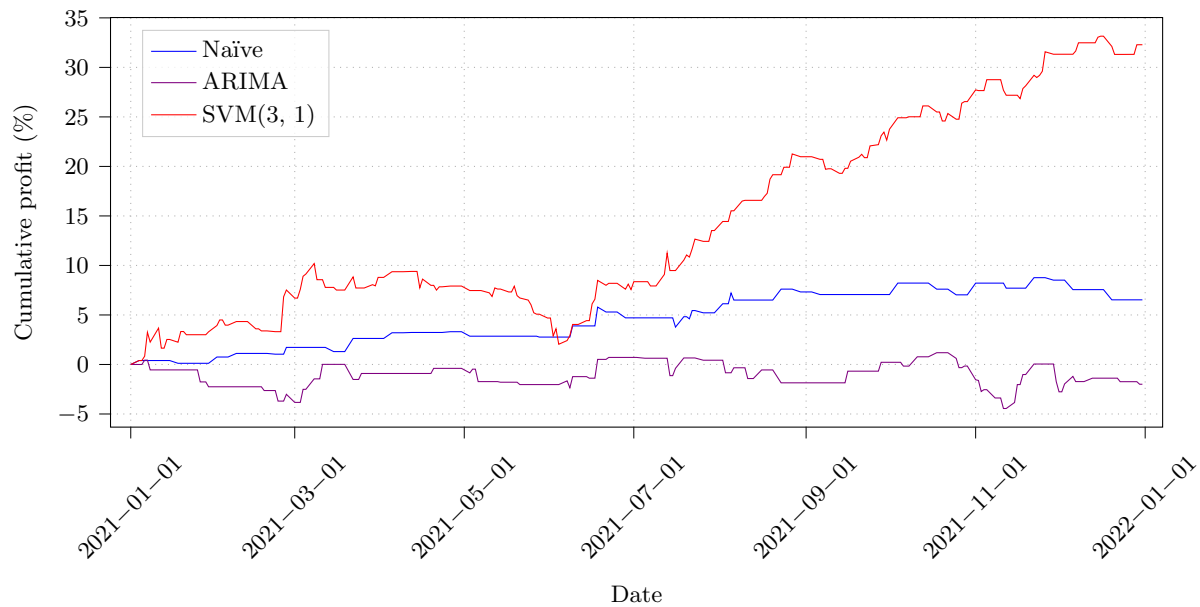


FIGURE B.70: The **SVM(3, 1)** model's (most profitable **SVM** model) cumulative profit for the **2021** test set compared to the naïve and Arima models.

B.3.5 Test set 2018–2021

The results obtained for the 2018–2021 testing period of the SVM model are given in this section. The figures and tables shown are summarised as follows:

- Figure B.71(a) and Figure B.71(b) shows the prediction accuracy metrics, MSE and MAPE, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.72(a), Figure B.72(b), Figure B.72(c), and Figure B.72(d) shows the prediction accuracy metric, threshold accuracy, for the different combinations of **retraining interval** and **sequence length**.
- Figure B.73(a) and Figure B.73(b) shows the cumulative profit and maximum drawdown for the different combinations of **retraining interval** and **sequence length**.
- Figure B.74(a), Figure B.74(b) and Figure B.74(c) shows the profit-related metrics, average profit per trade, profitable trades percentage and number of trades, for the different combinations of **retraining interval** and **sequence length**.
- Table B.15 contains the average accuracy and profit metrics of the different combinations of **retraining interval** and **sequence length** for the entire testing period 2018–2021.
- Figure B.75 shows the cumulative profit of the most profitable SVM model, SVM(6, 1), compared to the naïve and ARIMA models.

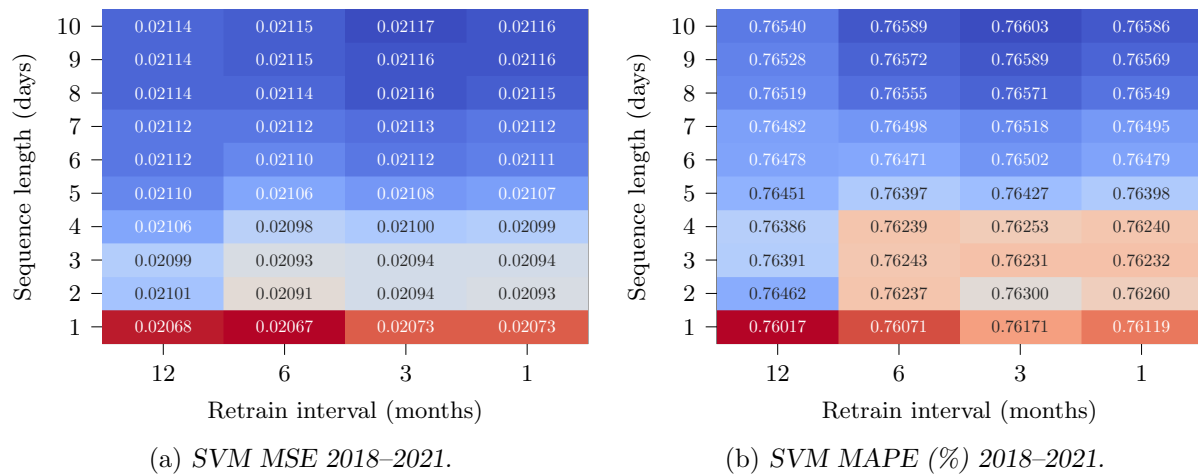


FIGURE B.71: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's MSE (left) and MAPE (right) for the **2018–2021** test set.

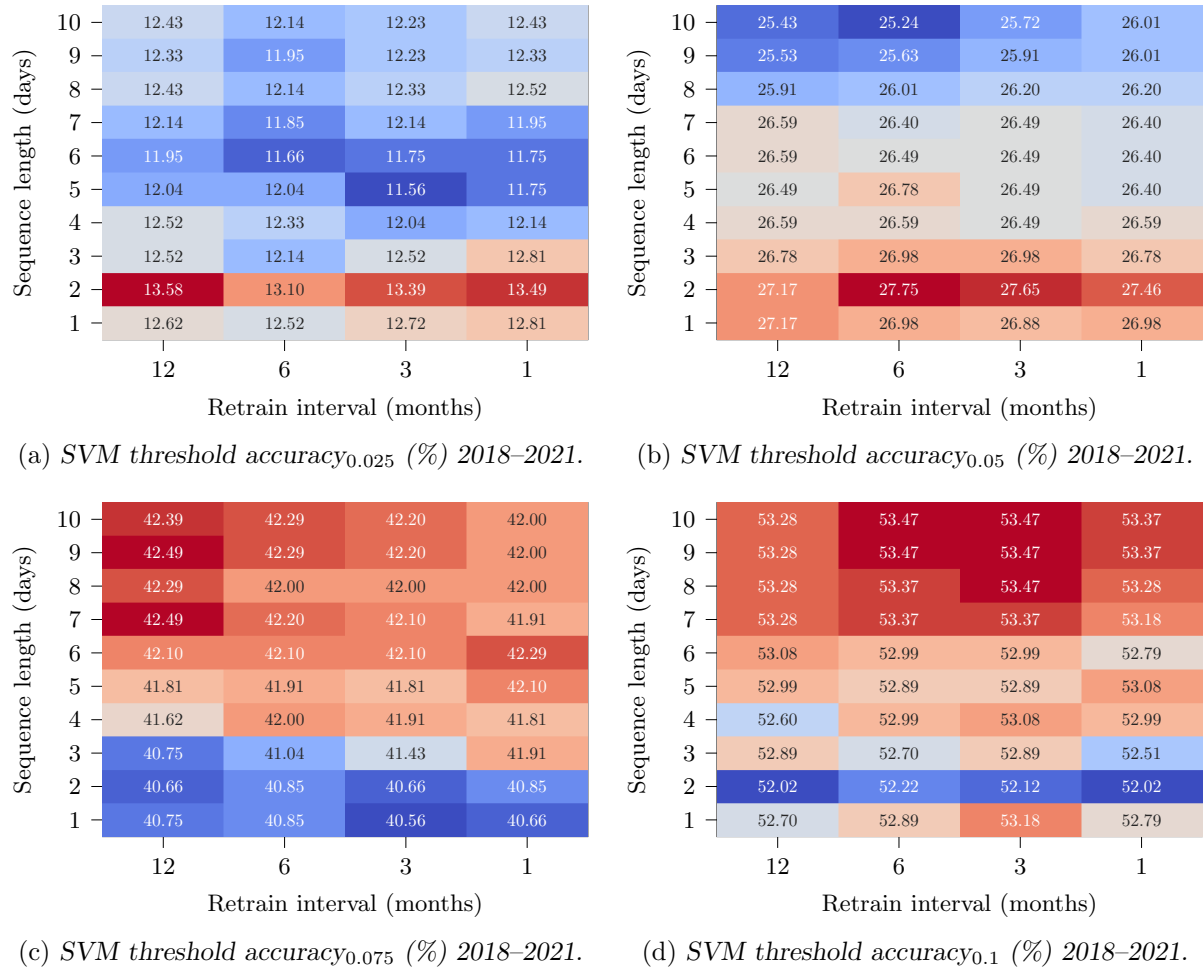


FIGURE B.72: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's different threshold accuracies for the **2018–2021** test set.

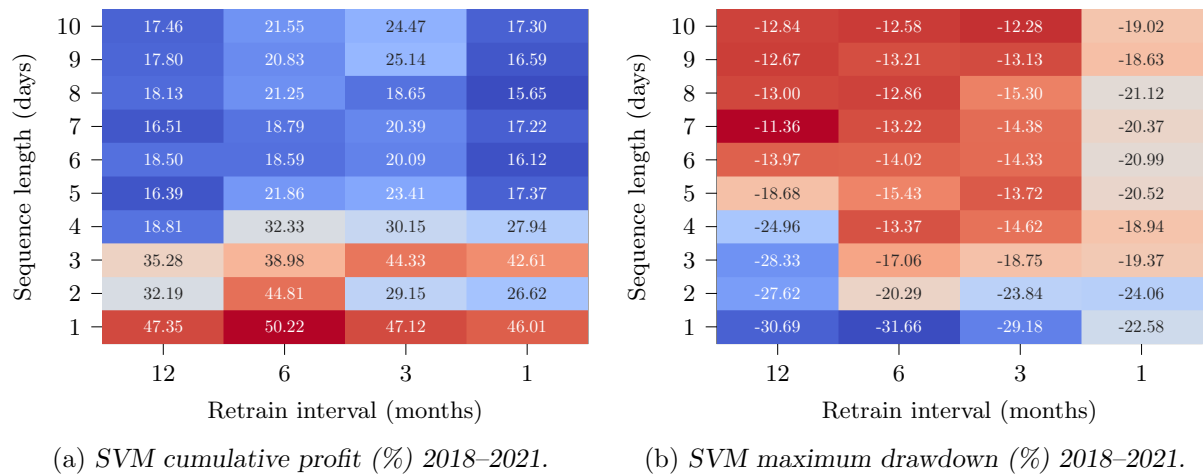


FIGURE B.73: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's cumulative profit (left) and maximum drawdown (right) for the **2018–2021** test set.

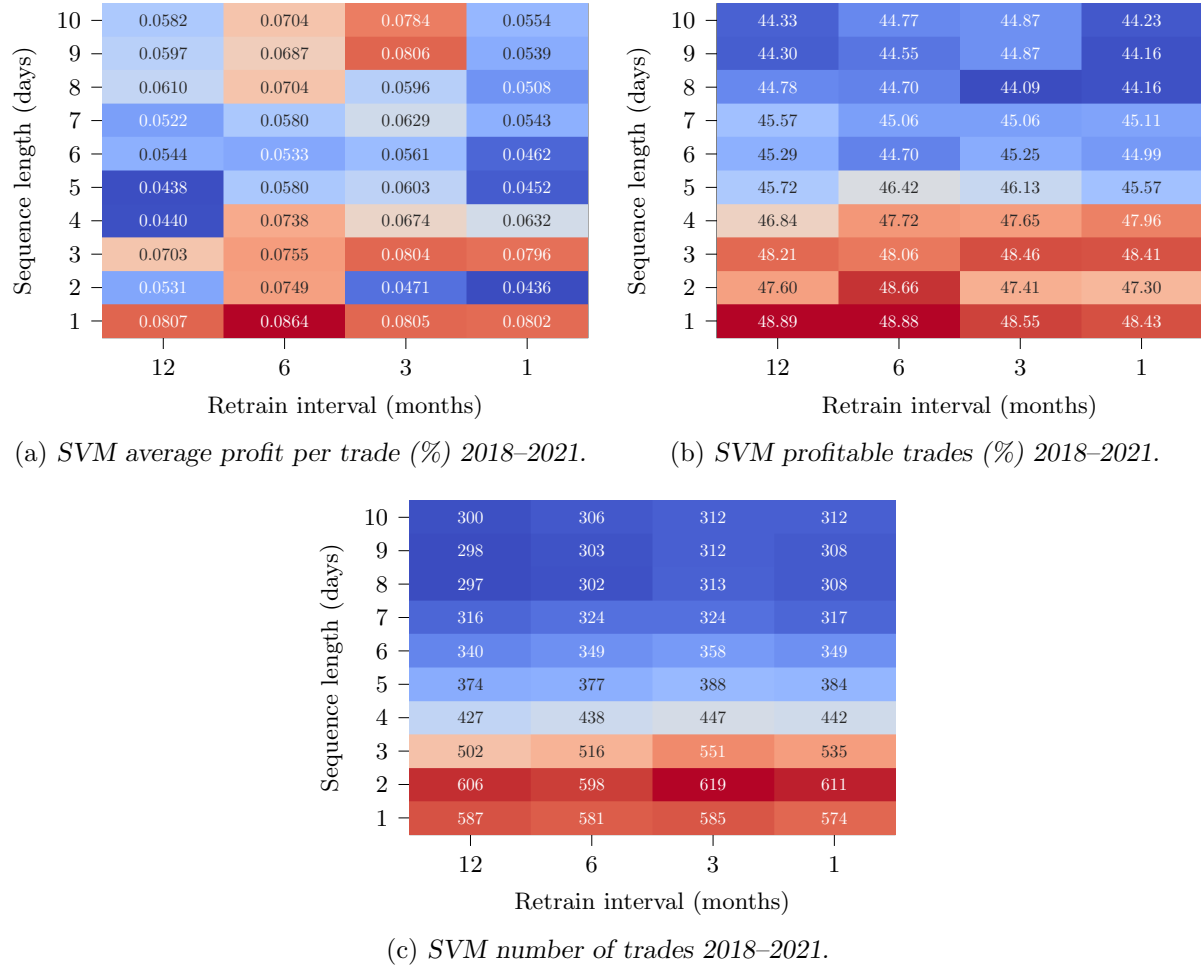


FIGURE B.74: The heatmaps displaying the effect of **retraining interval** and **sequence length** on the **SVM** model's average profit per trade (top left), number of profitable trades (top right), and total number of trades (bottom) for the **2018–2021** test set.

Parameter	Value	Average metric					
		MSE	MAPE	Profit (%)	Max draw (%)	Profit trades (%)	Num trades
Ret int	12	0.02105	0.76425	23.84	−19.41	46.15	405
	6	0.02102	0.76387	28.92	−16.37	46.35	409
	3	0.02104	0.76416	28.29	−16.95	46.23	421
	1	0.02104	0.76393	24.34	−20.56	46.03	414
Seq length	1	0.0207	0.76094	47.67	−28.53	48.69	582
	2	0.02095	0.76315	33.19	−23.95	47.74	608
	3	0.02095	0.76274	40.3	−20.88	48.28	526
	4	0.02101	0.7628	27.31	−17.98	47.54	438
	5	0.02108	0.76418	19.76	−17.09	45.96	381
	6	0.02111	0.76482	18.33	−15.83	45.06	349
	7	0.02112	0.76498	18.22	−14.83	45.2	320
	8	0.02115	0.76548	18.42	−15.57	44.43	305
	9	0.02115	0.76564	20.09	−14.41	44.47	305
	10	0.02115	0.7658	20.2	−14.18	44.55	308

TABLE B.15: The **SVM** model's aggregated results for each tested parameter value on selected performance metrics for the **2018–2021** test set.

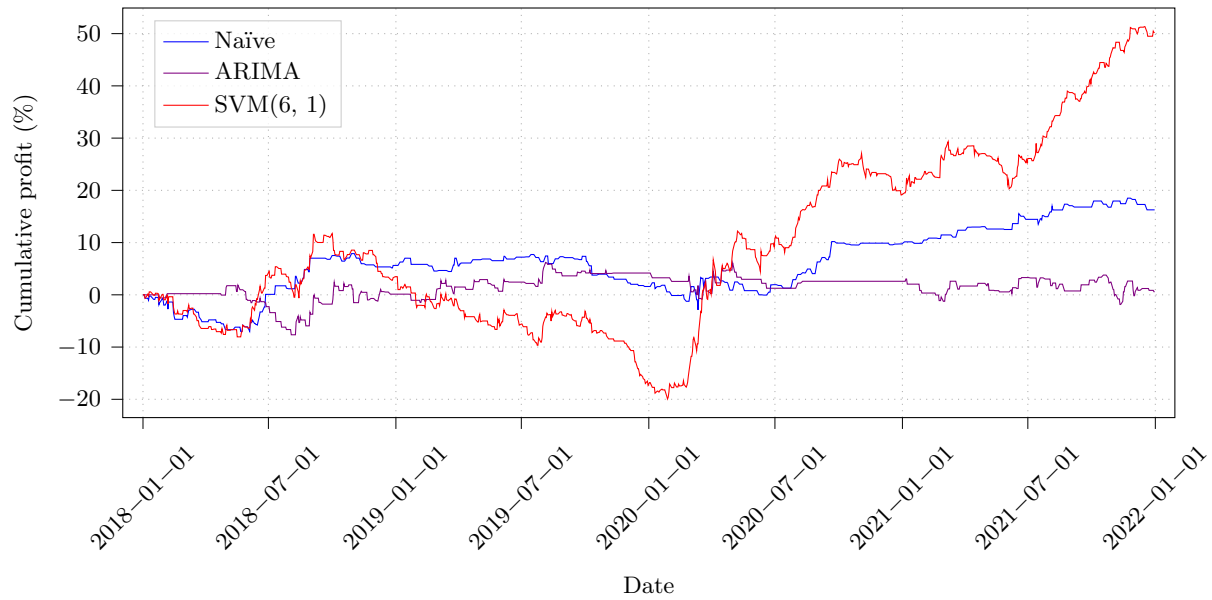


FIGURE B.75: The **SVM(6, 1)** model's (most profitable **SVM** model) cumulative profit for the **2018–2021** test set compared to the naïve and ARIMA models.

Bibliography

- [1] APPEL G, 2005, *Technical analysis: power tools for active investors*, FT Press, Upper Saddle River (NJ).
- [2] APPEL G and DOBSON E, 2007, *Understanding MACD*, Volume 34, Traders Press, Greenville (SC).
- [3] BASAK S, KAR S, SAHA S, KHAIDEM L and DEY SR, 2019, *Predicting the direction of stock market prices using tree-based classifiers*, The North American Journal of Economics and Finance, **47**(1), pp. 552–567.
- [4] BERGMEIR C and BENÍTEZ JM, 2012, *On the use of cross-validation for time series predictor evaluation*, Information Sciences, **191**(1), pp. 192–213.
- [5] BERGSTRA J and BENGIO Y, 2012, *Random search for hyper-parameter optimization.*, Journal of machine learning research, **13**(2), pp. 281–305.
- [6] BLAU W, 1991, *True strength index*, Technical analysis of stocks and commodities, **11**(1), pp. 438–446.
- [7] BLYTH S, 1994, *Karl pearson and the correlation curve*, International Statistical Review/Revue Internationale de Statistique, **62**(3), pp. 393–403.
- [8] BOLLINGER J, 2002, *Bollinger on Bollinger bands*, McGraw-Hill, New York (NY).
- [9] BOSER BE, GUYON IM and VAPNIK VN, 1992, *A training algorithm for optimal margin classifiers*, Proceedings of the 5th annual workshop on computational learning theory, Pittsburgh, Pennsylvania, United States, pp. 144–152.
- [10] BOX GE, JENKINS GM, REINSEL GC and LJUNG GM, 2015, *Time series analysis: forecasting and control*, John Wiley & Sons, Hoboken (NJ).
- [11] BREIMAN L, 2001, *Random forests*, Machine learning, **45**(1), pp. 5–32.
- [12] CHEN J, 2022, *Forex (FX): How Trading in the Foreign Exchange Market Works*, [Online], [Cited September 25th, 2023], Available from <https://www.investopedia.com/terms/f/foreign-exchange.asp>.
- [13] CHOLLET F *et al.*, 2015, *Keras*, [Online], [Cited November 26th, 2021], Available from <https://keras.io>.
- [14] COLBY RW, 2003, *The Encyclopedia of Technical Market Indicators*, 2nd Edition, McGraw-Hill, New York (NY).

- [15] DERBENTSEV V, MATVIYCHUK A, DATSENKO N, BEZKOROVAINYI V and AZARYAN A, 2020, *Machine learning approaches for financial time series forecasting*, Proceedings of the CEUR Workshop, Odessa, Ukraine, pp. 434–450.
- [16] DEVLIN J, CHANG MW, LEE K and TOUTANOVA K, 2018, *Bert: Pre-training of deep bidirectional transformers for language understanding*, Proceedings of the NAACL, Minneapolis, Minnesota, pp. 4171–4186.
- [17] DEVOS O, RUCKEBUSCH C, DURAND A, DUPONCHEL L and HUVENNE JP, 2009, *Support vector machines (svm) in near-infrared (nir) spectroscopy: Focus on parameters optimization and model interpretation*, Chemometrics and Intelligent Laboratory Systems, **96(1)**, pp. 27–33.
- [18] DOLADO JJ, GONZALO J and MAYORAL L, 2002, *A fractional dickey–fuller test for unit roots*, Econometrica, **70(5)**, pp. 1963–2006.
- [19] DU TOIT E, HALL JH and PRADHAN RP, 2018, *The day-of-the-week effect: South african stock market indices*, African Journal of Economic and Management Studies, **9(2)**, pp. 197–212.
- [20] FAROOQ O, BOUADDI M, AHMED N *et al.*, 2013, *Day of the week and its effect on stock market volatility: Evidence from an emerging market*, Journal of Applied Business Research (JABR), **29(6)**, pp. 1727–1736.
- [21] FRYZLEWICZ P, 2005, *Modelling and forecasting financial log-returns as locally stationary wavelet processes*, Journal of Applied Statistics, **32(5)**, pp. 503–528.
- [22] FXPRO, *Online trading broker*, [Online], [Cited September 27th, 2021], Available from <https://www.fxpro.com/>.
- [23] FXTM, *Online trading broker*, [Online], [Cited March 15th, 2023], Available from <https://www.forextime.com/>.
- [24] GÉRON A, 2019, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*, 2nd Edition, O'Reilly Media, Inc., Sebastol (CA).
- [25] GLOROT X and BENGIO Y, 2010, *Understanding the difficulty of training deep feedforward neural networks*, Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, Sardinia, Italy, pp. 249–256.
- [26] HE K, ZHANG X, REN S and SUN J, 2015, *Delving deep into rectifiers: Surpassing human-level performance on imagenet classification*, Proceedings of the IEEE international conference on Computer Vision, Washington DC, United States, pp. 1026–1034.
- [27] HOCHREITER S and SCHMIDHUBER J, 1997, *Long short-term memory*, Neural computation, **9(8)**, pp. 1735–1780.
- [28] HU Z, ZHAO Y and KHUSHI M, 2021, *A survey of forex and stock price prediction using deep learning*, Applied System Innovation, **4(1)**, p. 9.
- [29] HUTTER F, KOTTHOFF L and VANSCHOREN J, 2019, *Automated machine learning: methods, systems, challenges*, Springer Nature, Switzerland (AG).
- [30] IG, *Online trading broker*, [Online], [Cited October 2nd, 2022], Available from <https://www.ig.com/za/forex>.

- [31] JIANG W, 2021, *Applications of deep learning in stock market prediction: recent progress*, Expert Systems with Applications, **184**, p. 115537.
- [32] KAMRUZZAMAN J and SARKER RA, 2004, *Application of support vector machine to forex monitoring*, IEEJ Transactions on Electronics, Information and Systems, **124(10)**, pp. 1944–1951.
- [33] KAMRUZZAMAN J, SARKER RA and AHMAD I, 2003, *Svm based models for predicting foreign currency exchange rates*, Proceedings of the 3rd IEEE international conference on data mining, Washington DC, United States, pp. 557–560.
- [34] KELTNER C, 1960, *How to Make Money in Commodities*, Keltner Statistical Service, Kansas City (MO).
- [35] KLEINBAUM DG, KLEIN M, KLEINBAUM DG and KLEIN M, 2010, *Logistic regression: a self-learning text*, 3rd Edition, Springer, New York (NY).
- [36] LANE GC, 1984, *Lane's stochastics*, Technical Analysis of Stocks and Commodities, **2(3)**, pp. 87–90.
- [37] LECUN Y, BOTTOU L, BENGIO Y and HAFNER P, 1998, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, **86(11)**, pp. 2278–2324.
- [38] LI Z, LIU F, YANG W, PENG S and ZHOU J, 2021, *A survey of convolutional neural networks: analysis, applications, and prospects*, IEEE transactions on neural networks and learning systems, **33(12)**, pp. 6999–7019.
- [39] LIOUDIS N, 2023, *3 Factors That Drive the U.S. Dollar*, [Online], [Cited September 13th, 2023], Available from <https://www.investopedia.com/articles/forex/09/factors-drive-american-dollar.asp>.
- [40] LJUNG GM and BOX GE, 1978, *On a measure of lack of fit in time series models*, Biometrika, **65(2)**, pp. 297–303.
- [41] LU CJ, LEE TS and CHIU CC, 2009, *Financial time series forecasting using independent component analysis and support vector regression*, Decision support systems, **47(2)**, pp. 115–125.
- [42] MALKIEL B, 1989, *Efficient Market Hypothesis*, pp. 1–44 in EATWELL J, MILGATE M AND NEWMAN P (EDS), *Finance*, Palgrave Macmillan UK, London.
- [43] MCCULLOCH WS and PITTS W, 1943, *A logical calculus of the ideas immanent in nervous activity*, The bulletin of mathematical biophysics, **5(1)**, pp. 115–133.
- [44] MITCHELL C, 2021, *Major pairs: definition in forex trading and how to trade*, [Online], [Cited July 7th, 2021], Available from <https://www.investopedia.com/terms/forex/m/majors.asp>.
- [45] MITCHELL C, 2023, *Forex (FX): Definition, How to Trade Currencies, and Examples*, [Online], [Cited May 16th, 2023], Available from <https://www.investopedia.com/terms/f/forex.asp>.
- [46] MUNKHDALAI L, MUNKHDALAI T, PARK KH, LEE HG, LI M and RYU KH, 2019, *Mixture of activation functions with extended min-max normalization for forex market prediction*, IEEE Access, **7(1)**, pp. 183680–183691.

- [47] MURPHY JJ, 1999, *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*, Penguin, New York (NY).
- [48] MUSTAPHA A, MOHAMED L and ALI K, 2021, *Comparative study of optimization techniques in deep learning: Application in the ophthalmology field*, Proceedings of the 1st Journal of Physics: Conference Series, Khouribga, Morocco, Volume 1743, p. 012002.
- [49] NI L, LI Y, WANG X, ZHANG J, YU J and QI C, 2019, *Forecasting of forex time series data based on deep learning*, Procedia computer science, **147(1)**, pp. 647–652.
- [50] NOGUEIRA F, 2014, *Bayesian Optimization: Open source constrained global optimization tool for Python*, [Online], [Cited October 5th, 2022], Available from <https://github.com/fmfn/BayesianOptimization>.
- [51] NVIDIA CUDNN, *GPU-Accelerated Library for Deep Neural Networks*, [Online], [Cited June 30th, 2022], Available from <https://developer.nvidia.com/cudnn>.
- [52] PATEL J, SHAH S, THAKKAR P and KOTECHEA K, 2015, *Predicting stock market index using fusion of machine learning techniques*, Expert Systems with Applications, **42(4)**, pp. 2162–2172.
- [53] PEDREGOSA F *et al.*, 2011, *Scikit-learn: Machine learning in Python*, Journal of Machine Learning Research, **12(1)**, pp. 2825–2830.
- [54] PONGSENA W, SITSAYABUT P, KERDPRASOP N and KERDPRASOP K, 2021, *Development of a model for predicting the direction of daily price changes in the forex market using long short-term memory*, International Journal of Machine Learning and Computing, **11(1)**, pp. 61–67.
- [55] PROBST P, WRIGHT MN and BOULESTEIX AL, 2019, *Hyperparameters and tuning strategies for random forest*, Wiley Interdisciplinary Reviews: data mining and knowledge discovery, **9(3)**, pp. 1–15.
- [56] PYTHON SOFTWARE FOUNDATION, *Python 3.9*, [Online], [Cited June 5th, 2022], Available from <https://www.python.org/downloads/release/python-390/>.
- [57] QI L, KHUSHI M and POON J, 2020, *Event-driven lstm for forex price prediction*, Proceedings of the IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), Gold Coast, Australia, pp. 1–6.
- [58] RABBI MF, MOON MH, DHONNO FT, SULTANA A and ABEDIN MZ, 2022, *Foreign Currency Exchange Rate Prediction Using Long Short-Term Memory, Support Vector Regression and Random Forest Regression*, pp. 251–267 in KÖSEOĞLU SD (EDS), *Financial Data Analytics: Theory and Application*, Springer Nature, Switzerland (AG).
- [59] RANA M, UDDIN MM and HOQUE MM, 2019, *Effects of activation functions and optimizers on stock price prediction using lstm recurrent networks*, Proceedings of the 3rd International Conference on Computer Science and Artificial Intelligence, Illinois, United States, pp. 354–358.
- [60] REFAELZADEH P, TANG L and LIU H, 2009, *Cross-Validation*, Springer US, Boston, (MA).
- [61] ROSENBLATT F, 1958, *The perceptron: a probabilistic model for information storage and organization in the brain.*, Psychological review, **65(6)**, pp. 386–408.

- [62] RUMELHART DE, HINTON GE and WILLIAMS RJ, 1986, *Learning representations by back-propagating errors*, nature, **323(6088)**, pp. 533–536.
- [63] SERLETIS A and ROSENBERG AA, 2009, *Mean reversion in the us stock market*, Chaos, Solitons & Fractals, **40(4)**, pp. 2007–2015.
- [64] SHAHRIARI B, SWERSKY K, WANG Z, ADAMS R and DE FREITAS N, 2016, *Taking the human out of the loop: A review of bayesian optimization*, Proceedings of the IEEE, **104(1)**, pp. 148–175.
- [65] SIAMI-NAMINI S and NAMIN AS, 2018, *Forecasting economics and financial time series: Arima vs. lstm*, Proceedings of the 17th IEEE international conferneec on machine learning and applications (ICMLA), Orlando, Florida, United States, pp. 1394–1401.
- [66] SONG YY and YING L, 2015, *Decision tree methods: applications for classification and prediction*, Shanghai archives of psychiatry, **27(2)**, pp. 130–135.
- [67] STEVENSON S, 2007, *A comparison of the forecasting ability of arima models*, Journal of Property Investment & Finance, **25(3)**, pp. 223–240.
- [68] SUTHEEBANJARD P and PREMCHAIWADI W, 2010, *Analysis of calendar effects: Day-of-the-week effect on the stock exchange of thailand*, International Journal of Trade, Economics and Finance, **1(1)**, pp. 57–62.
- [69] VEZERIS D, KYRGOS T and SCHINAS C, 2018, *Take profit and stop loss trading strategies comparison in combination with an macd trading system*, Journal of risk and financial management, **11(3)**, p. 56.
- [70] VON STEIN H, 2020, *The importance of input features on deep neural networks when predicting foreign exchange rates*, MComm Thesis, University of Stellenbosch, Stellenbosch.
- [71] WILDER JW, 1978, *New concepts in technical trading systems*, Trend Research, Greensboro (NC).
- [72] WILSON JH and KEATING B, 2002, *Business forecasting with ForecastX*, 6th Edition, McGraw-Hill Irwin, New York (NY).
- [73] WITS, *South Africa Trade*, [Online], [Cited June 5th, 2023], Available from <https://wits.worldbank.org/CountrySnapshot/en/ZAF/textview>.
- [74] WORLD GOVERNMENT BONDS, 2023, *South africa credit rating*, [Online], [Cited February 6th, 2023], Available from <https://www.worldgovernmentbonds.com/credit-rating/south-africa/>.
- [75] YADAV A, JHA C and SHARAN A, 2020, *Optimizing lstm for time series prediction in indian stock market*, Procedia Computer Science, **167(1)**, pp. 2091–2100.
- [76] YILDIRIM DC, TOROSLU IH and FIORE U, 2021, *Forecasting directional movement of forex data using lstm with technical and macroeconomic indicators*, Financial Innovation, **7(1)**, pp. 1–36.
- [77] ZHENG A and CASARI A, 2018, *Feature engineering for machine learning: principles and techniques for data scientists*, O'Reilly Media, Inc., Sebastopol (CA).
- [78] ZHOU SK, GREENSPAN H and SHEN D, 2017, *Deep learning for medical image analysis*, Academic Press, New York (NY).